



Guidewire InsuranceSuite™ for Guidewire Cloud

InsuranceSuite Server Administration

Release: 2026.03.0



© 2026 Guidewire Software, Inc.

For information about Guidewire trademarks, visit <https://www.guidewire.com/legal-notices>.

Guidewire Proprietary & Confidential — DO NOT DISTRIBUTE

Product Name: Guidewire InsuranceSuite application for Guidewire Cloud

Product Release: 2026.03.0

Document Name: InsuranceSuite Server Administration

Document Revision: 01-April-2026

Contents

Support.....	13
---------------------	-----------

Part 1

Server administration overview.....	15
--	-----------

Part 2

InsuranceSuite server configuration.....	17
---	-----------

Important server configuration files.....	17
Understanding the configuration <registry> element.....	18
The <registry> element.....	18
Example syntax for the <server> element.....	20
Example syntax for the <systemproperty> element.....	21
Defining a new server role.....	21
JVM options and server properties.....	22
JVM options for gwb build commands.....	22
JVM options specific to the runServer build command.....	23
Setting JVM options for an InsuranceSuite application server.....	24
Ways to override configuration parameters.....	24

Part 3

InsuranceSuite application server administration.....	27
--	-----------

How to start a Guidewire InsuranceSuite application.....	27
Start an InsuranceSuite application on QuickStart (Jetty).....	28
Start the application server from Guidewire Studio.....	28
How to stop a Guidewire InsuranceSuite application.....	29
Stop a Guidewire InsuranceSuite application.....	29
Server startup tests.....	29
Server modes.....	30
Features available in each server mode.....	30
Setting the server mode.....	31
Determining the server mode.....	31
Server run levels.....	31
Server modes and server run levels.....	32
Determining the server run level.....	35
Use system tools to determine the server run level.....	35
Use web services to determine the server run level.....	36
Setting the QuickStart server run level.....	36
Set the QuickStart run level at server start.....	36
Set the server run level through system tools.....	36
Set the server run level through web services.....	37
Server maintenance mode.....	37

Place the server in maintenance mode.....	37
Monitoring server performance in an InsuranceSuite application.....	38
Monitoring and managing event messages.....	38
About message failure.....	38
The Messages screen.....	39
The Message Queues screen.....	39
Access the messaging editor.....	40
Purge completed messages.....	41
Session timeout.....	41
User session replication.....	42
Cache management.....	42
Caching and stickiness.....	43
Concurrent data change prevention.....	43
Cache behavior and performance.....	43
Cache thrashing.....	44
Detect cache thrashing.....	44
Cache impact on memory utilization.....	44
Server cache tuning parameters.....	45
Understanding cache metrics.....	46
Special caches.....	46

Part 4

Server tools.....49

The Batch Process Info screen.....	49
Processes table columns.....	50
Chart and History tabs.....	51
Download a batch process history report.....	51
The Work Queue Info screen.....	52
Work Queue table columns.....	52
Item Statistics tabs and columns.....	53
The By Writers tab.....	53
The By Executors tab.....	53
The Work Items tab.....	54
Work queue reports.....	55
The Work Queue report.....	55
Download a work queue report.....	55
The Work Queue Raw Data report.....	56
Download the Work Queue Raw Data report.....	56
The Work Queue History report.....	56
Download the Work Queue History report.....	56
The Set Log Level screen.....	57
The View Logs screen.....	58
The Intentional Logging screen.....	58
Info pages.....	59
The Configuration screen.....	59
The Archive Info screen.....	60
The Domain Graph Info screen.....	61
The Consistency Checks screen.....	61
The Run Consistency Checks tab.....	62
The View consistency checks definitions tab.....	63

Understanding consistency check severity.....	64
Run a consistency check from an InsuranceSuite application.....	64
Run a consistency check using system tools.....	65
Correct a consistency check SQL failure.....	65
Consistency check frequency.....	66
The Database Table Info screen.....	66
Database Table Info reports.....	66
Understanding the Database Table Info reports.....	67
View the Database Table Info reports.....	68
The Database Parameters screen.....	68
The Database Storage Information screen.....	69
The Data Distribution screen.....	70
Generate and view a data distribution report.....	70
Download comparison and combined data distribution reports.....	70
The Database Statistics screen.....	71
Database statistics generation.....	71
Working with database statistics reports.....	71
Generate and download a database statistics report.....	71
The Execution History tab.....	72
The PostgreSQL Performance Report screen.....	72
The Load History Information screen.....	73
View a load history report.....	74
The load history detail report.....	74
The Load Integrity Checks screen.....	75
The Load Errors screen.....	76
The Runtime Environment Info screen.....	76
The Safe Persisting Order screen.....	76
The Loaded Gosu Classes screen.....	77
The Serialization Info screen.....	77
Serialization list files.....	77
The Management Beans screen.....	78
The Startable Services screen.....	78
The Cluster Members screen.....	79
Cluster Members (Application Server instances).....	79
Cluster Members (Application Server instances).....	80
Cluster Members (Components tab).....	80
Cluster Members (History tab).....	81
The Components screen.....	81
Download a server component report.....	83
About planned server shutdowns.....	83
Schedule a planned cluster member shutdown.....	84
The Upgrade and Versions screen.....	85
Upgrade reports.....	88
View an upgrade report.....	89
Understanding Guidewire software versioning.....	89
File customer-version.properties.....	90
Create a custom version label file.....	90
Deploy a custom version label file on Tomcat.....	90
The About Guidewire < <i>InsuranceSuite Application</i> > pop-up.....	91
The Cache Info screens.....	91
The Cache Summary screen.....	92

The Historical Performance screen.....	93
The Cache Details screen.....	93
Understanding the cache data report.....	93
The Guidewire Profiler screens.....	94
Basic profiler concepts.....	96
Starting and stopping Guidewire Profiler.....	96
About the different Guidewire profilers.....	97
Web (UI) profiler.....	97
Entry-point profilers.....	99
Database upgrade profiler.....	100
General considerations for using Guidewire Profiler.....	100
Working with Guidewire profilers.....	102
The Profiler Configuration screen.....	102
Enable web profiling.....	103
Enable an entry-point profiler.....	104
Disable an entry-point profiler.....	104
Enable profiler for work queues.....	104
Save a profiler analysis report.....	105
View profiler analysis reports.....	105
View historical profiler reports.....	106
View uploaded profiler reports.....	106
View profiler upgrade information.....	106
Working with Profiler data.....	106
The Profiler Analysis screen.....	106
Understanding application profiling.....	110
Profiler tags.....	110
Profiler frames.....	113
Profiler stacks.....	113
Using custom profiler tags with Guidewire Profiler.....	114
Understanding properties and counters on a frame.....	115
Understanding the Guidewire Profiler API.....	115
Profiler-related configuration parameters.....	115
The Product Model Info screen.....	116
The JProfiler screen.....	116
The Metro Reports screen.....	116
The Funds Tracking screen.....	117
The Commission Drilldown screen.....	117
The Prioritized Preupdate screen.....	117

Part 5

InsuranceSuite application clusters..... 119

1 About InsuranceSuite application clusters.....	121
2 Monitoring cluster health.....	123
Monitoring InsuranceSuite application clusters.....	124
Using the InsuranceSuite application server ping utility.....	124
Using the ping utility with a production server.....	125
Example output from the ping utility.....	125
3 Working with a InsuranceSuite application server cluster.....	129
Create an InsuranceSuite application cluster.....	129
Enabling clustering.....	130

Adding a InsuranceSuite application server to cluster.....	130
Add a server to a InsuranceSuite application cluster.....	130
Restarting the InsuranceSuite application servers in a cluster	131
Server restart after server maintenance.....	131
Server restart after full InsuranceSuite application upgrade.....	131
Server restart after database restore.....	131
4 Working with component leases.....	133
Simple lease management lifecycle for a batch process.....	133
Simple lease management lifecycle for a message destination.....	134
Component lease failover.....	134
Automatic failover of a component lease.....	134
The background task failover plugin.....	136
Component load balancing.....	136
Plugin BackgroundTaskLoadBalancingPlugin.....	137
5 Deploying changes in a cluster.....	141

Part 6

Geocoding, catastrophe search, and heat maps.....143

Understanding geocoding.....	143
Configuring geocoding	144
About Guidewire geocoding parameters.....	145
Enable the Geocode plugin (Azure Maps).....	146
Enable the Geocode plugin (Bing Maps).....	147
Schedule Geocode batch processing runs.....	147
Configure the number of worker instances for Geocode batch processing.....	148
Catastrophe search and heat maps.....	148
Enable the catastrophe heat map.....	149
Enable the catastrophe policy location download batch process.....	149

Part 7

External server configuration.....151

About external server configuration.....	152
Substitution placeholders.....	152
Understanding namespace.....	154
Files that support configuration substitution.....	154
Processes that support configuration substitution.....	154
Stand-alone InsuranceSuite application servers.....	155
Clustered InsuranceSuite application servers.....	155
InsuranceSuite application system tools.....	155
InsuranceSuite application server gwb tasks.....	155
Dynamic configuration parameters.....	156
Dynamic product URLs.....	158
Additional configuration file replacements.....	159
Free-text search engine configuration files.....	159
Logging property files.....	159
Web service collection files.....	159
Logging substitution activity.....	162
Verifying external configuration.....	162

Working with external configuration files.....	163
Setting the path to a local file.....	164
Local development environments.....	164
Loading configuration parameters.....	164
Working with Guidewire Settings Vault.....	165
Plugin ExternalConfigurationProviderPlugin.....	166
ExternalConfigurationProviderPlugin implementation.....	166
Accessing properties from code.....	167

Part 8

Setting InsuranceSuite application properties at run time.....169

Working with Runtime Properties.....	169
Import runtime properties from a file.....	171
Configuring runtime properties.....	171
Class RuntimePropertyRetriever.....	172

Part 9

Work queues and batch processing..... 173

Administering InsuranceSuite application processes.....	173
Work queues.....	174
Work queue writers.....	174
Work queue workers.....	175
Work queue scheduling and processing intervals.....	175
Work queue architecture.....	176
Batch processes.....	177
Configuring the different processing types in Studio.....	178
Executing processes in Guidewire Studio.....	178
Working with work queue writers and batch processes.....	178
Run a writer from an InsuranceSuite application.....	179
Run a batch process from an InsuranceSuite application.....	179
Run a writer or batch process from the command prompt.....	180
Terminate a writer or batch process from the command prompt.....	180
Check status of a writer or batch process from the command prompt.....	180
Scheduling work queue writers and batch processes.....	181
Defining a schedule specification.....	182
Determine if it is possible to schedule a batch process.....	183
View a batch process schedule.....	184
Using an external batch scheduler.....	184
Configuring work queues.....	184
The work queue configuration file.....	184
General work queue configuration.....	185
Worker configuration.....	186
Worker task management.....	187
Work queues and server roles.....	187
Performing custom actions after a process completes.....	188
Schedule the Process Completion Monitor batch process.....	188
Implement the IBatchCompletedNotification interface.....	189
Register a custom batch notification plugin.....	190

Troubleshooting work queues.....	190
InsuranceSuite application base processes.....	190
Common base configuration processes.....	191
Activity Escalation work queue.....	191
Data Distribution batch process.....	191
Database Consistency Check work queue.....	192
Database Statistics work queue.....	193
Deferred Upgrade Tasks batch process.....	193
Destroy Contact for Personal Data work queue.....	194
Geocode Writer work queue.....	195
Group Exception work queue.....	195
Inbound Chunk work queue.....	195
Inbound File batch process.....	196
Inbound File Purge work queue.....	196
Notify External System for Personal Data work queue.....	196
Outbound File batch process.....	197
Outbound File Purge work queue.....	197
Outbound Record Purge work queue.....	197
Phone Number Normalizer work queue.....	198
Populate Search Columns batch process.....	198
Process Completion Monitor batch process.....	199
Process History Purge batch process.....	199
Purge Cluster Members batch process.....	199
Purge Failed Work Items batch process.....	200
Purge Message History batch process.....	200
Purge Old Contact Destruction Request work queue.....	201
Purge Old Transaction IDs batch process.....	201
Purge Profiler Data batch process.....	202
Purge Workflows batch process.....	202
Purge Workflow Logs batch process.....	203
Solr Data Import batch process.....	203
User Exception work queue.....	204
Work Item Set Purge batch process.....	204
Work Queue Instrumentation Purge batch process.....	204
Workflow work queue.....	205
PolicyCenter base configuration processes.....	205
Account Holder Count work queue.....	205
Account Withdraw Evaluation work queue.....	206
Apply Pending Account Data Updates work queue.....	206
Asynchronous Quoting work queue.....	207
Asynchronous Rating work queue.....	208
Audit Task work queue.....	209
Bound Policy Exception work queue.....	209
Bulk Submission Job batch process.....	209
Clean Up Account Contact Role Table work queue.....	210
Clean Up PurgedRootInfo work queue.....	210
Clear Policy Renewal Check Dates batch process.....	210
Closed Policy Exception work queue.....	211
Create UW Rules for UW Issue Types work queue.....	211
Extract Rating Worksheets work queue.....	212
Fix Job Group On Moved Policies work queue.....	212

Form Text Data Delete batch process.....	212
Handle Unresolved Contingency work queue.....	213
Impact Testing Export work queue.....	213
Impact Testing Test Case Preparation work queue.....	213
Impact Testing Test Case Run work queue.....	214
Job Expire work queue.....	214
Make util uppercase work queue.....	215
Open Policy Exception work queue.....	215
Overdue Premium Report work queue.....	215
Policy Hold Job Evaluation work queue.....	216
Policy Locations Risk Assessment Temporary Store work queue.....	216
Policy Renewal Start work queue.....	217
Premium Ceding work queue.....	217
Purge work queue.....	217
Purge Orphaned Policy Periods work queue.....	217
Purge Quote Clones work queue.....	218
Purge Rate Book Export Result work queue.....	218
Purge Rating Worksheets work queue.....	218
Purge Risk Assessment Temporary Store work queue.....	219
Purge Temporary Policy Periods work queue.....	219
Rate Book Export work queue.....	220
Recalculate Contingency Action Start Date work queue.....	220
Reset Purge Status and Check Dates work queue.....	220
Retire Activities work queue.....	220
Team Screens batch process.....	221
ClaimCenter base configuration processes.....	221
Address Delete work queue.....	221
Aggregate Limit Calculations work queue.....	222
Bulk Invoice Escalation batch process.....	223
Bulk Invoice Workflow Monitor batch process.....	223
Catastrophe Claim Finder batch process.....	223
Catastrophe Policy Location Download batch process.....	224
Claim Contacts Calculations batch process.....	225
Claim Exception work queue.....	225
Claim Health Calculations batch process.....	226
ClaimCenter (SPM) Completed Review Synchronization work queue.....	226
Contact Auto-synchronization work queue.....	227
Contact Retire work queue.....	227
Dashboard Statistics batch process.....	228
Encryption Upgrade work queue.....	228
Exchange Rate batch process.....	228
Financials Calculations batch process.....	229
Financials Escalation work queue.....	230
Idle Claim Exception work queue.....	230
Idle Closed Claim Exception work queue.....	231
Invoice Item Processing work queue.....	231
Invoice Processing work queue.....	232
Recalculate Claim Metrics work queue.....	233
Service Request Metric Escalation work queue.....	233
Statistics batch process.....	234
T-account Escalation batch process.....	234

User Workload Update work queue.....	235
BillingCenter base configuration processes.....	235
Account Inactivity work queue.....	235
Advance Expiration work queue.....	236
Agency Suspense Payment work queue.....	236
Allot Funds for Funds Tracking work queue.....	237
Automatic Disbursement work queue.....	237
Charge ProRata Transaction work queue.....	237
Collateral Requirement Effective work queue.....	238
Collateral Requirement Expiration work queue.....	239
Commission Payable Calculations work queue.....	239
Commission Payment work queue.....	240
Delete Account Commissions Expense TAccount work queue.....	241
Disbursement work queue.....	241
Full Pay Discount work queue.....	242
Invoice work queue.....	243
Invoice Due work queue.....	245
Legacy Agency Bill work queue.....	245
Legacy Collateral work queue.....	245
Legacy Delinquency work queue.....	246
Letter of Credit work queue.....	247
New Payment work queue.....	247
Payment Request work queue.....	247
Policy Closure work queue.....	248
Post Batch Payments work queue.....	249
Premium Reporting Report Due work queue.....	250
Producer Payment work queue.....	250
Release Charge Holds work queue.....	254
Release Trouble Ticket Hold Types work queue.....	254
Reverse Batch Payments work queue.....	254
Statement Billed work queue.....	255
Statement Due work queue.....	255
Suspense Payment work queue.....	256
Trouble Ticket Escalation work queue.....	257
Upgrade Producer Statement Hold Limits batch process.....	257
Workflow Writer work queue.....	258
Write-off Staging work queue.....	258
Other processes.....	258
Unused and internal batch processes.....	258
Archive-related processes.....	259
Custom processing.....	260
Batch processing typecodes.....	260
About scheduling InsuranceSuite application processes.....	260
Night-time processing.....	261
Daytime processing.....	261
Custom work queues.....	262
About custom work queue classes.....	262
Work queues and work item entity types.....	262
Work queues that use StandardWorkItem.....	263
Lifecycle of a work item.....	263
Considerations for developing a custom work queue.....	264

Define a typecode for a custom work queue.....	265
Define a custom work item type.....	266
Creating a custom work queue class.....	267
Developing the writer for your custom work queue.....	267
Developing workers for your custom work queue.....	268
Bulk insert work queues.....	270
Overview of bulk insert work queues.....	270
Bulk insert work queue declaration and constructor.....	271
Querying for targets of a bulk insert work queue.....	271
Eliminating duplicate items in bulk insert work queue queries.....	271
Processing work items in a custom bulk insert work queue.....	272
Example work queue that bulk inserts its work items.....	272
Examples of custom work queues.....	273
Example work queue that extends WorkQueueBase.....	273
Example work queue for updating entities.....	274
Example work queue with a custom work item type.....	275
Example of the real-time creation of work items.....	276
Delayed processing of real-time work items.....	283
Developing custom batch process.....	285
Create a custom batch process.....	286
Define a typecode for a custom batch process.....	286
Batch process base class.....	287
Useful properties on class BatchProcessBase.....	287
Useful methods on class BatchProcessBase.....	288
Examples of custom batch processes.....	292
Example batch process for a background task.....	292
Example batch process for unit of work processing.....	293
Working with custom processing.....	295
Categorizing a process typecode.....	295
Updating the work queue configuration.....	295
Implementing IProcessesPlugin	296
Retrying failed batch process items.....	297
Monitoring InsuranceSuite application processes.....	299
Monitoring InsuranceSuite application processing using administrative screens.....	300
Monitoring InsuranceSuite application processes for completion.....	300
Monitoring batch processes using maintenance tools.....	300
Managing user entity updates in batch processing.....	301
Set the external user field in a batch process.....	301

Support

For assistance, visit the Guidewire Community.

Guidewire customers

<https://community.guidewire.com>

Guidewire partners

<https://partner.guidewire.com>

Server administration overview

InsuranceSuite server management includes the following types of activities:

- Server configuration
- Server administration
- Cluster administration
- Geocode configuration and management (ClaimCenter, PolicyCenter, and ContactManager)
- InsuranceSuite application process monitoring and management

Server configuration

Using file `config.xml`, you can configure items such as the following:

- Server environment
- Server roles
- System properties whose values you specify at server start up

Server administration

Server administration includes such items as the following:

- Starting and stopping the application server
- Setting server modes and run levels
- Monitoring server performance
- Managing server memory

Cluster administration

To improve performance and reliability in production planets (environments), Guidewire installs multiple InsuranceSuite application servers in a configuration known as a cluster. A InsuranceSuite application cluster distributes client connections among multiple InsuranceSuite application servers, reducing the load on any one server. If one server fails, the other servers seamlessly handle its traffic.

Geocode configuration and management

InsuranceSuite supports the ability to configure and manage geocoding for ClaimCenter, PolicyCenter, and ContactManager. You can also enable ClaimCenter to use geocoding in producing ClaimCenter catastrophe searches and heat maps.

InsuranceSuite application process management

The InsuranceSuite application server contains the ability to process a batch of items in parallel (as a work queue), or, sequentially (as a batch process). Typically, many of these processes operate on a fixed schedule. Others you must manually execute. The InsuranceSuite application provides a means to monitor and manage many of these processes on the following Server Tools screens:

- **Batch Process Info** screen
- **Work Queue Info** screen

InsuranceSuite server configuration

During startup, the InsuranceSuite application server calculates key environment and server properties. These property values describe the environment in which the server runs. After you set environment properties, you can access these properties through the following methods:

- InsuranceSuite commands
- Gosu code
- Java code

It is possible to specify environment property values and server roles either through JVM (Java Virtual Machine) options or through the use of the `<registry>` element:

- You define environment and server properties in the `<registry>` element in file `config.xml`.
- You set environment and server properties through command prompt JVM options as you to start a InsuranceSuite application server.

IMPORTANT: Guidewire recommends that you specify server IDs and server roles for individual application servers using JVM options as you start the application server rather than defining these values using the `<registry>` element in `config.xml`.

Important server configuration files

Guidewire Studio places several important configuration files at the root of the following **Project** directory:

configuration > **config**

If you do not see this directory path, expand the Studio **Project** folder. The **config** directory contains the following important files.

config.xml

File `config.xml` contains global system parameters that you use to control the behavior of the application. These configuration parameters govern large-scale system options, such as authentication, server clustering, and the business calendar.

database-config.xml

File `database-config.xml` stores database connection information and Data Definition Language (DDL) options.

Internal configuration files

Directory **configuration** > **config** also contains internal file `config-service.yml`. This file is for Guidewire internal use only. Do not delete this file or modify it in any way.

Understanding the configuration <registry> element

It is possible to set the application server environment using the `<registry>` element in `config.xml`. Through the `<registry>` element, you can define the following:

- The set of server roles that are valid for this cluster.
- The set of server roles that are valid for each individual server instance in the cluster.
- The environment or environments in which a specific server operates, a development environment and/or a production environment, for example.

Also through the `<registry>` element, you can redefine certain system properties that you specify at server startup.

IMPORTANT: Guidewire recommends that you specify server IDs and server roles for individual application servers using JVM options as you start the application server rather than defining these values using the `<registry>` element in `config.xml`.

The <registry> element

The `<registry>` element in file `config.xml` has the following syntax.

```
<registry roles="role1, role2, ..." >
  <server env="environment1, environment2, ..." roles="role1, role2, ..." serverid="serverID" />
  <systemproperty name="name" value="value" default="default" />
</registry>
```

File `config.xml` contains exactly one required `<registry>` element. The `<registry>` element can contain zero to many `<server>` and `<systemproperty>` elements.

The attributes on the various elements have the following meanings.

Element	Attribute	Required	Description
registry	roles	Yes	List of valid cluster roles.
server	env serverid roles	No	The <code><server></code> attribute entries are legacy and not required. These attributes have the following meanings:

Element	Attribute	Required	Description
			- env - The name of the environment in which the server operates. The default is null. The value of env is immutable while the server is running. - serverid - Unique server name of a server instance. If you do not specify this value explicitly, the application server sets it to the hostname of the application server. - roles - Specifies role or roles to assign to the server specified by serverid. It is a better practice to set these server attributes using JVM options during server startup. For example, the following JVM option sets both the server ID and server roles: -Dserverid="dev2#messaging,scheduler" Use the # syntax to indicate the following text is a server role and not a server ID.
systemproperty name		Yes	Specifies the name of the Java -D system property that you want to redefine. This value can be one of the following: - env - serverid
	value	Yes	New name of the system property.
	default	Yes	Default property value to use if you do not specify a value at the command prompt.

The following code shows an example <registry> element.

```
<registry roles="batch, scheduler, workqueue, messaging, startable, ui, custom1">
  <systemproperty name="env" value="my.env" default="production"/>
  <systemproperty name="serverid" value="my.id" default="myDefault"/>
</registry>
```

Notice that:

- The set of valid server roles includes a custom role (custom1).
- There are two system property redefinitions, one for env and one for serverid.

Default server roles

If you run the InsuranceSuite application as a single server, Guidewire assigns all of the defined server roles to that server by default.

Property serverid.noroles

In standard usage, the value of property serverid includes also the list of server roles defined for that server. If you want to use this property in code, without the list of server roles, use serverid.noroles instead. For example, for the ClaimCenter application server, instead of using gw.cc.serverid, use gw.cc.serverid.noroles to suppress the list of server roles associated with this server ID.

Logging environment

The env property has special significance for logging behavior. If the env value is non-null, the application server tries to obtain the logging configuration from a config/logging/env-log4j2-json.xml file. If this file does not exist or if env is null, then the logging configuration is taken from the default log4j2-json.xml file.

Creating special server roles

It is possible to define new server roles by adding the role to the list of roles specified by the roles attribute on the <registry> element.

Accessing system properties in code

Gosu class gw.api.system.server.ServerUtil contains methods for working with system properties associated with servers.

Example syntax for the <server> element

The following example illustrates how to use the <server> element of the <registry> element in config.xml to set various server properties.

```
<registry roles=".....">
  <server env="..." serverid="..." roles="..." />
</registry>
```

Use the roles attribute to list the valid server roles for your installation. However, the existence of the <server> element is a legacy artifact. Guidewire discourages the use of the <server> element. Instead, use JVM options to set these values.

Using a JVM option to set server properties

Guidewire recommends that you set server system properties using a -D JVM option from a command prompt at server startup. For example, you can use JVM options to set the environment variable (env) and the server ID and server roles (serverid) at runtime using the syntax listed in the following table. The option syntax varies by server type. Notice the entries for serverid. The use of a starting # mark indicates that the following text is a server role and not a server ID.

Server type	Syntax	Example
QuickStart (Jetty)	-Denv=...	gwb runServer -Denv=test2
	-Dserverid=...	gwb runServer -Dserverid=t2#messaging
Tomcat	-Dgw.xx.env=...	-Dgw.pc.env==test2
	-Dgw.xx.serverid=...	-Dgw.pc.serverid=t2#messaging

Note: In the filepath, "xx" is the product code which can be, for example pc.

If you are starting the Jetty development server in Guidewire Studio™, use the syntax for Tomcat in the **Run - Server** configuration dialog, for example:

```
-Dgw.xx.serverid=testServer
```

For more information, see “Start the application server from Guidewire Studio” on page 28.

How the application server uses a -D JVM option

The values of env and serverid are immutable while the server is running.

The InsuranceSuite application server determines the value of a -D option in the following manner, using -Dserverid (on Jetty) as an example:

- If you specify a -Dserverid=prodserver JVM option at the command prompt at server startup, the application server sets the value of serverid for that server to prodserver.
- If you do not specify a -Dserverid JVM option at server start, the application server checks the server registry for a serverid value defined by a server entry. If found, the application server uses that value. In the example, the serverid value is prodserver.
- If you do not specify the JVM option, and no serverid value defined by a server entry exists, the application server sets serverid to the host name of the computer. Under some extreme security settings, this value is not available, in which case the application server sets the serverid to localhost.

Note: Log entries display only the first 10 characters of the serverid value.

Viewing server information in the InsuranceSuite application

The InsuranceSuite application shows the serverid and role values for each server on the Server Tools **Cluster Members and Components** screens. You must log in as an administrative user to access the Server Tools.

Example syntax for the <systemproperty> element

The following example illustrates how to set the application server environment using the <systemproperty> element of the <registry> element in config.xml.

```
<registry roles="..." >
  <systemproperty name="env" value="my.env" default="production"/>
</registry>
```

Notice that the defined <systemproperty> element:

- Redefines the name of the env property to be my.env. Thus, in places in which you formerly used env, you now use my.env. For example, -Denv becomes -Dmy.env.
- Sets a default value (production) for the system property.

To use this system property, you specify its value as a -D JVM option at server startup. The option syntax varies by server type.

QuickStart (Jetty) -Dmy.env=...

Tomcat -Dgw.xx.my.env=... , where xx is the application product code.

The value of my.env is immutable while the server is running.

The server determines the value of the redefined env system property in the following manner, using -Dmy.env (on Jetty) as an example:

- If you specify the -Dmy.env=test JVM option at the command prompt at server startup, the application server sets the value of env to test.
- If you do not specify a -Dmy.env option at server start, the application server sets env to the value of default that you specified in the systemproperty, in this example, production.
- If you do not set the value of my.env either through a default registry property or with a JVM option, the application server sets the value of the env property to null.

The application server ignores any attempt to set the environment property through the command prompt if you have previously defined that property using a <systemproperty> element. For example, suppose that you set the following <registry> element in config.xml:

```
<systemproperty name="env" value="my.env" default="standalone" />
```

Then, at server startup, you specify a -Denv="test" JVM option. The application server ignores any -Denv option that you specify on the command prompt and sets the env value to standalone.

Defining a new server role

You define server roles using the roles attribute on the <registry> element in file config.xml. In the base configuration, Guidewire defines the following default server roles:

```
<registry roles="batch, workqueue, scheduler, messaging, startable, ui" />
```

To add a specialized server role, say, one to use in managing activities, add the new server role to the list of roles:

```
<registry
  roles="batch, workqueue, activity, scheduler, messaging, startable, ui"
/>
```

Note: Some server roles do not apply to High Volume Quoting nodes.

JVM options and server properties

It is possible to set certain InsuranceSuite application server properties by using `-D` JVM options on the `gwb` build commands. Some of these JVM options work with all the `gwb` build commands. Other JVM options work only with the `gwb runServer` build command. In all cases, the exact manner in which you define and use the JVM option depends on the type of server involved.

Once set, a server property is immutable while the server is running.

JVM options for gwb build commands

The following JVM options work with many of the InsuranceSuite application server `gwb` build commands.

-Denv=aaaa

Starts the application server using the specified environment variable (env).

-Dgw.passthrough.systemProperty=dddd

Starts the application server using the provided system property value. The option parameters have the following meanings:

- *systemProperty* – The name of a system property defined in the `<registry>` element in file `config.xml`.
- *dddd* – The value of the system property.

JVM env string value always lower case

If you use a JVM option to set the server environment, the InsuranceSuite application server converts the provided string to lower case automatically. For example, the conversion to lower case occurs if you use the following JVM options:

- Jetty - `-Denv`
- Tomcat - `-Dgw.xx.env`
- Studio - `-Dgw.xx.serverid`

Thus, if you enter `-Dgw.xx.env=TEST`, the application server converts the string `TEST` to the string `test`. Be aware that the automatic conversion of the env string to lower case can cause a referenced env value to not match the defined environment name.

Note: If you start the Quickstart server from the command line, use `-Doption`. If you use any other application server, including the server that starts from within Guidewire Studio, use `-Dgw.xx.option`, where `xx` is the application product code.

Reference table of core gwb commands

It is possible to use the `-Denv` and the `-Dgw.passthrough` JVM options with some, but not all, of the `gwb` build commands. The following table indicates whether the listed JVM command options work with the core `gwb` commands (tasks).

Core task	Can use JVM option	Cannot use JVM option
clean		•
cleanIdea		•

Core task	Can use JVM option	Cannot use JVM option
codegen		•
compile		•
dropDb	•	
genDataDictionary	•	
idea		•
runServer	•	
stopServer	•	
studio		•

JVM option examples

For example, to pass `-DmySystemProperty=someValue` to build command `dropDB`, use the following command option.

```
gwb dropDb -Dgw.passthrough.mySystemProperty=someValue
```

Then, to make the `dropDB` command specific to a test environment, use the following command (for a Jetty server).

```
gwb dropDb -Denv=test -Dgw.passthrough.mySystemProperty=someValue
```

The following build command illustrates the use of a pass through system property.

```
gwb runServer -Dgw.passthrough.javax.net.ssl.keyStore=/Java/jdk1.8.0_74/jre/lib/security/cacerts
```

For information on the `gwb` build commands, see the *Developer Setup Guide*.

JVM options specific to the runServer build command

The following JVM options work with the InsuranceSuite application server `gwb runServer` command only.

-Dgw.port=nnnn

Starts the InsuranceSuite application server on the specified port (*nnnn*). The server URL reflects this port number, for example:

`localhost:nnnn/xx/InsuranceSuiteApplication.do`, where *xx* is the application product code.

-Dgw.server.mode=xxxx

Starts the InsuranceSuite application server in the specified server mode. Valid values are:

- dev
- prod

The default is dev.

-Dserverid=aaaa, -Dserverid=aaaa#bbbb

Sets the server ID, and possibly, one or more server roles for the InsuranceSuite application server:

- Server ID – Without the hash mark (*#*), *aaaa* represents a server ID only.
- Server role – With the hash mark, *#bbbb* assigns the *bbbb* server role to the server with *aaaa* server ID. Use a comma-separated list, with no spaces, to list multiple server roles.

The exact JVM syntax to use depends on the server type, for example:

Quickstart (Jetty)

Use `-Dserverid=aaaa`

Tomcat

Use `-Dgw.xx.serverid=aaaa`, where `xx` is the application product code.

Studio

Use `-Dgw.xx.serverid=aaaa`, where `xx` is the application product code.

Note: If you start the Quickstart server from the command line, use `-Doption`. If you use any other application server, including the server that starts from within Guidewire Studio, use `-Dgw.xx.option`, where `xx` is the application product code.

Setting JVM options for an InsuranceSuite application server

It is possible to set server system properties using the `-D` JVM option syntax. How you set a command option depends on the server type.

QuickStart (Jetty)

Set the options at server start using the following syntax:

```
gwb runServer -Denv=...
gwb runServer -Dserverid=...
```

Studio

Set JVM options in the **Run** dialog in Studio. Use the following syntax:

`-Dgw.xx.option=...`, where `xx` is the application product code.

Tomcat

Set the options using the `CATALINA_OPTS` environment variable. Use the following syntax:

```
-Dgw.xx.env=..., where xx is the application product code.
-Dgw.xx.serverid=..., where xx is the application product code.
```

For information on the `gwb` build commands, see the *Developer Setup Guide*.

Ways to override configuration parameters

Guidewire provides the following mechanisms for overriding configuration values defined in file `config.xml` from a command prompt at server start up:

- `--systemProperty name=value`
- `--configParam name=value`
- `gw.config.paramName=value`

You must supply the name (*name*) and value (*value*) of the property to change, for example:

```
gwb runServer --configParam OverrideJSONSchemaWarnings=on
gwb runServer --systemProperty OverrideJSONSchemaWarnings=on
gwb runServer -Dgw.config.OverrideJSONSchemaWarnings=on
```


If you use `--systemProperty` and `--configParam` to redefine the same configuration parameter at the same time, InsuranceSuite uses the value set by `--configParam`.

These overrides work on the QuickStart (Jetty) application server only. Thus, you can only use these override in development or testing environments. It is not possible to use these overrides in a production environment. Also, if you attempt to override configuration parameters that Guidewire defines as permanent, or semi-permanent (one-way), the override will fail.

IMPORTANT: If you are using external server configuration, do not use these configuration override mechanisms.

Overriding parameters by environment

You can make the configuration override active in a single environment only. Set the environment from the command prompt as you start the application server using the following syntax:

```
env={dev|test|prod}
```

For example:

```
gwb runServer env=dev --systemProperty OverrideJSONSchemaWarnings=on
```


InsuranceSuite application server administration

Management of the InsuranceSuite application server includes a number of tasks, including the following:

- Stopping and starting the application server
- Setting the server mode and run level
- Monitoring server performance
- Monitoring and managing event messages
- Tuning server caching
- Managing server memory

How to start a Guidewire InsuranceSuite application

How you start InsuranceSuite application depends on the application server type.

QuickStart (Jetty)

See the following:

- “Start an InsuranceSuite application on QuickStart (Jetty)” on page 28
- “Start the application server from Guidewire Studio” on page 28

Tomcat

See the *Developer Setup Guide* for working with Tomcat.

Start an InsuranceSuite application on QuickStart (Jetty)

About this task

Guidewire recommends the following sequence when starting an InsuranceSuite application on the QuickStart server. For information on the `gwb` build commands, see the *Developer Setup Guide*.

Procedure

1. Open a command prompt and navigate to the root of the InsuranceSuite application directory.
2. Run the following command to compile the needed application resources and move them to the correct location in the application server:

```
gwb compile
```

3. Run the following command to start the application server.

```
gwb runServer -x compile
```

There is a dependency between the `runServer` command and the `compile` command. Guidewire recommends that you always use the `-x compile` option with the `runServer` command to remove that dependency after you initially run the `compile` command. Otherwise, InsuranceSuite must first verify what resources, if any, need to be recompiled, then perform an incremental recompile of those resources before starting the server.

After starting the application server, log in to InsuranceSuite application as described in the *Application Guide*.

Start the application server from Guidewire Studio

You can start the development server (Jetty) from Guidewire Studio™.

Procedure

1. In the Guidewire Studio™ instance for your InsuranceSuite application, click **Run > Run....** Studio opens a **Run** drop-down with a list of server-related options.
2. Select **Edit Configurations..** from the list. Studio opens the **Run - Server** dialog.
3. Select **Server** in the left-hand navigation pane and verify the configuration options set for the server.

It is possible to modify the base configuration server settings by adding additional VM options with the format `-Dgw.xx`. If you do so, use the following format (using server ID an example):

For BillingCenter: `-Dgw.bc.serverid=testServer`

For ClaimCenter: `-Dgw.cc.serverid=testServer`

For PolicyCenter: `-Dgw.pc.serverid=testServer`

4. Click **Run**. Studio opens a pane at the bottom of the screen to display the server log. This area also has controls (icons) for stopping and starting the server.

How to stop a Guidewire InsuranceSuite application

Before you stop a Guidewire InsuranceSuite application, you must stop all work queues. Distributed workers run on daemon threads. As the JVM (Java Virtual Machine) exits, it destroys these threads. This can cause issues if the JVM destroys a thread while that thread is processing a work item. For example, suppose that a work queue calls a plugin that makes a blocking call to an external system or otherwise take a long time to return. In that case, if you do not shut down the work queue threads correctly, it is possible to end up with inconsistent data.

Stop a Guidewire InsuranceSuite application

Procedure

1. Log into a Guidewire InsuranceSuite application as a user with administrative privileges.
2. Within the InsuranceSuite application, press ALT+SHIFT+T to display the **Server Tools** screens.
3. Navigate to **Batch Process Info** screen:
 - a) For any process that has a **Last Run Status** that indicates that the process is running, click the **Stop** button in the **Action** column.
 - b) For any process that has a **Next Scheduled Run** time that is before the time that you intend to stop the application, click **Stop** in the **Schedule** column.

All processes must have a **Status** of **Completed** before you stop the InsuranceSuite application server.

4. Stop the Guidewire InsuranceSuite application server:
 - To stop an InsuranceSuite application in a production environment, stop the server on which it is running.
 - To stop an InsuranceSuite application in a development environment, run the following command from the InsuranceSuite application installation directory:

```
gwb stopServer
```

For information on the `gwb` build commands, see the *Developer Setup Guide*.

Server startup tests

The InsuranceSuite application server performs a series of tests during start up. For some of these tests, a test failure prevents the server from starting. For other tests, a test failure is simply a warnings and InsuranceSuite permits the server to start. In many cases, these checks warn about potential problems with the domain graph, but, which might not be an issue depending on business logic.

In ClaimCenter, these checks verify that the current data model can support the following operations:

- Claim archiving

- Claim export
- Claim purging

Server modes

It is possible to start the InsuranceSuite application server in different modes. A *server mode* is a server setting that determines whether development and testing features are enabled during server start. There are three server modes:

- dev - Development
- test - Test
- prod - Production

For these modes:

- In development mode, Guidewire enables all implementation and testing features.
- In test mode, Guidewire disable all implementation and testing features except for the ability to enable and advance the testing clock. Guidewire designs test mode for testing functionality in a production-like environment with the one exception of being able to advance the clock to test time-based processes.
- In production mode, Guidewire disables all implementation and testing features.

Features available in each server mode

Functionality	Development mode	Test mode	Production mode
Reload modified PCF files using the ALT + SHIFT+ L keyboard shortcut	Available	Not available	Not available
Open the current page in Studio using the ALT + SHIFT+ E keyboard shortcut	Available	Not available	Not available
System clock	You can adjust the testing system clock by using the <code>setCurrentTime</code> method on the <code>ITestingClock</code> plugin.	You can adjust the testing system clock by using the <code>setCurrentTime</code> method on the <code>ITestingClock</code> plugin.	Not available
Tools screens	Server Tools and Internal Tools are available.	Server Tools and Internal Tools are available.	Only Server Tools is available.
Warnings	The following warning is written to the log and the console: The server is in "development" mode. This mode should never be used for a production server.	The following warning is written to the log and the console: The server is in "test" mode. This mode should never be used for a production server.	No warning is written to the log or console.
Application title bar	When connected to the application, the browser title bar has a "DEV mode" prefix.	When connected to the application, the browser title bar has a "TEST mode" prefix.	When connected to the application, the browser title bar has no prefix.

Setting the server mode

To set the server mode, specify the following system parameter at server start:

```
-Dgw.server.mode={dev|prod|test}
```

The server mode can be specified only before the server is started. Once a server is running, you cannot change the server mode unless you stop and restart the server.

Non-QuickStart servers

For non-QuickStart servers, the first time you start the server, the application database is marked with the server's mode. From that point, the database can be used only by servers in that mode. For example:

- When you start a server for the first time, if it is started in development mode, its database can be used only by servers in development mode. Its database cannot be used by servers in production mode.
- When you start a server for the first time, if it is started in production mode, its database can be used only by servers in production mode. Its database cannot be used by servers in development mode.

QuickStart servers

By default, the QuickStart server starts in development mode. For QuickStart servers, the application database is not marked with the server's mode. You can start a QuickStart server in one mode and then later start it in a different mode using the same database.

IMPORTANT: Guidewire does not support the QuickStart server for production installations.

Determining the server mode

You can determine the server mode through one of the following methods:

- By checking the console or the log for warnings shortly before the "<InsuranceSuiteApplication> ready" message.
 - If there is a "development" mode warning, the server is in development mode.
 - If there is a "test" mode warning, the server is in test mode.
 - If there is no warning pertaining to the server mode, the server is in production mode.
- By checking the title bar of a browser when it is connected to the application.
 - If there is a "DEV mode" prefix before the application name, the server is in development mode.
 - If there is a "TEST mode" prefix before the application name, the server is in test mode.
 - If there is no prefix before the application name, the server is in production mode.

Server run levels

InsuranceSuite supports the concept of a server *run level*. Guidewire divides the complete set of server functionality into a number of sequential levels. Each server run level contains more server functionality than the previous run level. As you start the InsuranceSuite application server, you can specify that the server starts at a certain run level. In this manner, you can restrict the server functionality to the subset of functionality that is available at that run level.

The ability to restrict server functionality is useful if you want to perform a server operation at a certain functional level. For example, in a server upgrade, you want the server to be at a specific level of functionality so that you can perform the upgrade, but, you also want to prevent users from logging into the server.

Different ways to view and set run levels

It is possible to view and define the server run levels in multiple ways.

QuickStart run level

The *QuickStart run level* applies only to the development QuickStart server. It is a numeric value from 0 to 5 that you enter on the command line as you start the QuickStart application server. The server starts at level 0 and proceeds to move through each server run level in the sequence until arriving at the requested run level.

You set the QuickStart run level using the following command as you start the QuickStart server, with *n* being a specific run level number between 0 and 5:

```
gwb runServer --run-level n
```

Server run level

The *server run level* designates each level of server functionality with a specific name. The server run levels map to the QuickStart numeric levels. For example, QuickStart level 3 maps to a server run level of NODAEMONS. The server run level in the server log lists each run level as it reaches that level of functionality.

See the *Developer Setup Guide* for information on how to start the various types of production application servers.

System run level

The *system run level* provides a way to set a specific run level for an actively running server.

You set the system run level using `system_tools` commands, for example:

```
system_tools -maintenance
```

Note: Contact Guidewire Support to set the run level to maintenance.

The following table lists the correspondence between the various ways of describing the run levels.

QuickStart run level	Server run level	System run level
0	NONE	—
1	GUIDEWIRE_STARTUP	—
2	SHUTDOWN	—
3	NODAEMONS	maintenance
4	DAEMONS	daemons
5	MULTIUSER	multiuser

Server modes and server run levels

The following tables show which functionality is available for the possible combinations of server modes and server run levels by application server. For a description of the numeric values used with the development QuickStart server, see “Server run levels” on page 31.

BillingCenter functionality by server mode and run level

QuickStart	Server run level	Production mode	Development mode
0	NONE	• Nothing available.	• Nothing available.
1	GUIDEWIRE_STARTUP	• User interface not available.	• User interface not available.

QuickStart	Server run level	Production mode	Development mode
		- Web services not available. - Database not available.	- Web services not available. - Database not available.
2	SHUTDOWN	- User interface not available. - Web services not available. - Database not available.	- User interface not available. - Web services not available. - Database not available.
3	NODAEMONS	- User interface not available. - Web services available. - Staging table loading available. - Maintenance-level batch processes and work queues available. Note: Contact Guidewire Support to enable NODAEMONS.	- User interface available. All logins allowed. - Web services available. - Staging table loading available. - Maintenance-level batch processes and work queues available.
4	DAEMONS	- User interface not available. - Web services available. - All remaining batch processes and work queues available. - Workflow Stat Manager available. - Scheduler available. - Daemons started by application available. - Messaging engine available to send messages from a batch server.	- User interface available. All logins allowed. - Web services available. - All remaining batch processes and work queues available. - Workflow Stat Manager available. - Scheduler available. - Daemons started by application available. - Messaging engine available to send messages from a batch server.
5	MULTIUSER	- User interface available. All logins allowed. - Server Tools available for users with admin permission only. - Internal Tools not available. - Web services available.	- User interface available. All logins allowed. - Server Tools available to all users if EnableInternalDebugTools is set to true in config.xml - Internal Tools available. - Web services available.

PolicyCenter functionality by server mode and run level

QuickStart	Server run level	Production mode	Development mode
0	NONE	Nothing available.	Nothing available.
1	GUIDEWIRE_STARTUP	- User interface not available. - Web services not available. - Database not available.	- User interface not available. - Web services not available. - Database not available.
2	SHUTDOWN	- User interface not available. - Web services not available. - Database not available.	- User interface not available. - Web services not available. - Database not available.
3	NODAEMONS	User interface not available.	User interface available. All logins allowed.

QuickStart	Server run level	Production mode	Development mode
		• Web services available. • Staging table loading available. • Product Model checked for illegal changes. • Maintenance-level batch processes and work queues available.	• Web services available. • Staging table loading available. • Maintenance-level batch processes and work queues available.
4	DAEMONS	• User interface not available. • Web services available. • Product Model checked for illegal changes. • All remaining batch processes and work queues available. • Workflow Stat Manager available. • Scheduler available. • Daemons started by the PolicyCenter application server, such as those used to dispatch messages.	• User interface available. All logins allowed. • Web services available. • All remaining batch processes and work queues available. • Workflow Stat Manager available. • Scheduler available. • Daemons started by the PolicyCenter application server, such as those used to dispatch messages.
5	MULTIUSER	• User interface available. All logins allowed. • Server Tools available for users with admin permission only. • Internal Tools not available. • Web services available. • Product Model checked for illegal changes.	• User interface available. All logins allowed. • Server Tools available to all users if EnableInternalDebugTools is set to true in config.xml • Internal Tools available. • Web services available.

ClaimCenter functionality by server mode and run level

QuickStart	Server run level	Production mode	Development mode
0	NONE	• Nothing available.	• Nothing available.
1	GUIDEWIRE_STARTUP	• User interface not available. • Web services not available. • Database not available.	• User interface not available. • Web services not available. • Database not available.
2	SHUTDOWN	• User interface not available. • Web services not available. • Database not available.	• User interface not available. • Web services not available. • Database not available.
3	NODAEMONS	• User interface not available. • Web services available. • Staging table loading available.	• User interface available. All logins allowed. • Web services available. • Staging table loading available. • Maintenance-level batch processes and work queues available.

QuickStart	Server run level	Production mode	Development mode
		Maintenance-level batch processes and work queues available.	
4	DAEMONS	- User interface not available. - Web services available. - All remaining batch processes and work queues available. - Workflow Stat Manager available. - Scheduler available. - Daemons started by the ClaimCenter application server, such as those used to dispatch messages.	- User interface available. All logins allowed. - Web services available. - All remaining batch processes and work queues available. - Workflow Stat Manager available. - Scheduler available. - Daemons started by the ClaimCenter application server, such as those used to dispatch messages.
5	MULTIUSER	- User interface available. All logins allowed. - Server Tools available for users with admin permission only. - Internal Tools not available. - Web services available.	- User interface available. All logins allowed. - Server Tools available to all users if EnableInternalDebugTools is set to true in config.xml - Internal Tools available. - Web services available.

Determining the server run level

Use any of the following tools to determine the server run level:

- System tools
- Web services
- Server ping

Each tool returns a value that indicates the server run level. The possible responses are:

- MULTIUSER
- DAEMONS
- MAINTENANCE
- STARTING

Use system tools to determine the server run level

Procedure

1. Ensure that the InsuranceSuite application server is running.
2. Open a command prompt and navigate to the following location in the InsuranceSuite application installation directory:

```
admin/bin
```

3. Enter the following command:

```
system_tools -user user -password password -ping
```

You must supply the user name (*user*) and password (*password*) for a user with administrative privileges on the InsuranceSuite application server.

Use web services to determine the server run level

Procedure

1. Ensure that the InsuranceSuite application server is running.
2. Call the following method on the SystemToolsAPI web service:

```
SystemToolsAPI.getRunLevel
```

Setting the QuickStart server run level

It is possible to set the run level of a QuickStart server to one of the following system run levels:

- daemons
- maintenance
- multiuser

Note: Contact Guidewire Support to set the server run level to maintenance.

Set the QuickStart run level at server start

Procedure

1. Open a command prompt and navigate to the InsuranceSuite application installation directory.
2. Run the following command:

```
gwb runServer -run-level n
```

The value of *n* is the numeric value of the run level as defined in “Server run levels” on page 31.

Set the server run level through system tools

Procedure

1. Ensure that the InsuranceSuite application server is running.
2. Open a command prompt and navigate to the following location in your InsuranceSuite application installation directory:

```
admin/bin
```

3. Enter one of the following commands:

```
system_tools -user user -password password -daemons
```

```
system_tools -user user -password password -maintenance
system_tools -user user -password password -multiuser
```

Contact Guidewire Support to set the run level to maintenance.

You must supply the user name (*user*) and password (*password*) for a user with administrative privileges on the InsuranceSuite application server. The run level is a value as defined in “Server run levels” on page 31.

Set the server run level through web services

Procedure

1. Ensure that the InsuranceSuite application server is running.
2. Call the following method on the SystemToolsAPI web service:

```
SystemToolsAPI.setRunLevel
```

What to do next

If you run an InsuranceSuite application in a clustered environment, you cannot place all the computers in a particular run level with a single method call. Instead, you must call the method individually on each cluster member.

Server maintenance mode

Periodically, you need to perform maintenance on an InsuranceSuite application. For example, you might need to import new security roles in PolicyCenter or batch import First Notice of Loss claims in ClaimCenter. To prevent users from logging into an InsuranceSuite application during these activities, place InsuranceSuite application into the maintenance run level.

Note: Contact Guidewire Support to enable maintenance mode.

The maintenance run level effectively disables the InsuranceSuite application web interface if the server is in production mode. The InsuranceSuite application stops allowing new user connections and halts existing user sessions to production mode instances while running at the maintenance run level.

The InsuranceSuite application still allows connections made through APIs or command prompt tools for any daemons with a minimum run level equal or lower than NODAEMONS. Restricting the run level permits integration processes to proceed without interference from non-administrator users.

Place the server in maintenance mode

Before you begin

Contact Guidewire Support to enable maintenance mode.

Procedure

1. Open a command prompt and navigate to the following InsuranceSuite application installation directory:

```
admin/bin
```

2. Run the following command:

```
system_tools -password password -user user -maintenance
```

You must supply the user name (*user*) and password (*password*) for a user with administrative privileges on the InsuranceSuite application server.

Monitoring server performance in an InsuranceSuite application

Use standard operating system tools to monitor memory usage, CPU usage, and disk space to verify that the servers run smoothly. In particular, monitor disk space for log files, so the InsuranceSuite application does not run out of disk space for logs. Archive and truncate system logs periodically to prevent the InsuranceSuite application logs from growing too large.

If the server crashes with the following JVM error, increase the maximum heap size (`-Xmx` setting) of the JVM.

`Internal Error (53484152454432554E54494D450E43505001A8)`

Refer also the documentation for the particular application server.

Monitoring and managing event messages

InsuranceSuite application generates a large number of events. In a InsuranceSuite application installation, it is often helpful, or even necessary, for InsuranceSuite application to notify other applications of these events. InsuranceSuite application integration developers create message destination objects that provide the means for passing information between InsuranceSuite application and a particular destination. Rule writers can write Gosu rules to generate messages in response to events of interest. InsuranceSuite application queues these messages and dispatches them to receiving systems by using the destination objects.

Guidewire recommends that you monitor message traffic to verify that the integration is running smoothly.

[See also](#)

- For more information about messages, including how to create message destination objects, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

About message failure

If the InsuranceSuite application receives an unrecoverable or unexpected exception from a send attempt, or reaches the retry limit, it does not send messages to that destination until you clear the error. If InsuranceSuite application receives a processing error that is not retryable, InsuranceSuite application also suspends the destination and waits for you to clear the error.

To clear a message error, do one of the following:

- Manually retry to send the message

- Skip the message

You can retry or skip messages:

- Through the **Message Queues** screen available to application administrators
- Through `messaging_tools` command prompt options

If InsuranceSuite application becomes completely out of synchronization with an external system, such that skipping or retrying a message is insufficient to re-synchronize the two systems, re-synchronize the entire destination. A re-synchronization causes InsuranceSuite application to drop all pending and failed messages and resend all the messages associated with a particular item.

The Messages screen

In PolicyCenter, use the **Messages** screen to review information about messages. An administrator can access the **Messages** screen in PolicyCenter by using the following navigation path:

Administration > Monitoring > Messages

You can search and filter message by any of the following:

- **Destination**
- **Transaction Type**
- **Transaction Number**
- **Product**
- **Policy Number**
- **Account Number**
- **Message Status** – For each status, the corresponding status is:
 - **Failed messages** – `MessageStatus.ERROR_STATES`
 - **Messages needing retry** – `MessageStatus.RETRYABLE_STATES`
 - **Unfinished messages** – Messages that do not fall into `ERROR_STATES` or `RETRYABLE_STATES`

The messages in **Results** have the following information:

- **Transaction Number**
- **Transaction Type**
- **Policy Number**
- **Product**
- **Primary Named Insured**
- **Message Status**
- **Error Description**

The Message Queues screen

InsuranceSuite applications lists each message destination in the **Message Queues** screen. To open this screen, click the **Administration** tab and then navigate in the left Sidebar to **Monitoring > Message Queues**. This screen contains cumulative information about message destinations.

From this screen, you can select a message destination and perform the following actions:

Suspend

Click to suspend the operation of the selected message destination.

Resume

Click to resume the operation of the selected message destination if that destination is in a state of suspension.

Restart

Click to restart the operation of a selected message destination.

You can also restart messaging by clicking **Restart Messaging Engine**.

If a message destination is running correctly, you do not see any accumulation of information in the columns on this screen. If there is a problem and messages begin to accumulate, you can drill down into a message destination by clicking the destination name. This action opens a **Destination** screen. From the **Destination** screen, you can see additional detail about any issues with a destination. This information can assist you in diagnosing the error. In particular, you can use the **Error Message** column to see the possible cause of a particular issue.

Destination screen

The **Destination** screen lists all failed or in-process messages for an object for all destinations. You can search for a particular object and then open the object's detail view. For example, in ClaimCenter, you can search for a particular claim and then open the claim's detail view. From this screen, you can select one or more objects and instruct the InsuranceSuite application to skip, retry, or re-synchronize any message that is still in process or in a failed state.

The Detailed Statistics table column headers have the following meanings.

Column header	Meaning
Safe Ordering Object Name	The InsuranceSuite application groups messages for each messaging destination based on their associated primary object. (The InsuranceSuite application processes messages associated with objects other than the primary object as non-safe-ordered messages.)
Send Time	Time the InsuranceSuite application sent the message.
Failed	A message can fail for several reasons, for example: - The message destination did not process the message successfully due to a processing error. - The message destination returns a negative acknowledge (NAK) indicating that the message delivery failed. - The message was part of a series of messages, one or more of which failed.
Retryable Error	Waiting to attempt a retry. The InsuranceSuite application attempted to send the message but the destination threw an exception. If the exception was retryable, the InsuranceSuite application automatically attempts to retry the send before turning the message into a failure. InsuranceSuite application attempts to send an event message several times. Typically, you can configure the number of retries and the interval between them for an integration. Review documentation for the specific destination to find out how to configure it.
In Flight	The InsuranceSuite application is waiting for an acknowledgment.
Unsent	The message has not been sent, for example: - The message is waiting on a prior message. - The destination is not processing messages because the destination is in a state of suspension. - The destination is falling behind in processing messages.
Error Message	Error message returned if a message fails.

It is possible to filter the messages that show in the table by selecting a filtering characteristic from the filtering drop-down list.

Access the messaging editor

About this task

You create and configure message environments and destinations in the file named `messaging-config.xml`.

Procedure

1. In the Guidewire Studio **Project** window, navigate to **configuration > config > Messaging**.
2. Double-click on the file `messaging-config.xml` to open it in the Studio **Messaging** editor.

Purge completed messages

About this task

If the InsuranceSuite application sends an event message, the expected response from the destination is a positive acknowledgment (ACK) indicating that the destination received and processed the message. The InsuranceSuite application retains completed messages until you purge them. Because the number of messages in the InsuranceSuite application can become very large, Guidewire recommends that you purge completed messages on a regular basis.

Procedure

1. Ensure that the InsuranceSuite application server is running.
2. Open a command prompt in the following location in the InsuranceSuite application installation directory:

```
admin/bin
```

3. Enter the following `messaging_tools` command, replacing the variables as needed:

```
messaging_tools -user user -password password -purge MM/DD/YY
```

For example, the following command purges all completed messages received prior to 11/30/18.

```
messaging_tools -user su -password gw -purge 11/30/18
```

You must supply the user name (*user*) and password (*password*) for a user with administrative privileges.

Session timeout

The InsuranceSuite application creates a session for each browser connection. The InsuranceSuite application uses the server's session management capability to manage the session. Each individual session receives a security token that the InsuranceSuite application server preserves across multiple requests. The server validates each token against an internal store of valid tokens.

Typically, the server determines the session timeout value according to the following hierarchy.

Level	Description
Server	The session timeout to use for all applications on the server if the timeout value is not set at a higher level.
Enterprise application	The session timeout specified at the enterprise application level. You can specify this value at the EAR file level. You can set the enterprise application session timeout value to override the server session timeout value.
Web application	The session timeout specified at the web application level. You can specify this value at the WAR file level. You can set the web application session timeout value to override the enterprise application and server session timeout values.
Application level	The session timeout specified in the application <code>web.xml</code> file. The InsuranceSuite application server does not specify a session timeout in <code>web.xml</code> .
Application code	An application can override any other session timeout value by setting the following configuration parameters:

Level	Description
	• AbsoluteMaxSessionLength • EnableAbsoluteMaxSessionLength • SessionTimeoutSecs
User	An administrator can set the session expiration timeout value on an individual user basis, using the Session timeout field on the User settings on the InsuranceSuite application Administration page. (Available in BillingCenter and ClaimCenter only)

User session replication

The InsuranceSuite application server does not implement or support user session replication for a number of reasons, including the following:

- InsuranceSuite application sessions are not serializable. Therefore, you cannot replicate a InsuranceSuite application session, either with or without persistence to the database.
- InsuranceSuite application sessions hold the user state in memory and contain large amounts of information. Guidewire estimates that this information amounts to 1 MB of data on average for a 32-bit server and close to 2 MB for a 64-bit server. Session replication would create significant cross-member communication that is detrimental to performance.
- InsuranceSuite application commits changes to the database on almost all transactions. Notable exceptions are some wizards for which InsuranceSuite application commits data changes only after the user completes all necessary entries.
- InsuranceSuite application scales horizontally almost linearly. The implementation of a session replication solution would very likely impede that linear scalability.

Cache management

Objects do not remain forever present or valid in the InsuranceSuite application database cache. Guidewire provides several caching mechanisms to verify that cache entries are still relevant. They are:

- Stale timeout
- Eviction timeout

Stale timeout

A stale timeout mechanism ensures that the server instance does not use old object entries excessively. An object is stale if it has not been refreshed from the database within a configurable amount of time. Upon accessing a cache entry, the server instance calculates the duration since the object was last read from the database. If that duration exceeds the stale time, the server instance refreshes the cache entry from the database.

To avoid increased complexity, the InsuranceSuite application prefers this mechanism over evicting objects upon stale timeout. You can set a default stale time by adjusting the `GlobalCacheStaleTimeMinutes` parameter in `config.xml`.

Eviction timeout

A InsuranceSuite application evict timeout mechanism removes old objects from the cache. For example, an object has an evict time of 15 minutes and a stale time of 30 minutes. If the server uses the object a single time every 14 minutes, the InsuranceSuite application never evicts the cache entry, but the entry does eventually become stale.

You can set the default evict time by adjusting the `GlobalCacheReapingTimeMinutes` parameter in `config.xml`. In the base configuration Guidewire sets the value of `GlobalCacheReapingTimeMinutes` to 15 minutes. The minimum value for this parameter is 1 minute. The effective maximum value for parameter `GlobalCacheReapingTimeMinutes` is the lesser of its set value or the value of `GlobalCacheStaleTimeMinutes` parameter.

Caching and stickiness

The InsuranceSuite application is fully leveraging the caching mechanism if a user returns to the same server instance across different HTTP requests. This mechanism, referred to as *stickiness*, enables a true horizontal scalability solution. For more information on load balancing options for the InsuranceSuite application, consult Guidewire Services.

Concurrent data change prevention

It is possible for different users, either on the same InsuranceSuite application server or on a different server, to attempt to change data objects concurrently. To prevent data corruption, InsuranceSuite implements a data object versioning mechanism. During the update to an object value to the database, InsuranceSuite application compares a counter associated with the object to the counter in the database. A counter value mismatch indicates that two different user changed the object concurrently. In such case, the InsuranceSuite application rejects the change and the cache mechanism throws a concurrent data change exception. The InsuranceSuite application presents the user who initiated the concurrent change with the error and reloads the latest data. The user can then reapply the changes.

Furthermore, the InsuranceSuite application commits changes in an atomic bundle, ensuring transactional integrity. Therefore, the InsuranceSuite application enforces protection against concurrent data changes across the whole transaction. This mechanism is a standard design pattern called optimistic locking.

Concurrent data change exceptions occur only if two users modify the same object. A proper organization of the user community avoids this. Nevertheless, if two users modify the same object, any automatic resolution carries a significant risk of causing unwanted modifications. The optimistic locking mechanism causes very few concurrent data change exceptions and users can easily resolve those exceptions.

Other design patterns exist for concurrent data changes. The pessimistic locking pattern prevents all other users from modifying an object while one user or batch process is making a change. In many cases, pessimistic locking becomes completely dysfunctional. For example, if a user or batch process cannot complete a change, the locking mechanism blocks any other user or batch process from making a change. Pessimistic locking systems generally become impractical. Therefore, the InsuranceSuite application uses the optimistic locking mechanism.

Cache behavior and performance

Guidewire generally distinguishes two types of caches:

- Server cache – A cache that is purely local to the application server
- Database cache – A cache used by a database to hold data retrieved from storage

Proper database caching behavior is critical to performance.

The server cache is purely local to each individual application server. Therefore, one server instance might contain information on a specific object while another server might not contain that information.

For example, if an InsuranceSuite application user works on an entity, the InsuranceSuite application loads corresponding objects on the server to which the user connects. If another user must approve the action of the first user, the approver user might interact with another InsuranceSuite application server. In that case, the second server likely does not have the corresponding information in cache. Therefore, the approver might experience slower performance as the server must populate the cache.

Cache content is lost every time that you stop an InsuranceSuite application server. Therefore, after you start a InsuranceSuite application server, expect lower performance during a ramp-up phase.

Batch processes leverage the cache mechanism. Batch jobs can work on many objects. Therefore, batch jobs can use the cache extensively. This can have the adverse effect of prematurely evicting objects from the cache, thereby forcing additional cache loads. If you run many intensive batch processes, Guidewire recommends that you have a dedicated server with the batch role with no online traffic directed to it.

Cache thrashing

Cache thrashing is a phenomenon whereby evictions remove cache entries prematurely and force additional database reloads that are detrimental to performance. There are several cases that can lead to cache thrashing:

- A single dataset can be too large to reside in the global cache. This forces the server to load the same data from the database and subsequently evict the data, potentially thousands of times, while loading a single web page. This results in serious performance issues.
- Some concurrent actions result in thrashing. For example, a user logs onto a server that has the batch server role. A batch job, which can load many objects into the cache, can remove objects from the cache. This forces the server to reload the cache as the user again needs those objects. For this reason, Guidewire recommends that you have separate servers for handling batch processing and user interface transactions.

If an individual cache reports hundreds or thousands of evictions and a low cache hit rate, then that cache is experiencing cache thrashing. If you notice cache thrashing on a server that is not processing batch jobs, re-size the cache. Otherwise, dedicate the server to batch jobs.

Detect cache thrashing

Procedure

1. Log into a Guidewire InsuranceSuite application as a user with administrative privileges.
2. Press ALT+SHIFT+T to open the **Server Tools** screens.
3. Navigate to the **Cache Info** screen.
4. Use the information in the **Cache Info** screen to analyze the number of evictions.
5. Click **Clear Global Cache** to clear the cache.
6. Reproduce the operation that you suspect caused the cache thrashing.
7. Reanalyze the information on the **Cache Info** screen.

What to do next

After you have taken the proper action, repeat the analysis to verify that the change yielded the results you expected to occur.

Cache impact on memory utilization

The maximum size of the cache is dependent on cache parameters. The server cache grows in size to reach a maximum specified by cache sizing parameters. Java does not provide a good means to estimate the memory usage of objects. Therefore, it is difficult to reliably estimate the maximum size of a cache. If the cache size exceeds the maximum heap size, the application eventually runs out of memory.

Larger caches increase memory starvation issues. Larger caches expand the memory footprint of the application. Performance decreases as garbage collection becomes more frequent and analyzes more objects.

Set the cache as large as needed, but no larger. Monitor garbage collection to extrapolate memory usage patterns and garbage collection statistics.

Server cache tuning parameters

File `config.xml` contains cache parameters for InsuranceSuite applications. Access this file in Guidewire Studio in the following location in the **Project** window:

```
configuration > config
```

This file contains the following parameters.

ExchangeRatesRefreshIntervalSecs

The number of seconds between refreshes of the exchange rates cache. The InsuranceSuite application uses this specialized cache for exchange rates only.

GlobalCacheActiveTimeMinutes

Time, in minutes, that the InsuranceSuite application considers cached objects active. You can think of this as a period in which the InsuranceSuite application is heavily using an item, for example, how long a user stays on a screen. The cache mechanism gives higher priority to preserving these higher-use objects.

IMPORTANT: The value of `GlobalCacheActiveTimeMinutes` must be less than that of `GlobalCacheReapingTimeMinutes`. If the value of `GlobalCacheActiveTimeMinutes` is greater than that of `GlobalCacheReapingTimeMinutes`, the cache does not start.

GlobalCacheDetailedStats

Boolean value that specifies whether to collect detailed statistics for the global cache. Detailed statistics are data that the InsuranceSuite application collects to explain why the caching mechanism evicted items from the cache. The InsuranceSuite application collects basic statistics, such as the miss ratio, regardless of the value of `GlobalCacheDetailedStats`. Disabling collection of detailed cache statistics can sometimes improve performance.

Guidewire sets the value of `GlobalCacheDetailedStats` to `false` by default. Set the parameter to `true` to help tune your cache.

If the `GlobalCacheDetailedStats` parameter is set to `false`, the **Cache Info** screen does not include the **Evict Information** and **Type of Cache Misses** graphs.

At runtime, use the **Management Beans** screen to enable the collection of detailed statistics for the global cache.

GlobalCacheReapingTimeMinutes

Time, in minutes, since the last use of a cached object before the InsuranceSuite application considers the object eligible for reaping. This can be thought of as the period during which the InsuranceSuite application is most likely to reuse an object.

An evict timeout mechanism removes old objects from the cache. Once per minute, a thread evicts cache entries that have not been used for a period equal to or greater than `GlobalCacheReapingTimeMinutes`. This mechanism differs from the stale timeout mechanism. The stale timeout mechanism refreshes from the database those cache entries that have exceeded the stale time. This process occurs as the server accesses a cached object. The evict timeout mechanism deletes any cache entries that are older than the default evict time. An object can become stale but not evicted if it is continually in use. For example, a bean has an evict time of 15 minutes and a stale time of 30 minutes. If the server uses the object once every 14 minutes, the InsuranceSuite application never evicts the cache entry, but the entry does eventually become stale.

`GlobalCacheReapingTimeMinutes` is initially set to 15 minutes. The minimum value for this parameter is 1 minute. Since the eviction thread only runs once per minute, a smaller value would not make sense. The maximum value for this parameter is 15 minutes.

GroupCacheRefreshIntervalSecs

The number of seconds between refreshes of the groups cache. The InsuranceSuite application uses this specialized cache for group-related data only.

GlobalCacheSizeMegabytes

Maximum amount of heap space used to store cached entities, expressed as a number of megabytes. This parameter supersedes the value of `GlobalCacheSizePercent`.

At runtime, you can use the **Cache Info** or **Management Beans** screen to modify this value.

GlobalCacheSizePercent

Maximum amount of heap space used to store cached entities, expressed as a percentage of the maximum heap size.

GlobalCacheStaleTimeMinutes

Time, in minutes, after which the InsuranceSuite application considers an object in the cache stale if it has not been refreshed from the database.

A stale timeout mechanism ensures that the server does not use excessively old object entries. An object is stale if it has not been refreshed from the database within a configurable amount of time. Upon accessing a cache entry, the server calculates the duration since the object was last read from the database. If that duration exceeds the stale time, the server refreshes the cache entry from the database. To avoid increased complexity, the InsuranceSuite application prefers this mechanism over evicting objects upon stale timeout.

At runtime, you can use the **Cache Info** or **Management Beans** screen to modify this value.

GlobalCacheStatsWindowMinutes

This parameter denotes a period of time, in minutes, that the InsuranceSuite application uses for two purposes:

- The period of time to preserve the reason that the InsuranceSuite application evicted an object, after the event occurred. If a cache miss occurs, InsuranceSuite application reports the reason on the **Cache Info** screen.
- The period of time to display statistics on the chart on the **Cache Info** screen.

ScriptParametersRefreshIntervalSecs

The number of seconds between refreshes of the script parameters cache. The InsuranceSuite application uses this specialized cache for script parameters only.

ZoneCacheRefreshIntervalSecs

The number of seconds between refreshes of the zones cache. The InsuranceSuite application uses this specialized cache for zones only.

Understanding cache metrics

Cache hit ratio metric information intrinsically depends on the workflow that is using the object. Some workflows involve reading an object only one time while others involve reading the object many times. The cache hit varies depending on these workflows.

Also, it is also for the data be appear skewed to the low side if a server started recently, or if the server has not had much user load. For example, if you recently started the server, and users have only visited a few screens, the hit rate is very low because the InsuranceSuite application encountered only a few cache hits. As users visit more pages, the hit rate increases.

Thus, there are no good default cache hit ratios. Experimentation combined with performance measurements constitutes the only approach to identifying appropriate cache sizes.

The Server Tools **Cache Info** screen contains information on cache performance metrics.

Special caches

The InsuranceSuite application includes specific individual caches for the following objects that rarely change:

- Exchange rates
- Groups
- Script parameters
- Zones

These specialized caches periodically refresh the entire set of the rarely changing object. This prevents the server from querying the database each time the InsuranceSuite application accesses one of these objects, thereby improving

performance. For each of these special caches, you can set the refresh interval through a configuration parameter in file `config.xml`.

The following table lists the objects with specialized caches, along with the configuration parameter that controls the refresh rate of each specialized cache.

Cache objects	Cache refresh rate set by...
ExchangeRate	ExchangeRatesCacheRefreshIntervalSecs
Group	GroupCacheRefreshIntervalSecs
ScriptParameter	ScriptParametersRefreshIntervalSecs
Zone	ZoneCacheRefreshIntervalSecs

Server tools

InsuranceSuite applications (PolicyCenter, ClaimCenter, and BillingCenter) provide a set of set of administration tools (screens) called Server Tools that you use to manage various aspects of a running an InsuranceSuite application server.

Accessing the Server Tools screens

You must have the `internaltools` permission to access the **Server Tools** screens. If the value of parameter `EnableInternalDebugTools` in `config.xml` is `true` and the server is running in development mode, all users have access to the **Server Tools** screens.

To access the **Server Tools** screens, press `Alt+Shift+T` on any InsuranceSuite application screen after you log into the application.

The Batch Process Info screen

Use the Server Tools **Batch Process Info** screen to view information about InsuranceSuite application batch processes, including writer threads for work queues.

Note: It is possible to run writers for work queues either from the **Work Queue info** screen or from the **Batch Process Info** screen.

Screen options

Use the drop-down on the **Batch Process Info** screen to filter the batch process list. This drop-down contains the following options.

Any

Shows all batch processing types.

Schedulable

Shows batch processing types marked as `Schedulable` in the `BatchProcesstype` typelist. You define a schedule for batch processes, including writer processes for work queues, in file `scheduler-config.xml`.

Runnable

Shows batch processing types marked as **APIRunnable** in the **BatchProcessType** typelist. It is possible to run these types of batch processes using APIs.

Screen actions

From the **Batch Process Info** screen, you can perform the following actions.

Refresh

Click to update the information in the **Processes** table.

Download

Click to download some of the information on this screen as a HTML report.

Suspend Scheduler

Stops the batch processing scheduler that triggers the execution of the batch processes. After stopping the scheduled, you can restart it using this same button.

Screen tabs

The **Batch Process Info** screen contains the following areas or tabs that contain batch process information.

Processes

Lists all the available batch processing types along with information about each individual batch process. It is also possible to run certain actions on a selected batch processing type from this screen.

Chart

Shows the execution time in seconds and the number of operations performed by the batch process over time in a graphical format.

History

Shows records of past runs of the selected batch process in tabular form.

Processes table columns

The **Processes** area of the Server Tools **Batch Process Info** screen contains a number of columns. The column labels have the following meaning:

Column	Description
Batch Process	Name of the batch processing type.
Description	Description of the batch processing type.
Action	Actions that you can perform on the selected batch processing type. These actions include: • Run – Runs a batch process. The Run button is active for all batch process types that belong to the BatchProcessTypeUsage category UIRunnable. • Stop – Stops an actively running batch process. • Download History – Downloads a batch process history report for the selected batch process in HTML format. You cannot start multiple runs of non-exclusive custom batch processes from the Batch Process Info screen. Instead, you must use the <code>maintenance_tools</code> command to start multiple runs of non-exclusive custom batch processes.
Last Run	Date on which this batch processing type last run.
Last Run Status	Completion status from the last run of this batch processing type. This field shows as Failed or Interrupted if the batch process <code>OperationsFailedReasons</code> is non-null.
Next Scheduled Run	Date of the next scheduled run for this batch processing type.

Column	Description
Schedule	Scheduling actions that you can perform on the selected batch processing type. These actions include: • Stop – Disables the scheduled runs for the selected batch processing type. • Start - Enables the scheduled runs for the selected batch processing type.
Cron-S M H DOM M DOW	Column header stands for seconds, minutes, hours, days of month, month, and day of week.

Chart and History tabs

The Server Tools **Batch Process Info** screen contains **Chart** and **History** tabs at the bottom of the screen. Select a batch processing type to view its chart or history.

The **Chart** tab shows the execution time in seconds and the number of operations performed by the batch process over time in a graphical format. The **History** tab includes a table of records of past runs of the selected batch process in tabular form.

The **History** table includes the following information:

Column	Description
Start Requested	The time at which the InsuranceSuite application received the request to start the process.
Started	The time that a batch process started.
Completed	The time that a batch process completed.
Scheduled	Yes indicates that the start request was the result of the regular execution of a schedule. No indicates that a user made the request manually.
Server	Server on which the start request was made. The server on which the request was made is not necessarily the server on which the InsuranceSuite application executed the batch process.
Description	Optional description.
Ops	For batch processes that are work queue writers, the value of Ops (operations) is the number of work items that the work queue processed. This number includes work items that failed.
Failed	For batch processes that are work queue writers, the value of Failed is a counter that the InsuranceSuite application increments each time work item processing encounters an exception. It is possible for a work queue to attempt to process a failed work item multiple times. Therefore, the Failed and Ops numbers do not necessarily match the total number of work items.
Failure Reason	For batch processes that are work queue writers, Failure Reason is the reason that a work item failed processing.

Download a batch process history report

Procedure

1. Navigate to the Server Tools **Batch Process Info** page.
2. Select a batch process in the **Processes** table.
3. Click **Download History** in the **Actions** column of the row for that particular batch process.
4. Select the date range for the records that you want to download.
5. Click **Complete Download**.
6. Select the location for the local file download and click **OK**.

7. Unzip the download file into a local directory.
8. Find and double-click `index.html` to open the report.

The Work Queue Info screen

Use the Server Tools **Work Queue Info** screen to control and view information associated with work queues. From this screen, you can track work queues as they process information. Each work queue has both a writer and one or more workers. You can run the writer to add a batch to the work queue, and you can monitor the progress of the workers processing the batch.

The **Work Queue Info** screen contains multiple tables that contain work queue information. These tables include:

- **Work Queue** table
- **Item Statistics** table

Besides the work queue information available on the **Work Queue Info** screen, it is possible to download a number of specialized reports from this screen as well.

Note: It is possible to run writers for work queues either from the **Work Queue info** screen or from the **Batch Process Info** screen.

Work Queue table columns

The **Work Queue** area of the Server Tools **Work Queue Info** screen includes a table containing information and functionality related to work queues. The table column headings have the following meaning:

Column	Description
Work Queue	Name of the work queue.
Available	Number of work items available for processing.
Checked Out	Number of work items checked out by workers.
Failed	Number of work items that failed during processing.
Executors Running	Number of executors processing the work queue.
Cluster-wide State	Work queue status, such as started or stopped.
Writer Status	Status of the writers.
Actions	Actions that you can perform on the selected work queue. These actions include: • Run Writer – Launches the writer to write work items for the work queue. • Notify – Wakes workers by notifying the executor that there are items to process. • Stop – Stops the selected work queue. • Restart – Restarts the selected work queue. • Download History – Downloads the historical instrumentation data for the work queue, in CSV format.

Item Statistics tabs and columns

The **Item Statistics** area of the Server Tools **Work Queue Info** screen provides information on work items related to the work queue selected in **Work Queue** table. This region has three tabs the contain a number of tables.

Tab	Description
By Writers	Tab contains information on writer item counts
By Executors	Tab contains information on active work queue executors
Work Item	Tab contains information on writer work items

The By Writers tab

The **By Writers** tab on the Server tools **Work Queue Info** screen contains item counts generated by the writer associated with the work queue selected in the **Work Queue** table. Each row represents one wake period for the writer. These table column headers have the following meanings.

Column	Description
Process ID	ID for the writer process.
Item Creation Time	Time at which the writer woke and began writing work items. The first item in the table for a queue has a creation time that matches the queue's current Last Execution Time for the Writer value.
Server	Name of the server running the work queue.
Scheduled	Yes indicates that the start request was the result of the regular execution of a schedule. No indicates that a user made the request manually.
Number of Items	Total work items in the queue regardless of status.
Worker End Time	Time at which the last worker completed the last item in the work queue.
Execution Time	Time, in minutes, since the start of the process (the execution time so far).
Available	Total number of available items in the queue.
Checked Out	Number of items checked out by workers for processing.
Succeeded	Number of items that completed successfully.
Failed	Number of items that failed.

The By Executors tab

The **By Executors** tab on the Server Tools **Work Queue Info** screen lists active executors. The table column headers have the following meanings:

Column	Description
Hostname	Server on which the executor is running.
Max. Number of Workers	Maximum number of workers available to the executor.
Processed Items	Number of items processed by the workers.
Exceptions	Number of exceptions encountered during processing.
Failed items	Number of work items that failed processing.
Status	Status of the executor, for example, running.

Column	Description
Started	Timestamp of the start of the executor.
Up For	Length of time since the start of the executor.

Under the **By Executors** tab is a **By tasks** tab. The **By tasks** columns have the following meanings:

Column	Description
ID	The unique identifier of the task.
Writer	The identifier of the writer process.
Success	Whether the worker succeeded in the processing of the work items.
Checked out items	The number of work items the worker checked out.
Processed items	The number of work items the worker processed.
Exceptions	The number of exceptions, if any, encountered during item processing.
Orphans Reclaimed	The number of orphaned work items the worker adopted for processing.
Failed items	The number of work items that failed.
Skipped items	The number of items that the process skipped.
Started	Timestamp of the start of the task.
Ended	Timestamp of the end of completion of the task.
Active	Whether the task is still active.
Consecutive Errors	If processing resulted in exceptions, the number of consecutive work items found that resulted in exceptions during processing by the worker.

The Work Items tab

The **Work Items** tab on the Server Tools **Work Queue Info** screen lists work items. The table column headers have the following meanings.

Column	Description
ID	Unique identifier of the work item.
Create time	Timestamp of the work item creation time.
Update time	Timestamp of the last update to the work item.
Available at	Timestamp of when the work item is available processing. This value is null for failed work items.
Server	Server that processed the work item.
Writer	Writer that created the work item.
Attempts	How many attempts a worker has made to process the item.
Activity ID	ID of the activity involved. This field is only visible if you select the Activity Escalation work queue.
Subject	Subject of the activity involved. This field is only visible if you select the Activity Escalation work queue.

The InsuranceSuite application shows data on this screen during the active execution of database checks only.

Work queue reports

Use the Server Tools **Work Queue Info** screen to generate and download multiple report types of work queue data. You can use the data from these reports to calculate different kinds of secondary data, work queue efficiency, for example.

The Work Queue Runs view

The Server Tools **Work Queue Info** screen also includes a view called **Work Queue Runs**. This view shows work queue statistics organized by writer run, including how long it took the writer to produce work items and how long the workers processed those items. Be aware that this view can appear confusing if you run the writer repeatedly before finishing the work items produced by the previous run.

About work queue efficiency

It is possible to use the information on the Server Tools **Work Queue Info** screen to generate additional types of data. For example, the Work Queue report contains information on the processing time for each item (**Item Processing Time**). Using this data, you can calculate the efficiency of a work queue by dividing the work item processing time by the total active time of a worker.

The Work Queue report

The **Work Queue** report available from the Server Tools **Work Queue Info** screen contains the following linked HTML files:

- Files that include a summary of the work queues
- Files that include detailed information for specific work queues

The report provides data for each worker by thread and by host, such as:

- Last wake up time
- Start and end times for the worker
- How long since the start of the worker
- How many items the worker processed
- Up-time (cumulative, average, and maximum)
- Execution time (cumulative, average, and maximum)
- Items processed for each thread (cumulative, average, and maximum)
- Throughput (items processed per minute of execution) per thread and per host

For each report, you can specify the following:

- The maximum number of writer runs, executors, and batches for each worker
- The number of hours for which to generate item distribution data in the report

Note: Internally, Guidewire sets the allowable maximum number of writer runs to show in the report at 150.

Download a work queue report

Procedure

1. Navigate to the Server Tools **Work Queue Info** page.
2. Click **Download** on the top-level menu.
3. Specify the parameters for the report.
4. Click **Complete Download**.

5. Select the location for the local file download and click **OK**.
6. Unzip the download file and double-click `index.html` to open the Work Queue report.
7. Click the **Work Queue Runs** link at the bottom of the Work Queue report to open the Queue Runs view.

The Work Queue Raw Data report

The **Work Queue Raw Data** report available from the Server Tools **Work Queue Info** screen is a set of CSV-formatted files. There is one CSV file for each work queue. The files contain time-sliced raw data from the `ProcessHistory` and `InstrumentedWorkerTask` tables.

You can perform analysis of this data in third-party tools such as Microsoft Excel.

Download the Work Queue Raw Data report

Procedure

1. Navigate to the Server Tools **Work Queue Info** page.
2. Click **Download Raw Data** on the top-level menu.
3. Select the date range for the records that you want to download.
4. Click **Complete Download**.
5. Select the location for the local file download and click **OK**.
6. Unzip the download file and open the work queue instrumentation reports individually.

The Work Queue History report

The Server Tools **Work Queue Info** screen includes the ability to download information on the history of a particular work queue. The report, in CSV format, includes the following information:

- Process ID
- Writer start and end times
- Duration
- Failures by workers

You can clear the instrumentation data for all work queues by running the Work Queue Instrumentation Purge process.

Download the Work Queue History report

Procedure

1. Navigate to the Server Tools **Work Queue Info** page.
2. Select a work queue in the **Work Queue** table.
3. Click **Download History** in the **Actions** column of the row for that particular work queue.
4. Select the date range for the records that you want to download.
5. Click **Complete Download**.
6. Select the location for the local file download and click **OK**.
7. Unzip the download file and open the **Work Queue History** report.

The Set Log Level screen

Use the Server Tools **Set Log Level** screen to set a specific logging level for the different logging categories. The logging levels that you specify in the **Set Log Level** screen persist until you change them or restart the server.

The **Logger** drop-down presents the following types of logging categories.

Guidewire default

Guidewire provides a number of default InsuranceSuite application logging categories. You can also see the list of Guidewire logging categories by running the `system_tools` command from a command prompt and adding the `-loggercats` option.

Guidewire internal classes

Guidewire provides a number of logging categories that apply to Guidewire internal classes. These logging categories start with `com.guidewire.*`. These logging categories generally look like a fully qualified class path.

Third-party software

Guidewire applications integrate with certain types of third-party software. The manufacturers of this software provide their own logging categories. These logging categories start with `org.*`, for example, `org.apache.*` or `org.eclipse.*`. These logging categories generally look like a fully qualified class path.

Making logging level changes permanent

To make your log level settings permanent, you must edit file `log4j2-json.xml` and set the log level there. Access this file from the **Project** window in Studio by navigating to **configuration > config > logging**, and then opening `log4j2-json.xml`. This file tells the logger to log output in JSON format rather than plain text log lines, which is then consumed and indexed correctly by Datadog when it arrives there.

Note: Do not use the `log4j2.xml` file to set the log level.

In the default InsuranceSuite application configuration, the `log4j2-json.xml` file does not contain a unique logger definition for each Guidewire logger.

Note: Some log4j loggers do not appear on the **Set Log Level** screen until actually used. This is a standard log4j behavior.

Setting logging levels through system tools

It is also possible to set a logging level through the following system tools command:

```
system_tools -updatelogginglevel logger level
```

The command prompt tools for administering the InsuranceSuite applications all require that you enter the password of an administrative user for the tool to work. The use of a user name is optional.

To use this command, you must supply the name of a specific logger category (*logger*) and the new logging level (*level*) for that logger. Use the `system_tools -loggercats` command option to see a list of valid InsuranceSuite application logger categories.

You must refresh the Server Tools **Set Log Level** screen after using the `system_tools` command to see your changes reflected in that screen.

The View Logs screen

You use the Server Tools **View Logs** screen to manage how you view the application log files in the InsuranceSuite application. This screen is only available in a development environment and is not available in a product environment. To view logs in a production environment, Guidewire recommends using Datadog. For more information about application logging, see *InsuranceSuite Observability*.

The **View Logs** screen contains the following items.

Log File

Use to select the log file to view.

Filter

Use to display the log file lines that contain the specified word or words.

Max Lines to Display

Use to set the maximum number of lines to show on the screen.

View Logs

Click to display the number of lines set in **Max Lines to Display** on the current screen.

Stream full log in popup

Click to display the complete log in a separate screen. This screen updates in real time with the application.

By default, the InsuranceSuite application writes log files to `tmp/gwlogs/<InsuranceSuiteApplication>/logs/`. To specify a different location in which the **View Logs** screen checks for log files, specify a different value for property `guidewire.logDirectory` in file `log4j2-json.xml`.

The Intentional Logging screen

You use the **Intentional Logging** screen to manage intentional logging elements, either individually or globally, as a group. The **Intentional Logging** screen contains the following elements.

Turn Intentional Logging On / Turn Intentional Logging Off

Set of buttons that enable (**On**) or disable (**Off**) the use of intentional logging in the Guidewire InsuranceSuite application. If you disable intentional logging, the InsuranceSuite application disables the use of any of the defined logging markers into the application log.

Settings Per Element table

Table that lists the defined logging markers and provides the means to enable or disable individual logging markers.

By default, Guidewire enables the use of logging markers globally in the base configuration. Guidewire also enables the use of each individual logging marker as well.

Marker visibility in the Intentional Logging screen

Initially, in a new installation, the InsuranceSuite application does not know all possible intentional logging markers that exist in the base configuration and in custom code. Consider that an intentional logging marker is an arbitrary string that you use to delineate a business or a system event. And, in fact, some markers are dynamic, meaning that the marker includes the name of a work queue, or a batch process, or that the marker name depends on the use of some data.

For example, imagine a custom batch process exists for importing some data into the database. Imagine that this batch process uses a custom marker for intentional logging as the batch process executes. In this example, suppose that the name of the marker is dynamic and that the marker name depends on the data the batch process imports into the database. In this situation, the InsuranceSuite application does not have a way of knowing the name of the marker before the use of that marker in the system.

At the first use of an intentional logging marker anywhere in a InsuranceSuite application cluster, the InsuranceSuite application stores the marker name in the database, which registers the marker name in the InsuranceSuite application. Registering a marker makes the marker name visible in the Server Tools **Intentional Logging** screen, which shows only registered markers.

Whenever you start an installation with a new database, there are no registered markers in the empty database. As the InsuranceSuite application server starts for the first time, the database upgrade process uses various intentional logging markers for logging the start up process. As the start up process unfolds, it registers each marker that it encounters in the database. Over time, as you use your installation (meaning running batch processes, work queues, messaging, and exercising SOAP/REST APIs), the more application uses the defined logging markers and consequently the more markers that it registers in database.

At some point, the application has used all of the possible markers, which register the markers in the database, and makes the full set of markers visible on the **Intentional Logging** screen.

Info pages

The Server Tools **Info Pages** provide information to help manage a InsuranceSuite application server and database. Use these screens to view information on the state of the server and to diagnose existing and potential database-related performance problems. Which screens you can access and view depend on the following:

- Type of application database
- Configuration parameter settings
- Specific user permissions

The Configuration screen

The Server Tools **Configuration** screen lists the values of the configuration parameters in your InsuranceSuite application environment. This screen also includes a **Download** button. Click **Download** to download a copy of the following configuration files:

- config.xml
- messaging-config.xml
- scheduler-config.xml
- work-queue.xml

You can find these files in the `config` folder within the downloaded ZIP file. The ZIP file also includes a current directory, which includes the in-memory state of `config.xml` and `work-queue.xml` parameters on the server. Guidewire makes the in-memory state available because it is possible to change certain configuration parameters using a web service or JMX APIs after server startup.

The Archive Info screen

IMPORTANT: Guidewire must enable data archiving for you to be able to view this screen.

Use the Server Tools **Archive Info** screen to view information about any archiving processing taken by the InsuranceSuite application.

Actions

On the **Archive Info** screen, you can take the following actions.

Refresh	Click to update the information on the Archive Info screen. Clicking the Refresh button also refreshes the <code>IArchiveSource</code> plugin.
Download	Click to download archive information to an HTML report. This report is a summary of the information shown on the Archive Info screen.
View Progress	Click to open the Work Queue Info screen. From this screen you can view the progress of the Archive work queue. You can also start an unscheduled run of the archive work queue from the Work Queue Info screen.
Export Upgrade Info	Click to download and save a <code>.dat</code> file that contains information on the database version for a specific archive operation.
Import Upgrade Info	Click to upload a <code>.dat</code> file that you previously exported. You must browse for a file (click Browse) to upload before the Import Upgrade Info button becomes active.
Browse...	Click to open a standard file picker dialog.

Tables

The **Archive Info** screen contains the following tables.

Overview	The Overview table includes the following: • The number of entities that the application archived • The number of items that the archiving process skipped or excluded • The number of items that failed the archiving process The screen information divides the excluded items into the following categories: • Items that the application excluded from archiving due to business logic • Items that the application excluded from archiving due to a failure Click Reset to set the total number of items excluded to zero.
Archive Source Information	The Archive Source Information table indicates the last refresh time of the Archive Info screen and the <code>IArchiveSource</code> plugin. The Archive Source Information also shows the availability of the store, retrieve and delete services. The application derives these values from the following variables in Gosu class <code>ArchiveSource</code>: • <code>storeStatus</code> • <code>retrieveStatus</code> • <code>deleteStatus</code> Possible values for these services include: • Available – The service is available.

- **Failure** – The last archive, restore, or delete operation failed.
- **Manually** – A user has manually flagged the service as unavailable.
- **Not configured** – The service is not yet configured.
- **Not enabled** – Archiving is not yet enabled.
- **Not started** – Archiving is not yet started.
- **Queue** – The service is not available. However, InsuranceSuite application permits the queueing of user requests.

Archive Summary by Data Model Version	As its name suggests, the Archive Summary by Datamodel Version table shows archiving information organized by data model version. For each database version, the table shows the following: • The number of entities the application succeeded in archiving. • The number entities that the application did not archive due to business logic. • The number of entities that the application did not succeed in archiving due to a failure. Each excluded category has a Reset button to set the count of excluded entities back to zero. Click a specific version to view additional archive information for that data model version.
--	---

The Domain Graph Info screen

IMPORTANT: Guidewire strongly recommends that you review the **Warnings** tab for possible issues every time you change the data model.

The Server Tools **Domain Graph Info** screen includes the following tabs.

Graphs

The **Graphs** tab of the Server Tools **Domain Graph Info** screen contains the following items:

- A DOT formatted version of the domain graph. The DOT format is a means of showing object relationships in plain text.
- A **Download** button, which if clicked, generates a ZIP file that contains the text version of the domain graph and the means to view the graph visually.

Warnings

The **Warnings** tab of the Server Tools **Domain Graph Info** screen shows non-fatal violations of the domain graph that the application detected while starting the server. The application provides warnings for these situations rather than preventing the server from starting because it is possible to prevent the erroneous situation through business logic.

The server also performs other graph checks while starting. If these checks fail, the server does not start.

If the server does not start, you cannot use the **Archive Info** screen to view errors detected by these validation checks. Instead, the server reports the error in the application log and prints the archive domain graph in DOT notation.

Excluded Entities

The **Excluded Entities** tab provides a list of all entities excluded from the domain graph.

The Consistency Checks screen

Use the Server Tools **Consistency Checks** screen to view and run consistency checks on the InsuranceSuite application database. The screen consists of two tabs:

- **Run consistency checks**
- **View consistency checks definitions**

The Server Tools **Consistency Checks** screen executes the Database Consistency Check batch process.

Note: If the server running a database consistency check fails for some reason, the InsuranceSuite application provides for a graceful recovery. After the server becomes operational again, the InsuranceSuite application starts the check at the place it last left off and completes the check.

The Run Consistency Checks tab

The **Run Consistency Checks** tab on the Server Tools **Consistency Checks Info** screen manages the process of running database consistency check batch processing. From this screen, you can:

- Start, stop, pause, restart, or cancel consistency check batch processing.
- Set the tables or table groups on which to run the consistency checks.
- Set the number of workers to use in the batch processing.
- Set the type of consistency checks to run.
- Review the results of each consistency check batch processing in tabular format.

The **Run Consistency Check** tab contains the following screen elements.

Item	Action
Download all selected	Click to download the consistency check results for all selected consistency check runs in the results table.
Delete	Click to delete the results of prior consistency check runs for those rows with a check mark in the results table.
Refresh	Click to update the information on the screen while processing is active.
Run Consistency Checks	Click to submit a batch processing job to perform one or more database consistency checks. This action executes the Database Consistency Check batch process. This button is not available if the Database Consistency Check work queue is not active,
Pause/Resume Consistency Checks	Click to pause a currently executing batch processing job. During a pause in processing, the application changes the button label to Resume . Clicking the button resumes processing execution. If this button is not visible, click the Refresh button.
Cancel Consistency Checks	Click to cancel all currently executing batch processes. The application displays this button only if there are currently executing consistency checks. If there are a large number of checks currently executing, the application displays a progress bar for the cancellation process. Understand that the cancel button changes the state of the executing checks overall to canceled. However, as the application cancels each consistency check individually, the progress bar can continue for some time if there are a large number of checks to cancel.
All tables	Click to select All tables (default) to run consistency check against all database tables.
Specify tables	If you select Specify tables , the application opens a table picker from which you can select the database tables against which the consistency checks run. You must select at least one table.
Specify table groups	If you select Specify table groups , the application opens a table groups picker from which you can select the table groups against which the consistency checks run. You must select at least table group.
Change	Click to change the number of workers used to execute the consistency check. Enter a positive integer value. If you change the number of workers in an active work queue, the application stops the existing work queue and restarts it with the new number of workers. This button is not available if the Database Consistency Check work queue is not active.
Check all types?	Click to select Specify Types to see the list of available check types from which you can select the type of consistency checks to run.

The consistency checks results table

After completing a consistency check, the results table columns have the following meanings.

Column	Description
Download	Click the Download icon to download a <code>ConsistencyCheckRundate.zip</code> file that contains the set of database reports generated by this consistency check run.
Download Errors	Click the Download Errors icon to download a <code>ConsistencyCheckRunErrorsOnlyRunDate.zip</code> file that contains the consistency check log file and stack trace. The application displays this column only if there are SQL errors in the database consistency check.
View	Click the View icon to open a pop-up from which you can view the same reports contained in the <code>ConsistencyCheckRundate.zip</code> file, after you supply your user credentials. The application displays this column in test and development mode only. The column is not visible in production mode.
Delete	Click the Delete icon to remove a consistency check row from the table.
Rerun	Rerun a consistency check that has a SQL error. The application displays this button only if there are SQL errors in the database consistency check.
Description	List of tables against which the consistency checks ran.
With Errors	Number of errors encountered by the consistency checks run.
Total Checks	Total number of consistency checks that ran.
Not started	Number of consistency checks that have not yet started in the current consistency checks run. Click the Refresh button to update this data during a currently executing check run.
In progress	Number of actively executing consistency checks at any given moment.
Finished	Number of consistency checks that completed in this run.
Workers	Number of workers that the application used in running the consistency check process.
Start time	Time at which this set of consistency checks started.
End time	Time at which this set of consistency checks ended.
Duration	Length of time that this set of consistency checks took to run.
Version	Database version, listing (in order): • Application major version • Application minor version • Platform major version • Platform minor version • Data model version
ID	Identifier (ID) of the stored results of this consistency check run.

If the consistency checks results table lists multiple check runs, use the check box next to a table row to select a consistency check run for further action.

The View consistency checks definitions tab

The **View consistency checks definitions** tab on the Server Tools **Consistency Checks Info** screen provides information on the consistency checks available for each database table. In this tab, you can undertake the following actions.

Download consistency check information

Click **Download** to download a ZIP file that contains a set of linked HTML files that describe the consistency checks that Guidewire provides in the InsuranceSuite application base configuration.

Search by table name

Search by table name to find the consistency checks related to a specified table. Most consistency checks operate on the specified table, but some checks, such as typelist table checks, operate on other tables as well.

To search, enter a complete or partial table name in the **Table name fragment** field and click **Search**. The results of the search show in a table that lists the table name, the consistency check name, and a description of the consistency check.

To clear the results of the search, click **Reset**.

Filter by check type

Filter the list of consistency check types to see a list of all tables for which that consistency check is available.

BillingCenter marks certain consistency checks as Blocker, Critical, or Standard and you see this designation in the consistency check name.

View SQL query

Review the SQL query used to generate a given database consistency check. First, select a consistency check, then:

- Select the **Command** tab at the bottom of the screen to view the SQL command of the consistency check. The SQL command retrieves a count of rows that violate the consistency check.
- Select the **Query to identify rows** tab at the bottom of the screen (if available) to view the SQL query used to identify rows that violate the consistency check. SQL queries to identify rows that violate consistency checks are not available for all check types.

Understanding consistency check severity

BillingCenter marks certain database consistency checks as Blocker, Critical, or Standard. To indicate the severity level, BillingCenter adds the severity level to the consistency check category name. Thus, you see **Accounting - Blocker Severity** as a consistency check name in the Server Tools **Consistency Checks** screen (**View Consistency Checks Definitions** tab).

BillingCenter associates the following consistency check categories with severity levels:

- Accounting
- Agency Bill
- Base
- Collateral
- Commission
- Contacts
- Funds Tracking
- Holds
- List Bill
- Multicurrency

You use consistency check categorization (category plus severity) to filter the list of available consistency checks:

- To view in the **Consistency Checks** screen, **View Consistency Checks Definitions** tab.
- To run on the **Consistency Checks** screen, **Run Consistency Checks** tab.

Run a consistency check from an InsuranceSuite application

Procedure

1. Navigate to the Server Tools **Consistency Checks** screen.
2. Select the **Run Consistency Checks** tab.
3. (Optional) Specify any, or all, of the following items:

- Description that the InsuranceSuite application prepends to the standard description of the tables and checks in the consistency check reports.
 - Tables or table groups on which to run the consistency checks.
 - Number of workers to use in executing the consistency check.
 - Type of consistency check to run.
4. Click **Run Consistency Checks**.
 5. After the batch process completes, select one of the following in the summary table:

Download arrow	Downloads a Zip file that contains the full set of consistency check reports.
Download Errors arrow	Downloads a Zip file that contains only the consistency checks that contain SQL errors.
View icon	Opens a pop-up window from which you can view the full set of consistency check reports.
 6. If you downloaded the consistency check file to your local system, unzip the file into its own directory.
 7. Locate the `index.html` file and double-click it to open it in a browser.
 8. In file `index.html`, do the following:
 - Click a table name to view all consistency checks related to that table.
 - Click a check name to view all tables that the consistency check runs against.

Run a consistency check using system tools

Procedure

1. Open a command prompt.
2. Navigate to the following location in the InsuranceSuite application installation:

`admin/bin`

3. Run the system tools command with the `-checkdbconsistency` option:

```
system_tools -password password -checkdbconsistency [...]
```

You must supply a value for *password*. You can optionally supply additional parameters for the `-checkdbconsistency` option.

Correct a consistency check SQL failure

Before you begin

If a database consistency check fails due to a SQL error, the InsuranceSuite application adds the following icons to the Server Tools **Consistency Checks** screen:

- **Download Errors**
- **Rerun**

Procedure

1. In the InsuranceSuite application Server Tools **Consistency Checks** screen, click **Download Errors** next to the check that generated the error.
2. Open the error report:
 - a) Click the number in the **With Errors** column.
 - b) On the details report that opens, click **Details**.

- c) Review the SQL query that generated the error.
- d) Correct any identified errors.

The listed table name indicates the object against which the consistency check ran. The check name indicates which consistency check actually generated the error.

3. In the InsuranceSuite application Server Tools **Consistency Checks** screen, click **Rerun** next to the consistency check that generated the SQL error.

Consistency check frequency

Run all database consistency checks periodically during implementation and during the stabilization period before going into live production. Running the consistency checks during the implementation and stabilization phases frequently helps to discover issues with application configuration.

Live production BillingCenter installations

In a live production BillingCenter installation, Guidewire recommends that you run the consistency checks at the following frequency.

Blocker	Every three months
Critical	Every six months
Standard	Once a year

As it is possible to run subsets of the InsuranceSuite application consistency checks, you can also set up a rolling schedule for these consistency checks. For example, run the account consistency checks in January, the base consistency checks in February, the commission consistency checks in March, and so on.

Upgrade

Run all consistency checks before and after a database upgrade. Running these checks before and after a database upgrade helps to verify the validity of the data and identify potential issues.

The Database Table Info screen

The Server Tools **Database Table Info** screen provides a way to access information about the tables used to store the data model. The screen contains the following buttons.

Verify

Click to have the application compare the database schema with the schema defined in the data model metadata files. The application shows any errors that it finds on the screen.

Download Database Schema Verification Errors

Click to download the results of the database schema verification. If there were no errors, the report is empty.

Download Database Table Info

Click to download a ZIP file containing a number of reports that document each table in the data model.

Database Table Info reports

The **Server Tools > Info Pages > Database Table Info** screen provides the following reports.

Report	Description
All Tables	Lists information about the InsuranceSuite application tables.
Array Membership	Lists the different tables that own an array, if any.

Report	Description
Array Ownership	Lists the different arrays that a table owns, if any.
Event Paths from Tables to Listening Objects	Shows paths from event-generating entities to non-event-generating entities. Each row in the table contains a non-event-generating entity <i>E</i> , along with one of the paths from an event-generating entity to <i>E</i> . The InsuranceSuite application uses each path to generate a query to find the event-generating entity instances that reference an instance of the non-event-generating entity.
Event Paths to Listening Tables	Shows the same paths as those in the Event Paths from Tables to Listening Objects report, but the entities in the second column are the event-generating entities. Each entry in the table contains an event-generating entity <i>E</i> , along with a path from <i>E</i> to a non-event-generating entity.
Foreign key columns with createBackingIndex="false" by Table	Lists the tables that have createBackingIndex set to false.
Foreign Key Constraints by Table	Lists the foreign key constraints on tables and provides information about the tables referenced by the keys.
Guidewire Version	Lists schema version and build information for InsuranceSuite application.
Implemented Interfaces	For each entity table, lists the interfaces that the entity implements.
Indexes by Table	Lists the indexes on a table and provides information about the associated key columns.
Indexes without a Description	Lists indexes that do not have a description.
Indexes with Shared Prefixes	Lists indexes that share multiple leading key columns. It is possible to use this information to find redundant indexes.
Indexes with the Same Key Columns	Lists indexes that have the same key columns.
Instrumentation Queries	Lists the queries that InsuranceSuite application executes against the database while building the data the comprises the download reports.
Materialized Views	Lists materialized views by name and the associated view definition, if any exist.
Number of columns	Displays the number of columns of various types in each table.
Possibly Redundant Backing FK Indexes	Lists foreign key indexes that may be redundant, including information about whether the index is unique and if it is an extension.
Primary Key Constraints by Table	Lists the primary key constraints on tables and provides information about the fields that reference the keys.
Schema Verification Errors	Lists database schema errors. The report is only present if there are errors.
Spatial Indexes	Provides information about spatial indexes.
Typekey Columns by Typelist	Lists the referencing typekey columns for each typelist.

The InsuranceSuite application copies table-specific information from each report category to the individual report for the respective table, excepting the **All Tables** and **Instrumentation Queries** reports.

Understanding the Database Table Info reports

The report information that you download from the **Server Tools Info Pages > Database Table Info** screen consists of the following categories of information:

- Summary reports with links to detailed table reports
- Copies of specific configuration files.

Configuration files

The download ZIP file contains several directories of configuration files:

- The `config` directory contains a number of InsuranceSuite application configuration files as defined at server startup.
- The current directory contains the in-memory state of the `batch-process-config.xml`, `config.xml`, and `work-queue.xml` parameters on the server. Guidewire makes the in-memory state available because it is possible to change certain configuration parameters using a web service or JMX APIs after server startup.

View the Database Table Info reports

Procedure

1. Log into the Guidewire InsuranceSuite application using an administrative account.
2. Navigate to the Server Tools **Info Pages Database Table Info** screen.
3. Click **Download Database Table Info**.
4. Save the download ZIP file.
5. Unzip the ZIP to a local directory.
6. Find and double-click file `index.html` to view a table of contents for the downloaded reports in a browser window.
7. Do one of the following:
 - Click a report type to open that report.
 - Click `config_files` on the table of contents page to access copies of certain metadata configuration files.

The Database Parameters screen

The Server Tools **Database Parameters** screen displays information about the database configuration. You can view the information on-screen, or you can download a set of linked HTML reports that contain the same information.

To download and access the HTML reports, click **Download Database Parameters Info**, unzip the resulting file, and click `index.html`.

To view the information on-screen, select a view type from the **View** drop-down list.

IMPORTANT: The list of views that you see depends on your database type.

View	Lists
Database and Driver	Versions of the database and its associated driver.
Database Connection Pool Settings	Connection pool settings as configured in <code>database-config.xml</code> .
Database Connection Properties	Properties related to the database connection.
Extensions	List of database extensions.
Guidewire Database Config	Guidewire-specific database configuration parameters. The table lists the <code><database></code> element attributes specified in file <code>database-config.xml</code> or lists the default value if file <code>database-config.xml</code> does not specify a value.
Guidewire Database Config Statistics Settings	Guidewire-specific database configuration parameters related to statistics gathering.
Guidewire Database Upgrade Configuration	Guidewire-specific database configuration parameters related to upgrade as defined by the <code><upgrade></code> element in file <code>database-config.xml</code> .
Guidewire Version	Information about this specific version of the InsuranceSuite application.
PostgreSQL Settings	Information about the Postgres database.
Queries Executed to Build Download	SQL queries used to generate the information in the HTML download reports.

The Database Storage Information screen

The Server Tools **Database Storage Information** screen provides information about the space and memory taken up by the database on the InsuranceSuite application server. You can both view and download database storage information from this screen.

After setting the desired filtering options, chose one of the following:

- To see the database storage information on the current InsuranceSuite application screen, click **Display Database Storage Info**.
- To download the database storage information to view later, click **Download Database Storage Info**.

After you click **Display Database Storage Info**:

- The screen provides an additional list of filter options.
- The **Display Database Storage Info** label changes to **Refresh**.
- The screen shows database information at the bottom of the screen

View database storage information

To view information about data storage within the InsuranceSuite application, set the desired filters, then click **Display Database Storage Info**. To update the information on the screen, click **Refresh**.

The **Database Storage Information** screen provides the following filter options.

Include Estimation of Compression Savings for Tables and Indexes

If checked (Yes), the InsuranceSuite application provides a number of additional filter options, for example:

Select Compression Level for Estimated Savings

If checked, you need to also select the compression level for estimating savings from the list of available options.

For all tables

You can select **Yes** (all tables) or **Specify tables** (to choose one or more specific tables for analysis).

Select Storage Set To Display

After you click **Display Database Storage Info**, the screen shows a set of additional filter options. Select one of the following options for additional information.

Storage Set to Display	Description
Database Usage Stats	Total space used in the database.
Guidewire Version	Guidewire product-specific information such as application and platform version.
Index Usage Stats	List of each index in the database along with related information.
Queries Executed to Build Download	List of SQL queries used to generate the data. The data includes the SQL used to generate the query and other information such as the number of rows returned the time it took for the query to run.
Summary of Queries Executed to Build Download	Simple summary of the number of queries involved in generating the data and the total database time that the queries took to execute.
Table Usage Stats	List of tables in the database along with related information.
Tablespace Usage Stats	List of each tablespace in the database along with related information.

Download database storage information

Set the on-screen options and click **Download Database Storage Info** to generate a DatabaseStorageInfo.zip file containing HTML reports. You can save this file to your local file system if desired. Open the Zip file and extract its contents to see the reports. Select file index.html to view the list of available reports.

The Data Distribution screen

Use the Server Tools **Data Distribution** screen to run a batch processing job that generates data on the distribution of various items in the database. You can then view this information on-screen or download a set of reports that details this information.

There are multiple categories of data distribution reports.

Comparison reports

The report shows data for the various items selected for inclusion in any of the comparison reports. It also shows the individual row count for the data, as of that date. The intent of this report is to provide a way to visualize data growth. The download ZIP file contains an HTML report plus separate CSV reports. The HTML report contains distinct columns representing the data from each report used for comparison.

Combined reports

The report combines information from multiple runs. The intent of the report is to provide a way to create smaller reports that require less generation time and then combine the information into one report. The download ZIP file contains an HTML report that contains information on each of the previous reports combined into this report plus tables that contain the combined data.

It is also possible to start the data distribution batch process (DataDistribution) directly from the command prompt in a non-production environment by using a `maintenance_tools` command option.

Generate and view a data distribution report

Procedure

1. Navigate to the Server Tools **Info Pages** and select **Data Distribution**.
2. Select from the available options listed under **Data Distribution Batch Job Parameters**.
3. Click **Submit Data Distribution Batch Job**.
4. After the batch job completes, select one of the following in the summary table:

Download arrow

Downloads an `DataDistribution.zip` file that contains the set of database reports.

View icon

Open a pop-up window from which you can view the same reports contained in the `DataDistribution.zip` file, after you supply your user credentials.

5. If downloaded the report to your local system, unzip the download Zip file into its own directory.
6. Locate the `index.html` file and double-click it to open it in a browser.
7. Use the links on the screen to navigate through the distribution reports.

Download comparison and combined data distribution reports

Before you begin

To create either a comparison or a combined data distribution report, two or more data distribution reports and their output must exist in the database.

Procedure

1. Navigate to the Server Tools **Info Pages** > **Data Distribution** screen.
2. In the summary table, select (check) at least two of the generated reports.
The InsuranceSuite application enables the following download buttons:

- **Download Comparison Zip File**
 - **Download Combined Zip File**
3. Click the appropriate button.

The Database Statistics screen

The Server Tools **Database Catalog Statistics Information** screen provides reports about out-of-date statistics in the database indexes, histograms, staging tables, and the InsuranceSuite application tables. This screen is not available with the development-only QuickStart database.

The **Database Statistics** screen contains the following tabs.

Database Statistics Information

Use to generate and download database statistics reports for the entire database or for specific tables.

Execution History

Use to view information about individual database statistics reports and take action with respect to each report.

Database statistics generation

Batch processing type Database Statistics (DBStats) generates the data available on **Database Catalog Statistics Information** screen.

Development mode

In development mode, it is possible to run the Database Statistics process in any of the following ways:

- From a command prompt, using the `-updatestatistics` option of the `system_tools` command
- From the **Execution History** tab of the Server Tools **Database Statistics** screen
- As a scheduled batch process

Production mode

In production mode, it is possible to run Database Statistics process in the following ways only:

- From a command prompt, using the `-updatestatistics` option of the `system_tools` command.
- As a scheduled batch process

Working with database statistics reports

To access the **Database Catalog Statistics Information** screens, click **Info Pages > Database Statistics** in the left-hand navigation pane of the **Server Tools** screen. Depending on the database type, you can access the following tabs from this screen:

- **Database Statistics Info**
- **Execution History**

Generate and download a database statistics report

About this task

Use the Server Tools **Database Statistics** screen to generate database statistics about how the InsuranceSuite application and data model interact with the physical database.

Procedure

1. Log into the Guidewire InsuranceSuite application using an administrative account.

2. Navigate to the Server Tools **Info Pages > Database Catalog Statistics Information** screen.
3. Select one of the following values for the **View database catalogs statistics on all tables** option:

Yes

View statistics data for all database tables. This selection can take a few seconds to complete, depending on the size of your database.

No

View statistics data for the selected tables only. If you select this option and do not select any tables, the application shows statistics data from the database metadata only.
4. Click **Download**.
5. Save the report ZIP file to a local directory.
6. Unzip the file.
7. Double-click file `index.html` to open the HTML report summary.
8. Use the report links to navigate through the various database statistics reports.

The Execution History tab

The **Execution History** tab of the Server Tools **Database Statistics** screen lists information about each run of the batch process that updates database statics. This information includes the following:

- Date and time of the start and stop of the database statistics generation
- Type of database statistics generation, either full or incremental
- Description of the particular database statistics generation

From this screen, you can also perform the following actions.

Action	Description
Refresh	Update the contents of the summary table.
Run Incremental Statistics	Generate incremental database statistics. This action updates database statistics for tables exceeding the change threshold only. This functionality is available in development mode only.
Run Full Statistics	Generate full database statistics. This functionality is available in development mode only.
Download	Click the download arrow to download the data from this report.
Delete	Click the trash can icon to delete this data row from the summary table.

The PostgreSQL Performance Report screen

From the Server Tools **PostgreSQL Performance Report** screen, you can do the following:

- Generate new PostgreSQL performance reports based on different parameters.
- View information about previously generated performance reports.
- Terminate a report generation that is taking a long time.
- Download an existing performance report.
- Delete an existing performance report.

After you generate and download a report, you can navigate through the various portions of the HTML report using the menu at the top of the screen. Each of these menu items provide further report choices.

PerformanceStatistics

Reports that contain database performance information including the top SQL statements, and database, table, and index activity.

Note: To view performance statistics, your installation must include the `pg_stat_statements` PostgreSQL extension, and optionally, the `auto_explain` extension.

RuntimeInfo

Reports that contain information on various application and system properties and variables.

AdditionalInfo

Reports that contain information on database catalog statistics and database parameters.

Specifying report parameters

You can specify the following report parameters while generating the performance report.

Include database statistics

If checked, the report includes database catalog statistics.

Read Auto Explain plans in logs

If checked, the report includes the SQL execution information generated by the `auto_explain` extension.

Note: For this option to be available, your application installation must include both the `auto_explain` extension and the Aurora PostgreSQL `log_fdw` (Foreign Data Wrapper) extension.

The Load History Information screen

The Server Tools **Load History Information** screen displays information about specific database operations. For example, loading data into the staging tables impacts the loader history information on this screen.

On this screen:

- Click **Refresh** to reload and update the table data.
- Click **Edit** to make the **Description** field for each load history writable.

The load history summary table contains the following information:

Download	Click the Download arrow to download a <code>LoadHistoryInfo.zip</code> file that contains a set of HTML reports. The reports consist of a summary table and a set of links to individual reports that load different views of the database operation data onto the screen. These are the same reports that are available by clicking the View icon.
View	Click the View icon to view the on-screen Load History Detail report for the selected database operation. The detail view consists of a summary table and a set of tabs that load different views of the database operation data onto the screen. These are the same reports that are available for download by clicking the Download icon.
Delete	Click the trash can icon to remove the data for this database operation.
Load Operation Type	Type of database operation. For example, this can be any of the following: • Database table load operations • Staging table clearing operations • Database statistics generation operations
Start Time	Start date and time of the database operation.
End time	Completion date and time of the database operation.
Duration	Length of time, in seconds, to complete the operation.
Error Count	Number of reported errors for this database operation..

Calling User	Name of user who initiated this database operation.
Description	Click Edit to make the Description field for each database operation row writable. Click Update after entering text to save your work and update the field.

View a load history report

Procedure

1. Log into the Guidewire InsuranceSuite application using an administrative account.
2. Navigate to the Server Tools **Load History Information** screen.
3. For the database load operation that interests you, select one of the following in the summary table:

Download

Select the arrow icon to download a LoadHistoryInfo.zip file that contains the set of database reports.

View

Select the view icon to open a new screen that displays the **Load History Detail** report.

4. If you downloaded the report file, unzip the file into its own directory.
5. Locate the index.html file and double-click it to open it in a browser.
6. Use the links on the screen to navigate through the linked reports.

The load history detail report

The Server Tools **Load History Detail** report contains a summary table for the selected database load operation. As with the main **Load History Information** table, the report summary view lists information about the type of database operation, the user who initiated the operation, and similar information.

Underneath the report summary table is a set of tabs, each of which loads a different view of the load operation data onto the screen. The following list describes these tabs.

Parameters	Lists the values of the configuration parameters used in generating the data for the database operations reports.
Steps	Lists the individual steps in the data load operation. Click a step to view detail data about that step.
Row Counts	Lists information about database tables impacted by the data load. Use the information on this screen to quickly assess whether the amount of data loaded by operation was the amount that you expected the operation to load.
Integrity Checks	Lists the integrity checks that the InsuranceSuite application ran against each affected database table before the data load operation.
Inserts	Lists the results of the SQL INSERT INTO queries that the InsuranceSuite application ran against the affected database tables. This tab lists all INSERT INTO run for this database load. In contrast, the Steps tab lists only the INSERT INTO queries that run for each particular step.
Callbacks	Lists the operations that the InsuranceSuite application executes before and after a staging table load operation. This tab lists all callback operations for this database load. In contrast, the Steps tab lists only the callback operations that execute for each particular step.
Statistics Commands	Lists the SQL commands used to generate database statistics.

Load history detail report - Parameters tab

The Server Tools **Load History Detail** report contains a summary table for the selected database operation. Directly underneath this table is a **Parameters** tab. Selecting this tab opens a table that lists the database parameters that affect the results of the **Load History Detail** report. The table shows the current values of these parameters.

Setting parameters for the load history report

The following table describes how to set the database parameter values that affect the results of the load history detail report.

allowRefsToExistingNonAdminRows

Use the `table_import -allreferencesallowed` command prompt option.

clearErrorTable

Use the `table_import -clearerror` command prompt option.

parallelism

Modify file `database-config.xml`.

populateExclusionTable

Use the `table_import -populateexclusion` command prompt option.

updateDBStatisticsWithEstimates

Use the `table_import -estimateorastats` command prompt option.

Load history detail report - Steps tab

The Server Tools **Load History Detail** report contains a summary table for the selected database operation. Directly underneath this table is a **Steps** tab. Selecting this tab opens a table that lists the steps involved in the operation in this particular load operation.

Steps and Ops link

If a step contains multiple sub-steps, that step becomes a clickable link that opens a **Steps and Ops** detail screen for that step. The **Steps and Ops** detail screen contains a summary table that describes the substeps involved in this step operation. It can also contain an additional table depending on the type of step. For example, both callback and `INSERT INTO` operations generate an additional table of database information.

Be aware that the **Table Name** column can contain the name of an actual database table or the name of a database callback. If it is a callback, then the associated **Callbacks** table contains specific information about that callback operation.

The Load Integrity Checks screen

The Server Tools **Load Integrity Checks** screen reports on the SQL integrity checks that run as a database load operation executes. For each integrity check, the screen lists the SQL query that the check performs, along with a description of the integrity check.

This screen has two tabs.

View by Staging Table

This tab lists the set of integrity checks that the InsuranceSuite application executes against a specific staging table. Use the **Staging Table** drop-down to select the table of interest.

View by Load Error type

This tab lists the set of tables against which the InsuranceSuite application runs a particular integrity check. Use the **Load Error Type** drop-down to select a specific integrity check.

In both tabs, you are able to set the value of **Allow Non Admin References** to true. If you do, the application checks foreign key references to administrative tables on load, such as users and groups. In the base configuration, Guidewire disables these references by default.

The Load Errors screen

The Server Tools **Load Errors** screen displays errors generated by failed integrity checks. You can use this screen to drill down through a table name to the specific error generated by a load operation. Errors relate to a particular staging table row. For each error, the **Load Errors** screen shows:

- The table name
- The row number
- The logical unit of work ID (LUWID)
- The error message
- The data integrity check query that failed

In some cases, the InsuranceSuite application cannot identify or store a single LUWID for the error. For example, in ClaimCenter, this might happen for some types of invalid financials imports.

The Runtime Environment Info screen

The Server Tools **Runtime Environment Info** screen lists information about the runtime environment for the InsuranceSuite application. This information includes Guidewire platform and application build information, system properties, and environment variables.

The Safe Persisting Order screen

The Server Tools **Safe Persisting Order** screen provides information on the following:

- The order in which the application writes object data to the database during a bundle commit.
- The order in which the application runs the Preupdate rules on a set of objects.

The screen contains a table with columns that show the following:

Column	Description
Entity	Entity name
Order	Entity ranking order
Table	Entity database table name
Preupdate	Whether the entity triggers a Preupdate rule set

To see only those objects affected by Preupdate rules, select **With rules only** from the drop-down filter.

Note: PolicyCenter does not use Preupdate rules.

Bundle commits to the database

The application uses the defined object ordering in every bundle write to the database for insert, write, or delete operations. Thus, during a commit to the database of a bundle containing multiple objects, the application writes the objects to the database in the order listed in the table **Order** column.

Preupdate rules

In running the rules in the Preupdate rule set, the application first computes the set of objects on which to run the Preupdate rules. Then, it runs the Preupdate rules for this set of objects as determined by the order listed in the table **Order** column.

The Loaded Gosu Classes screen

The Server Tools **Loaded Gosu Classes** screen no longer shows loaded Gosu classes. However, Guidewire still supports the ability to preload Gosu classes.

The Serialization Info screen

The Server Tools **Serialization Info** screen (under **Info Pages**) shows, for any specific server in the cluster, the entire set of Java objects (classes) deserialized by that server instance. This screen contains an optional filter (**Including listed in the serialization allow list classes**) that filters the list of classes:

Filter enabled

The list of class names visible on the screen includes the names of all Java classes encountered and deserialized by the local server. This list includes the names of classes that exist in the serialization allow list as well.

Filter disabled

The list of class names visible on the screen includes only the names of classes encountered and deserialized that are not on the serialization allow list. Guidewire recommends that you add these classes to the serialization allow list as well. After you complete your updating of Java class allow list, the class listing on this screen will be empty.

Enabling object deserialization

Configuration parameter `SerializationAllowlistEnabled` in `config.xml` determines whether InsuranceSuite application permits only those Java classes placed on a serialization allow list to be deserialized. Before you enable the use of the allow list, ensure that you first add any additional types needed due to customizations. Primarily, this is due to creating batch processes that accept custom objects as arguments. Such objects must be serialized if the batch process is invoked from a server other than the server with the batch role. If you do not add these objects to the allow list, it is possible that the related services that uses these objects to not function properly.

Serialization list files

In the Guidewire Studio **Project** window, you can access the serialization list files in the following location:

configuration > config > security

This folder contains the following serialization-related classes.

serialization-allowlist.lst

To permit an InsuranceSuite application the ability to serialize a Java class, add an entry for that class to file `serialization-allowlist.lst` in the **security** folder.

serialization-denylist.lst

To deny an InsuranceSuite application the ability to serialize a Java class, add an entry for that class in file `serialization-denylist.lst` in the **security** folder.

In making entries in these files, use the following syntax:

- Place a # symbol at the beginning of a line to indicate that the line is a comment.
- Use a separate line for each class or package name, for example, `gw.api.myPackage.*`.
- Do not place the * separator in the middle of a class or package name. For example, do not do the following:

```
#Incorrect example  
gw.api.*.myClass
```

- Use blank lines and leading spaces as desired to enhance readability of the file.

CVE-2018-8088

About this task

Due to an issue with `org.slf4j:slf4j-ext` version 1.7.5 as described in <https://nvd.nist.gov/vuln/detail/CVE-2018-8088>, Guidewire recommends that you explicitly prohibit this class to prevent it from being serialized.

Procedure

1. In Guidewire Studio, navigate to the following location:

Configuration > **config** > **security**

2. Open the following file for editing:

`serialization-denylist.lst`

3. Add the following text to this file on its own line:

`org.slf4j.ext.EventData`

The Management Beans screen

Note: The **Management Beans** screen is accessible to users with the `soapadmin` permission only.

The Server Tools **Management Beans** screen lists the InsuranceSuite application management beans, which are objects that represent different resources. Click the name of a resource to open the **Guidewire Managed Bean Properties** screen for the selected resource.

The **Bean Properties** screen that opens most often contains a **MBean Property** table that lists the properties associated with the selected resource bean. Property values are either read-only or editable. After modifying an editable field, you can either save your work or cancel your changes by clicking **Save** or **Cancel**.

A few **Bean Properties** screens also contain an **Operation** table that lists available operations associated with this resource bean. Click **Execute** to execute a selected operation. The **Result** field shows the result of the operation.

The Startable Services screen

The Server Tools **Startable Services** screen contains summary information on all of the startable plugin services in the InsuranceSuite application cluster. A startable plugin is a special type of plugin. At runtime, the application creates a background process, or service, for each startable plugin. The summary table provides the following information for each service.

Name Name of the plugin service.

Status Current status of the service.

Host One of the following:

- Name of the host on which the plugin service is running.
- Distributed, if cluster configuration allows the startable plugin services to run on all cluster members.

Action Available actions, either starting or stopping the selected service.

The Cluster Members screen

The Server Tools **Cluster Members** screens provide information on the server cluster installation. The screen consists of the following areas:

- **This Application Server instance**
- **(Cluster Members) Application Server instances**
- **Components** tab
- **History** tab

Available actions

The following list describes the actions that you can take in **Cluster Members** screen.

Download

Click to access the **Configure Cluster Members** report. From this screen, you can configure various parameters that affect the down-loadable cluster members report.

Refresh Cluster Members

Click to refresh the information on the **(Cluster Members) Application Server instances** table.

Start planned shutdown

Click to access the **Schedule Planned Shutdown** screen. From this screen, you can also schedule (or cancel) a planned shutdown of a specific server in the InsuranceSuite application cluster.

Cluster Members (Application Server instances)

The Server Tools **Cluster Members (Application Server instance)** screen provides the following information about the local server instance from which you access the screen.

Host	Name of the machine on which this InsuranceSuite application server instance is running.
Cluster ID	Name of the cluster to which the InsuranceSuite application server belongs.
Server ID	Name for this server instance. You specify the server ID by either adding an entry for this cluster member in the <registry> element in config.xml, or, by setting a JVM option at server startup. If you do not specify a server ID, InsuranceSuite application uses the host (machine) name as the server ID.
UUID	Universally Unique ID for this server machine. InsuranceSuite application randomly generates a UUID for each machine at each machine startup.
Server Roles	List of server roles assigned to this InsuranceSuite application server.

Cluster Members (Application Server instances)

The Server Tools **Cluster Members (Application Server Instances)** table provides the information about individual server instances recognized by the cluster. To update this information, click **Refresh Cluster Members**. A review of this information is one way to determine if a cluster member has become unreachable, and therefore, missing from the list.

This screen provides the following information.

Server ID	Name for each InsuranceSuite application server in the cluster.
Status	Status of each server instance in the cluster.
Host	Name of the machine on which each InsuranceSuite application server is running.
User Sessions	Number of user sessions active on each server instance in the cluster.
Run Level	Run level for each server instance in the cluster.
Version	Version information for each application installation. The InsuranceSuite application provides the version information in the following format: Guidewire build.customer build (<i>n,n,n,n,n</i>) These numbers have the following meaning: • Guidewire build – Guidewire application version. • Customer build– Custom label, defined in file <code>customer-version.properties</code>. If not defined, this field is empty. • (<i>n,n,n,n,n</i>) – Numbers that have the following meaning: ◦ Platform major version ◦ Platform minor version ◦ Application major version ◦ Application minor version ◦ Data model version
Server Roles	Specific roles assigned to each server instance in the cluster.
Server Started	Date and time at which this server instance started.
Connection Started	Date and time this cluster member first started.
Last Update	Date and time of the last update on this server instance.
Planned Shutdown	Date and time of any planned shutdown of a cluster member.
Actions	Action button to manage a planned shutdown of a cluster server instance. The button label is either Start Planned Shutdown or Cancel Planned Shutdown. You can also initiate a server shutdown using the administrative <code>system_tools</code> command option - <code>scheduleshutdown</code>.

Cluster Members (Components tab)

The Server Tools **Cluster Members – Components** tab provides information on the components running on the server that you select in the **Cluster Members (Application Server instances)** table. These components, with the exception of the work queues, are all singleton components. You can view all components, or, filter the list to see only those components of a specific type.

The **Components** table provides the following information about the components running on the selected server.

Type	Type of component, one of the following:
-------------	--

- Batch Process
- Message Destination
- Startable Service
- System
- Work Queue

Name	Name of component. This name is the work queue name, or, the message destination name, for example.
Started	Date and start time for each component.
State	State of the component, for example, Assigned.
Retry Failover	Date and time of the deadline in which to complete the failover process. If a lease manager detects the expiration of a component lease owned by another cluster member, the first lease manager starts a failover process for this lease. This failover process is not instantaneous. It is possible that during the failover process the cluster member performing the failover itself might fail, or lose database connection, or encounter other potential problems. To be able to handle this situation gracefully, the cluster member starting the failover gives itself a deadline to complete the failover process. If the current failover process does not finish by the specified timestamp, some other cluster member needs to start the failover process for the same component lease.

You can view similar information to that of the **Components** table on the **Cluster Components** screen.

Cluster Members (History tab)

The Server Tools **Cluster Members – History** tab provides information on the history of the server that you select in the **Cluster Members (Application Server instances)** table.

The **History** table provides the following information for the selected server.

Host	Name of the machine on which the InsuranceSuite application server is running.
Cluster ID	Name of the cluster to which the InsuranceSuite application server belongs.
UUID	Universally Unique ID for this server machine. The InsuranceSuite application randomly generates a UUID for each machine at each machine startup.
Env	Environment set for this InsuranceSuite application server.
Last Run Level	Run level for this InsuranceSuite application server at the point it stopped
Server Roles	Specific roles assigned to this server instance.
Server Started	Date and time at which this server started.
Server Stopped	Date and time at which this server stopped.

The Components screen

The Server Tools **Cluster > Components** screen provides the following information.

Type	Type of component to show in the table, one of the following: • All types • Batch Process • Message Destination • Startable Service
Name	Name of component. This name is the work queue name, or, the message destination name, for example.

State	State of the component, such as Assigned, meaning that there is a server that has the lease to run this component.
Start Requested	Date and time that the server received the component start request.
Started	Date and start time of the component.
Owner	Server ID of the InsuranceSuite application server on which the component runs.
Lease Expiration	Date and time at which the component lease expires.
Stopped	Stop date and time of the server instance on which the component is running, due to one of the following reasons: • Server instance stopped as intended • Server instance failed due to lost power or network connection and automatic recovery failed or is still in progress
Actions	The InsuranceSuite application shows a Complete Failover button if a component lease expires and a custom failover plugin implementation instructs the InsuranceSuite application not to perform an automatic failover.
Terminate Requested	Date and time at which the server instance received a terminate request for this component.
Transfer Requested	Date and time at which the server instance received a request to transfer this component to another server instance.
Transfer Target	Server ID of the server to which the transfer request was made.
Retry Failover	Date and time of the deadline in which to complete the failover process. If a lease manager detects the expiration of a component lease owned by another cluster member, the first lease manager starts a failover process for this lease. This failover process is not instantaneous. It is possible that during the failover process the cluster member performing the failover itself might fail, or lose database connection, or encounter other potential problems. To be able to handle this situation gracefully, the cluster member starting the failover gives itself a deadline to complete the failover process. If the current failover process does not finish by the specified timestamp, some other cluster member needs to start the failover process for the same component lease.

You can view similar information to that of the **Components** table on the **Cluster Members** screen, the **Components** tab.

Available actions on this screen

The following list describes the actions that you can take in **Components** screen.

Action	Description
Download cluster server report	Click Download to download an HTML report of the cluster components and component history.
Filter by component type	Select a component type from the Types drop-down list to filter the information by a specific component type.
Filter by component state	Select a component state from the State drop-down list to filter the information by a specific component state.
Filter by component	Click Filter by Component to open a Select Components screen. In this screen, you can select individual components of all available types to show in the component table.
Refresh the component information	Click Refresh to update the server component information in the table.
Review component history detail	Click the name of a component to open a component history detail screen.

Download a server component report

Procedure

1. Log into the Guidewire InsuranceSuite application using an administrative account.
2. Navigate to the Server Tools **Cluster > Components** screen.
3. Click **Download** to open the **Specify record limits for the report** screen.
4. Enter the maximum number of days to include in the generated component history report.
5. Click **Complete Download** to generate a ZIP file that contains the actual report.
6. Save the download file to your local machine.
7. Extract the Zip file to a local directory.
8. Search for and double-click file `index.html` to open the report in a web browser.
9. Click the **Component History** link for the component history report.

About planned server shutdowns

The Server Tools **Cluster Members** screen provides the ability to start a planned or scheduled shutdown of a specific server in the cluster. This action affects only the server on which you start or stop the scheduled shutdown. The **Actions** column of the **Application Server instances** table contains a button that toggles between **Start planned shutdown** and **Cancel planned shutdown**, depending on whether a server shutdown is currently scheduled.

Scheduling a server shutdown

Clicking **Start planned shutdown** opens the **Schedule Planned Shutdown** screen. From this screen, you can set the details of the planned server shutdown. In this screen, you need to set the following items.

Banner text

Choose the text of the banner message to show on the InsuranceSuite application screen to warn users of the server shutdown. The banner message occurs immediately as you schedule the shutdown and contains a countdown to the time of the scheduled shutdown. You can choose from several listed messages or create your own custom message.

Shutdown date and time

Set the date and time of the server shutdown.

Action to take with respect to running batch processes

Decide how to handle any currently running batch processes

After you initiate a scheduled shutdown, InsuranceSuite application does the following on the affected server:

- It does not start any new batch processes.
- It manages the stopping of any currently running batch processes depending on the setting for **Terminate Batch Processes** field.
- It requests that all work queues and message destinations stop processing immediately.
- It completes the transmission of any current messages without starting any new message transmissions.
- It completes the processing of any current work items without beginning work on any new work items.

After you click **OK** in the **Schedule Planned Shutdown** confirmation dialog, InsuranceSuite application reopens the **Cluster Members** screen. In this screen, you now see the following:

- The **Actions** entry for the affected server contains a **Cancel planned shutdown** button.
- The **Planned Shutdown** entry for the affected server provides information about the planned shutdown.
- The screen contains a banner with the chosen shutdown message and a countdown timer to the server shutdown date and time.

Shutdown message details

You have several choices for shutdown messages in the **Schedule Planned Shutdown** screen. If you do not elect to create a custom message, you can select one of several default messages. The InsuranceSuite application stores shutdown messages as display keys in file `display.properties`.

Web.TabBar.SystemAlertBar.PlannedShutdown.ScaleInMessage

Please save your work and log out to re-direct to the new server.

After you initiate a planned shutdown, the InsuranceSuite application stores only the current time and the shutdown time in the database. It stores other information, such as the shutdown message itself, in a single-threaded atomic reference in memory. The InsuranceSuite application clears this message reference under the following conditions:

- At the restart of the shutdown InsuranceSuite application server
- At the cancellation of the planned server shutdown

Logging into InsuranceSuite application during a planned server shutdown

Guidewire does not restrict the ability of a credentialed user to log into a InsuranceSuite application server that is undergoing a planned shutdown. However, upon logging into the server, that user sees the alert banner that a planned shutdown is in progress with a message instructing the user to log out. If you want to redirect users away from the server undergoing shutdown, Guidewire recommends that you delegate this function in the load balancer configuration.

Terminating running batch processes

The **Schedule Planned Shutdown** screen contains a **Terminate Batch Processes** checkbox. The InsuranceSuite application terminates the running batch processes differently depending how you set this field.

Checked

The InsuranceSuite application sets a flag on all currently running batch processes (including workqueue writers) assigned to this server to terminate their operation. To the extent that the batch process logic permits, the process attempts to stop as gracefully and as safely as possible.

Unchecked

The InsuranceSuite application does not instruct the currently running batch processes on this server to terminate prematurely. All batch processes assigned to this server continue until completed, regardless of how long it takes to complete the work.

In either case, InsuranceSuite application does not show a **Planned Shutdown** status of ready on the **Cluster Members** screen until all the affected processes have stopped on the server.

Note: Batch processes run as non-daemon threads. As such, it is not possible for the server to perform a graceful shutdown if any batch processes are still running on the server at the time of the shutdown.

Using system tools to schedule a shutdown

Note: You must have the required permissions and access to use the System Tools commands.

It is also possible to initiate a scheduled shutdown of a server using either the `SystemToolsAPI` web service or its associated `system_tools` command prompt option. For example, the `system_tools` command uses the following syntax:

```
system_tools -user user -password password -scheduleshutdown serverId [-terminatebatchprocesses  
-shutdowndelay minutes]
```

If you include the `-terminatebatchprocesses` command option, the shutdown process is the same as if you checked the **Terminate Batch Processes** check box in the **Schedule Planned Shutdown** screen.

Schedule a planned cluster member shutdown

About this task

You schedule a planned shutdown of a cluster member from the Server Tools **Cluster Members** screen.

Note: It is also possible to initiate a planned server shutdown using the administrative `system_tools - schedulshutdown` command option if you have the required permissions and access.

Procedure

1. Navigate to the Server Tools **Cluster Members** screen.
2. Find the table row that corresponds to the cluster member for which you want to schedule a shutdown.
3. If necessary, scroll the screen all the way to the right until you see the **Actions** column.
4. Click **Start Planned Shutdown**.
5. In the **Schedule Planned Shutdown** screen, select the message that you want to show to the application user about the planned shutdown.
Select one of the provided messages, or, enter custom text in the provided field.
6. Select the date and time of the planned shutdown.
7. Decide how you want the InsuranceSuite application to handle the currently running batch processes (including work queue writers) on the server.
8. Click **Schedule Shutdown**.

Results

After you schedule the shutdown, the **Cluster Members** screen shows the following for your selected cluster member:

- The **Planned Shutdown** column shows the date and time that you initiated the planned shutdown. It also shows the date and time of the actual planned shutdown.
- The **Actions** column shows a **Cancel Planned Shutdown** button. To cancel a planned server shutdown, click **Cancel Planned Shutdown**.

All InsuranceSuite application screens, for all users logged into the affected cluster member, display a banner indicating the date and time of the planned shutdown and your selected message.

The Upgrade and Versions screen

Note: Some functionality on the **Upgrade and Versions** screens is visible only if configuration parameter `ClusteringEnabled` is set to `true` in `config.xml`.

The Server Tools **Upgrade and Versions** screen displays information about any upgrade process that is run on the cluster. You can view this InsuranceSuite application screen from any server in the cluster. The InsuranceSuite application stores the information that generates this page in the application database.

The **Upgrade and Versions** screen has the following areas.

Function buttons

The function buttons provide the means to take specific upgrade actions. Individual buttons become visible depending on various aspects of the cluster, its members, and any ongoing upgrade process.

Upgrade information table

The summary table contains information about each upgrade operation, with a separate row for each operation. Each row has several icons that you can click to access additional functionality related to that specific operation. Click **Refresh** to update the on-screen information.

Guidewire Cloud Platform

Those with access to Guidewire Cloud Platform (GWCP), and the necessary permissions, will use Guidewire Home and its related microservices to start either a full upgrade or a blue/green upgrade. Each tenant planet or environment (development, staging, production) requires different permissions to start an upgrade. If you have the required access and permissions, use GWCP to start an upgrade. Do not use either **Start Full Upgrade** or **Start Blue/Green Upgrade** on the **Upgrade and Versions** screen to start an upgrade.

Full upgrade

A full upgrade of a Guidewire application cluster requires that you shut down all nodes in the cluster before you start the upgrade work. The following buttons refer to a full upgrade of the application cluster.

Start Full Upgrade

Click to open a **Start Full Upgrade** screen. Review the information on this screen before proceeding with the upgrade.

Clicking **Yes** on this screen sets a flag in the database that you intend to start a full database upgrade. You must set this database flag before deploying your new configuration to members of the cluster. If you do not set this flag, the InsuranceSuite application does not start and does not alter the database.

After you complete the full upgrade, the InsuranceSuite application deletes this flag from the database automatically. You must set the flag again before starting a new full upgrade.

Cancel Full Upgrade

Click to cancel a full upgrade operation. The **Cancel Full Upgrade** button is visible if the upgrade process has not yet started the database work. In this case, the cancel operation is a safe operation that does not require any further work after the upgrade stops.

Blue/green upgrade

In contrast to a full upgrade, a blue/green upgrade requires minimal downtime of the Guidewire cluster. This type of upgrade requires two fully functional Guidewire clusters:

- A Blue server cluster that is configured to run the existing (base) configuration. This cluster is active initially.
- A Green server cluster that is configured to run the upgraded (target) configuration. This cluster is offline initially.

After the upgrade, the Green server cluster uses the upgraded (target) configuration and becomes the active cluster. The Blue server cluster is offline.

The following buttons refer to a blue/green upgrade of the application cluster.

Start Blue/Green Upgrade

Click to open the **Start Blue/Green Upgrade** screen. To start the upgrade:

- You must actively confirm the conditions listed on the screen.
- You must select the specific form of the upgrade.

Guidewire recommends that you run the schema verification process before the upgrade if you are deploying changes that affect the database schema. See the documentation on `system_tools ... -verifydbschema` for more information.

Cancel

Click to cancel the blue/green upgrade. The **Cancel** button is visible after the start of the blue/green upgrade and before the schema upgrade completes.

Force Cancel

Click to open the **Force Cancel** screen. This screen lists the nodes that are running the new configuration. Before you can force a cancellation of a schema report or schema upgrade, you must actively affirm that each listed node is inactive. The **Force Cancel** button is visible only if the blue/green upgrade operation is in the process of generating a schema report or performing a schema upgrade.

Initiate Backout

Click to stop the blue/green upgrade after the upgrade process has already started upgrading the database. The backout operation is relatively less safe than a cancel operation as it does not revert database schema changes that have already occurred. Typically, an upgrade backout requires a manual cleanup of affected schema changes by an administrator. The **Initiate Backout** button is visible after the blue/green upgrade process has started its database work and before the completion of the blue/green upgrade (meaning before you click **Blue/Green Upgrade Complete**).

Enable Schema Migration

Click to enable the application database to upgrade to the new data model.

Enable Green Cluster

Click to start the new Green cluster in read-only mode. The **Enable Green Cluster** button is visible only after the schema upgrade completes.

Blue/Green Upgrade Complete

Click to indicate the completion of the blue/green upgrade. The **Blue/Green Upgrade Complete** button is visible if all nodes in the Blue cluster have shut down and at least one UI node in the Green cluster has started.

Upgrade information

The upgrade summary table contains the following information. All times are in hh:mm:ss format.

Version

Version information for each application installation. The InsuranceSuite application provides the version information in the following format:

Guidewire build.customer build (n,n,n,n,n)

For example, 50.13.0.4564.(50,1300,50,1314,517)

These numbers have the following meaning:

- Guidewire build – InsuranceSuite application version, such as 50.13.0
- Customer build– Custom label, defined in file customer-version.properties, such as 4564. If not defined, this field is empty.
- (n,n,n,n,n) – Numbers that have the following meaning:
 - Platform major version, such as 50
 - Platform minor version, such as 1300
 - Application major version, such as 50
 - Application minor version, such as 1314
 - Data model version, such as 517

Status

Status of each upgrade, upgraded, canceled, or new schema, for example.

Type

Type of each upgrade, install, full, or blue/green, for example.

Start Time

Date and time of the beginning of the upgrade.

End Time

Date and time of the completion of the upgrade.

Schema Report

Time that it takes to generate the schema report.

Database

Time that it takes to upgrade only the database.

Total

Time that it takes for the upgrade (either full or blue/green) to complete.

Deferred Upgrade Tasks Status

Execution status of the Deferred Upgrade Tasks batch process. A check mark indicates that the process has completed its work.

View Details

(Development mode only) Click the **View** icon to open a secondary screen that contains the same set of reports that you access using the **Download** icon.

Download Details

Click the **Download** icon to download an UpgradeInfo.zip file to your local machine. The download file contains a set of linked HTML reports describing various aspects of the selected upgrade.

Remove Detail Data

Click the trash can icon to remove this row of data.

Upgrade reports

The Server Tools **Upgrade and Version** screen contains the means to view details of a specific upgrade, or, to download the data for local viewing. The name of the report is **Upgrade Info**. The upgrade report contains the following types of information.

Upgrade Instance

Provides information on the database upgrade configuration, along with additional configuration reports.

Database Parameters

Provides information on various database parameters, including the following:

- Database driver
- Database connection pool settings
- Database upgrade configuration properties
- Database connection properties

The Upgrade Instance report

After you select **Upgrade Instance** from the drop-down at the upper-left hand side of the screen, you see additional reports listed under **Additional Information**, for example:

File Mismatches

Blue/green upgrade – Provides the name of each file that has changed (which becomes a mismatch between file versions) and whether the mismatched files contain data model changes that are compatible with a blue/green upgrade.

Download Raw Profiler Data

Provides the ability to download data about the upgrade process. The download file has a gwprof file extension. You can only open this type of file in Guidewire Profiler.

Database Upgrade Steps

Blue/green upgrade – Provides information on which database tables have changed. This report is available only if the generation of the schema report completes successfully.

Compact Database Upgrade Steps

Blue/green upgrade – Provides similar information to the **Database Upgrade Steps** report but in a more compact form. This report is available only if the generation of the schema report completes successfully.

View an upgrade report

Procedure

1. Navigate to the Server Tools **Upgrade and Versions** screen.
2. For the upgrade process that interests you, click one of the following:

Download Details icon

Downloads a Zip file that contains various types of upgrade information.

View Details icon

Opens a pop-up window from which you can view the upgrade reports. This option is available in development mode only.

3. If you downloaded the report file, unzip the file into its own directory.
4. Locate file `index.html` and double-click it to open it in a browser.
5. Use the links on the screen to navigate through the linked reports.

Understanding Guidewire software versioning

WARNING: In a production environment, Guidewire requires that you increment the data model version number before you restart the application server whenever you make changes to the data model. Otherwise, unpredictable results can occur. Use of the `extensions.properties` file in a development environment is optional.

Guidewire application versioning is a way to label a particular snapshot of a Guidewire application. It is a string label that Guidewire applies to its software. You see this version number primarily in Server Tools, in the **Cluster Members** screen and the **Upgrade and Versions** screens.

Guidewire manages software versioning through internal classes that provide the following information:

- Guidewire application name and version
- Guidewire module name and version

The InsuranceSuite application writes version information to the application log at InsuranceSuite application start.

Application version numbering

Both the Server Tools **Cluster Members** screen and the **Upgrade and Versions** screens show the InsuranceSuite application version as the following sequence of numbers:

A.B (a,b,c,d,e)

These numbers have the following meaning:

Version #	Meaning
A	Guidewire application version plus the application release build version, such as 50.13.0
B	Custom version label, such as 4564
a	Platform major version, such as 50
b	Platform minor version, such as 1300
c	Application major version, such as 50
d	Application minor version, such as 1314

Version #	Meaning
e	Data model version number, such as 517

The following example illustrates these concepts.

50.13.0.4564.(50,1300,50,1314,517)

File customer-version.properties

Use file `customer-version.properties` to define a custom build version number. If you create this file and populate it correctly, the InsuranceSuite application appends your custom version number to the Guidewire software version label.

File `customer-version.properties` contains the following single property:

`customer.build`

The following example illustrates how to use this property:

`customer.build=202111-PCF-upgrade`

The InsuranceSuite application does not contain the file `customer-version.properties` in the base configuration. Instead, you must create this file and place it in a location that the InsuranceSuite application recognizes. Create your version of this file in the following location in the Studio **Project** window:

configuration > **res**

Create a custom version label file

About this task

Place any `customer-version.properties` file that you create in the `res` directory.

Procedure

1. Open Guidewire Studio™.
2. In the **Project** window, expand **configuration** > **res**.
3. Select **res** and right-click to open the context menu.
4. Select **New** > **File**.
5. Name the file `customer-version.properties`.
6. Enter a value for `customer.build`.
For example:

`customer.build=20160914-PCF-upgrade`

7. Save your work.

Deploy a custom version label file on Tomcat

Before you begin

Before starting this procedure, ensure that you complete all steps in “Create a custom version label file” on page 90.

Procedure

1. Create a Tomcat WAR file:
 - a) Open a command prompt in the InsuranceSuite application installation directory.
 - b) Execute the following command:

`gwb warTomcatDBcp`

The InsuranceSuite application adds file `customer-version.properties` to the following JAR file in the generated WAR file:

```
WEB-INF/lib/configuration.jar
```

2. Deploy the WAR file to the application server.

Results

After starting the server, the Server Tools **Upgrade and Versions** screen shows the custom version label, appended to the standard application version number.

The About Guidewire *InsuranceSuite Application* > pop-up

You can access information about your Guidewire application version by selecting the following path in Guidewire InsuranceSuite application:

Options > About

You access the **Options** menu by clicking the Gear icon at the right-hand side of the banner area at the top of the InsuranceSuite application screen.

Copyright information

The pop-up provides the current software copyright information. For more information about Guidewire trademarks, refer to the following Guidewire site:

<https://www.guidewire.com/legal-notices>

Application version information

The pop-up lists the following application version information:

- Application version, such as 2025.03.0
- Application build, such as 50.13.0.4564
- Platform version, such as 50.13.2
- Schema version, such as 1314
- Customer version
- Server instance

The Cache Info screens

The Server Tools **Cache Info** screens provide information in both table and chart form of the InsuranceSuite application server cache information. Guidewire recommends that you use this information to help you monitor how well the cache is performing.

The **Cache Summary** screen:

- Contains charts and data such as the cache size and cache eviction information
- Provides the ability to clear the global cache
- Download a report with detailed cache information

The **Historical Performance** screen:

- Contains charts and data from a historical perspective, the cache hits and miss information over a period of time, for example

The **Cache Details** screen:

- Contains charts that show the age distribution of objects in the cache
- Contains graphs that show percentage information on objects in the cache over time

The Cache Summary screen

The Server Tools **Cache Summary** screen includes the following information:

Max Cache Space (KB)	Maximum amount of space to allot to the global cache.
Stale Time (mins)	Maximum time allowed for an object to be in the cache without a database refresh.
Page Loaded at	Date and time of the last refresh of the data on this screen.

Available actions on this screen

From the **Cache Info** screen, you can do the following. Note that all actions apply only to the particular cluster node on which you run the action.

Edit	Click Edit to enter different values for Maximum Cache Space and Stale Time . If you change these parameters from the Cache Summary view, the values you specify apply only to the InsuranceSuite application server to which you are connected. If you restart the server, your changes are lost. For your changes to persist, edit their values in file <code>config.xml</code> .
Refresh	Click Refresh to update the information shown on this screen. This action also updates the date and time values shown for the Page Loaded at field.
Download	Click Download to download a CSV-formatted file containing detailed cache information.
Clear Global Cache	Click Clear Global Cache to clear the cache of all entities. Note, however that the cache always contains some objects to support an active server.

Cache summary graphs

The **Cache Summary** screen provides the following graphs.

Graph	Description
Cache Size	The memory used by the cache over time.
Hits and Misses (Stacked)	The number of cache hits (an object was found in the cache) and misses (object was not found in the cache) and the miss percentage.
Type of Cache Misses	The number of cache misses caused by the InsuranceSuite application evicting an object because the cache was full and the number of missed caused by the InsuranceSuite application evicting an object due to reaping. This graph is not visible if configuration parameter <code>GlobalCacheDetailedStats</code> in <code>config.xml</code> is set to <code>false</code>.
Evict Information	Information about cache evictions over time, including: • Number of times no entry was found to evict when cache was full • Number of evictions within active time when cache was full • Number of evictions when cache was full • Number of evictions due to reaping

Graph	Description
	This graph is not visible if configuration parameter GlobalCacheDetailedStats in config.xml is set to false.
Current Age Distribution	The number of objects of various ages in the cache.
Current Cache Contents for Age All	The percentage of types of objects present in the cache for all ages.

The Historical Performance screen

Note: For the **Historical Performance** screen to be visible, configuration parameter GlobalCacheDetailedStats in file config.xml must be set to true.

The Server Tools **Historical Performance** screen provides the following graphs:

Graph	Description
Space Retained	Memory used by the cache over the past couple days. The time shown is a much longer period than the cache size graph on the Cache Summary tab, which only displays the past 15 minutes. In this case, the x-axis represents the average values for each time period during each of the past eight days. This is to allow for comparison of cache behaviors against hourly trends.
Hits and Misses (stacked)	Number of hits (object was found in the cache) and misses (object was not found in the cache) and the miss percentage over the past day and past seven days.
Miss %	The percentage of cache read attempts in which the object was not found in the cache over the past day and the past seven days.
Number of Misses because item was evicted when cache was full	The number of misses over the past day and the past seven days due to the InsuranceSuite application having evicted an object from the cache because the cache was full.

The Cache Details screen

The Server Tools **Cache Details** screen includes graphs of the following:

Graph	Description
Age Distribution by time	A number of graphs that show the age distribution of objects in the cache. The Cache Details tab shows age distributions for zero to 30 minutes ago.
Current Cache Contents by age	A number of graphs that show the percentage of types of objects in the cache over time.

Understanding the cache data report

Clicking **Download** on the Server Tools **Cache Info** screen downloads a CSV-formatted file containing information on the current state of the server cache.

In looking at the cache data provided in the downloaded report, Guidewire recommends that you first calculate the cache miss ratio around the time of the performance degradation. The cache miss ratio is the ratio of (misses) / (misses + hits).

The cache miss ratio is a useful metric in that it normalizes the cache values. For example, suppose that you have the following hit and miss cache values that you use to calculate each individual cache miss ratio.

Timestamp	# cache misses	# cache hits	Cache miss ratio
Time 1	12345	2345	$12345 / (12345 + 2345) = 0.8404$
Time 2	1234	23456	$1234 / (1234 + 23456) = 0.4998$

The calculation of the cache miss ratio enables you to compare the data in way that is not possible by simply examining the raw data.

High miss ratio

If the miss ratio is high, it is very likely that the performance issue involves the cache. Thus, you need to look at the reasons for the cache misses, especially the following:

- Number of misses because item was evicted when cache was full
- Number of misses because item was evicted due to reaping

If the number of misses is high due to the cache being full, you can try increasing the size of the cache and observing the result. However, keep in mind that the cache uses the heap. Thus, increasing the size of the cache can potentially reduce the amount of heap memory available for other computations.

If the number of cache misses is high due to reaping, you can try increasing the reaping time and observing the result. Another approach would be to examine ways to reduce the time since the last access of the cache objects. For example, how long does it take to reuse objects that have already been fetched from the database? Is it possible to hold onto the objects without having to fetch the objects again from the database?

Cache misses and hits

The cache miss count and the cache miss ratio are probably the most important data points in the download cache data report. Essentially, the more cache hits the better, as it means the cache was able to service that many requests.

Each cache miss is expensive. It means that the looked-for object was not in the cache. Regardless of the hit count, a high miss count can indicate a problem with the cache. However, keep in mind that the miss count is relative. A cache miss can occur simply because it is the first time a request is made to the cache for an object. Thus, the object is not in the cache. A cache miss can also occur as the sought-after object may have already been evicted from the cache. The downloaded cache report provides data on the various reasons for object eviction from the cache.

Using the assumption that there is a constant cost for each cache miss, it is possible to compare the miss counts directly. The cost of a cache hit is essentially zero, or very, very small compared with the cost of a cache miss. A cache miss may take a 10 millisecond trip to the database to access the data whereas a cache hit is essentially zero time.

The Guidewire Profiler screens

The Server Tools **Guidewire Profiler** screens provide access to a set of tools that are useful in gathering and analyzing information on the runtime behavior and performance of the Guidewire InsuranceSuite application. The Profiler records the time spent in specific processing areas done by application code, as well as the configured rules and PCF screens. Use of this information can help narrow down performance issues with InsuranceSuite application components such as PCF screens, rules, custom code, batch processes, and work queues.

Profiler code is useful to record the following types of InsuranceSuite application operations and information:

- Database SQL queries
- Parameters passed to those SQL queries

- Table row counts
- Rule execution information
- Gosu code execution

IMPORTANT: Guidewire recommends that you exercise care in using Guidewire Profiler. Storing too much profiling information can cause the **Profiler** screen to become too cluttered, require extensive amount of space for storage, and, for long-running processes, hold on to too much memory at runtime.

Using Guidewire Profiler

One of the best uses of Guidewire Profiler is to review application data for a specific component or action in a Guidewire InsuranceSuite application that has a quantifiable performance problem. Examples of the types of performance issues that are quantifiable include the following:

- Response time that is poor (excessive wait times) for a particular InsuranceSuite application application screen
- Batch process or work queue that has a low throughput/minute
- Database activity that is high for a particular web service method

Guidewire Profiler does not collect memory usage statistics. If necessary, use a third-party tool to gather memory usage and garbage collection information.

Using Guidewire Profiler during development and testing

Guidewire Profiler is not only useful in diagnosing existing performance issues, it is also useful in preventing performance issues. For example, you can use Guidewire Profiler to do the following in the development and testing phases of an InsuranceSuite deployment:

- Predict the scalability of a large operation by testing it with two or three datasets of increasing size.
- Identify unused or unnecessary database queries that slow down performance so that you can remove them.
- Test the performance of a PCF screen with a custom widget by profiling the screen both before and after you add the custom widget.

Required permissions to access Guidewire Profiler

To access the Server Tools **Guidewire Profiler** screens, you must have one of the required permissions.

Server mode	Access
All server modes (production, development, testing)	All users that have a role with the <code>internaltools</code> permission can access Guidewire Profiler. Some administrative accounts have this role permission by default.
All server modes except production (development, testing)	All users in a development mode environment have access to the Server Tools if the value of parameter <code>EnableInternalDebugTools</code> in <code>config.xml</code> is set to <code>true</code> . This type of access is useful as it is sometimes necessary for a user to run certain tests of the application screens as a non-administrative user.

Useful Knowledge Base article

Guidewire makes available a Knowledge Base articles that discusses Guidewire Profiler. To view this article, access Guidewire Community at one of the following links:

- **Guidewire Customers** - <https://community.guidewire.com/>
- **Guidewire Partners** - <https://partner.guidewire.com/>

After logging into Guidewire Community, search for the following article:

- 6394 - Collecting Data to Diagnose UI Performance Issues

Basic profiler concepts

The following list describes basic concepts that are important to an understanding of how to configure and use Guidewire Profiler.

Guidewire Profiler

Guidewire Profiler records the time spent in specific processing areas by the application code, as well as the configured rules and PCF screens.

Web profiler

The web profiler records information on rendering a InsuranceSuite application screen, entering the screen, taking action on the screen, and similar types of operations.

Entry-point profilers

The profiler entry point indicates the type of application component that you select to profile, a batch process, for example. InsuranceSuite applications provide the following entry point profilers:

- Batch process
- Work queue
- Message destination
- REST operation
- Web service
- Startable service
- Gosu servlets

Profiler tag

A profiler tag represents a section of code that Guidewire Profiler can track and analyze. A profiler tag is an alias for a piece of code in the Guidewire application for which you want to gather performance information.

The InsuranceSuite application base configuration contains a number of profiler tags. However, you are free to add your own custom profiler tags in addition to the existing profiler tags.

Profiler frame

A profiler frame is the unit of measurement for a profiled section of code. Thus, one profiler frame contains details of a single invocation of the profiled section of code, such as its start and finish times, for example.

Profiler stack

A profiler stack records information about one particular instance of an application thread. Examples include the following:

- One run of a batch process
- One server round-trip from a PCF screen
- One processing cycle for a (work queue) work item

Starting and stopping Guidewire Profiler

There are several different ways that you can access Guidewire Profiler and start and stop a profiler:

- Alt-Shift-T
- Alt-Shift-P

Alt-Shift-T

If available, pressing Alt-Shift-T within a Guidewire InsuranceSuite application screen opens the main **Server Tools** screen. From this screen, you can navigate to the **Guidewire Profiler** screen and its sub-screens. From these sub-screens, you can start and stop all Guidewire profilers and review their output. However, Guidewire does not recommend that you start web profiling sessions using Alt-Shift-T. Use Alt-Shift-P instead.

Alt-Shift-P

If available, pressing Alt-Shift-P within a Guidewire InsuranceSuite application screen opens the main **Guidewire Profiler** screen directly from that screen. Guidewire recommends that you always use Alt-Shift-P to access Guidewire Profiler if you are working with the InsuranceSuite application screens. Using Alt-Shift-P to jump directly from a InsuranceSuite application screen to Guidewire Profiler removes the need to navigate through InsuranceSuite application screens and **Server Tools** screens to reach the **Guidewire Profiler** screen. It thus removes the need to profile the navigation path to the Profiler and back to the InsuranceSuite application screen, making the final profiling information much smaller and more easily understood.

To stop an active web profiler, press Alt-Shift+P from within the profiled InsuranceSuite application screen to return to the **Guidewire Profiler > Configuration** screen, from which you can stop the profiler.

Starting and stopping individual entry-point profilers

You start and stop the individual entry-point profilers directly in the **Guidewire Profiler > Configuration** screen:

- To start an entry-point profiler, click the **Enable Profiling** button for that particular profiler.
- To stop an entry-point profiler, click **Edit Configuration** (in the right-most column) for the entry-point profiler that you want to stop and then select **Disable Profiling**.

Be aware of any on-screen instructions that the InsuranceSuite application provides as you enable an entry-point profiler. Certain profilers only take effect after you stop and restart the relevant component.

About the different Guidewire profilers

There are multiple types of Guidewire profilers, including:

- A Web (UI) profiler
- Multiple entry-point profilers of various types
- A database upgrade profiler

You use the Server Tools **Guidewire Profiler > Configuration** screen to configure profiler operation. Use the **Configuration** to select the specific type of profiler to execute, and the circumstances under which to execute the profiler. The one exception is the Upgrade profiler, which you do not have to configure as it collects data automatically during an application upgrade.

Differences between profilers

The following list describes some of the main differences between web and entry-point profilers.

Web profiler	Entry-point profiler
Captures the current user session in which you enabled the profiler for a single node in the InsuranceSuite application cluster until you disable the profiling session.	Captures all instances of targeted components across all nodes in the InsuranceSuite application cluster for each profiler run.
Provides a set of results for all round-trips to the server (browser-server-browser) while the profiler is active, for a single node in the InsuranceSuite application cluster.	Provides a separate result for each individual profiler run on each node in the InsuranceSuite application cluster. This information includes a separate result for each instance of the profiled action, for example: • Each individual run of the profiled batch process • Each processing cycle of every profiled work item • Each call to the profiled web service operation
Saves profiler information in the current user session in memory so that the data disappears at session termination.	Stores profiler information in the database until deleted.

Web (UI) profiler

After you enable web profiling, InsuranceSuite application records all subsequent round-trips to the application server as a separate profiler stack. Thus, the profiler records information on rendering the screen, entering the screen, taking

action on the screen, updating a widget on the screen, and similar types of operations. The web profiler captures only the current user session in which you enable the web profiler. Exiting the profiler removes the profiler information from memory. Logging out of the current InsuranceSuite application session also ends the web profiler session.

Types of web performance issues

Remember that web performance issues can be any of the following issue types.

Issue type	Example	Profiler
Operation on a single screen	Widget update on a screen or button click on the screen	Press Alt+Shift+P to open the profiler directly from the screen on which the operation occurs.
Moving between screens	Excessive time in rendering a screen	Press Alt+Shift+P to open the profiler directly on the screen before the problematic screen. In other words, start the profiler, then navigate into the problematic screen.

How to use web profiling

In profiling a PolicyCenter application screen, press Alt+Shift+P to open the profiler popup directly from within Guidewire InsuranceSuite application. In setting up the profile test scenario, remember the following:

- Enable Guidewire Profiler right before the screen click that you are trying to profile.
- Disable the web profiler immediately after the screen click.

Specifically, you want to capture each separate screen click as a unique profiling operation so that you can analyze each profiling run separately.

About web session profiling

Web session profiling records timing information for specific sections of application code on the server only. Thus, the profiler starts tracking from when the application servlet on the server first receives a HTTP POST request and stops tracking when the servlet sends back its response. During this process, the profiler tracks the timing of Guidewire application code as well as the response time of application calls to the database or to external services.

Data persistence

InsuranceSuite application does not persist the stacks from web requests to the database. Instead, InsuranceSuite application stores these stacks in the user session. Logging out of the current user session deletes the profiler data. Logging out of the InsuranceSuite application application also ends the user session. It is only possible to enable web profiling for the current user session.

It is possible to download and save a copy of the data for a current web profiling session using the functionality on the **Web Profiler** screen.

Profiler recommendations

In using web profiling, Guidewire recommends the following:

- Enable the web profiler only as absolutely needed as it can possibly degrade performance.
- Clearly identify the web UI operation that you want to profile.
- Profile only a single-click operation on an application screen in each profiler run.
- Collect data about each individual operation separately.
- Log out of the InsuranceSuite application application to free the memory used by the profiler after you have analyzed or downloaded the profiler data.

Web profiler results

Guidewire automatically displays the web profiling information in the Server Tools **Guidewire Profiler > Profiler Analysis > Web** screen after you disable the current web profiling session.

The web profiler results include the following types of information:

- Timing information on specific sections of the InsuranceSuite application application code.
- Location of the time spent on server-side execution of InsuranceSuite application application code.
- Response time of application calls to the database, external services, and other similar items.

The web profiler results do not include the following types of information:

- Time spent in the browser.
- Network latency between the browser and the application server.
- Additional details on external calls other than the response time. The exception is calls to the database, for which the profiler records the SQL, bind variables, and stack trace that generated the query.

Entry-point profilers

Guidewire Profiler collects profile data based on the type of interaction with the application that you request to be profiled. Guidewire refers to the different types of possible interactions as entry points, with the entry point type indicating the application component to profile. The following list describes the various entry point types that you can access from Guidewire Profiler **Configuration** screen:

- Batch process
- Work queue
- Message destination
- REST operations
- Web service
- Startable service
- Gosu servlets

Integration with Digital applications

It is possible to use the web service profiler in Guidewire Profiler to measure the performance of JSON-RPC requests in Guidewire Digital Portals™. For more information, see "Using the Guidewire Web Service Profiler for Digital Services" in the *Developer's Guide* for Digital applications.

About entry-point profiling

Guidewire Profiler stores the data collected by the entry-point profilers in the database. This information is visible to all InsuranceSuite application servers in the cluster.

After you enable a profiler entry point, the next time that InsuranceSuite application runs an instance of that component, it checks to see if the profiler is enabled for that component. If profiling is enabled, Guidewire Profiler records the profiling data in the form of a profiler stack. The profiler records multiple stacks if the initial thread spawns more threads and the developer profiles the spawned threads. InsuranceSuite application persists the data from entry-point profilers to the database, making it possible to retrieve the data for later use.

Batch process profilers

Guidewire Profiler specifically does not profile a batch process that is executing at the time of you enable the profiler. Instead, the profiler begins operation the next time that the batch process executes, whether on schedule or started manually.

Work queue and message destination profilers

In order for a change in configuration to take effect for the following components, you need to perform additional work:

Component	InsuranceSuite application screen	Additional steps
Work queue	Server Tools Work Queue Info screen	Stop, then restart, the selected work queue.
Message destination	InsuranceSuite application Administration > Monitoring > Message Queues	Suspend, then restart the selected message destination.

Any change in configuration can take some time to propagate through the cluster. Also, it can take up to the full amount of the entire cache stale time for a configuration change to become visible.

Purging profiler data

The Purge Profiler Data batch process works in conjunction with configuration parameter `ProfilerDataPurgeDaysOld` to determine when to purge profiler data from the database. For more information, see *InsuranceSuite Database Administration*.

Entry-point profiler results

The Server Tools **Guidewire Profiler > Profiler Analysis** screen shows the results of each profiler run. To see the data for a specific profiler, select the profiler name under the **Profiler Analysis** link in the Server Tools left-hand navigation pane. The **Profiler Result** area of the **Profiler Analysis** screen that opens provides a means to view different reports (views) of the profiler data. Selecting a different report type updates the screen data to reflect your choice.

Database upgrade profiler

InsuranceSuite application collects upgrade data automatically during an application upgrade. Guidewire stores the upgrade information in the database so that the information is available for each upgrade in the Server Tools **Guidewire Profiler > Profiler Analysis > Upgrade** screen until deleted.

General considerations for using Guidewire Profiler

Guidewire recommends that you keep the following considerations in mind as you design your profiler test runs:

- Clear the global cache (ClaimCenter and BillingCenter)
- Capture a limited amount of data
- Use Alt+Shift+P for web profiling operations
- Enable stack trace tracking
- Create comparable test cases for before and after comparisons
- Thoroughly document each test case

Clear the global cache in ClaimCenter and BillingCenter

If you are performing web profiling and you are working in a test environment, Guidewire recommends that you consider clearing the global application cache before the start of each profiler run. The advantage of clearing the global application cache is that it forces all profiler queries for that node to go to the InsuranceSuite application database.

However, be aware that clearing the global application cache affects all users active on that node in the cluster. For this reason, Guidewire does not recommend that you clear the global cache in a production environment.

To clear the global application cache, navigate to the Server Tools **Cache Info > Cache Summary** screen and click **Clear Global Cache**.

Capture a limited amount of data

If you are performing web profiling, Guidewire recommends that you structure your profiler test cases to capture data for a single round-trip to the application server only. The following are example of a round-trip sequence from the client browser (in which InsuranceSuite application runs) to the server (which processes the request) and back to the browser again.

- Load (render) an application screen
- Click a button on an application screen
- Tab out of a screen widget that performs a post-on-change operation

The more information that the profiler captures, the more data it must store and load. Thus, it is important to minimize the number of actions or operations that occur during a single run of the profiler. Guidewire recommends that you profile a single button click or action only in the InsuranceSuite application screen in each profiler run.

Multi-trip profiler runs create the following problems:

- A multi-trip profiler run has the potential to create large datasets that are difficult for the analyzer tool to load and analyze effectively.
- More importantly, in a large dataset, it is often very difficult to associate specific data with the specific action by the user that caused the profiler data.

Use Alt-Shift-P for web profiling operations

If you are performing web profiling, Guidewire recommends you always use Alt+Shift+P to open the web profiler directly from the InsuranceSuite application screen that you want to profile. If you use Alt+Shift+P to open the web profiler directly from an application screen, you minimize the number of actions that the profiler records. Otherwise, if you use Alt+Shift+T to access Server Tools, from which you can navigate to the **Guidewire Profiler > Configuration** screen, you must then navigate back to the application screen that you want to profile. This adds many intermediate actions to the profiler data before you actually return to the screen and operation of interest.

Enable stack trace tracking

Stack trace tracking provides the following benefits:

- A stack trace shows the exact widget or code that generates a specific database query.
- Without this type of information, it is almost always difficult to pinpoint the root cause of a problem.

Guidewire recommends that you almost always enable stack trace tracking for the following profiler types:

- Web profiler
- REST API profiler
- Web service profiler

Guidewire recommends that you carefully consider the use of stack trace tracking in profiling batch processes before you enable it. The use of stack trace tracking with batch process profilers can impact application performance.

Create comparable test cases

After you isolate and fix an issue identified by a profiler run, rerun the test case that identified that issue to determine if the provided fix indeed corrected the issue. However, keep in mind that the test cases that you run (before and after the issue fix) need to compare the same items. Thus, ensure that you do the following for both the before and after profiler test runs:

- Use the same environment (server, server mode, database)
- Use the same data
- Run the before and after test cases as the same user
- Access Guidewire Profiler in the same manner
- Perform the exact same steps in both the before and after test cases.

For example, if you ran the first profiler in a test environment on a specific database as an administrative user, use the same test environment, database, and the same user to run the second profiler test.

Thoroughly document each test case

To create easily repeatable test cases, Guidewire recommends that you thoroughly document each test case:

- Include screen shots of the affected application screen for Web profiling.
- Document the environment, the exact data used in the test, and the user who performs the action in web profiling.
- Document the exact sequence of steps to perform in the test case.

Working with Guidewire profilers

The Profiler Configuration screen

You use the Server Tools **Guidewire Profiler > Configuration** screen to enable different types of trace options for the different Guidewire profilers. Some of these options are quite expensive to compute in terms of processing power. Thus, Guidewire recommends that you narrow down your performance issues first before trying these options.

The exact number and type of tracking options available for profiling depend on the following:

- The database type
- The chosen entry point
- The operating system of the application server

Setting web profiler options

The **Web Profiler** pane of the **Guidewire Profiler > Configuration** screen contains one or more profiler options that affect the web profiling session. To enable a profiler option, click (check) its selector square.

Setting entry-point profiler options

After you select a specific entry point to profile, the InsuranceSuite application inserts a row for that profiler under **Entry Point Configuration**. For each table row, you can select one or more profiling options. Only those options with a red X in the table cell are available for that entry point. Options with a gray circle are not available for that profiler.

To enable an option, do one of the following:

- Click the red X in the table cell for that option. The InsuranceSuite application changes the icon to a green check mark indicating that the option is now active.
- Select the option from the **Edit Configuration** drop-down at the end of the profiler table row.

Profiler trace options

The following table lists the potentially available profiler options. Many of these options are only available for entry-point profilers.

Trace option	Availability	Description
Individual Stacks	Work queue profiler	This option stores one stack for each work item and does not roll up the data. Use caution with the Individual Stacks option as the amount of data that the profiler collects can become unbounded. Guidewire recommends that you use this profiler option to collect data only if there are a few items in the work queue.
Stack Trace Tracking	All profilers	This option captures the Java stack trace and the PCF trace at the point at which a query executes. Use caution with this tracing option as it can be very performance intensive.
Query Optimizer Tracing	Not supported by all databases	This option generates data on how the database arrived at a specific execution path. This stack trace creates a trace file on the database server.

Trace option	Availability	Description
High Resolution Clock	Microsoft Windows only	This option uses the Microsoft Windows 100 nanosecond clock.
Extended Query Tracing	All profilers	This option tracks the parse-execute-fetch actions of a SQL statement. This stack trace creates a trace file on the database server.
Diff DBMS Instrumentation Counters	Not supported by all databases	This option captures the DBMS counters at the beginning of the profiling session. It also includes analysis of the differences in the DBMS-specific report.
DBMS Instrumentation Capture Threshold for each Action (millis)	Not supported by all databases	This option generates a DBMS report if an action exceeds the threshold value set by this option. This option is only available if you first enable Diff DBMS Instrumentation Counters tracing. Click Edit Configuration and select Edit DBMS Instrumentation Capture Threshold (millis) to edit the default value of 0.

Enable web profiling

Web profiling collects data associated with the InsuranceSuite application screens.

About this task

Guidewire recommends that you do not profile excessive amounts of data in a single web profiler run as it increases the difficulty of analyzing and interpreting the gathered data. The best use of the web profiler is to profile a single click operation on an application screen.

Procedure

1. Navigate within the application to just before the click operation that you want to profile.
For example, if there are one or more post-on-change operations that you want to profile on a screen, you need to profile each individual post-on-change separately. Thus, you need to navigate in the screen to just before the post-on-change operation before activating the profiler. You then repeat this process for each screen click that you want to profile.
2. Press Alt-Shift-P.
This action opens the Server Tools **Guidewire Profiler > Configuration** popup screen directly in the InsuranceSuite application, without additional application navigation necessary.
3. Under **Web Profiler**, select the profiler configuration options that you want to enable.
Guidewire recommends that you always enable stack trace tracking with web profilers, as, without this option, it is often difficult to determine specific actions of the profiled code.
4. Click **Enable Web Profiling for this Session**.
This action returns you to the application screen from which you started.
5. Perform the web operation that you want to profile.
6. Press Alt+Shift+P to return to Guidewire Profiler.
7. Click **Disable Web Profiling** to end the current Web profiling session.

Results

The Profiler generates the **Web Profiler** report screen automatically after the current web profiling session ends.

Enable an entry-point profiler

An entry point is a specific application component about which the profiler collects data.

Procedure

1. In Server Tools, navigate to **Guidewire Profiler > Configuration**.
2. In the **Entry Point Configuration** pane, select (click) the application entry point that you want to profile.
For example, to enable application profiling for message destinations, click **Enable Profiling for Message Destinations**.
3. In the secondary configuration screen that opens, choose the specific element that you want to profile, then click **OK** to close this screen.
For example, if you clicked **Enable Profiling for Message Destinations**, select a specific message destination to profile and click **OK** to return to the **Configuration** screen. In the **Configuration** screen, InsuranceSuite application adds a message destination row to the **Entry Point Configuration** table, with a different column for each profiler option.

IMPORTANT: Be aware of any on-screen instructions that the application provides as you enable an entry-point profiler. Certain profilers only take effect after you stop and restart the relevant component.

4. Select the profiler configuration options that you want to enable

Disable an entry-point profiler

About this task

Although it is possible to leave many of the entry-point profilers running indefinitely, Guidewire does not recommend that you do so for more than a few hours, or days, at most. The more data that Guidewire Profiler collects, the more storage the data requires and the more load on the Profiler itself.

Procedure

1. In Server Tools, navigate to **Guidewire Profiler > Configuration**.
2. Click **Edit Configuration** (in the right-most column) for the entry-point profiler that you want to stop.
3. Select **Disable Profiling**.

Results

After you disable an application profile entry type, the Profiler removes the row for that entry point from the **Entry Point Configuration** table.

What to do next

To review the collected profiler data, use the Server Tools left-hand navigation pane to access the correct **Profiler Analysis** screen. For example, if you profiled message destinations, navigate to **Guidewire Profiler > Profiler Analysis > Message Destinations** to view the data for the chosen message destination.

Enable profiler for work queues

About this task

You need to restart the work queue that you are profiling after you start Guidewire Profiler.

Procedure

1. Within the Guidewire InsuranceSuite application, press **Alt+Shift+T** to access the Guidewire Server Tools screens.
2. Navigate to **Guidewire Profiler > Configuration**
3. Under **Entry Point Configuration**, click **Enable Profiling for Work Queues**.
4. In the work queue selection screen that opens, select the work queue that you want to profile and click **OK**.
Guidewire Profiler now shows a **Work Queue** row for the work queue that you selected.
5. Click the red Xs in the **Individual Stacks** and **Stack Tracing** columns.
This action turns the icons in those two columns to green check marks, indicating that these two options are now active.
6. Navigate to Server Tools **Work Queue Info**.
7. Restart the work queue that you want to profile.
8. At the completion of your profiling run, return to Guidewire Profiler and disable profiling for the chosen work queue.

Save a profiler analysis report

About this task

This procedure downloads a copy of the data from the selected profiler run in a **.gwprof** file that you can store outside of Guidewire Profiler. The download file contains all data (all views) for the selected profiler run.

Procedure

1. Navigate to the Server Tools **Guidewire Profiler > Profiler Analysis > Profiler Analysis** screen.
2. Select the profiler entry point.
3. In the **Profiler Source** area at the top of the screen, select the profiling run that interests you.
4. Click **Download**.
5. In the dialog that opens, select **Save File** and click **OK**.

View profiler analysis reports

Use Guidewire Profiler to access different views (reports) of the data that a specific profiler run records.

About this task

Procedure

1. Navigate to the Server Tools **Guidewire Profiler > Profiler Analysis** screen.
2. If not already visible, expand the list of profilers under **Profiler Analysis**.
3. Select the profiler of interest.
The application opens a **Profiler Analysis** screen specific to the chosen profiler type.
4. Under **Profiler Source**, select the profiler result that you want to view.
5. Under **Profiler Result**, use the **View Type** drop-down to select a specific report type.

Results

The application populates the **Profiler Result** tables with the requested profiler information.

View historical profiler reports

Procedure

1. Navigate to the Server Tools **Guidewire Profiler > Profiler Analysis > By Time Range** screen.
2. Click **Search**.
The screen shows two calendar date pickers.
3. Enter a start and stop date in which to search for existing profiler analysis reports.
The InsuranceSuite application prints the results of the search to the screen:
 - If the application finds no existing profiler reports that occurred within the specified time frame, it prints a message to that effect.
 - If the application does find one or more profiler reports that occurred within the specified time frame, it lists the reports.
4. In the **Profiler Source** area at the top of the screen, select the report that interests you.
5. In the **Profiler Result** area, select a value from the **View Type** drop-down list.

View uploaded profiler reports

About this task

The file to upload must be a file with a **.gwprof** extension.

Procedure

1. Navigate to the Server Tools **Guidewire Profiler > Profiler Analysis > Saved File** screen.
2. In the **Restore Snapshot** field, browse to find a **.gwprof** file for upload.
3. Click **OK**.

Results

The saved profiler data loads in the **Saved File** screen, which then becomes the **Profiler Analysis** screen. If you navigate away from this screen, Guidewire Profiler deletes the uploaded data from the screen.

View profiler upgrade information

About this task

Use the Server Tools **Guidewire Profiler > Profiler Analysis > Upgrade** screen to review information on InsuranceSuite application database upgrades, beginning from initial server upgrade and including all subsequent upgrade operations thereafter. The upgrade profiler collects this upgrade information automatically.

Procedure

1. Navigate to the Server Tools **Guidewire Profiler > Profiler Analysis > Upgrade** screen.
2. In the **Profiler Source** area at the top of the screen, select the report that interests you.
3. In the **Profiler Result** area, select a value from the **View Type** drop-down list.

Working with Profiler data

The Profiler Analysis screen

After you disable the current application profiling session, the InsuranceSuite application generates a **Profiler Analysis** screen. This screen contains the following areas:

- **Profiler Source**
- **Profiler Result**

Profiler Source

The **Profiler Source** area of the **Profiler Analysis** screen behaves differently depending on whether you are looking at data for web profiling or entry-point profiling.

Web profiling

The **Name** column lists each run of the web profiler in the current user session. This data persists until you either delete the profiler data manually, quit the current user session, or log out of the application.

Entry-point profiling

The **Name** column lists each profiling run stored in the database for that particular component. This data persists until you either delete the profiler data manually or run the Purge Profiler Data batch process.

To see the results for a particular run, select it from the **Name** list. This action causes the Profiler to load the data for the selected profiling run into the tables under **Profiler Result**.

To delete a profiler result, select it from the list and click **Delete**. The Profiler removes the row entry for that profiler result from the table under **Profiler Source**.

Profiler Result

The **Profiler Result** area of the **Profiler Analysis** screen provides a means to view different views (reports) of the profiler data for the run that you select in the **Name** list. Selecting a different view type from the drop-down list updates the screen data to reflect your choice.

View type	Result
Stack Queries	This view lists the database queries that each separate stack executes. The first table shows the stacks in the profiled session. The second table shows the queries executed in that stack. You can obtain more details by opening the various tabs.
Aggregated Queries	This view lists all queries executed as part of the profiled session and some statistics about them, including number of times executed, average time, and similar data.
Search by Query	This view provides a means to search all the stored sessions for this profiler for a given query. Use this view to determine the source of a particular query.
Elapsed	This view lists each profiler frame in chronological order within its stack along with the time in seconds between the profiler PUSH and POP operations for that frame.
Chrono	This view lists each frame in chronological order within its stack along with the time in seconds between the profiler creating the stack and then pushing the frame.
Group Frames	This view lists frames in each stack aggregated by tag and presented in order of total aggregate time on the stack.
Group Stacks	This view presents data similar to that shown in the Group Frames view, except Guidewire Profiler aggregates the frames across all stacks in the session instead of aggregating the frames by individual stacks.
Stacks Grouped	This view lists the stacks in the profiling session grouped by name, along with timing information on each stack group.
Rule Execution	This view lists any rules that application executed during the profiling session. If no rules fired during the session, the Profiler Result region contains the following message: NO PROFILER STACKS FOUND

Profiler data download

Clicking **Download** under **Profiler Result** enables you to save a copy of the data for the selected profiler run to your local machine. The download data includes the data from all the types of views. You cannot download the data from a specific view only.

The download file has an extension of `.gwprof`. Only Guidewire Profiler tools can read and display the data in files of this type.

About query types

A number of reports (views) in the **Guidewire Profiler > Profiler Analysis** screen display SQL query statements. Guidewire uses the following information to indicate the source of the SQL query.

Query	Operation type	Meaning
qRoot	Row	Generic query of a database table row.
<code>/* KeyTable:TableName */</code>	Key load	Embedded comment in the initial query to retrieve the IDs of beans (objects) prior to loading them. See “Key loads” for more information.
bRoot	Bean	Query to load the actual beans (objects) themselves.
rbRoot	Rule	Query generated by a rule.
gRoot	Gosu Query API	Query generated by the Query API.
vbRoot	View Bean	Query to retrieve the <code>ViewEntity</code> object that displays the result of a database query.
aRoot	Array load	Query for loading the keys of all the entities in an array.
amRoot	Array (multiparent) load	Query to load a multiparent array.
lcRoot	Link chain	Query to load an entity link chain. See “Link chains” for more information.

Example SQL statement

The following code is a PolicyCenter example of a SQL statement returned by the profiler. Notice the use of elements from the table in the SQL code.

```
SELECT
  /* KeyTable:pc_profilerconfig; */
  gRoot.ID          col0,
  gRoot.Subtype     col1,
  gRoot.EntryPoint  col2,
  gRoott10.S_en_US col1s
FROM
  pc_profilerconfig gRoot
INNER JOIN
  pctl_profilerconfig gRoott10
ON
  gRoott10.ID = gRoot.Subtype
WHERE
  gRoot.ProfilerEnabled = ?
ORDER BY
  col1s ASC ,
  col2 ASC ,
  col0 ASC
FETCH NEXT ? ROWS ONLY
```

Key load

As the application loads a bean (object), it executes a query to retrieve the bean ID, and sometimes the bean subtype. Using this information, the application queries the bean cache to determine if the bean already exists in the cache. If the bean is not in the cache, the application loads the bean into the cache. Guidewire inserts this type of comment in the initial query to retrieve the bean IDs.

Link chains

Whenever a direct change happens to an event-aware object, the action automatically generates an `xxxChanged` event for that object instance. However, many event-aware objects have sub-objects as well. And, often, it is the case that a user changes the sub-object, but, not the parent object itself. In this case, the application generates a change event for the top-level object, even if the top-level object has not changed directly. Depending on the complexity of the data model, this type of action can lead to an excessive number of link-chain queries, which, in turn, can degrade the performance of bundle commit operations. (The application issues these types of queries during a commit operation.)

It is possible to exclude certain links (foreign keys or arrays) from participating in these queries, if you decide that a particular relationship is not important for the purpose of determining events. It is also possible to exclude entire entities, if you determine that any change to instances of that entity need not cause the generation of a changed event for any top-level entity that references that entity.

To exclude these types of links from the SQL queries, set the `ignoreforevents` attribute on the entity you want the changed event operation to ignore. See the *Configuration Guide* for more information on the `ignoreforevents` attribute.

About the Stack Queries view

The **Stack Queries** view is the default view for all profiler results. To access this view directly, select **Stack Queries** from the **View Type** drop-down list of available views in the Server Tools **Guidewire Profiler** > **Profiler Analysis** screen. The **Stack Queries** view displays information about all the database queries that the profiler ran during its profiling session. The view presents this information in different tables in the **Profiler Results** area of the **Profiler Analysis** screen.

Stack/Query Summary table

The **Stack/Query Summary** table lists information about the stacks that the profiling run generated, along with timing and query information. Each separate row in the table contains information about a different profiler stack (**Stack Name**). Selecting a different stack updates the data shown in the **Aggregated Queries** table. You can ignore any stack row that references the profiler.

The stack name itself provides important information:

- For the web profiler, the stack name lists the beginning and ending screen of each server trip.
- For entry-point profilers, the stack name lists the name of the method, process, or other item about which the profiler collected data.

Aggregated Queries table

The **Aggregated Queries** table lists the database queries that occurred in the selected profiler stack.

About the Chrono view

A profiler **Chrono** view shows information for all profiler frames (including database queries) in chronological order. To access the **Chrono** view of the results of a profiler run, select **Chrono** from the **View Type** drop-down list of available views in the Server Tools **Guidewire Profiler** > **Profiler Analysis** screen. The **Profiler Result** area for a **Chrono** view contains two tables.

Stack information

The first table shown in the **Profiler Result** area lists stack information starting from a fixed point in time (**Start Time**). Guidewire Profiler calculates the start time as the number of milliseconds from Thursday, January 1, 1970 00:00:00 UTC.

Each separate row in this table contains information about a different profiler stack (**Stack Name**). Selecting a different stack updates the data in the second table in the **Profiler Result** area. You can ignore any stack row that references the profiler.

The stack name itself provides important information:

- For the web profiler, the stack name lists the beginning and ending screen of each server trip.
- For entry-point profilers, the stack name lists the name of the method, process, or other item about which the profiler collected data.

The table also includes information on the number of profiler frames (**Frames**) that each stack contains.

Frame information

The second table shown in the **Profiler Result** area shows the frames generated in the stack selected in table 1, in chronological order. This information also includes frames that contain database queries. The following list describes the information in each column of the table.

Time

Column shows a running total of the cumulative time elapsed by the profiler relative to a start time of 0.

Name

Column lists the name of the profiler tag used in profiling a specific piece of code.

Frame (ms)

Column shows the amount of time a specific frame took.

Elapsed (ms)

Column shows the total elapsed time for all frames (code invocations) in that node up to that point of time.

Properties and Counters

Column indicates that type of operation that occurred in each individual frame.

Ways to use query information

A useful way to view query information in Guidewire Profiler is to use the column header sort functionality to look for the top few queries in the following areas.

Avg Time

Look for the individual queries that took the longest average time.

Total Time

Look for the individual queries that took the longest cumulative time.

Total Rows

Look for the individual queries that contain a large number of rows, which indicates a large amount of processing.

Avg Rows

Look for a query with one or more executions with a row count of 1000 (or a similar round number). Then, look for another query with seemingly identical SQL and the same stacks but with a different value for the row count. In most cases, the two queries are the same query. However, the profiler treats the two queries differently as the queries have a different number of arguments (indicated by a ? in the reported SQL statements). Typically, the row numbers differ because the IN or WHERE clause for the query retrieves 1000 rows at a time and thus the row total equals 1000. The query then retrieves the remainder of the rows, which is less than 1000.

Understanding application profiling

Profiler tags

Profiler tags represent sections of code that Guidewire Profiler can profile. A profiler tag is an alias for a piece of code in the Guidewire application for which you want to gather performance information.

The code represents profiler tags by instances of the `gw.api.profiler.ProfilerTag` class. The constructor for the `ProfilerTag` takes a `String` parameter defining the `ProfilerTag` name.

Always create a static final `ProfilerTag` object and preserve it. If you attempt to create more than one instance of the same `ProfilerTag` object, the application generates a warning message in the application log that is similar to the following:

```
WARN Duplicate tag name: tag_name
```

Common profiler tags

Guidewire provides a number of defined profiler tags in the base configuration. The following list describes some of these default tags.

Profiler tag	Meaning
AcquireLock	Captures timing for the database lock
BatchProcess	Captures timing for the profiled batch process
BeanProxy_CopyRow	<i>number of beans => clones bean for bundle transfer</i>
BuildLocation	Captures timing for generating location tags on all PCF elements
BuildSection	Captures timing for building a PCF Section, including all Widgets
BundlerWriter_PrepareBundleForWritePostIDGeneration	Captures timing for preparing the objects in memory for writing to the database after the generation of their ID's
BundlerWriter_PrepareBundleForWritePreIDGeneration	Captures timing for preparing the objects in memory for writing to the database prior to the generation of their ID's
BundleWriter_ExecuteCallbacks	Wrapper tag that captures timing for the operations on objects in memory, including the following: • BundlerWriter_PrepareBundleForWritePreIDGeneration • BundleWriter_ExecuteInternalFieldsCallbacks • BundleWriter_SetInsertedBeanIDs • BundlerWriter_PrepareBundleForWritePostIDGeneration
BundleWriter_ExecuteInternalFieldsCallbacks	Captures timing for executing the following operations on the appropriate objects in memory: • beforeRemove • beforeUpdate • beforeInsert
BundleWriter_SetInsertedBeanIDs	Captures timing for creating object ID's for all beans in memory
BundleWriter_WriteBeans	Captures timing for writing all the entities to the database in the safe-insert / update order
ClaimHealthUpdater	Captures timing for the following: • Loading ClaimMetrics • Updating the Claim Indicators • Preparing Tiers • Updating the Claim Metrics (ClaimCenter only)
ClaimIndicatorUpdate	Captures timing for updating the Claim Indicators (ClaimCenter only)
ClaimMetricUpdate	Captures timing for updating the Claim Metrics (ClaimCenter only)
CloseConnection	Captures timing for closing a database connection
Commit	Captures timing for committing to the database
CreateIteratorDataList	Captures timing for creating an iterator to back a PCF IteratorWidget
CreateIteratorDataQueryProcessor	Captures timing for creating the iterator for a data query processor to back a PCF IteratorWidget
DBCPActivateConn	Captures timing for activating a connection in the database connection pool

Profiler tag	Meaning
DBCPPassivateConn	Captures timing for deactivating a connection in the database connection pool
DisplayNameForBean	Captures timing for calculating the <code>DisplayName</code> for an entity
DisplayNameForQuery	Captures timing for calculating the <code>DisplayName</code> for a <code>QueryRow</code>
DistributedKeyAllocator	Captures timing for generating object keys
ExecuteBatchUpdate	Captures timing for executing a full batch update
ExecuteDatabaseQuery	Captures timing for executing database queries
ExecuteEventsStep	Captures timing for a web request event
ExecuteQuery	Captures timing for executing the following: • Stored procedure • <code>PreparedSelectStatement</code>
ExecuteRule	Captures timing for executing a single rule
ExecuteRuleSet	Captures timing for executing a rule set
ExecuteUpdate	Captures timing for executing the following SQL operations on the database: • Delete • Insert • Update
Flush	Captures timing for writing a bundle to the database, without committing the bundle
GenerateEvents	Captures timing for generating integration events.
GenerateMessages	Captures timing for generating all integration message payloads for the bundle
GetConnection	Captures timing for creating a new connection in the database connection pool
GetReadConnection	Captures timing for obtaining a checking a connection for read from the database connection pool
GetWriteConnection	Captures timing for obtaining a connection from the database connection pool and checking whether it has been used for a previous write operation
LoadBeans	Captures timing for populating all beans from database query results
LoadVirtualArrayKeyList	Captures timing for loading the key list of a virtual array from database query results
Menu	Captures timing for rendering a menu for display
Plugin	Captures timing for executing a plugin
PreRenderLocation	Captures timing for a web request to render the context of the current screen for screen navigation
ReadHandler	Captures timing for reading in the results of a database read query
ReadRows	Captures timing for reading multiple rows from the database, including the instantiation of Java and Gosu objects.
RefreshBeans	Captures timing for refreshing an array of entities from a database query
RenderLocation	Captures timing for rendering a frame of a HTML page
Request	Captures timing for HTTP request
RetireInsertUpdateRemoveBeans	Captures timing for a database update across all needed updates for a transaction

Profiler tag	Meaning
RunLifecycleSteps	Captures timing of the lifecycle steps of a web request
Session	Captures timing for executing a block of code within a transaction: • If the code runs without error, InsuranceSuite application commits the transaction • If the code throws an exception, InsuranceSuite application rolls back the transaction and rethrows the exception
SetBundleToNull	Captures timing for nulling out a bundle, which occurs in the following circumstances, for example: • After deleting (retiring) a bean and InsuranceSuite application removes the bean from the commit bundle so that it no longer possible to reference the bean • After rolling back a commit bundle, either because the user clicked Cancel or because of a validation failure, for example
Update	Captures timing of various update operations
Upgrade	Captures timing and other information about upgrade operations
UpdateDenormClaimContactFromClaimContactRole	Captures timing for updating the denorm fields on the claim contact role (ClaimCenter only)

Profiler frames

A profiler frame contains information corresponding to a specific invocation of profiled code, such as its start and finish times.

In each Profiler session:

- Whenever the code executes a PUSH operation on the profiler stack, Guidewire Profiler creates a profiler frame and pushes the frame onto the stack.
- Whenever the code executes a POP operation on the profiler stack, Guidewire Profiler removes the profiler frame from the stack. The Profiler continues to store the frame information. This makes the information available for future examination.

The sequence of PUSH and its the associated POP operation define the boundaries of a single frame.

The profiler code represents profiler frames by instances of a `gw.api.profiler.ProfilerFrame` object.

Profiler stacks

The purpose of the profiler stack is to store the profiling information for a specific application thread. The name of the stack itself provides important information.

Web profiler

The stack name lists the beginning and ending screen of each server trip. You can ignore all stacks that include profiler pages.

Entry-point profilers

The stack name lists the name of the method, process, or other item that the profiler is tracking.

Each profiler stack implements the standard PUSH and POP functionality of a programming stack. The PUSH and POP operations correspond to the beginning and end, respectively, of a piece of code represented by a profiler tag. Thus, at any time, the current contents of the profiler stack reflect all profiler tags whose code the application is currently executing within that particular thread.

Gosu code represents profiler stacks by instances of a `gw.api.profiler.ProfilerStack` object.

If the current thread initializes the profiler stack, the call to `Profiler.push(ProfilerTag.MYTAG)` pushes a new frame with tag `MYTAG` onto that profiler stack. Otherwise, the call has no effect. Similarly, `Profiler.pop(frame)` is just a pass-through to calling a POP operation on the profiler stack of the current thread. The sequence of PUSH and its the associated POP operation define the boundaries of a single frame.

Using custom profiler tags with Guidewire Profiler

It is possible to profile custom code in the Server Tools **Guidewire Profiler** by creating your own custom profiler tags. Use only your custom class to add new profiler tags.

To define a custom profiler tag, create a globally-accessible Gosu class using one of the following the Java-based classes as a model:

- In `BillingCenter`, use `BCProfilerTag` as a model.
- In `ClaimCenter`, use `CCProfilerTag` class as a model.
- In `PolicyCenter`, use `PCProfilerTag` class as a model.

Defining a custom profiler tag

The following sample code illustrates how to create a custom Gosu class for new profiler tags.

```
package gw.profiler

uses gw.api.profiler.ProfilerTag

class MyProfilerTags {
  public static final var MY_TEST_TAG1 = new ProfilerTag("MyTestProfiler1")
  public static final var MY_TEST_TAG2 = new ProfilerTag("MyTestProfiler2")
  public static final var MY_TEST_TAG3 = new ProfilerTag("MyTestProfiler3")
  //..

  private construct() {
    // Do not instantiate}
  }
}
```

Using a custom profiler tag

To profile a block of custom code, use the following pattern to push and pop profiling information onto the profiler stack.

```
uses gw.api.profile.Profiler
...

ProfilerFrame frame = Profiler.push(ProfilerTag.MY_TEST_TAG1)
try {
  // CODE TO PROFILE
} finally {
  Profiler.pop(frame)
}
```

Notice that the frame PUSH operation is outside the try block. The POP operation occurs in the try finally code. If you include the PUSH and POP operations inside code that iterates (a for loop, for example), each iteration of the code creates a new frame.

Profiling spawned threads

Some processes spread their workload across multiple threads. If you want to use Guidewire Profiler to profile these threads, use the following pattern:

```
gw.api.profiler.Profiler.createPotentiallyProfiledRunnable(ProfilerTag entryPointTag,
  String entryPointDetail, GWRunnable block)
```

This code generates a new Runnable object that executes the given block. This Runnable object profiles the block if the calling thread is also being profiled. If this is the case:

- The Profiler associates the stack for that thread with the stack of the calling thread.

- The Profiler persists that thread along with the stack of the calling thread.

See the Javadoc for the `Profiler.createPotentiallyProfiledRunnable` method for more details.

Understanding properties and counters on a frame

Guidewire profiler frames can hold custom properties and counters that provide more information about system events. Consider the following example:

```
uses gw.api.profiler.Profiler
uses gw.api.profiler.ProfilerTag

public class MyProfilerTags {

    public static var paramValue : String
    public static var ctrValue : Integer

    public function profileCode() {

        public static var myProfilerTag: ProfilerTag = new ProfilerTag("MY_TAG")

        var frame = Profiler.push(MyProfilerTags.myProfilerTag)

        try {

            //Some paramter value, set by method code...
            var param = "some parameter value"

            //Some counter value, set by method code
            var ctr = 5

            frame.setPropertyValue("PARAMETER", param)
            frame.setCounterValue("COUNTER", ctr)

        } finally {

            Profiler.pop(frame)

        }

    }

}
```

After the sample code pops a profiler frame off the stack, the frame contains information about the calculated values of `PARAMETER` and `COUNTER`. The Server Tools **Profiler Analysis** screen then shows these values as well.

Notice also that the frame `PUSH` operation is outside the `try` block. The `POP` operation occurs in the `try finally` code. If you include the `PUSH` and `POP` operations inside code that iterates (a `for` loop, for example), each iteration of the code creates a new frame.

Understanding the Guidewire Profiler API

The `ProfilerAPI` web service enables or disables the InsuranceSuite application profiler framework for various components, such as batch processes or web services. In addition to enabling and disabling profiling for the various application entry points, you can enable web profiling for all subsequent sessions. This API does not provide the ability to profile a specific session or to profile the current active session.

Profiler-related configuration parameters

Guidewire provides the following parameter in configuration file `config.xml` that affect Guidewire Profiler.

ProfilerDataPurgeDaysOld

For use with the Purge Profiler Data batch process in determining the length of time in storing profiler data before deletion.

The Product Model Info screen

In PolicyCenter, the Server Tools **Product Model Info** screen contains a single **Reload Availability** button. If you click this button, PolicyCenter attempts to reload availability data from the directory specified in the `ExternalProductModelDirectory` parameter defined in `config.xml`.

The server attempts to synchronize the lookup entities and the existing product model availability data with the XML files stored in the external lookup directory:

- If the reload is successful, PolicyCenter displays an informational message.
- If reload is not successful, PolicyCenter displays an error message.

In either case, you can check the server log files for details of the reload operations or problems that occurred.

To access the **Product Model Info** screen in PolicyCenter:

- The server must be in development mode.
- The `EnableInternalDebugTools` parameter in `config.xml` must be set to `true`.
- The logon user role must have the **View ProductModelInfo tools screen** permission. The code for this permission is `toolsProductModelInfoview`.

The JProfiler screen

For BillingCenter and ClaimCenter applications, JProfiler is a third-party tool that is available from ej-technologies. It is a Java profiler for use with CPU, memory, and thread profiling. Consult the ej-technologies product documentation for details on how to use JProfiler.

The Metro Reports screen

In ClaimCenter, the Server Tools **Metro Reports** page tracks the Metropolitan Reporting Bureau reports that the InsuranceSuite application is processing. This is a cumulative reporting showing all the reports and their current status. You can filter the list of reports by date, status, and step. You can also use this page to resume the processing of reports.

The Funds Tracking screen

In BillingCenter, funds tracking is designed to be enabled once and then to remain enabled indefinitely. Guidewire recommends that you do not attempt to disable funds tracking once enabled.

By default:

- Guidewire enables funds tracking if the BillingCenter server is in development mode.
- Guidewire disables funds tracking if the BillingCenter server is in production mode.

IMPORTANT: Do not enable funds tracking while performing a BillingCenter upgrade or conversion operation. Instead, complete the upgrade/conversion process and then enable funds tracking.

The Commission Drilldown screen

In BillingCenter, the Server Tools **Commission Drilldown** screen facilitates internal Guidewire development of early releases of BillingCenter. Guidewire does not support this tool for external use. It is possible for the data shown on this page to not reflect commission data accurately. Do not rely on this data.

The Prioritized Preupdate screen

The Prioritized Preupdate screen contains information related to the Prioritized Preupdate Handler. For information on this screen, see the *Configuration Guide*.

InsuranceSuite application clusters

To improve performance and reliability in production planets (environments), Guidewire installs multiple InsuranceSuite application servers in a configuration known as a cluster. A InsuranceSuite application cluster distributes client connections among multiple InsuranceSuite application servers, reducing the load on any one server. If one server fails, the other servers seamlessly handle its traffic.

Cluster terminology

Guidewire uses the following terminology in discussions involving Guidewire InsuranceSuite application clusters.

Host

The physical machine on which one or more Guidewire applications run.

Application instance

An individual InsuranceSuite application deployment. It is possible to run multiple application instances on the same host.

InsuranceSuite application server

The application server associated with each application instance. Guidewire supports only Tomcat for production environments. If running multiple servers on the same host, each server must map to a different physical port.

Server role

A categorization of each application instance in the cluster by its function, as defined by its role. Examples of server roles are `ui` (manages user interface requests), `batch` (manages batch processing), and `messaging` (manages messaging and message destinations).

You define server roles using the cluster `<registry>` element in `config.xml`.

Cluster

A grouping of two or more application instances that have a common configuration and function as an integrated unit. Typically, each server in the cluster has one or more roles or function. Most often, if there are multiple servers assigned the `ui` role, the cluster contains a third-party load balancer as well.

The individual application instances in the cluster must all connect to a common database.

Cluster member

A single application instance within a cluster.

About InsuranceSuite application clusters

The typical InsuranceSuite application cluster consists of multiple InsuranceSuite application servers, with each server having one or more server roles, and a load balancer. In general, a InsuranceSuite application server cluster contains the following types of servers:

- One or more user interface servers (online servers) that process web requests, perform business transactions, and render web pages.
- One or more non-user interface servers (offline servers) that manage batch processing, work queues, scheduling, message destinations, and startable services (plugins).
- A load balancer that manages user interface requests if there are multiple user interface servers.

Technically, non-user interface servers can also process web requests from business users, meaning that they can also act as user interface servers. However, InsuranceSuite typically does not redirect business user web requests to non-user interface servers in order to simplify the management of the cluster.

Cluster membership

As a InsuranceSuite application server joins the cluster, it updates a membership table in the InsuranceSuite application database. All cluster servers periodically poll this table to determine cluster membership.

Cluster availability

To ensure a high degree of availability, the InsuranceSuite cluster configuration include two or more servers with each specific server role. In addition, the cluster also provides ample capacity for running role-constrained items such as message destinations or batch processes.

Cluster monitoring

InsuranceSuite provides cluster monitoring screens that are available to those with privileges to view the Server Tools screens:

- The Server Tools **Cluster Members** screens provide information on each server in the cluster.
- The Server Tools **Cluster Components** screen provides information on the components running on a given server.

Cluster cache usage

Cluster inter-communication ensures that if an object changes in one cluster member cache, that member sends a cache invalidation message to the other members of the cluster. This message instructs the other cluster members to tag the

cache entry for the object as obsolete and evict it from the cache. The next time a InsuranceSuite application server needs the object, it reloads the object value directly from the database. This mechanism is different from full cache synchronization, in which a server broadcasts the new value of the object to other cluster members.

It is possible to lose message packets. Network failures or other issues also can disrupt communication between cluster members. Such cases can result in cache eviction messages not being propagated to all cluster members. As a result, one or more cluster members can contain stale cache entries.

InsuranceSuite applications implement a data versioning mechanism to prevent data corruption. One or more version mismatches indicates that one or more objects have changed since the InsuranceSuite application last accessed the entities. This mismatch results in InsuranceSuite issuing a Concurrent Data Change Exception (CDCE). The user or batch job can then re-issue a change based on the latest values entered.

Monitoring cluster health

Typically, hardware or software load balancers check the health of the various cluster members and stop directing traffic to a cluster member that stops responding. This check simply verifies that the corresponding network port responds.

Therefore, it is possible that a load balancer redirects traffic to a cluster member that is not capable of processing that traffic appropriately. For example, the application is:

- Not fully started yet
- At the MAINTENANCE run level
- Experiencing significant issues, such as an out-of-memory condition

Or, some other infrastructure constraints exist. For example, some environments cannot use source IP stickiness to load-balance conversational SOAP calls. In this situation, if using Guidewire ContactManager, you can instantiate one ContactManager instance on each server machine hosting one or more InsuranceSuite application instances. You then configure each InsuranceSuite application instance on that machine to direct contact requests to that local ContactManager instance. In this scenario, if the local ContactManager instance is not functioning properly, it is advisable to stop directing traffic to any of the InsuranceSuite application instances on that server.

InsuranceSuite applications include a simple HTTP ping utility that enables you to check the application status with a web browser. For instance, to check the status of an instance of an InsuranceSuite application running on port 8080 of the local computer, you would enter the following URL into a web browser:

`http://localhost:8080/xx/ping`, where `xx` is the application product code.

There are three possible responses by the web browser:

- If the application is running at the default MULTIUSER run level, the browser displays the number 2.
- If the application is running in any mode other than MULTIUSER, the browser displays a specific ASCII character, depending on the circumstances.
- If the InsuranceSuite application server is not running, the browser displays an HTTP failure message, depending on the configuration of the server.

The following list describes the ASCII characters that the browser can return.

ASCII character in ping URL result	Run level
No response to the HTTP request	Server not running
ASCII character 30, the Record Separator character	DB_MAINTENANCE

ASCII character in ping URL result	Run level
ASCII character 40, the character "("	MAINTENANCE
ASCII character 50, the character "2"	MULTIUSER
ASCII character 0. A null character result might not be returnable for some combinations of HTTP servers and clients.	GW_STARTING

Invoking the ping utility programmatically provides more granular information on the server's status.

It is possible to configure the load balancers access this URL on a regular basis to determine the health of each member of the cluster. You can then use these results to create redirection logic.

For example, suppose that you have an environment in which ClaimCenter directs traffic to a local ContactManager instance. You then configure the load balancer to only redirect traffic to a ClaimCenter instance if both that instance and the ContactManager instance on that server are accessible for user requests.

Monitoring InsuranceSuite application clusters

The Server Tools **Cluster** screens, accessible to system administrators, provide information on your InsuranceSuite application cluster installation. For the InsuranceSuite application to make this screen visible, the value of cluster configuration parameter `ClusteringEnabled` in `config.xml` must be `true`.

The **Cluster** screens provide information on the server cluster installation that includes the following items.

Information about the local server member	Its server ID, the server roles assigned to this InsuranceSuite application server, and similar information
Information about individual members recognized by the cluster	Their server IDs, number of active user sessions on each server instance, the server roles assigned to each application instance, date and time of each server start, and similar information
Information on the components running on each cluster member	The component state, date and time of each component start, and similar information
Information on any component lease failover in progress	The date and time of the deadline in which to complete the failover process
History of each member in the cluster	The start and stop times for each cluster member, server roles, run level, and similar information

From this screen, on any cluster member, you can start or cancel a planned shutdown for any recognized server instance in the cluster.

Using the InsuranceSuite application server ping utility

Guidewire provides an unauthenticated web page that you can ping to access information about an InsuranceSuite application server. This is the second version of the ping utility, hence, `v=2` in the URL.

To access this web page, use the following URL:

`http://server:port/xx/ping?v=2`, where `xx` is the application product code.

The ping utility returns the following types of information on the return HTML page, depending on various factors, including whether the server is a production server and whether the server start was successful.

Information type	Result description
Planned shutdown status	Shutdown status (for example, planned, ready) if the server is involved in a planned shutdown.
Server run level code	ASCII decimal code. For example, 45, represents the '(' character.
Server run level name	Server run level, such as MULTIUSER.
Server run level ordinal	QuickStart (Jetty) run level. For example, 5, corresponds to the MULTIUSER run level.

Information type	Result description
Server ID	Name of the server host.
Server up time	Time, in seconds, for which the server has been operational.
Server start up exception	Exception that caused the server to fail start up, if applicable. By default, the ping utility does not display this information on a production server.
Thread stack trace	Stack trace of the thread performing the transition from one server run level to another. By default, the ping utility does not display this information on a production server.

Using the ping utility with a production server

Due to security concerns, the InsuranceSuite application server ping utility does not return certain information if used with a production server. In production mode, the ping utility does the following:

- It replaces the text of any exception that occurs at server start up with <not null>.
- It removes the stack trace of the thread that performs the work of transitioning from one server run level to another entirely.

To view this information for a production mode server, start the production server with the following JVM command option:

```
-Dgw.ping.servlet.show.stack=true
```

Example output from the ping utility

The following examples illustrate the kind of information that the server generates for the different server scenarios:

- Successful server start
- Failed server start
- Server transition between run levels
- Planned server shutdown

Successful server start

The following code is an example of the ping utility return values if the PolicyCenter server starts successfully.

```
{
  "runLevelCode": 50,
  "runLevelName": "MULTIUSER",
  "runLevelOrdinal": 5,
  "serverId": "PolicyCenterServer1",
  "uptimeSeconds": 45
}
```

Failed server start

The following code is an example of the ping utility return values if the BillingCenter development server fails to start.

```
{
  "runLevelCode": 40,
  "runLevelName": "NODAEONS",
  "runLevelOrdinal": 3,
  "serverId": "BillingCenterServer1",
  "startupException": "java.lang.RuntimeException: Test Startup Exception\n\tat\n\t\tcom.guidewire.pl.system.server.PingServerServletTest.\ntestInitTabStateJSONObjectShowsStartupException(PingServerServletTest.java:52)\n\n\tat... ",
  "uptimeSeconds": 40
}
```

The following code is an example of the ping utility return values if the ClaimCenter production server fails to start. By default, the InsuranceSuite application does not show the actual exception text and instead replaces the text with <not null>.

```
{
  "runLevelCode": 40,
  "runLevelName": "NODAEMONS",
  "runLevelOrdinal": 3,
  "serverId": "ClaimCenterServerPROD1",
  "startupException": "<not null>",
  "uptimeSeconds": 40
}
```

Server transition between run levels

The following code is an example of the ping utility return values on a development server while it is transitioning from one run level to another. In this example, the PolicyCenter server is transitioning from MULTIUSER run level to the DAEMONS run level.

```
{
  "attemptingTransition": {
    "fromRunLevelName": "MULTIUSER",
    "fromRunLevelOrdinal": 5,
    "threadStackTrace": "Thread-142:TIMED_WAITING\n\tat    sun.misc.Unsafe.park(Native Method)\n\tat    java.util.concurrent.locks.LockSupport.parkNanos(LockSupport.java:215)\n\tat...",
    "toRunLevelName": "NODAEMONS",
    "toRunLevelOrdinal": 3
  },
  "runLevelCode": 45,
  "runLevelName": "DAEMONS",
  "runLevelOrdinal": 4,
  "serverId": "PolicyCenterServer1",
  "uptimeSeconds": 6814
}
```

Note: Use `system_tools` command options to transition a InsuranceSuite application server from one run level to another.

Planned server shutdown

The following code is an example of the ping utility return values.

```
{
  "runLevelCode": 50,
  "runLevelName": "MULTIUSER",
  "runLevelOrdinal": 5,
  "serverId": "testsrv1",
  "uptimeSeconds": 1820
}
```

The following code is an example of the ping utility return values after starting a planned server shutdown from the (Server Tools) **Info Pages > Cluster Members** page.

```
{
  "plannedShutdownStatus": "activated",
  "runLevelCode": 50,
  "runLevelName": "MULTIUSER",
  "runLevelOrdinal": 5,
  "serverId":
    "testsrv1",
  "uptimeSeconds": 1825
}
```

The following code is an example of the ping utility return values after the server shutdown completes.

```
{
  "plannedShutdownStatus": "ready",
  "runLevelCode": 50,
```

```
"runLevelName": "MULTIUSER",  
"runLevelOrdinal": 5,  
"serverId": "testsrv1",  
"uptimeSeconds": 1830  
}
```


Working with a InsuranceSuite application server cluster

Plan the cluster so that if any one server fails, the other servers in the cluster can handle its traffic without being overwhelmed. The InsuranceSuite application servers in the cluster can run on separate computers, or you can run multiple servers on the same computer. Guidewire recommends you maintain at least three InsuranceSuite application servers in the cluster, whether on the same or different physical computers. With multiple servers running on the same computer, in the event of a failure, then all servers are unusable. Of course, the exact configuration depends on specific usage needs. InsuranceSuite includes an integrated load balancer that bridges client connection requests to a InsuranceSuite application server.

Create an InsuranceSuite application cluster

To create an InsuranceSuite application server cluster, first define the individual servers in configuration, then deploy your configuration to each server in the cluster.

Procedure

1. In your source configuration, for use on all InsuranceSuite application servers in the cluster, open `config.xml` for editing.
2. In `config.xml`, set the following clustering-related configuration parameters appropriately:
`ClusteringEnabled`
`ClusterMemberPurgeDaysOld`
`ConfigVerificationEnabled`
`PDFMergeHandlerLicenseKey`
3. Define the set of valid server roles for use in this cluster using the `<registry>` element in file `config.xml`.
Note: Guidewire recommends that you do not set the server roles on individual cluster members using the `<registry>` element in `config.xml`. Instead, use JVM system properties at server start to set the server roles for that server.
4. In `config.xml`, set the value for `KeyGeneratorRangeSize`.
5. Create a deployment WAR or EAR file for the Guidewire InsuranceSuite application.
6. Install a InsuranceSuite application cluster server in the same way that you install a standalone InsuranceSuite application server.

IMPORTANT: If you install multiple InsuranceSuite application servers on the same host machine, each InsuranceSuite application server must run in its own JVM instance.

7. Start each member of the cluster in turn.
As you start each server, set the server roles for that server using a JVM system parameter.
8. In InsuranceSuite application, navigate to the Server Tools **Cluster Members** screen and verify the information.

Enabling clustering

To enable clustering in an InsuranceSuite installation, set the `ClusteringEnabled` parameter in `config.xml` to `true` on all cluster members, for example:

```
<param name="ClusteringEnabled" value="true"/>
```

To disable clustering and remove a server from a cluster, set this parameter to `false` on that server. After the server is no longer in a cluster, it behaves as any other standalone InsuranceSuite application server.

Adding a InsuranceSuite application server to cluster

InsuranceSuite does not require that you use the cluster registry in file `config.xml` to define cluster members. Its use is optional in that regard. Instead:

- You can specify the server ID and server roles from the command prompt at server startup using JVM command options.
- Or, you can specify a subset of the cluster server members in the cluster registry and set other server properties from the command prompt.

For example, you can define the set of servers assigned the batch server role in the cluster registry and specify the remaining server IDs through JVM options at server start.

There is no issue in adding a server to the cluster that is not specified in the cluster registry in `config.xml`. You can add a server to the cluster without any change of configuration or stopping and restarting servers.

Add a server to a InsuranceSuite application cluster

Procedure

1. Deploy the current cluster configuration WAR or EAR file to the server that you intend to add to the InsuranceSuite application cluster.
2. Verify that this configuration has parameter `ClusteringEnabled` set to `true` in the configuration's `config.xml` file.
3. If using the `config.xml` server registry, you must set the `serverid` attribute for this server to a unique value within the cluster.
If you use a non-unique `serverid` value, the server with the duplicate ID does not start and does not join the cluster.
4. Start the new server.

Results

After you start the new server, it connects to the cluster and compares its configuration with the cluster configuration stored in the InsuranceSuite application database. It performs a checksum of the `config.xml` file and checks the `config` subdirectories. If the configurations differ, the server fails startup and the InsuranceSuite application writes failure messages to the log file.

Restarting the InsuranceSuite application servers in a cluster

There are several different scenarios that require you to bring down one or more members of the InsuranceSuite application cluster and then restart the servers.

Server restart after server maintenance

In performing maintenance on the servers in the InsuranceSuite application server cluster, do the following for each InsuranceSuite application server in the cluster, as required:

- Set the server run level to maintenance.
 - Contact Guidewire Support to set the server run level to maintenance.
- Perform the needed work.
- Restart the server.

Server restart after full InsuranceSuite application upgrade

In a full upgrade, you bring down all members of the InsuranceSuite application server cluster completely. Before starting a full upgrade, click **Start Full Upgrade** in the Server Tools **Upgrade and Versions** screen. You can do this on any server in the cluster. This action sets a flag in the InsuranceSuite application database to indicate that a full upgrade is in progress. As long as the upgrade-in-progress flag exists, it is not possible to start a cluster member that uses the old (non-upgrade) WAR/EAR file.

In a full database upgrade, you must start a single cluster server and allow it to complete the full upgrade cycle before starting the upgrade on the remaining servers. After all the servers in the cluster start with the upgrade WAR/EAR file, the InsuranceSuite application automatically deletes the upgrade-in-progress flag from the InsuranceSuite application database.

Use the following guidelines as you bring up the individual cluster members:

- After the first server completes the upgrade cycle, it is possible to bring up all other servers in the cluster in parallel. However, if starting a large numbers of servers causes resource contention, insert a short interval of time between each server start, for example, 10 seconds.
- As a general rule, start servers that manage back-end processes first. For example, start servers with the batch and messaging roles before starting servers with the ui role.

Server restart after database restore

It is sometimes the case that you want to restore the cluster database from one of the following:

- A non-cluster server
- A server in a different cluster environment but with the same code base

Attempting to start the server after the database restore can generates database errors sufficient to cause the server to not start. To prevent these errors, it is necessary to start a single cluster node and let it cycle through a full database upgrade before starting the other members of the cluster.

To set the full upgrade flag on a server as you start the server, use the following JVM system property, with `YYYYMMDD` being today's date:

`-Dgw.xx.full.upgrade.intended.date=YYYYMMDD`, where `xx` is the application product code.

For example:

```
gwb runServer -Dgw.bc.full.upgrade.intended.date=20180726
```

After the first server completes the database upgrade operation, start the other servers in the cluster.

Working with component leases

A lease represents the right to perform some job for some period of time. Within a InsuranceSuite application cluster, a lease specifically represents one of the following:

- A single run of a batch process
- An instance of a message destination
- An instance of a startable plugin, if it is a single instance plugin

Each server instance in the InsuranceSuite application cluster has a lease manager for each lease type. However, some functionality require a server with a specific server role. For example, only a server with the `messaging` role can acquire and manage a lease for a message destination.

Each type of lease manager can perform the following actions:

- Create, acquire, renew, or retire a lease
- Release a lease or request the release of a lease from another lease manager
- Transfer a lease to, or request the transfer of a lease from, another lease manager

Lease management contention

InsuranceSuite application cluster lease management provides for minimal contention if two lease managers attempt to acquire the same lease at approximately the same time. Guidewire guarantees that only one server can acquire a specific lease. Guidewire also guarantees that, for an exclusive batch process, only a single such lease can exist at any one time.

Simple lease management lifecycle for a batch process

The following is a general description of the lease management lifecycle for a batch process:

- The InsuranceSuite application translates the request to execute a batch process into the creation of an available lease and broadcasts the lease availability to all servers in the cluster.
- A lease manager on a server with the batch role discovers the available lease and acquires it. No other server in the cluster can acquire the lease. The lease is exclusive.
- The server with the active lease executes the batch process until processing is complete, with the server extending the lease as needed.
- At the completion of the batch process, the server lease manager releases the lease.

Simple lease management lifecycle for a message destination

The following is a general description of the lease management lifecycle for a message destination:

- At message initialization, the InsuranceSuite application creates each enabled destination as an available lease.
- A lease manager on a server with the messaging role discover the available leases and attempts to acquire a lease.
- As a server succeeds in acquiring a lease, it initializes the corresponding destination and places the destination into the proper state (started, stopped, or suspended).
- A destination runs as intended, based on its state.
- A server periodically extends its destination lease until the messaging daemon stops, such as during server shutdown or if you set the server run level to a level below DAEMONS.
- The server releases its destination lease.

Component lease failover

In the course of standard cluster operation, meaning no network issues, no lost cluster members or similar problems, the InsuranceSuite application periodically and automatically renews all component leases. Thus, the expiration of a lease is an exceptional condition that requires attention. If a component lease expires, the InsuranceSuite application considers the owner of that lease to have failed. To detect this situation, the cluster lease managers periodically search for expired leases and initiate failover of the expired lease to another member of the cluster.

After a lease manager detects an expired lease, it does the following:

- It terminates the lease.
- It updates the lease history.
- It deletes the lease or recreates the lease for acquisition by another cluster member, depending on the type of the lease.

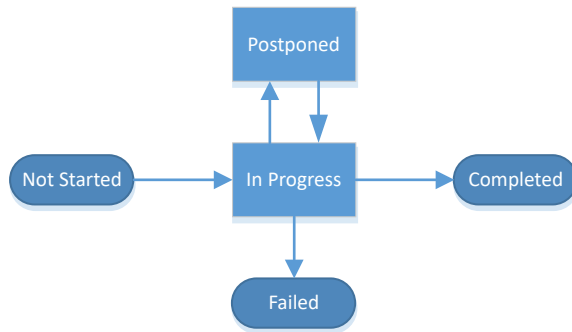
Automatic and manual failover

The InsuranceSuite application supports both automatic and manual failover of a component lease from one cluster member to another member of the cluster. However, there are situations in which an automatic component failover is not desirable, for example:

- There is a need for further additional configuration of an external third-party product before it is possible to start a destination or plugin on a different computer.
- There is a need for further diagnostic testing to determine the exact cause of failure before initiating the failover process.

Automatic failover of a component lease

The following state diagram illustrates the different states in the automatic failover of a component lease. The InsuranceSuite application defines these states in the `FailoverState` typelist. The `FailoverState` typelist does not contain the 'Completed' state. The 'Completed' state is implicit after the failover process completes and the lease manager deletes the original lease.



Initially, each component lease starts in the Not Started failover state. If a lease expires, the first lease manager that discovers the expired lease does the following:

- It sets the lease to the 'In Progress' failover state. After set to this state, the component associated with the lease cannot run anywhere until there is a resolution of the issue that caused the lease to expire.
- It sets the **Retry Failover** field in the Server Tools **Cluster Components** screen to the following value:
`CurrentTime + BackgroundTaskFailoverPlugin.FailoverTimeout`

If more than one lease manager discovers the expired lease at the same time, only the first lease manager continues the failover handling. The other lease managers detect that their SQL updates do not change anything and do not continue the failover process for that lease.

The lease manager that started the failover calls the `handleComponentNameFailover` method on the `BackgroundTaskFailoverPlugin` plugin to determine what to do next with the lease. The method returns one of the following actions to handle the component lease failover.

Complete the failover

The `BackgroundTaskFailoverPlugin` plugin logic confirms the lease failure and instructs the lease manager to complete the failover. In this case, the lease manager completes the failover process, either by deleting or expiring the lease.

Postpone the failover

It is possible that the `BackgroundTaskFailoverPlugin` plugin logic cannot reliably confirm the lease failure. In this case, it can postpone the failover process by returning an associated action to take and the time duration to wait before taking that action. The lease manager updates the **Retry Failover** field in the Server Tools **Cluster Components** screen with the following value:

`Current Time + FailoverHandlingResult.Duration`

After the updated retry failover time expires, the lease manager considers the lease expired and starts the process of lease failover again.

Dismiss the failover

It is possible that the `BackgroundTaskFailoverPlugin` plugin logic decides the specified background task did not fail, or, that this particular task requires some manual action. In this case, the `BackgroundTaskFailoverPlugin` plugin logic dismisses or fails the automatic failover of the lease. The lease with its `FailoverState` set to 'Failed' remains in the database until there is some kind of manual intervention. The failover process does not attempt to retry the automatic failover.

Use an external tool

The `BackgroundTaskFailoverPlugin` plugin logic returns a failover handled action. This action instructs the lease manager to do nothing with the lease. An external tool either deletes or renews the lease.

Calling an external tool to complete the failover can happen in any of the following ways:

- Programmatically calling the `SystemToolsAPI.nodeFailed` method.
- Programmatically calling the `SystemToolsAPI.completeFailedFailover` method.

- Clicking the **Complete Failover** button on the Server Tools **Cluster Components** screen.

If the cluster member that started the failover does not complete the failover in the specified retry failover time, another cluster member detect this condition. The second cluster member then restarts the failover.

If at any point the original lease manager for the lease takes action to renew the lease, it does the following:

- It sets the `FailOver` state for the lease to `Not Started`.
- It resets the `Retry Failover` value to null.

At this point, the renewal of the lease resets the automatic failover process and negates any previous failover action undertaken for the renewed lease.

The background task failover plugin

In the base configuration, Guidewire provides a `BackgroundTaskFailoverPlugin` plugin implementation that you configure to manage component lease failover. The default implementation class for this plugin is Gosu class `DefaultBackgroundTaskFailoverPlugin`. This default implementation makes the implicit assumption that there is no manual cleanup work required for any batch process, message destination, or startable plugin after a component lease failover.

At the failover of a component lease, the failover logic postpones the start of the failover process by several minutes, the value of static variable `INITIAL_POSTPONE_TIMEOUT`. Failover logic implements the timeout to handle the case in which the database was unavailable for a relatively long period of time, and then comes back online. The failover postponement provides some time for the cluster members to recover and renew their leases.

After the failover postponement completes:

- If the cluster member that owns the lease does not return to the cluster, the automatic failover process continues.
- If the cluster member that owns the lease does return to the cluster, the automatic failover process fails.

Active external monitoring

Some clusters use external monitoring and management software to watch the JVM processes of cluster members. Guidewire provides Gosu class `ActiveExternalMonitoringBackgroundTaskFailoverPlugin` as a template that you can use in such installations. You must implement your own logic for the `notifyExternalMonitoringAboutExpiredlease` method in this class.

Passive external monitoring

Guidewire provides Gosu class `PassiveExternalMonitoringBackgroundTaskFailoverPlugin` as a template to use in requesting a cluster member status report from external monitoring and management software. In this case, the external monitoring and management software is not actively watching the JVM processes of cluster members. Instead, it provides cluster member data on request.

Any monitoring and management software that you use for this purpose needs to be able to do the following:

- Check if a JVM process on a specified cluster member is alive and return the process uptime in seconds if the process is running.
- Terminate a specified JVM process.
- Start a new JVM process.

The plugin implementation manages the failover.

Component load balancing

In the base configuration, Guidewire provides several default strategies for managing the load balancing of messaging destinations, message processors, and startable services within the InsuranceSuite application cluster. Use plugin `BackgroundTaskLoadbalancingPlugin` to manage the load balancing process.

Load balancing strategies

In the base configuration, Guidewire provides the following strategies for load balancing components.

Work stealing

The work stealing strategy periodically transfers a component lease from an over-utilized server to a server that is under-utilized.

Work acquisition

The work acquisition strategy provides under-utilized servers with a chance to acquire a lease on an available component.

Plugin BackgroundTaskLoadBalancingPlugin

Guidewire provides the `BackgroundTaskLoadBalancingPlugin` plugin to manage the dynamic load balancing of long-running tasks such as startable services or message destinations. You access the plugin configuration in Studio at the following location:

configuration > config > Plugins > registry > BackgroundTaskLoadBalancingPlugin

In the base configuration, Guidewire provides the following default implementation class for the `BackgroundTaskLoadBalancingPlugin` plugin:

```
gw.api.system.cluster.DefaultBackgroundTaskLoadBalancingPlugin
```

This class implements the `BackgroundTaskLoadBalancingPlugin` interface. It is possible to modify the implementation of the default Gosu plugin class to meet your business needs.

In the base configuration, Guidewire sets the following Mode plugin parameters.

Name	Value
startablePluginLoadBalancingMode	notransfer
messageDestinationLoadBalancingMode	notransfer
messageProcessorsLoadBalancingMode	dynamic

Possible Mode values

Each of the Mode plugin parameters can take one of the following values.

Value	Description
disabled	Disables both the work acquisition and work stealing strategies.
dynamic	Enables both the work acquisition and work stealing strategies.
notransfer	Enables the work acquisition strategy only.

Methods on class DefaultBackgroundTaskLoadBalancingPlugin

Class `DefaultBackgroundTaskLoadBalancingPlugin` is the default implementation class for the `BackgroundTaskLoadBalancingPlugin` plugin. The `DefaultBackgroundTaskLoadBalancingPlugin` class implements the `BackgroundTaskLoadBalancingPlugin` interface, which contains the the following methods:

- `rebalanceStartablePlugins`
- `rebalanceMessageDestinations`
- `rebalanceMessageProcessors`
- `selectStartablePluginsToStartNow`
- `selectMessageDestinationsToStartNow`

- `selectMessageProcessorsToStartNow`

Method reference

`rebalanceStartablePlugins(LoadBalancingContext context)`

This method analyzes the startable plugin services that exist on the other member nodes of the InsuranceSuite application cluster and requests the transfer of those services to the current server, if the current server is underloaded.

This method returns the load balancing result as a `LoadBalancingResult` object. The result cannot be null.

`rebalanceMessageDestinations(LoadBalancingContext context)`

This method analyzes the message destinations that exist on the other member nodes of the InsuranceSuite application cluster and requests the transfer of those message destinations to the current server, if the current server is underloaded.

The method returns the load balancing result as a `LoadBalancingResult` object. The result cannot be null.

`rebalanceMessageProcessors(LoadBalancingContext context)`

This method analyzes the message processors that exist on the other member nodes of the InsuranceSuite application cluster and requests the transfer of those processors to the current server, if the current server is underloaded.

The method returns the load balancing result as a `LoadBalancingResult` object. The result cannot be null.

`selectStartablePluginsToStartNow(List<ComponentInfo> availableStartablePlugins, LoadBalancingContext context)`

This method selects the startable plugins to start. Use this method to set which components start on the current server and which components start on the other member nodes of the InsuranceSuite application cluster. The principal use of this method is to set a more uniform load distribution. If something delays the start of a component, and no other server starts it, InsuranceSuite application calls this method on the current server again.

This method returns the selected components to start as a list of `<ComponentInfo>` objects.

`selectMessageDestinationsToStartNow(List<ComponentInfo> availableMessageDestinations, LoadBalancingContext context)`

This method selects the message destinations to start. Use this method to set which components start on the current server and which components start on the other member nodes of the InsuranceSuite application cluster. The principal use of this method is to set a more uniform load distribution. If something delays the start of a component, and no other server starts it, the InsuranceSuite application calls this method on the current server again.

This method returns the selected components to start, as a list of `<ComponentInfo>` objects.

`selectMessageProcessorsToStartNow(List<ComponentInfo> availableProcessors, LoadBalancingContext context)`

This method selects the message processors to start. Use this method to set which components start on the current server and which components start on the other member nodes of the InsuranceSuite application cluster. The principal use of this method is to set a more uniform load distribution. If something delays the start of a component, and no other server starts it, the InsuranceSuite application calls this method on the current server again.

This method returns the selected components to start, as a list of `<ComponentInfo>` objects.

The LoadBalancingContext object

All `DefaultBackgroundTaskLoadBalancingPlugin` methods require that you pass a `LoadBalancingContext` object as you execute the method. Public interface `LoadBalancingContext` provides the following methods.

getAllComponents()

Method returns a `List<ComponentInfo>` object, which is a list of all components running in the cluster.

requestTransferToThisServer(*component*)

Method requests the transfer of the given component to this server. Use the `getAllComponents` and the `ComponentInfo.getServerId` methods to obtain the current owner of a component. The component transfer itself is an asynchronous process. This method returns `true` if the transfer request completed successfully, which does not mean the transfer itself completed successfully.

isAllowedToStartOnThisServer(*component*)

Method returns `true` if it is possible to start the given component on the current server.

getTransferInfo(*component*, *unit*, *timeout*)

Given a specific component (*component*), the method returns information on the component transfer or `null` if the transfer attempt timed out. Parameters *timeout* and *unit* provide the time to wait before the transfer request times out.

The return `TransferInfo` object has two useful methods:

- `getCurrentOwner` - Method returns the current owner of the component. If the method returns `null`, it means that the component is already stopped.
- `getTargetOwner` - Method returns the owner of the target component. It cannot return `null`.

scheduleComponentAvailabilityCheck(*delay*, *unit*)

Method schedules a component availability check. If a component is available to start, this method calls either of the following methods on the `BackgroundTaskLoadBalancingPlugin` plugin:

- `selectMessageDestinationsToStartNow`
- `selectStartablePluginsToStartNow`

Parameters *delay* and *unit* provide the time to wait before performing the availability check.

The ComponentInfo object

The `ComponentInfo` object contains information about a distributed component instance in the InsuranceSuite application cluster. Many of the `DefaultBackgroundTaskLoadBalancingPlugin` methods return a `ComponentInfo` object. For example, the `selectStartablePluginsToStartNow` method returns a `ComponentInfo` object for each startable plugin that the method starts. Public class `ComponentInfo` provides a number of getter and setter methods for the following component properties.

Property	Data type	Description
type	ComponentType	Type of the component, one of the following: • Startable service • Message destination • Message processor
uniqueId	String	UUID (universally unique ID) of the server on which the component is running.
code	String	Code designation of the component, email for a message destination, for example.
name	String	Name of the component, Email for a message destination, for example.
state	ComponentState	Assignment state of the component, for example: • Assigned • Unassigned • Ended
serverId	String	ID of the server that has the current lease on the component.
startRequested	Date	Date and time that the server received the component start request.

Property	Data type	Description
started	Date	Date and start time of the component.
leaseExpiration	Date	Date and time at which the component lease expires.
transferRequested	Date	Date and time at which the server received a request to transfer this component to another server.
transferTarget	String	The ID of the server to which the transfer request was made.
terminateRequested	Date	Date and time at which the server received a terminate request for this component.
stopped	Date	Date and time that the component stopped.
retryFailover	Date	Date and time of the deadline in which to complete the failover process.
replacementId	String	UUID of the server to which to transfer the component.

It is possible to see many of the values for these properties in the Server Tools **Cluster** screens, especially the **Components** screen.

The LoadBalancingResult object

A LoadBalancingResult object is the result of a load balancing operation. For example, the `rebalanceStartablePlugins` method on plugin `DefaultBackgroundTaskLoadBalancingPlugin` returns a LoadBalancingResult object. Public class LoadBalancingResult provides the following methods.

Method	Returns	Description
<code>getRepeatDelay()</code>	long	Method returns the delay value from load balancing result as a long integer.
<code>getRepeatUnit()</code>	unit	Method returns the delay time unit from load balancing result.
<code>isNeverRepeat()</code>	Boolean	Method returns one of the following: • true - The load balancing operation never repeats. • false - The load balancing operation repeats.
<code>repeatAfter(<i>unit</i>, <i>delay</i>)</code>	LoadBalancingResult	Static method that repeats the load balancing operation after the specified delay. Parameters <i>delay</i> and <i>unit</i> provide the time to wait before repeating the load balancing operation.. The <i>delay</i> value cannot be negative.
<code>neverRepeat()</code>	LoadBalancingResult	Static method that disable the load balancing operation after one attempt.

Deploying changes in a cluster

Blue/green deployment is a method of deploying changes in a clustered server environment. This method gradually transfers user traffic from a prior version of an application or microservice to a new version, and thereby minimizes production downtime.

You can use blue/green deployment to deploy supported changes to a InsuranceSuite application cluster, while avoiding the need to shut down all InsuranceSuite application production servers.

Very large changes, or changes that are not supported by blue/green deployment, must still be deployed using the full deployment process, with the associated full deployment downtime and safeguards.

For related information, see "Blue/green deployment" in the [Cloud Platform](#) documentation.

Geocoding, catastrophe search, and heat maps

Geocoding is the process of assigning a latitude and longitude to an address. Guidewire supports geocoding in ClaimCenter, PolicyCenter, and ContactManager.

If enabled, ClaimCenter uses geocoding in producing ClaimCenter catastrophe searches and heat maps.

Understanding geocoding

The *geocoding* process assigns latitudes and longitudes to addresses. Software then uses geocoded addresses to present users with geographic information, such as the distance between two addresses. All primary addresses in ClaimCenter, ContactManager, and PolicyCenter can be geocoded.

The Geocode plugin interface

To implement geocoding, the InsuranceSuite application provides a default GeocodePlugin plugin interface. Implementations of the plugin connect with specific external geocoding services, which provide geocode coordinates for specific addresses. Typically, plugin implementations also use an external mapping service to calculate and return proximity information, driving instructions, and maps. You enable the plugin implementation in Guidewire Studio and specify the parameters for the implementation you choose.

InsuranceSuite application provides two fully functioning and supported GeocodePlugin implementations:

- The BingMapsPluginRest Gosu class. This plugin implementation connects to the Microsoft Bing Maps Geocode Service. If you intend to use the BingMapsPluginRest implementation, your organization must have a valid account with Microsoft and an application key with Microsoft Bing Maps. To obtain these items, access the [Bing Maps Dev Center](#). After you create an application key, the application name is arbitrary and there is no need for an application URL. The free version of Bing Maps will not work.
- The AzureMapsPlugin Gosu class. This plugin implementation connects to the Microsoft Azure Maps Geocode Service. If you intend to use the AzureMapsPlugin implementation, your organization must have a valid account with Microsoft. For more information, see [The Azure Maps map control](#).

Working with Geocode batch processing

Guidewire implements geocode batch processing as a work queue. The Geocode writer in the InsuranceSuite application and the ABGeocode writer in ContactManager use BatchGeocode and other criteria to filter which addresses to pass to the plugin for geocoding.

- ContactManager verifies that the BatchGeocode property is true and the GeocodeStatus property is none.
- ClaimCenter verifies that the BatchGeocode property is true and the contact for the address has a subtype of UserContact.
- PolicyCenter verifies the following:
 - The address is an AccountLocation or subtype.
 - The address is a primary address of a UserContact.
 - The BatchGeocode property is true and the GeocodeStatus property is none.

The geocoding process submits qualifying addresses to your implementation of the GeocodePlugin plugin. After the GeocodePlugin plugin adds geocode coordinates to an address, the geocoding process updates the address in the database.

Geocode status typelist

The GeocodeStatus typelist defines the set of status codes returned from the default GeocodePlugin plugin implementation. As the typelist is final, you cannot edit it. You access the GeocodeStatus typelist in Guidewire Studio in the following location:

configuration > config > Metadata > Typelist

Configuring geocoding

Configuring geocoding in ClaimCenter and PolicyCenter involves the following tasks:

- Enabling your implementation of the GeocodePlugin plugin
- Setting geocoding feature parameters
- Scheduling the Geocode work queue
- Configuring the number of Geocode batch processing workers

If you configure geocoding for ClaimCenter and you integrate ContactManager with ClaimCenter, you can also configure geocoding for ContactManager. For more information, see *Contact Management Guide*.

Enabling the Geocode plugin

By default, the base configuration GeocodePlugin plugin implementation uses the Azure Maps implementation. Guidewire disables this plugin in the base configuration. You must enable the geocode plugin before you can use the geocoding functionality. If you intend to use geocoding in multiple Guidewire applications, you must make these changes in each application separately.

Scheduling the Geocode plugin

By default, Guidewire does not schedule geocode batch processing in the base configuration. To schedule geocode batch processing, you need to uncomment the relevant section in file `scheduler-config.xml`. Guidewire recommends that you schedule the geocoding process to run during periods of minimal activity on the PolicyCenter or ClaimCenter servers.

IMPORTANT: Schedule geocode batch processing for PolicyCenter, ClaimCenter, and ContactManager with sufficient processing windows between runs to assure sufficient time for runs to fully process the work items in the work queues. If you find duplicate work items in the work queues for the same address ID, extend the interval between runs.

Configuring the number of geocoding batch processing workers

At the time you first start an InsuranceSuite application, it is possible for your database to have many addresses to geocode, especially if you imported many new addresses into your production database. A large number of new addresses can cause the GeocodePlugin plugin implementation to take a long time to process these new addresses. Guidewire recommends that you configure the geocoding process with a sufficient number of worker instances before you start your production servers.

The default configuration specifies one worker instance. Worker instances pass addresses from the work queue to the GeocodePlugin plugin implementation. Consider increasing the number of worker instances to improve throughput. To further improve throughput, assign worker instances to run on multiple servers.

About Guidewire geocoding parameters

Use the following parameters in file `config.xml` to configure the geocoding features in PolicyCenter and ClaimCenter.

ClaimCenter geocoding parameters

Parameter	Description
UseGeocodingInPrimaryApp	Set to <code>true</code> to enable geocoding in the ClaimCenter user interface, such as in assignment or user search screens. The default is <code>false</code> .
UseGeocodingInAddressBook	Set to <code>true</code> if you have InsuranceSuite application integrated with ContactManager and ContactManager has geocoding enabled for vendors. This setting enables vendor search in the ClaimCenter and ContactManager user interfaces. The default is <code>false</code> .
UseMetricDistancesByDefault	If <code>true</code> , ClaimCenter uses kilometers and metric distances instead of miles and United States distances for driving distance or directions. Set this parameter identically in Guidewire applications that use geocoding. The default is <code>false</code> .
ProximitySearchOrdinalMaxDistance	A distance that provides an approximate bound to improve performance of an ordinal (nearest <i>n</i>) proximity search. This distance is in miles, unless you set <code>UseMetricDistancesByDefault</code> to <code>true</code> . The search can return results that are farther away than the distance specified. Set this parameter identically in Guidewire applications that use geocoding. This parameter has no effect on <i>radius</i> (within <i>n</i> miles or kilometers) proximity searches or walking-the-group-tree-based proximity assignment. The default is 300.
ProximityRadiusSearchDefaultMaxResultCount	The maximum number of results to return if performing a radius (within <i>n</i> miles or kilometers) proximity search. This parameter has no effect on ordinal (nearest <i>n</i> items) proximity searches. This parameter does not have to match the value of the corresponding parameter in other Guidewire applications.

Parameter	Description
	The default is 1000.

PolicyCenter geocoding parameters

Parameter	Description
UseGeocodingInPrimaryApp	If true, PolicyCenter enables searching for nearby locations in the Reinsurance Management user interface. ContactManager does not respond to this parameter. The default is false.
UseMetricDistancesByDefault	If true, PolicyCenter uses kilometers and metric distances instead of miles and United States distances for location searches. Set this parameter identically in Guidewire applications that use geocoding. The default is false.
ProximitySearchOrdinalMaxDistance	The maximum distance to use while performing an ordinal (nearest <i>n</i> items) proximity search for locations. This distance is in miles, unless UseMetricDistancesByDefault is true. The default is 300.
ProximityRadiusSearchDefaultMaxResultCount	The maximum number of results to return if performing a radius (within <i>n</i> miles or kilometers) proximity search. This parameter has no effect on ordinal (nearest <i>n</i> items) proximity searches. The default is 1000.

Enable the Geocode plugin (Azure Maps)

About this task

These steps enable the Geocode plugin (Azure Maps) for PolicyCenter and ClaimCenter. For information about configuring geocoding for ContactManager, see *Contact Management Guide*.

Procedure

1. In Guidewire Studio, navigate to the following location in the **Project** window:
configuration > config > Plugins > registry
2. Double-click GeocodePlugin to open it.
3. If the **Disabled** check box is checked, un-select the check box to enable the plugin.
4. Ensure that the **Class** field specifies the Azure Maps implementation class:
`gw.plugin.geocode.impl.AzureMapsPlugin`
5. Under **Parameters**, specify the following:

applicationKey	The application key that you obtained from Azure Maps.
geocodeDirectionsCulture	The locale for geocoded addresses and routing instructions returned from Azure Maps. For example, use the locale code <code>ja-JP</code> for addresses and instructions for Japan. The plugin uses <code>en-US</code> if you do not specify a value.
imageryCulture	The language for map imagery. For example, use the language code <code>ja</code> for maps labeled in Japanese. The plugin uses <code>en</code> if you do not specify a value.

mapUrlHeight	Height of maps, in pixels. The plugin uses 500 if you do not specify a value.
mapUrlWidth	Width of maps, in pixels. The plugin uses 500 if you do not specify a value.

6. Save your changes.

Enable the Geocode plugin (Bing Maps)

About this task

These steps enable the Geocode plugin (Bing Maps) for PolicyCenter and ClaimCenter. For information about configuring geocoding for ContactManager, see *Contact Management Guide*.

Procedure

1. In Guidewire Studio, navigate to the following location in the **Project** window:
configuration > config > Plugins > registry
2. Double-click GeocodePlugin to open it.
3. If the **Disabled** check box is checked, un-select the check box to enable the plugin.
4. Ensure that the **Class** field specifies the Bing Maps implementation class:
`gw.plugin.geocode.impl.BingMapsPluginRest`
5. Under **Parameters**, specify the following:

applicationKey	The application key that you obtained from Bing Maps.
geocodeDirectionsCulture	The locale for geocoded addresses and routing instructions returned from Bing Maps. For example, use the locale code <code>ja-JP</code> for addresses and instructions for Japan. The plugin uses <code>en-US</code> if you do not specify a value. For a current list of codes that Bing Maps supports, refer to the following web site: http://msdn.microsoft.com/en-us/library/cc981048.aspx
imageryCulture	The language for map imagery. For example, use the language code <code>ja</code> for maps labeled in Japanese. The plugin uses <code>en</code> if you do not specify a value. For a current list of codes that Bing Maps supports, refer to the following web site: http://msdn.microsoft.com/en-us/library/cc981048.aspx
mapUrlHeight	Height of maps, in pixels. The plugin uses 500 if you do not specify a value.
mapUrlWidth	Width of maps, in pixels. The plugin uses 500 if you do not specify a value.

6. Save your changes.

Schedule Geocode batch processing runs

About this task

By default, Guidewire schedules geocode batch processing to run at 1:30 AM daily. If you regularly add many new contacts, especially in ContactManager, tune the schedule to match your expected daily load of new addresses.

These steps apply to PolicyCenter and ClaimCenter. For information about configuring geocoding for ContactManager, see *Contact Management Guide*.

Procedure

1. In Guidewire Studio, navigate to **configuration > config > scheduler** and open `scheduler-config.xml` for editing.

2. Uncomment the following section in `scheduler-config.xml` file.

```
<ProcessSchedule process="Geocode">
  <CronSchedule hours="1" minutes="30"/>
</ProcessSchedule>
```

Configure the number of worker instances for Geocode batch processing

About this task

These steps apply to PolicyCenter and ClaimCenter.

Procedure

1. In Guidewire Studio, navigate to **configuration > config > workqueue** and open `work-queue.xml` for editing.
2. Modify the number of worker instances in the following section in file `work-queue.xml` to meet your business needs.

ClaimCenter example:

```
<work-queue workQueueClass="com.guidewire.pl.domain.geodata.geocode.GeocodeWorkQueue"
  progressinterval="600000">
  <worker instances="1"/>
</work-queue>
```

PolicyCenter example:

```
<work-queue workQueueClass="com.guidewire.pc.domain.geodata.geocode.PCGeocodeWorkQueue"
  progressinterval="600000">
  <worker instances="1"/>
</work-queue>
```

3. Update database statistics by running the Database Statistics batch process.

Catastrophe search and heat maps

Before you can see the **Catastrophe Search** screen in ClaimCenter, and view, edit, and administer catastrophes (**Administration > Business Settings > Catastrophes**) using its catastrophe heat map, you must enable the use of heat maps in Guidewire ClaimCenter.

Catastrophe sample data

If working in a development environment, you can use a command in the **QuickJump** box in ClaimCenter to load catastrophe sample data. This includes catastrophes and a set of homeowners policies and claims in a specific geographic area associated with these catastrophes that appear on heat maps. Use this sample data to view information in the Catastrophe Search page and heat maps before you load any actual production claims and catastrophe data.

Run the following **QuickJump** command to load this sample data:

```
Run NewClaim catHOPClaims
```

See also

- *Configuration Guide*

Enable the catastrophe heat map

Before you begin

In the base ClaimCenter application, Guidewire supports two map provider services for catastrophe heat maps:

- Microsoft Azure Maps
- Microsoft Bing Maps

The use of catastrophe heat maps with Bing Maps requires a license from Microsoft Corporation for the Bing Maps Ajax Control. This license provides the necessary credentials that you need to complete the following procedure.

Note: For information on using and configuring a geocoding service other than Microsoft Bing Maps or Azure Maps, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Procedure

1. Decide which geocoding service you want to use, then configure a geocoding plugin implementation to work with that service.

If you use the Microsoft Bing Maps or Azure Maps service, you can use the default geocoding plugin interface supplied with InsuranceSuite application.

2. Enable your implementation of the geocode plugin.
3. In Guidewire Studio **Project** window, expand **configuration > config**:
 - a) Double-click `config.xml` to open it.
 - b) Locate the following parameters and set them according to the maps provider service you are using:

Azure Maps:

```
<param name="HeatMapServiceTemplate" value="gw.api.heatmap.AzureMap"/>
<param name="HeatMapCredential" value="Credential"/>
```

Set *Credential* to the value that you obtained during the licensing process for Azure Maps.

Bing Maps:

```
<param name="HeatMapServiceTemplate" value="gw.api.heatmap.BingMap2"/>
<param name="HeatMapCredential" value="Credential"/>
```

Set *Credential* to the value that you obtained during the licensing process for the Bing Maps Ajax Control.

4. Save your changes.

What to do next

You must restart the application server before this change goes into effect.

Enable the catastrophe policy location download batch process

About this task

If your installation of ClaimCenter integrates with PolicyCenter, enable batch process `CatastrophePolicyLocationDownload`. This batch process retrieves policy locations for a specific geographic area of interest from the policy system. The catastrophe heat map shows retrieved policies as policy locations if PolicyCenter stores the addresses with geocode information.

Procedure

1. In Guidewire Studio, navigate to the following location in the **Project** window:
configuration > config > scheduler
2. Double-click scheduler-config.xml to open it.
3. Remove any comments around the CatastrophePolicyLocationDownload configuration parameter. The following default entry schedules the batch process to run every day at 2 a.m.

```
<ProcessSchedule process="CatastrophePolicyLocationDownload">  
  <CronSchedule hours="2"/>  
</ProcessSchedule>
```

4. Set the schedule for the batch process to met your business requirements.

What to do next

You must restart the application server before this change becomes effective. And, optionally, you can load the catastrophe sample data supplied with ClaimCenter. This data enables you to see the Catastrophe Search page and heat maps before you load any production claims and catastrophe data.

External server configuration

External server configuration provides the means to modify designated server properties at runtime without the necessity to rebuild and redeploy a server WAR or EAR file. Using external server configuration, you can do the following:

- Modify application log levels dynamically.
- Modify configuration options such as database connection information, URLs to external services, and similar items.
- Enable or disable specific feature options at runtime, in specific environments, without starting the application server.
- Securely store sensitive information (secrets) used by Guidewire applications, such as system and service passwords, private keys, and similar items.

Guidewire implements external server configuration in plugin `ExternalConfigurationProviderPlugin`. The implementation class for the `ExternalConfigurationProviderPlugin` plugin provides several different mechanisms for accessing this feature:

- File-based external configuration
- Service-based external configuration

You can enable both types of external configurations at the same time.

IMPORTANT: Guidewire designs the `ExternalConfigurationProviderPlugin` plugin implementation to work with both Guidewire Settings Vault and with Guidewire authentication services. Do not try to extend or modify the default implementation of plugin `ExternalConfigurationProviderPlugin`.

[See also](#)

- **Environment variables** in the Cloud Platform documentation.

About external server configuration

External server configuration contains the following core elements:

- Placeholders in configuration files.
- A plugin that determines the substitution value for placeholder replacement using, in part, a key embedded in the placeholder.
- A Substitutor process that manages the steps in the substitution process.

Configuration placeholders

It is possible to insert placeholders into configuration files that support external configuration. The InsuranceSuite application reads the substitution values from a source external to the application WAR or EAR file and replaces the entire placeholder in the configuration file with the new value.

External configuration provider plugin

Plugin `ExternalConfigurationProviderPlugin` determines the substitution value of the configuration placeholder. Given information about the placeholder, the plugin returns the substitution value for the placeholder. In the base configuration, the InsuranceSuite application provides a default implementation class for this plugin. However, it is also possible to create your own custom version of this plugin.

Substitutor process

An InsuranceSuite application internal Substitutor process manages the actual substitution process for external server configuration. The Substitutor process performs the following tasks:

- It parses the configuration files that support externalized configuration and extracts the embedded placeholders from those files.
- It extracts a key from each placeholder that it uses to identify each placeholder.
- It queries the external configuration provider plugin for the substitution value for each placeholder.
- It replaces the placeholder in the configuration file (in its entirety) with the value returned by the plugin.

It is not possible to create your own custom Substitutor process.

Substitution placeholders

It is possible to use system variables to start the Guidewire application server with specific server properties. Each provided system variable is a placeholder for the actual value that exists in a configuration file stored in a location that is external to the Guidewire application installation. Each substitution placeholder called by a system variable must exist in a specific configuration file and the placeholder text must conform to the following syntax and guidelines.

About placeholders

Guidewire uses the following syntax for a placeholder in a configuration file:

```
${key:defaultValue}
```

In this string:

- `key` is an identifier that the Substitutor uses, in part, to query the `ExternalConfigurationProviderPlugin` plugin for the substitution value that replaces the placeholder.

- *defaultValue* is an optional default value to use for substitution if the query of the `ExternalConfigurationProviderPlugin` plugin does not return a value.

Separate the key and its default value with a colon. The initial curly brace { must follow the initial \$ character directly, meaning that there cannot be a space between the \$ and the { brace.

As InsuranceSuite application encounters a placeholder in a configuration file:

- If the `lookupValue` method returns a value, InsuranceSuite application replaces the entire placeholder string with the returned value.
- If the `lookupValue` method does not return a value InsuranceSuite application uses the provided default value in the placeholder, if the default value exists.

Placeholder syntax

A placeholder must contain the following types only:

- Letters
- Digits
- Dots
- Underscores

As you construct a key, observe the following rules:

- A key cannot contain spaces.
- A key cannot start with a digit.
- A key cannot be an empty string (`${ }`).
- A key cannot contain special characters, such as \$.

Also keep in mind:

- A default value is optional.
- A placeholder in an XML configuration file cannot span multiple nodes.
- A placeholder in a Java property file cannot span multiple properties.

Placeholder examples

The following are all examples of valid placeholders.

`${key}`

The placeholder contains a key value only.

`${key:}`

The placeholder contains an empty default value.

`${key:defaultValue}`

The placeholder contains both a key value and a default value.

`${key:onlyFirstColon:counts:all:further:colons:are:part:of:default:value}`

The placeholder contains a default value with multiple colons. The first colon indicates the beginning of the default string. Subsequent colons are part of the default value. For example, this can be a database URL in file `database-config.xml`.

`${keyW1thAD1g1t}`

The placeholder contains a key that contains a digit.

Understanding namespace

Each group of files that Guidewire supports for external configuration has a unique namespace. This namespace is a hard-coded string that uniquely identifies a given file group. A file group can consist of a single member, `config.xml`, for example. Or, a file group can contain multiple members, the set of all `*.gwp` files, for example. The Substitutor process infers the file group namespace from the file that it is parsing.

The Substitutor uses the namespace (along with the placeholder key) as a composite argument as it queries the `ExternalConfigurationProviderPlugin` plugin for the substitution value for the placeholder. Because each key is unique, it is possible to reuse a key in different file groups. For example, one can use the same URL key in file `database-config.xml` and in file `myPlugin.gwp` as the namespace for each is unique.

Files that support configuration substitution

Static reload

The following categories of the InsuranceSuite application configuration files all support configuration substitution at server start. This is known as a *static reload* as it can only occur at application server start and only once. To load a different substitution value, you must restart the application server again.

Configuration file group	Namespace
<code>config.xml</code>	<code>config</code> ¹
<code>database-config.xml</code>	<code>database</code>
<code>inbound-integration-config.xml</code>	<code>inbound-integration</code>
<code>ISO.properties</code>	<code>iso</code> (ClaimCenter only)
<code>messaging-config.xml</code>	<code>messaging</code>
<code>Metro.properties</code>	<code>metro</code> (ClaimCenter only)
<code>published-apis.yaml</code>	<code>published-apis</code>
<code>scheduler-config.xml</code>	<code>scheduler</code>
<code>suite-config.xml</code>	<code>suite</code>
<code>work-queue.xml</code>	<code>work-queue</code>
<code>*.gwp</code> ²	<code>plugin</code>

Dynamic reload

The following files also support the reloading of very specific configuration values while the application server is running. This is known as *dynamic reload* (or hot reload) as it can occur at any time while the application server is running, for as many times as you like.

File	Valid substitution value	More information
<code>config.xml</code>	value field on specific <code><parameter></code> elements	"Dynamic configuration parameters" on page 156
<code>suite-config.xml</code>	url field on <code><product></code> elements	"Dynamic product URLs" on page 158

Processes that support configuration substitution

Variable substitution touches every InsuranceSuite application process that involves parsing configuration files that support externalization. It is possible to divide these processes into the following areas:

- ¹ Except for the registry node in this file
- ² Plugin registry definition files

- Stand-alone application server
- Clustered application servers
- System tools
- Build tasks

Stand-alone InsuranceSuite application servers

If you enable external server configuration on a stand-alone InsuranceSuite application server, the InsuranceSuite application performs a series of steps in working with placeholders:

1. Initially, at server start, the InsuranceSuite application reads the affected configuration files, once, during one of the various phases of server start-up.
2. The InsuranceSuite application Substitutor process then performs an in-place substitution step on all affected configuration files.
3. Finally, the InsuranceSuite application parses the contents of the modified configuration files and stores the results in memory.

Clustered InsuranceSuite application servers

During the start of the first server node in the InsuranceSuite application server cluster, InsuranceSuite application creates a digest of all the configuration files and their parameters that are present on the start-up node. InsuranceSuite application then stores this configuration information in the database. This information becomes the official configuration information for the entire cluster.

In the usual course of events, as each subsequent InsuranceSuite application node starts, InsuranceSuite application performs a configuration verification for that node. If the new configuration is non-compatible with the existing cluster configuration, InsuranceSuite application requires a database upgrade.

It is possible to have two seemingly identical configuration files that differ in their effect by including or excluding parts of each configuration through the use of `env` or `serverid` settings. InsuranceSuite application creates different digests from these configuration files. Be aware that InsuranceSuite application considers these digests to be different from each other and potentially non-compatible.

InsuranceSuite application system tools

After you run the `system_tool -verifyconfig` command against a running InsuranceSuite application server, the command uses the `remote ExternalConfigurationProviderPlugin` plugin implementation to determine the substitution values for the local configuration files, if any. The command then compares the digest extracted from the local WAR/EAR file configuration files with the digest of the remote cluster configuration. Finally, the command prints the result of this comparison to the command prompt in which you ran the `system_tool` command.

InsuranceSuite application server gwb tasks

The following gwb build tasks all parse the configuration files that support substitution:

- `dropDb`
- `runServer`
- `diffDisplayKeys`
- `exportLocalizations`
- `importLocalizations`
- `exportWsd1`
- `genFromWsc`

- `genExternalSchemas`
- `genWsiLocal`
- `genDataDictionary`
- `genEntityModelXml`
- `genDataMapping`
- `genImportAdminDataXsd`
- `genPcfMapping`
- `verifyExtConfig`
- `verifyResources`

Build command `verifyExtConfig` checks that the `ExternalConfigurationProviderPlugin` plugin provides all necessary values for configuration substitution in the affected files. The command output reports any issues that it finds.

Dynamic configuration parameters

Guidewire provides the ability to reload the value of an external configuration parameter on a running application server through Guidewire Settings Vault. Hot reloading of external properties provides for the following:

- Substitution changes that you make using this method persist between application upgrades and server shutdowns.
- Substitution changes that you make propagate to all servers in the cluster at roughly the same time, meaning the change is nearly simultaneous in all cluster members.

Thus, it is possible to edit any dynamic configuration parameter that uses the 'config' namespace and have that change picked up and broadcast simultaneously to all running members of a Guidewire application cluster.

Server run level

The application server must be at, or below, the `MULTIUSER` run level for dynamic (hot-reloading) of substitution parameters to work. This means that if you start the application server at a lower run level, the application server does not perform hot reloading of substitution parameters.

If you want to test dynamic substitution in a development or test environment, while testing at a lower run level, do the following:

1. Start the application server using the following command to set the server run level, with `x` being at, or below, the `MULTIUSER` run level:

```
gwb runServer -Dserver.runLevel=x
```
2. Navigate to **Startable Services** in the InsuranceSuite application Server Tools.
3. Manually start the `ExternalConfigurationProviderPlugin` plugin.

Substitution value restrictions

The parameter to change must fulfill the following conditions:

- The parameter is defined in file `config.xml`.
- The parameter is defined as a dynamic in the application code that defines that parameter.

IMPORTANT: You can only perform dynamic reloading of an external parameter that is specifically defined as dynamic in the parameter metadata.

You must use the 'config' namespace in your external configuration file. The only parameter value that you can change using this method is the parameter 'value' field.

Change monitoring

Guidewire Configuration Service monitors the values of external properties that are changed, added, or deleted by Guidewire Settings Vault. Upon discovery of a configuration change in the 'config' namespace, the Configuration Service reloads those values to the running application server.

Substitution examples

The parameter definitions in the example config.xml file use the properties defined in the following external configuration file.

```
config.myExtProp1=true
config.foo=hello
config.bar=world
```

The following config.xml examples contain both valid and invalid entries for dynamic parameters. In this case, invalid means that the parameter definition can cause a server to not be able to start.

```
<!-- Example config.xml -->
<!-- ++++++VALID EXAMPLES+++++ -->
<!-- Valid; Substitution file also specifies a default value of true. -->
<param name="MyBooleanParam" value="${myExtProp1:false}"/>
<!-- Valid use of two external properties;
-- parameter definition also specifies a default value of "goodbye-planet". -->
<param name="MyStringParam" value="${foo:goodbye}-${bar:planet}"/>
<!-- Valid: Property myExtProp2 was not given a value in the external configuration file,
-- its value will revert to the default value of "hello" specified in this file.-->
<param name="MyParamHasExtPropDefault" value="${myExtProp2:hello}"/>
<!-- Valid: Property myExtProp2 was not given a value in the external configuration file,
-- so it will revert to its Guidewire-specified default value. -->
<param name="MyParamHasDefault" value="${myExtProp2}"/>
<!-- Valid: External property {baz} was not given a value, but it will revert
-- to the customer-specified default.
-- So, the value of MyTwoPropParamHasExtPropDefault will be "hello-planet". -->
<param name="MyTwoPropParamHasExtPropDefault" value="${foo}-${baz:planet}"/>
<!-- Valid: External property {baz} was not given a value, so it will revert to null
-- as it has no Guidewire-specified default. -->
<param name="MyParamNoDefaultButNotRequired" value="${foo}-${baz}"/>
<!-- Valid: Despite the fact that {baz} is given a default substitution, {qux} is not.
-- Therefore the value of this parameter will not be "hello". Instead it will be null
-- or the Guidewire-specified default.-->
<param name="MyParamHasOneExtPropDefault" value="${baz:hello}${qux}"/>
<!-- ++++++INVALID EXAMPLES+++++ -->
<!-- Invalid: Property myExtProp1 is given a non-integer value,
-- which does not revert to the substitution default of 1234!.
-- So, MyIntegerParam3 will not be able to parse its value.
-- Therefore the server will be unable to start -->
<param name="MyIntegerParam" value="${myExtProp1:1234}"/>
<!-- Invalid: External property {baz} was not given a value, and there is no default.
-- Since this parameter is required, the server will be unable to start -->
<param name="MyParamNoDefaultRequired" value="${foo}-${baz}"/>
```

Best practices

Guidewire recommends that you incorporate the following best practices if configuring your server to work with the hot reloading of configuration parameters.

Carefully monitor all substitution changes

Carefully monitor the server environment during and after a hot reload of a configuration parameter value. This action ensures that you are able to recognize and respond to any mistakes quickly. In particular, be aware that a bad substitution value can cause an invalid property value, which can cause any new servers that you attempt to add to the cluster to not be able to start.

Always specify a default substitution value

Guidewire recommends that you always specify a default substitution value for all external property usage, for example:

```
$$${myExtProp:myDefault}
```

This action prevents issues on server start up if the hot reload mistakenly removes an external property entry. Moreover, it ensures that you have full control over your parameter values. If you do not specify a default value, then removing an external property will cause the parameter to reset to Guidewire-specified defaults. The resulting server behavior may not be what you intend or desire. Thus, always specify your own default values for substitution values.

See also

- *Configuration Guide* – Information on configuration parameters marked as dynamic in file `config.xml`.

Dynamic product URLs

In InsuranceSuite application servers, almost all placeholder substitutions are static, meaning that the substitution of a configuration value occurs once, as the server starts, and that configuration value remains the same until a restart of the server. The single exception is the use of special syntax that permits a change to the `url` attribute value on `<product>` elements in certain configuration files while the server is running. The provided URL must be a InsuranceSuite application server URL.

IMPORTANT: Guidewire permits the use of the dynamic substitution syntax in file `suite-config.xml` only.

Dynamic substitution syntax

You can only use dynamic substitution for `<product>` elements that take the following form:

```
<product name="..." url="..." />
```

Use the following syntax to indicate a `url` attribute value that supports dynamically changing values:

```
$$${...}
```

The following code example illustrates how to create a dynamic URL.

```
<suite-config xmlns="http://guidewire.com/config/suite/suite-config">
  <product name="bc" url="$$${bc.url}"/>
</suite-config>
```

In this example, the `url` attribute defines `bc.url` as the placeholder value. The `bc.url` placeholder maps to the value for `suite.bc.url` in an external configuration file.

The placeholder substitution value of `bc.url` is an example only. You can define other URL names as well, for example:

```
<product name="bc" url="$$${MyBillingCenterURI}"/>
```

where `xx` is the application product code.

In this example, Guidewire Settings Vault maps `MyBillingCenterURI` to the value for `suite.MyBillingCenterURI` contained in an external configuration file.

Dynamic substitution errors

Do not attempt to use the dynamic substitution syntax in files or elements that do not support that type of substitution. For example, suppose you attempt to use the following code in a file that does not support dynamic substitution.

```
<param name="SomeConfigParam" value="$$${some.param.value}"/>
```

In this case, Guidewire Settings Vault does not make a substitution replacement and the value of `SomeConfigParam` becomes `$$${some.param.value}`, the placeholder text itself.

Reloading dynamic values

You set dynamic substitution values in the exact same manner as you do static substitution values. This means that you create an external configuration file and add substitution values to the file using the correct syntax. You then point the server to the substitution file as you start the server. Thereafter, the server periodically polls Settings Vault server for any updated dynamic placeholder values. If the polling request returns a new value for a dynamic placeholder, the server uses that value for the placeholder until a new polling request returns a different value.

Additional configuration file replacements

It is also possible to substitute values, in whole or in part, in the following configuration files.

Type	File	Affects
Free-text search engine configuration files	solrserver-config.xml	Placeholder (PolicyCenter and ClaimCenter only)
Interface extension	ExtensionInterfacemapper.gs	Enables additional interfaces not listed in file
Logging property file	log4j2-json.xml	Entire file
Web service collection files	*.wsc	Override URL

Free-text search engine configuration files

In ClaimCenter and PolicyCenter, the free-text search engine batchload.bat (batchload.sh) script launches a Java program that uses file solr-config.xml to identify where and how the free-text search engine is running.

It is possible to use configuration placeholders in file solr-config.xml in a similar manner to other Guidewire configuration files. This means that you can insert configuration placeholders into file solr-config.xml and use a separate configuration file to replace the original placeholder. The substitution file contains only single-line entries of the following type:

```
solr.key=placeholderValue
```

Set the path to the substitution file using the gw.config.external.property.file system property as you start the free-text search engine. Upon starting, the free-text search engine loads and uses the contents of the given file for substitution in file solrserver-config.xml. The free-text search engine logs the substitution activity in the free-text search engine log.

Logging property files

Guidewire supports the substitution of the entire logging configuration file log4j2-json.xml by using a system property to specify the full path to an alternate log4j2-json.xml file. To make this change, add the following system property to the server start command:

```
gw.config.external.logging.file
```

Example of logging property file replacement

The following example replaces the log4j2-json.xml file packaged with the installation WAR/EAR file with a logging file that exists on a local machine:

```
gwb runServer -Dgw.config.external.logging.file="C:\tmp\loggingfile\MyLoggingFile.xml"
```

Web service collection files

You can use a web services collection file (.wsc) to customize the SOAP web service client configuration. You can create and edit the file in Studio or manually. Guidewire recommends that you create WSC files under wsi.remote. For more information on using the web service editor in Studio see, the *Configuration Guide*.

Each WSC file supports the following setting types that you can configure for a given environment, server, WSDL service, and port:

Configuration Provider

You can create multiple custom implementations of interface `IWsiWebserviceConfigurationProvider`. Each custom implementation binds to web service collection files (.wsc files) through the `<configuration-provider>` node.

You can also set or override specific values of the `Wsd1Config` object, such as http headers, authentication information, and `ServerOverrideUrl`. If you think that the parameter might be set externally before the call, check if `config.Default` is true.

Override URL

You can override the URL for local testing. The applications defined in `suite-config.xml` can be used to support substitution. You can override a URL with a configuration provider as well.

Note: Although there are web service collection files for one application in the configuration for another application, you must override the URL for these services in the suite configuration file, rather than the WSC file. See "Guidewire suite configuration file" in *Plugins, Prebuilt Integrations, and SOAP APIs*.

Example: Override WSDL URL

To override URLs, use the following items:

- Public interface `IWsiWebserviceConfigurationProvider`
- Public API `gw.external.configuration.SubstitutionProperties`
- Property `ServerOverrideUrl` on the `Wsd1Config` object.

You can use API method `SubstitutionProperties.lookupValue` to retrieve the override URL then set the `Wsd1Config.ServerOverrideUrl` property to this new value.

In the following example, `testClient.wsc` provides one WSDL URL for the production environment and a separate WSDL URL for use in all other environments. It performs the following:

- The `<configuration-provider>` node binds class `TestConfigurationProvider` to this WSC file. Class `TestConfigurationProvider` manages the URL substitution process.
- The code provides for two override URLs. One URL is active in the production (cloud-dev) environment. The other URL is active in all other environments.

```
<webservice-collection ... >
  ...
  <settings>

    <configuration-provider>
    <type-name>gw.util.TestConfigurationProvider</type-name>
    </configuration-provider>

    <override-url>
    <env>cloud-dev/env>
    <url>http://someUrl</url>
    </override-url>

    <override-url>
    <url>url_to_be_rewritten</url>
    </override-url>

  </settings>
  ...
</webservice-collection>
```

The following code provides an implementation of class `gw.util.TestConfigurationProvider` and it performs the following:

- The `ConfigurationProvider` class implements the `IWsiWebserviceConfigurationProvider` interface.
- The class uses the `lookupValue` method on the `SubstitutionProperties` API to retrieve the URL override value.
- The `lookupValue` method parameter `ServiceName` is a placeholder for an actual web service name.

- The `ServerOverrideUrl` statement writes the new URL to the non-production `<overrideUrl>` node in file `testClient.wsc`.

```
package gw.util

uses gw.external.configuration.SubstitutionProperties
uses gw.xml.ws.IWsiWebserviceConfigurationProvider
uses gw.xml.ws.WsdlConfig
uses javax.xml.namespace.QName

class TestConfigurationProvider implements IWsiWebserviceConfigurationProvider {
  override function configure(serviceName: QName, portName: QName, config: WsdlConfig) {
    If (config.Default) {
      //Access the plugin through the public API
      var newValue = new SubstitutionProperties().lookupValue("ServiceName", "urlForTestWsc")
      config.ServerOverrideUrl = newValue
    }
  }
}
```

Example: Use token authentication

You can configure multiple implementations of `IWsiWebserviceConfigurationProvider`. Guidewire recommends OAuth bearer token authentication in production (cloud-dev) environments.

In the following example, you can use one authentication method for local environments and another for the cloud-dev environment.

```
<webservice-collection ... >
...
<settings>
<configuration-provider>
<type-name>wsi.remote.gw.webservice.ab.ABBasicConfigurationProvider</type-name>
</configuration-provider>

<configuration-provider>
<env>cloud-dev</env>
<type-name>gw.util.gw.wsi.pl.OauthTokenConfigurationProvider.</type-name>
</configuration-provider>
</settings>
...
</webservice-collection>
```

local environments

For local environments, replace the current implementation of `IWsiWebserviceConfigurationProvider` with one that uses the `CredentialsPlugin` to extract the user name and password.

The following code shows an implementation of class `ABBasicConfigurationProvider`:

```
package wsi.remote.gw.webservice.ab
class ABBasicConfigurationProvider implements IWsiWebserviceConfigurationProvider {
  override function configure(serviceName : QName, portName : QName, config : WsdlConfig ) {
    var plugin = Plugins.get(CredentialsPlugin)
    var credentials = plugin.retrieveUsernameAndPassword("guidewire.suiteinteg.auth.ab")
    If (credentials != null) {
      config.Guidewire.Authentication.Username = credentials.Username
      config.Guidewire.Authentication.Password = credentials.Password
    } else if (ServerUtil.LowerPlanet) {
      config.Guidewire.Authentication.Username = "gw"
      config.Guidewire.Authentication.Password = "su"
    }
  }
}
```

cloud-dev environments

For cloud-dev environments, if the web service uses the same OAuth token as this application's registry, `gw.wsi.pl.OauthTokenConfigurationProvider` provides the necessary functionality to authenticate.

If the web service is configured to use a different token source, extend `OauthTokenConfigurationProvider` and provide the product code associated with this application. `OauthTokenConfigurationProvider` extracts the client secret, client id, and authorization url and scope from the secret and variables that you set in Guidewire Home.

```
package wsi.remote
class ABAuthConfigProvider extends OauthTokenConfigurationProvider {
  construct() {
    super("AB")
  }
}
```

```
}
}
```

In this example, AB is the product code. Include this prefix when you set up Okta variables and secrets. For example: AB__OKTA_SCOPE, AB__OKTA_CLIENT_ID, AB__OKTA_AUTH_SERVER_URL, and AB__OKTA_CLIENT_SECRET.

Example: Set the transaction ID

The following example code sets the transaction ID in the SOAP header:

```
package wsi.remote
class TransactionIdConfigProvider implements IWSIWebserviceConfigurationProvider {
    override function configure(serviceName : QName, portName : QName, config : WsdlConfig ) {
        var transId = new gw.xsd.guidewire.soapheaders.TransactionId()
        transId.Value = UUID.randomUUID().toString()
        config.RequestSoapHeaders.add(transId)
    }
}
```

Example: Set trace headers

The following example code sets the trace header:

```
package wsi.remote
uses gw.logging.TraceabilityIDConstants
uses gw.xml.ws.WsdlConfig
uses org.slf4j.MDC
override function configure(serviceName : QName, portName : QName, config : WsdlConfig ) {
    var traceId = MDC.get(TraceabilityIDConstants.TRACEABILITY_ID_KEY)
    if (traceId != null) {
        config.Http.RequestHeaders.setHeader(TraceabilityIDConstants.TRACEABILITY_ID_HEADER, traceId)
    }
}
```

Logging substitution activity

The InsuranceSuite application outputs all substitution-related messages to the application log using the `Server.PropertySubstitutor` logger. The logging output also includes information on the initial loading of the `ExternalConfigurationProviderPlugin` plugin. The logging messages are verbose and sufficient to track all steps and problems of any substitution operation.

Verifying external configuration

Guidewire provides a `gwb` build command that you can use to verify your externalized server configuration. Use the following syntax at a command prompt to execute the command:

```
gwb verifyExtConfig
```

This command generates a report that includes the following types of information:

- The number of files affected
- The number of substitutions involved
- The status of plugin `ExternalConfigurationProviderPlugin` if disabled

Example

Running the `verifyExtConfig` command generates a report similar to the following text, if there are no issues with the externalized server configuration:

```
Verification results: 14 key substitutions would occur in 3 files.
```

Path to the substitution file

Notice in the `gwb verifyExtConfig` command that you do not specify a path to the substitution file. Instead, you set the path to the external substitution file as you start the application server, for example:

```
gwb runServer -Dgw.config.external.property.file="C:cc10_1853\modules\configuration\etc\ext-properties.properties"
```

The gwb build commands

For information on the gwb build commands, see the *Developer Setup Guide*.

Working with external configuration files

The internal InsuranceSuite application Substitutor can process the following types of substitution files:

- config/config.properties
- config/config.env.properties
- fileName.properties

Files config.properties and config.env.properties exist in the following location in the Studio **Project** window:

```
configuration > config
```

These files must exist in that specific Studio location for the Substitutor process to recognize and automatically load their substitution values. These files exist on the InsuranceSuite application application server.

File fileName.properties exists on your local development file system. You can choose the file location and file name. You set the path to the local substitution file as a system parameter as you start the application server. Use the following syntax.

```
gw.config.external.property.file=FileName
```

Populating the substitution file

Each group of configuration files that support substitution has a unique namespace value. You must incorporate the file group namespace into each name/value entry in the external substitution file. Use the following syntax for each entry in this file.

```
namespace.key=substitutionValue
```

Substitution file example

For example, file database-config.xml contains a <dbcp-connection-pool> element with a jdbc-url attribute. Suppose that you want to provide a value at server start for this URL. Creating a placeholder for a configuration file is a multi-step process:

1. Replace the jdbc-url="..." attribute string in file database-config.xml with \${jdbc_url}. Thus, the jdbc-url entry in database-config.xml looks similar to the following example:

```
<dbcp-connection-pool jdbc-url=${jdbc_url} />
```
2. Make an entry for the jdbc-url placeholder in the external substitution file that looks similar to the following entry:

```
database.jdbc_url="someURL"
```

After the Substitutor process makes the substitution for the placeholder, the <dbcp-connection-pool> element looks similar to the following example:

```
<dbcp-connection-pool jdbc-url="someURL" />
```

Setting the path to a local file

If using a substitution file for development and testing on your local file system, set the path to *fileName* using the following system property command as you start the server:

```
gw.config.external.property.file=fileName
```

However, to use this system property with the `gwb runServer` command, you must use the following construction:

```
gwb runServer -Dgw.passthrough.key=value
```

The following example illustrates this syntax.

```
gwb runServer -Dgw.passthrough.gw.config.external.property.file="C:\bc\configuration\etc\ext-properties.properties"
```

Local development environments

IMPORTANT: If you use a local *fileName.properties* file for development, create the file for use in specific, non-production, environments only. This prevents the substitution values present in this file from being used in a production environment.

It is possible to store substitution values in a properties file with a name of your choice on your local file system, for use during development. The primary use of this file is to pass bootstrap values to the application server during server start. This could be authentication parameters. For example:

```
plugin.authenticationissuer.current.secretkey=...
```

Setting configuration by environment

Use the following syntax to create a `config.properties` file for different environments:

```
config.env.properties
```

Using the local development file

The InsuranceSuite application server does not load a local substitution file automatically. You must specify the file name at server start from a command prompt. Use the following system parameter as you start the application server to point to the file:

```
gw.config.external.property.file=fileName
```

Loading configuration parameters

The InsuranceSuite application internal Substitutor process loads external configuration parameters from multiple sources into a single string of type:

```
Map<String, String>
```

The configuration parameters that the Substitutor process loads last override any previously defined configuration parameters.

Load order by file type

The Substitutor process loads configuration parameters from multiple file sources in the following order. The use of any of the substitution files is optional.

Server file `config.properties`

The Substitutor process loads server file `config.properties` automatically, if the file exists in the following location in Studio **Project** window:

configuration > config

This file generally holds default values for key configuration parameters. The InsuranceSuite application server includes this file with the other server files in creating a WAR file.

Server file `config.env.properties`

The Substitutor process loads server file `config.env.properties` automatically, if the file exists in the following location in Studio **Project** window:

configuration > config

This file contains environment-specific (*env*) defaults. The parameters in this file override any previously loaded configuration parameters if there is any overlap between the parameters. InsuranceSuite application server includes this file with the other server files in creating a WAR file.

Local file *fileName*

The Substitutor process loads the parameters in this configuration file from the local file system (from outside the server WAR file). You can name this file as you choose. Set the path to this file at server start from a command prompt, using the following syntax:

```
gw.config.external.property.file=fileName
```

The parameter values listed in this file override any previously loaded parameter values.

Guidewire Settings Vault

If configured to use Guidewire Settings Vault, the Substitutor process uses the parameters provided by the service to override any matching parameters that were previously loaded from substitution files, either server-based or on the local file system.

To summarize:

- If multiple substitution files define a value for the same configuration parameter, the server accepts the parameter value loaded last.
- If InsuranceSuite application Settings Vault is enabled and working, the value that Settings Vault supplies for a parameter overrides any value for that parameter set in a substitution file.

Working with Guidewire Settings Vault

Guidewire Settings Vault provides a means for Guidewire applications (and other services) to externalize configuration settings and to vary application configurations between deployment environments. An application that uses Settings Vault no longer needs to load configuration settings from multiple sources such as:

- Files
- Database content
- Environment variables
- Secret managers
- SSM
- Other sources

Instead, the application uses Settings Vault to load the necessary configuration settings based on the deployment environment. At runtime, Settings Vault captures any updates to application configuration that occur and makes the updated configuration available to the application without having to restart or redeploy the application.

It is possible to use both file-based configuration substitution and service-based configuration substitution at the same time. If there are conflicting substitution values, the values obtained through Settings Vault override values supplied through external file configuration.

Enabling Guidewire Settings Vault

To enable the Settings Vault client for InsuranceSuite, set environment variable `DEPLOYMENT_ID` to an appropriate value. The `DEPLOYMENT_ID` value defines the GRN (Guidewire Resource Notation) value for the current InsuranceSuite deployment.

You can set the GRN value using the following system property as you start the application server:

```
deployment.id
```

Plugin

ExternalConfigurationProviderPlugin

Guidewire implements external server configuration in plugin `ExternalConfigurationProviderPlugin`. Guidewire provides a default implementation class for the plugin:

`PropertyServiceExternalConfigurationProvider`

Implementation class `PropertyServiceExternalConfigurationProvider` supports both of the following mechanisms for external configuration:

- File-based external configuration
- Service-based external configuration

IMPORTANT: Do not try to extend or modify the Guidewire implementation of the `ExternalConfigurationProviderPlugin` plugin. Guidewire designs the default plugin implementation to work with both Guidewire Settings Vault and with Guidewire authentication services.

Starting ExternalConfigurationProviderPlugin manually

It is possible to start the `ExternalConfigurationProviderPlugin` manually, from the **Startable Services** screen in InsuranceSuite application Server Tools. This is useful if you want to test dynamic (hot) reloading of substitution parameters in a testing or development environment in which the server run level is at, or less than, the `MULTIUSER` run level.

ExternalConfigurationProviderPlugin implementation

Plugin implementation class `PropertyServiceExternalConfigurationProvider` extends class `ExternalConfigurationProviderPluginBase`, which contains the following public method:

```
lookupValue(String namespace, String key)
```

In this method:

- `namespace` is a hard-coded string that uniquely identifies a given file group.
- `key` is the identifier for the configuration placeholder .

The Substitutor process calls the `lookupValue` method for each placeholder in a supported configuration file. The plugin method call then returns a substitution value for placeholder, which the Substitutor process uses to replace the entire placeholder in the configuration file.

Placeholder examples

The following examples illustrate how to work with the `lookupValue` method:

- The Substitutor replaces placeholder `${url}` in file `config.xml` with the value returned by method `lookupValue("config", "url")`.
- The Substitutor replaces placeholder `${url}` in file `myPlugin.gwp` with the value returned by method `lookupValue("plugin", "url")`.

Lookup failure

If there is no default value and the plugin does not return a value:

- InsuranceSuite application leaves the placeholder strings unchanged.
- InsuranceSuite application places warning messages in the application log.

If a placeholder remains in a file used by the application server, it can cause the application server to fail, if not excluded from consideration due to an `env` or `server` configuration.

Accessing properties from code

Guidewire provides the following public API that provides access to the `ExternalConfigurationProviderPlugin` plugin:

```
gw.external.configuration.SubstitutionProperties
```

This API contains a single `lookupValue` method that internally delegates the API method call to the identical method in the `ExternalConfigurationProviderPlugin` plugin implementation. The following example illustrates how to work with this API.

```
uses gw.external.configuration.SubstitutionProperties
var value = SubstitutionProperties.lookupValue(namespace, key)
```


Setting InsuranceSuite application properties at run time

Guidewire InsuranceSuite applications provides the ability to set and modify application properties in real-time without restarting the application server. You do this through the use of what Guidewire calls *runtime properties*. You manage runtime properties through the InsuranceSuite application **Runtime Properties** screen. You must have administrative privileges to access this screen.

The Runtime Properties screen

To access the **Runtime Properties** screen, navigate to the following location in Guidewire InsuranceSuite application:

Administration > Utilities > Runtime Properties

From the **Runtime Properties** screen, you can do the following:

- Create a new property
- Edit the details of a property
- View the list of application properties
- Drill-down into the details of a property
- Import new properties into the InsuranceSuite application from a file
- Export the defined properties from the InsuranceSuite application to a file

Guidewire Home

If you have the required access, you can manage runtime properties also through Guidewire Home. See the [Cloud Platform](#) documentation.

Working with Runtime Properties

To work with runtime properties, navigate to the following location in Guidewire InsuranceSuite application:

Administration > Utilities > Runtime Properties**Viewing property details**

The initial **Runtime Properties** screen contains a table that lists all of the currently defined runtime properties. You can filter the contents of this table using the **Property Group** drop-down list to select what you want to see. Use the entries in this table to do the following:

- View the details of a specific property by clicking its name to open the **Property Detail** screen for that property.
- Edit the details of a property by clicking **Edit** on the **Property Detail** screen of the selected property.
- Delete a property by clicking **Delete** on the **Property Detail** screen of the selected property.

Creating a new runtime property

To create a new runtime property, click **New Property** on the **Runtime Properties** screen. In the **New Property** screen that opens, enter the following information.

Group

Group identifier of the property. This parameter is used by the InsuranceSuite application runtime property retriever. In the base configuration, InsuranceSuite applications provide two groups:

- Configuration
- Integration

These values are typecodes from typelist RuntimePropertyGroup.

Name

Original identifier of the property.

Description

(Optional) Additional information about the property.

Value

Property value. The InsuranceSuite application infers the data type of the value that you enter in the **Value** field, for example:

- If you enter a numeric value without a decimal point, the InsuranceSuite application infers an Integer value.
- If you enter a numeric value with a decimal point, the InsuranceSuite application infers a Long value.
- If you enter a text value surrounded by quotation marks, the InsuranceSuite application infers a String value.
- If you enter either true or false, the InsuranceSuite application infers a Boolean value.

Example

For example, to create a new property named `myImportantProperty`, click **New Property** and enter the following values.

Group

Configuration

Name

`myImportantProperty`

Description

Optional text string

Value

Required text string

Processing runtime properties

During start of the application server, the InsuranceSuite application inspects file `RuntimeProperties.xml` for new properties. You can access this file in the following **Project** directory in Guidewire Studio:

configuration > config > runtimeproperties

InsuranceSuite application processes properties defined at the application level before it processes any properties defined in file `RuntimeProperties.xml`. Thus, while processing the file, InsuranceSuite application does the following:

- It ignores any modifications to existing application properties in the file.
- It adds any new properties defined in the file to the list of application properties.
- It ignores (does not delete) any property removed from the file that currently exists as an application property.

Import runtime properties from a file

By default, the InsuranceSuite application loads new runtime properties from file `RuntimeProperties.xml` file during server start up. Separately, you can load runtime properties from any external properties file that you choose to import.

About this task

These instructions apply only to a development environment.

Procedure

1. Log into a Guidewire InsuranceSuite application as an administrative user.
2. Navigate to the following location in the InsuranceSuite application:
Administration > Utilities > Runtime Properties
3. Click **Import**.
4. In the **Import Runtime Properties** screen that opens, browse to select an `.xml` properties file to import.
Example `.xml` file:

```
<?xml version="1.0"?>
<Properties xmlns="http://guidewire.com/psc/properties">
  <Property>
    <Group>integration</Group>
    <Name>TestOne</Name>
    <Description></Description>
    <Value>Value</Value>
  </Property>
  <Property>
    <Group>integration</Group>
    <Name>TestTwo</Name>
    <Description></Description>
    <Value>Value</Value>
    <Value>Value Two</Value>
  </Property>
</Properties>
```

5. Select **Update Existing Values** if you want an imported property to override its matching application property.
6. Click **Import**.

Configuring runtime properties

To be useful, each defined runtime property must link to the InsuranceSuite application application code. The associated application code then:

- Calls an instance of the `RuntimePropertyRetriever` class.
- Calls a `RuntimePropertyRetriever` class method to retrieve a property value.
- Uses the retrieved property value to affect business logic in the InsuranceSuite application.

Class `RuntimePropertyRetriever`

In the base configuration, InsuranceSuite provides a `RuntimePropertyRetriever` class with public getter methods to retrieve the different types of runtime properties. Class methods include the following, for example:

- `getBooleanProperty(propertyName)`
- `getIntegerProperty(propertyName)`
- `getStringProperty(propertyName)`

The `RuntimePropertyRetriever` class contains a constructor method that takes a single `Group` parameter and returns an instance of the `RuntimePropertyRetriever` class. For example:

```
var runtimePropertyRetriever = new RuntimePropertyRetriever(RuntimePropertyGroup.TC_CONFIGURATION)
```

Notice the use of a typecode from the `RuntimePropertyGroup` typelist to define the `Group` argument. You can only access a runtime property from the specified group passed to the constructor. In the base configuration, the `RuntimePropertyGroup` typelist contains the following typecodes:

- `configuration`
- `integration`

Example

The following code is an example of how to access a `String` runtime property in Gosu application code:

```
uses gw.api.properties.RuntimePropertyRetriever
...

var runtimePropertyRetriever = new RuntimePropertyRetriever(RuntimePropertyGroup.TC_CONFIGURATION)
var myImportantProperty = runtimePropertyRetriever.getStringProperty("myImportantProperty")
```

In this code, notice the following:

- The code uses a `myImportantProperty` runtime property, with the assumption that this application property already exists.
- The code sets the `Group` argument using a typecode from the `RuntimePropertyGroup` typelist.

Work queues and batch processing

Guidewire InsuranceSuite application processes provide a wide variety of functionality and they are divided into the following categories.

Work queue

Provides for the processing of a batch of items in parallel, in a distributed manner.

Batch process

Provides for the processing of a batch of items sequentially, one after the other

To manage these processes, Guidewire provides a number of management tools, including the following InsuranceSuite application Server Tools screens:

- **Batch Process Info** screen
- **Work Queue Info** screen

Administering InsuranceSuite application processes

InsuranceSuite supports two modes of processing:

- Work queue
- Batch process

Work queue

A work queue operates on a batch of items in parallel. InsuranceSuite distributes work queues across all servers in a InsuranceSuite application cluster that have the appropriate role. In the base configuration, Guidewire assigns this functionality to the workqueue server role.

A work queue comprises the following components:

- A processing thread, known as a *writer*, that selects a group (batch) of business records to process. For each business record the writer creates an associated work item.
- A queue of selected work items.
- One or more tasks, known as *workers*, that process the individual work items to completion. Each worker is a short-lived task that exists in a thread pool. Each work queue on a cluster member shares the same thread pool. By default, each work queue starts a single worker on each server with the appropriate role, unless configured otherwise.

Work queues are suitable for high volume batch processing that requires the parallel processing of items to achieve an acceptable throughput rate.

Batch process

A batch process operates on a batch of items sequentially. Batch processes are suitable for low volume batch processing that achieves an acceptable throughput rate as the batch process processes items in sequence. For example, writers for work queues operate as batch processes because they can select items for a batch and write them to their work queues relatively quickly.

Process history

Database table `xx_processhistory` stores information on all completed work queues and batch processes. In the table name, `xx` represents the application product code, such as `pc`, `cc`, or `bc`.

Work queues

A work queue comprises the following components.

Work queue

A work queue is a database table that holds work items for processing. Guidewire work queues share the standard work item table: `StandardWorkQueue` (`xx_standardworkqueue`, where `xx` is the application product code).

Writer

A writer thread does the following:

- It selects units of work for processing.
- It writes the identities of the work items to a work queue table.
- It notifies the workers that there are work items to process.

Worker

A worker retrieves one or more work items from the work queue and processes the work items to completion. A worker only processes work items from its associated writer. By default, each work queue starts a single worker on each cluster member with the `workqueue` role, unless configured otherwise.

Executor

The work queue executor manages the worker tasks on each application server in a InsuranceSuite application cluster. If a work queue has no more remaining items to process, the executor shuts down all workers on a server node except for one, which periodically checks to see if there is new work to perform.

Starting the writer initiates a run of processing on a work queue. The work is complete if the workers exhaust the queue of all work items, except those that they failed to process successfully.

Work queue writers

Whenever the writer thread awakes or starts on demand, it checks the work queue table for any work items that remain from a prior batch. The specific configuration of each work queue determines how the work queue creates and processes work items.

The following workflow examples provides a simple description of how InsuranceSuite application work queues operate.

Work items remaining

If work items remain from a previous batch, the following sequence of events occur:

1. The writer thread notifies the workers that they have work to process.
2. The application terminates the writer thread.

No work items remaining

If no work items remain from a previous batch, the following sequence of events occurs:

1. The writer thread selects items for a new batch.
2. The writer writes the identifiers of the selected items to the work queue table.
3. The writer notifies the workers that they have work to process.
4. The application terminates the writer thread.

Work queue workers

Typically, each work queue shares the standard work item table (`StandardWorkQueue`) for its work items. However, a worker task operates only on work items that its associated writer inserts into the table. For example, suppose that the InsuranceSuite application distributes the Activity Escalation work queue across a cluster, with six workers operating on three different servers. Those workers work only on activity escalation items in the standard work item table. Typically, you configure work queues for multiple workers. Thus, typically, some number of workers operate throughout the day on work items in the standard work queue table.

At specified intervals (defined through the `maxpollinterval` attribute on `worker` in `work-queue.xml`), a worker awakens and checks the work item table for work items from its associated writer. If a worker finds work items available for processing, the worker checks out its quota from the work queue. For each item, the worker sets the following attributes.

Status	Set to <code>checkedout</code> . The value of the Status attribute can be any one of the following: • <code>available</code> • <code>checkedout</code> • <code>failed</code>
LastUpdateTime	Set to the time at which the worker checks out the work item.
CheckedOutBy	Set to the worker.

After it checks out a quota of work items, the worker task processes them sequentially. Whenever a worker completes a work item successfully, it deletes the item from the table and begins to process the next item. The standard work item table (`StandardWorkQueue`) is retireable, so successfully completed work items remain in the table for historical reference.

Note: In rare cases, it is possible for the InsuranceSuite application to notify a worker of work, but, the worker finds no work is available after it awakens. For example, for small batch runs, a worker can check out all items in the batch with its first check out quota of items. This action can occur between the time the writer notifies the workers and other workers awaken. If a worker awakens and finds no work items, the worker goes back to sleep.

Work queue scheduling and processing intervals

A writer for a work queue starts at the interval specified in `scheduler-config.xml`. Typically, you schedule the writer to start several times during the day or once at night. Access schedule configuration file `schedule-config.xml` in Guidewire Studio at the following location:

```
configuration > config > scheduler
```

Worker tasks awaken much more frequently than their writers start. One writer awakens at least as frequently as a specified maximum polling interval, if not more frequently.

You do not schedule worker tasks. Instead, they awaken in response to notification from the writer or upon expiration of the polling interval. After a worker awakens, if there are work items to process, it processes up to the number of batchsize items, as defined in `work-queue.xml`. If there are more items than the batch size to process, the worker awakens another worker. This worker repeats the process of checking out work items and waking up another worker if necessary, until maximum allowed number of workers are active.

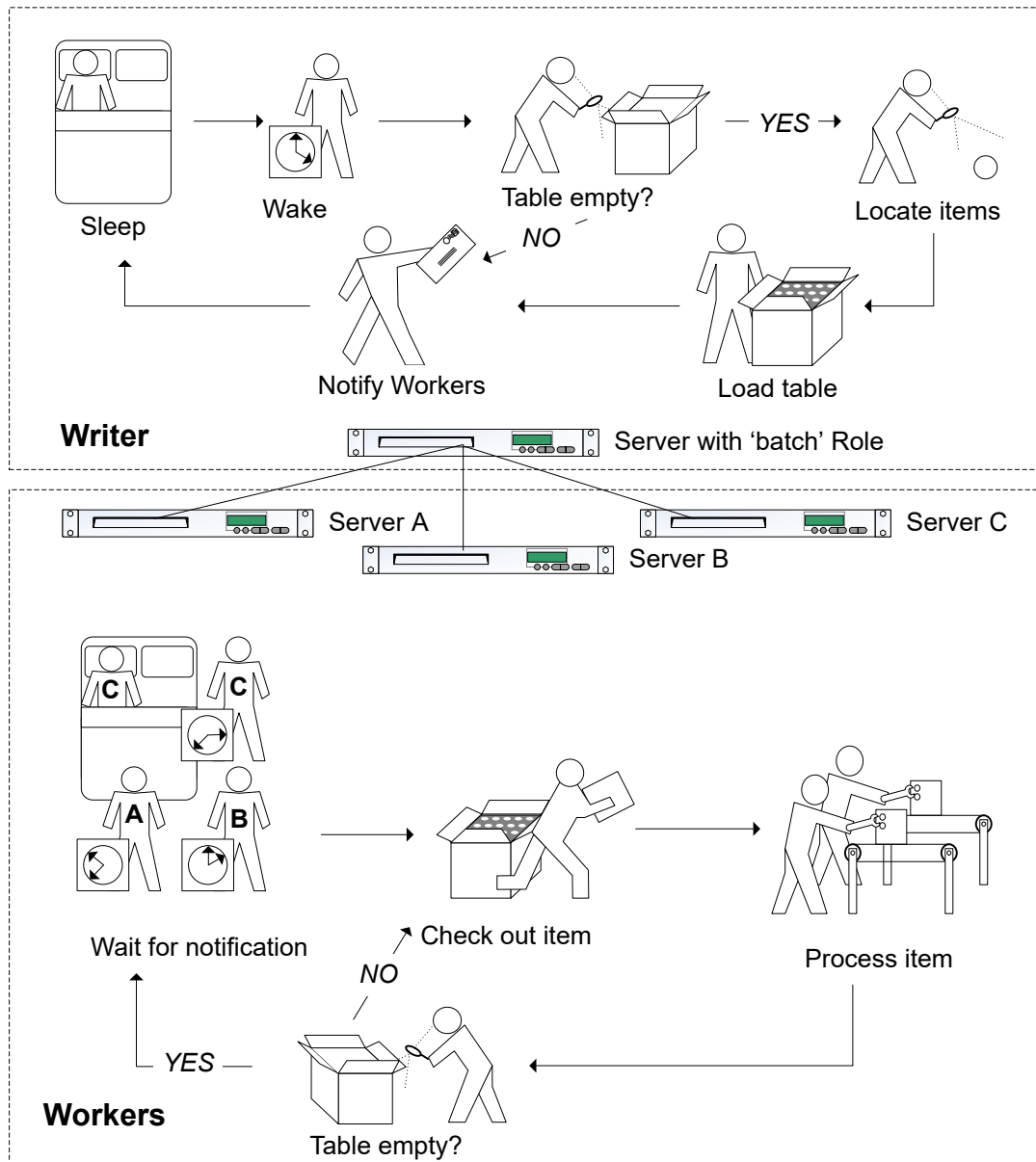
You configure the number of workers, polling interval, and batch size in `work-queue.xml`. Access work queue configuration file `work-queue.xml` in Guidewire Studio at the following location:

configuration > config > workqueue

You can view and manage work queues from the Server Tools Work Queue Info screen in the InsuranceSuite application, if you have the appropriate administrative privilege.

Work queue architecture

The following diagram illustrates the components of a work queue and how they function.



Batch processes

The InsuranceSuite application distributes batch processes across all cluster members that have the batch server role. Each server with the batch role also has a batch process lease manager that acquires and manages the batch process leases on that server. In this context, a lease represents a single run of a single batch process.

Available servers with the batch role compete for available batch processing leases. After a server acquires a lease, that server runs the batch process to completion.

How aggressively the cluster servers compete with each other depends on how many batch processes each one is individually already running. Those servers running fewer or no batch processes are more likely to acquire a new batch process lease than other servers already busy running processes. It is possible to configure this behavior.

For scheduled batch processes, a scheduler component, running on a cluster member with the scheduler role, decides to start a batch process according to the published schedule. The scheduler first creates a new lease for the batch

process in the database. All cluster members with the batch server role then compete to acquire that lease. The cluster member that wins the competition starts the batch process.

The Batch Process Info screen

Generally, a batch process runs to completion and then reports its result back to a log or to the administrative user interface. You can view and manage batch processing from the Server Tools **Batch Process Info** screen in the InsuranceSuite application, if you have the appropriate administrative privilege. From this screen, you can view the batch process schedule, if any, and start or stop a scheduled batch process.

Batch process caching

Batch processes that run concurrently on the same server share a common cache. The cache demands of each process end up flushing the cache more frequently, so fewer cache hits occur for each process. That increases the amount of physical reads from the relational database, thus degrading performance. Concurrent data change exceptions can occur also if multiple batch processes (on the same or different servers) attempt to update the same cached entity instances. This requires one or the other batch process to retry an item, leading to further performance degradation.

Configuring the different processing types in Studio

To access information about InsuranceSuite application processing types, open the following files in Studio:

- `BatchProcessType.ttx` – Provides the name and codes for each processing type.
- `work-queue.xml` – Provides the name of the backing class and number of work instances for a processing type.
- `scheduler-config.xml` – Provides the schedule for any process that is Schedulable.

The number of worker tasks is important if zero (0). Setting the number of worker tasks to 0 disables the work queue as there are no workers to perform the work. To enable the work queue, set the number of workers to greater than zero.

Executing processes in Guidewire Studio

The Test server (TestServer) in Guidewire Studio runs in the NODAEMONS (no daemons) run level by default. At the 'no daemons' run level, batch processes and work queues do not run in the test server. This prevents batch processes and work queues from running while a test is executing and interfering with the test.

To set the run level for the test server to a different value, edit the VM options for the test server and set the server run level to a higher level, one that supports process execution. For example:

`-Dgw.xx.runlevel=5`, where `xx` is the application product code.

Working with work queue writers and batch processes

You can run many batch processes, including writers for work queues, from the InsuranceSuite application or from the command prompt.

How to run a writer from the Work Queue Info screen

It is possible to run a writer for a work queue from the Server Tools **Work Queue Info** screen. The InsuranceSuite application enables the **Run Writer** button on this screen for all work queue types that belong to the `BatchProcessTypeUsage` category `UIRunnable`.

To access the **Work Queue Info** screen, you must have the `internaltools` permission. The Admin role has this permission by default. Alternatively, if the `EnableInternalDebugTools` parameter is set to `true` in `config.xml` and the server is running in development mode, all users can access the **Work Queue Info** screen.

How to run a batch process from the Batch Process Info screen

It is possible to run batch processes from the Server Tools **Batch Process Info** screen in the InsuranceSuite application. The application enables the **Run** button on this screen for all batch process types that belong to the BatchProcessTypeUsage category UIRunnable.

To access the **Batch Process Info** screen, you must have the `internaltools` permission. The Admin role has this permission by default. Alternatively, if the `EnableInternalDebugTools` parameter is set to true in `config.xml` and the server is running in development mode, all users can access to the **Batch Process Info** screen.

How to terminate a writer or batch process from the command prompt

It is possible to terminate some in-progress processes, including writers for work queues, from a command prompt. To determine if it is possible to terminate an in-progress batch process, consult the reference topic for that particular batch processing type. The information for each individual batch process includes whether it is a single phase or multiphase process. You can only stop those processes listed as being multiphase.

Note: It is not possible to terminate a *single phase* batch process. Single phase processes run in a single transaction. Thus, there is no convenient place to terminate the process.

Run a writer from an InsuranceSuite application

Start or stop a work queue from the **Work Queue Info** screen.

Before you begin

You must have the `internaltools` user permission to access the **Work Queue Info** screen.

About this task

It is possible to run writers for work queues from either the **Work Queue info** screen or the **Batch Process Info** screen.

Procedure

1. Log in to the InsuranceSuite application.
2. Press ALT+SHIFT+T to display the **Server Tools** tab.
3. Navigate to **Work Queue Info**.
4. Click **Run Writer** in the **Actions** column of the work queue that you want to run.

Run a batch process from an InsuranceSuite application

Start, or stop, a batch process from the **Batch Process Info** screen.

Before you begin

You must have the `internaltools` user permission to access the **Batch Process Info** screen.

Procedure

1. Log in to the InsuranceSuite application.
2. Press ALT+SHIFT+T to display the **Server Tools** tab.
3. Navigate to **Batch Process Info**.
4. Click **Run** in the **Action** column of the batch process that you want to run.

Run a writer or batch process from the command prompt

Start a process using a `maintenance_tools` command option.

About this task

It is possible to run many processes in a local development environment from a command prompt.

Procedure

1. Start the InsuranceSuite application server if it is not already running.
2. Open a command prompt.
3. Navigate to the following location in the InsuranceSuite application installation directory:

```
admin/bin
```

4. Run the following command:

```
maintenance_tools -password password -startprocess process
```

The command prompt tools for administering the InsuranceSuite applications all require that you enter the password of an administrative user for the tool to work. The use of a user name is optional.

For the *process* value, specify a valid process code. To determine the correct process code, consult the reference topic for the individual batch processing type.

Terminate a writer or batch process from the command prompt

Stop a process using a `maintenance_tools` command option.

About this task

It is not possible to use following procedure to terminate the operation of the `table_import` command.

Procedure

1. Start the InsuranceSuite application server if it is not already running.
2. Open a command prompt.
3. Navigate to the following location in the InsuranceSuite application installation directory:

```
admin/bin
```

4. Run the following command:

```
maintenance_tools -password password -terminateprocess process
```

The command prompt tools for administering the InsuranceSuite applications all require that you enter the password of an administrative user for the tool to work. The use of a user name is optional.

For the *process* value, specify a valid process code. To determine the correct process code, consult the reference topic for the individual batch processing type.

Check status of a writer or batch process from the command prompt

Check the status of a process using a `maintenance_tools` command option.

About this task

It is possible to check the status of processes in a local development environment from a command prompt, including the writer processes for work queues.

Procedure

1. Start the InsuranceSuite application server if it is not already running.
2. Open a command prompt.
3. Navigate to the following location in the InsuranceSuite application installation directory:

admin/bin

4. Run the following command:

```
maintenance_tools -password password -processstatus process
```

The command prompt tools for administering the InsuranceSuite applications all require that you enter the password of an administrative user for the tool to work. The use of a user name is optional.

For the *process* value, specify a valid process code. To determine the correct process code, consult the reference topic for the individual batch processing type.

Results

For work queues, executing this command returns the status of the writer process. The command does not check whether any work items remain in the work queue. Thus, it is possible for the process status to report as being complete after the writer finishes adding items to the work queue. However, it is possible that there are work items that need processing that remain in the work queue.

Scheduling work queue writers and batch processes

The InsuranceSuite application scheduler launches many processes, including writer processes for work queues, according to a schedule defined in `scheduler-config.xml`. Access this file in Guidewire Studio at the following location:

configuration > config > scheduler

The `scheduler-config.xml` file contains entries in the following format:

```
<ProcessSchedule process="process_code" env="environment">
  <CronSchedule schedule_attributes>
</ProcessSchedule>
```

The `process` attribute sets the process to run. The `env` attribute is an optional attribute that specifies the environment in which the schedule definition for the process applies. The `schedule_attributes` value is a valid schedule specification.

If needed, you can list multiple `ProcessSchedule` entries for the same process. The process then runs according to each specified schedule. If you schedule a process to run while the same process is already running, then the InsuranceSuite application skips the overlapping process. If the `scheduler-config.xml` file does not list a process, the process does not run.

Generally, schedule the amount of time between process runs in hours as opposed to minutes. This is because some batch processes require a lot of server resources. Schedule such processes to wake infrequently or at times that the server is less busy, such as late at night or very early in the morning.

You may want to schedule some InsuranceSuite application processes to run periodically throughout the business day. For example, the default configuration of the InsuranceSuite application schedules the `ActivityEsc` work queue to run every 30 minutes. Exclude running such processes periodically during your nightly processing window. Instead, wait until the end of the processing window to run them. For example, schedule the `ActivityEsc` process to run every 30 minutes except during your nightly processing window. Alternatively, run such processes at prescribed places in your chain of nightly processes.

The InsuranceSuite application scheduler uses the InsuranceSuite application server time for reference.

Defining a schedule specification

The `<CronSchedule>` element in file `scheduler-config.xml` describes when the InsuranceSuite application is to run the process.

```
<CronSchedule schedule_attributes/>
```

Use this element to define a `schedule_attributes` value to specify the exact timing, such as once every hour or every night at a certain time. The `schedule_attributes` value is a combination of one or more of the following attributes:

Attribute	Standard Values	Default	Example
seconds	0-59	0	seconds="0"
minutes	0-59	0	minutes="15"
hours	0-23	*	hours="12"
dayofmonth	1-31	*	dayofmonth="1"
month	1-12 or JAN-DEC	*	month="2"
dayofweek	1-7 or SUN-SAT	?	dayofweek="1"

IMPORTANT: If you do not provide a value for a defined schedule attribute, the scheduler uses its default value in determining the work queue schedule. For example, if you do not specify a value for the hours attribute, the scheduler assumes a value of * and InsuranceSuite application runs the work queue process every hour. Thus, Guidewire recommends that you provide a value for each scheduler attribute. If you do not provide a value for a specific attribute, carefully review that attribute's default value and determine if the default value meets your business needs.

Useful attribute characters

Along with the standard values listed, there are some special characters that give you more flexible options.

Character	Meaning
*	Indicates all values. For example, minutes="*" means run the process every minute.
?	Indicates no specific value. Used only for dayofmonth and dayofweek attributes. See the examples for clarification.
-	Specifies ranges. For example, hour="6-8" specifies the hours 6, 7 and 8.
,	Separates additional values. For example, dayofweek="MON,WED,FRI" means every Monday, Wednesday, and Friday.
/	Specifies increments. For example, minutes="0/15" means start at minute 0 and run every 15 minutes.
L	Specifies the last day. Used only for dayofmonth and dayofweek attributes. See the examples for clarification.
W	Specifies the nearest weekday, use only with dayofmonth. For example, if you specify 1W for dayofmonth, and that day is a Saturday, the trigger then fires on Monday the 3rd. You can combine this with L to schedule a process for the last weekday of the month by specifying dayofmonth="LW".
#	Specifies the nth day of the week within a month. For example, a dayofweek value of 4#2 means the second Wednesday of the month (day 4 = Wednesday and #2 = the second Wednesday in the month).

These represent only some of the values that you can use for setting schedule.

Scheduler examples

The following table lists a few examples of how to work with the `<CronSchedule>` element.

Example	Description
<code><CronSchedule hours="10" /></code>	Run every day at 10 a.m.
<code><CronSchedule hours="0" /></code>	Run every night at midnight.
<code><CronSchedule minutes="15,45" /></code>	Run at 15 and 45 minutes after every hour.
<code><CronSchedule minutes="0/5" /></code>	Run every five minutes.
<code><CronSchedule hours="0" dayofmonth="1" /></code>	Run at midnight on the first day of the month.
<code><CronSchedule hours="12" dayofweek="MON-FRI" dayofmonth="?" /></code>	Run at noon every weekday (without regard to the day of the month).
<code><CronSchedule hours="22" dayofmonth="L" /></code>	Run at 10 p.m. on the last day of every month.
<code><CronSchedule hours="22" dayofmonth="L-2" /></code>	Run at 10 p.m. on the second-to-last day of every month.
<code><CronSchedule minutes="3" hours="8-18/2" dayofweek="1-5" dayofmonth="?" /></code>	Run 3 minutes after every other hour, 8:03 a.m. to 6:03 p.m., Monday through Friday.
<code><CronSchedule minutes="*/15" hours="0-8,18-23" /></code>	Run every 15 minutes after the hour, 12:15 a.m. to 8:45 a.m. and 6:15 p.m. to 11:45 p.m.
<code><CronSchedule hours="0" dayofweek="6L" /></code>	Run at midnight on the last Friday of the month.
<code><CronSchedule hours="4" dayofweek="4#2" dayofmonth="?" /></code>	Run at 4 a.m. on the second Wednesday of the month.

The Quartz Enterprise Job Scheduler

These characters represent only some of the values that you can use for setting a schedule. Guidewire bases the InsuranceSuite application scheduler on the open source Quartz Enterprise Job Scheduler. The scheduler therefore uses the same specification for schedule attributes that Quartz uses. To determine the exact Quartz version, check the file name of the Quartz JAR file in `<InsuranceSuiteApplication>/admin/lib`.

Determine if it is possible to schedule a batch process

Use the **Batch Process Info** screen to determine if it is possible to schedule a batch process.

About this task

It is possible to schedule many batch processes, including work queue writers. However, it is not possible to schedule all batch processes.

Procedure

1. Log into an InsuranceSuite application as an administrative user.
2. Press ALT+SHIFT+T to access Server Tools.
3. Navigate to the **Batch Process Info** screen.
4. Select **Schedulable** from the processes drop-down filter.
The InsuranceSuite application displays only those batch processes, including work queue writers, that it is possible to schedule in the `scheduler-config.xml` file.

View a batch process schedule

Review information in the InsuranceSuite application **Batch Process Info** screen.

Procedure

1. Log in to InsuranceSuite application as an administrative user.
2. Press ALT + SHIFT + T to access Server Tools.
3. Navigate to the **Batch Process Info** screen.
4. Click the **Next Scheduled Run** column header to sort processes by schedule.

If there is no current schedule for a process, the **Next Scheduled Run** field is blank.

Using an external batch scheduler

It is possible to disable the internal scheduler by setting the `SchedulerEnabled` configuration parameter to `false` in file `config.xml`. If you do so, you need to implement your own mechanism for running InsuranceSuite application processes. For a discussion of using an external scheduler in the Guidewire Cloud environment, see *IS-INT-1274 Batch Schedulers in Guidewire Cloud* at the following location:

<https://docs.guidewire.com/cloud/standards/latest>

If the search does not immediately return a result, try changing the selected filters.

Configuring work queues

In working with work queues, there are multiple general areas that you can configure.

Configuration area	Configuration file
Work queue scheduling	<code>scheduler-config.xml</code>
Workers and work queues	<code>work-queue.xml</code>
Work queue configuration parameters	<code>config.xml</code>

The work queue configuration file

You may want to modify the configuration of Guidewire-provided work queues to improve performance. You configure attributes of a work queue and its workers in file `work-queue.xml`. For custom work queues, you must modify `work-queue.xml` to enable your work queue to operate.

File `work-queue.xml` contains one top-level element, which is `<work-queues>`. This element has one required attribute, `defaultServer`. In the base configuration, Guidewire sets the value of this attribute to `workqueue`.

```
<work-queues
  xmlns="http://guidewire.com/work-queue"
  defaultServer="#workqueue">
```

Note: The hash mark in front of `workqueue` (`#workqueue`) indicates that the value that follows the hash mark is a server role and not a server ID.

Attribute `defaultServer` requires a value. There is no default. The InsuranceSuite application server refuses to start if you do not provide a value for this attribute. The server also refuses to start if you set `defaultServer` to a role that does not exist in `<registry>` element in `config.xml`.

Work queue definitions

Within the top-level `<work-queues>` element in `work-queue.xml`, use subelement `<work-queue>` to define individual work queues.

```
<work-queue workQueueClass="string" progressinterval="decimal">
  <worker instances="integer"
    throttleinterval="decimal" env="string1, string2, ..."
    server="string"/>
</work-queue>
```

The `<work-queue>` subelement has attributes to configure a named work queue in general. The `<worker>` subelement has attributes to configure worker tasks on specific servers. You can declare as many workers as you want for a work queue by specifying on which servers the workers run. You can declare a set of possible environments for a worker by specifying a value or multiple comma-separated values for the `env` attribute.

Access `work-queue.xml` in Guidewire Studio at the following location:

configuration > config > workqueue

General work queue configuration

The `<work-queue>` element in `work-queue.xml` contains attributes for configuring the general characteristics of a work queue.

Attribute	Description
<i>Required attributes</i>	
<code>progressinterval</code>	The <code>progressinterval</code> value is the amount of time, in milliseconds, that the InsuranceSuite application allots for a worker to process the number of <code>batchsize</code> work items. If the time a worker has held a batch of items exceeds the value of <code>progressinterval</code>, then the application considers the work items to be orphans. The application reassigns orphaned work items to a new worker instance. The <code>progressinterval</code> value must be greater than the time to process the slowest work item, or that work item never completes. Guidewire recommends that you set the <code>progressinterval</code> value greater than the processing time for an entire <code>batchsize</code> of work items: • If a worker takes more time than the time specified by <code>progressinterval</code> to processes its assigned work items, the application reverts the remaining work items to available from checkedout. • If many worker batches take longer than the time specified by <code>progressinterval</code>, the repeated checking out and reverting to available of work items can have a negative impact on performance.
<code>workQueueClass</code>	(Required) The <code>workQueueClass</code> value must be one of the following: • A Guidewire-provided work queue class listed in the base configuration version of <code>work-queue.xml</code> • A custom work queue class derived from Gosu class <code>WorkQueueBase</code> You cannot configure Guidewire-provided batch processes or custom batch processes derived from the Gosu class <code>BatchProcessBase</code>.
<i>Optional attributes</i>	
<code>blockWorkersWhenWriterActive</code>	If the work queue workers start execution before the work queue writer completes writing work items to the work queue, it can possibly cause performance issues under certain circumstances. If set to <code>true</code>, InsuranceSuite application blocks the work queue workers from acquiring new work items until the writer completes writing work items to the queue. After the writer completes writing any new work items, the workers automatically start acquiring work items again.

Attribute	Description
	The default is <code>false</code> . Only enable this attribute for the work queues for which you require this capability. Guidewire recommends that you consider setting this attribute to <code>true</code> if the work queue writer can run for extensive periods of time due to the work load generated.
<code>logRetryableCDCEsAtDebugLevel</code>	If the value of <code>logRetryableCDCEsAtDebugLevel</code> is set to <code>true</code> for a work queue, InsuranceSuite application logs any retryable Concurrent Data Change Exception (CDCE) at the <code>DEBUG</code> level. The log message includes a prepended string to indicate that the error is non-fatal. The application logs any CDCE that pushes the retry count over the value of <code>retryLimit</code>, or the value of <code>workItemRetryLimit</code> if <code>retryLimit</code> is not set, at the <code>ERROR</code> level.
<code>retryInterval</code>	How long in milliseconds to wait before retrying a work item that threw an exception. The default value is 0, meaning the InsuranceSuite application retries processing the item immediately. In the case of a <code>ConcurrentDataChangeException</code> , InsuranceSuite application resets the value to 0.
<code>retryLimit</code>	The number of times InsuranceSuite application retries a work item that threw an exception or that became an orphan for this work queue. If you do not specify a <code>retryLimit</code> value for a work queue, the application uses the value of the <code>WorkItemRetryLimit</code> configuration parameter in <code>config.xml</code> as the default value. IMPORTANT: Guidewire generally recommends that you increase, never decrease, the number of retries for a work queue.

Worker configuration

The use of the `<worker>` element in `work-queue.xml` is optional. However, in actual practice, it is necessary for there to be at least one `<worker>` element for each `<work-queue>` element for the work queue to operate properly. The `<worker>` element contains an `instances` attribute that has a default value of 1. Without a `<worker>` element to provide this default, the processing logic does not allocate any workers for the work queue.

All of the following attributes are optional.

Attribute	Description
<code>instances</code>	The number of workers to create. By default, the InsuranceSuite application sets the values of this attribute to 1. If a worker wakes up and detects work items, it checks out those work items from the work queue. If there are more work items than the value specified by the <code>batchsize</code> attribute, the worker starts another worker. Each new worker checks out up to the maximum <code>batchsize</code> number of work items. If there are more work items remaining, the new worker starts another worker. The creation of workers continues until the number of workers reaches the maximum limit of workers as specified by the <code>instances</code> attribute.
<code>maxpollinterval</code>	How often a worker wakes up automatically and queries for work items, even if the worker receives no notification. You might need to increase the value of <code>maxpollinterval</code> to prevent excessive numbers of queries for work items. The default value of <code>maxpollinterval</code> is 60,000 milliseconds.
<code>throttleinterval</code>	The delay between processing work items in milliseconds. The value controls how long the process sleeps. A value of 0 (zero) means worker tasks process work items as rapidly as possible. To reduce the CPU load, set the value of <code>throttleinterval</code> to a positive non-zero value.
<code>batchsize</code>	How many work items the worker attempts to check out while searching for more work items. Larger batch sizes are more efficient, but might not result in good load distribution. The default value for <code>batchsize</code> is 10.
<code>env</code>	The one or more environments in which this particular worker configuration is active. To specify multiple values for the <code>env</code> attribute, use a comma-separated list.

Attribute	Description
server	The serverid of the server on which this particular worker is active.

Example

Suppose that you want to create a worker with two instances on each server that has an assigned role of workqueue. You define this work in a manner similar to the following code:

```
<worker server="#workqueue" instances="2"/>
```

In this case, the designation #workqueue (with the hash mark) indicates a specific server role and not the server ID. The worker definition sets instances to 2. This action creates two instances of the designated worker on each application server with the workqueue role.

Worker task management

An *executor* manages the worker tasks on each cluster server with the appropriate role. In the base configuration, Guidewire assigns this functionality to the workqueue server role. Each server with the workqueue role creates one executor for each work queue on that server.

Each work queue executor periodically creates a worker. (The executor can also create a worker upon receiving a notification from the writer.) This worker checks the work queue for items to process. If necessary, the initial worker creates an additional worker if there is more work to process than it can handle. This new worker can also create a worker if there is still more work to process. After there is no more work to process, all active workers stop.

It is possible to control the maximum number of workers, for all work queues on a server, by setting the value of configuration parameter WorkQueueThreadPoolMaxSize in config.xml. It is possible to set this value individually on each InsuranceSuite application server in the cluster.

Work queues and server roles

In the base configuration, Guidewire assigns work queue functionality to servers with the workqueue role. File workqueue.xml associates the workqueue server role with the application work queue functionality.

Note: The hash mark in front of workqueue (#workqueue) indicates that the value that follows the hash mark is a server role and not a server ID.

The workqueue role is merely the default role, however. You are free to create and assign new work queue management roles. You can also use server roles to enable or disable certain work queues on a specific InsuranceSuite application server.

Defining a new work queue role

You define server roles using the roles attribute on the <registry> element in file config.xml. In the base configuration, Guidewire defines the following server roles:

```
<registry roles="batch, workqueue, scheduler, messaging, startable, ui" />
```

To add a specialized work queue role, say, one to use in managing activity work queues (activitywq), add the new server role to the list of roles:

```
<registry roles="batch, workqueue, activitywq, scheduler, messaging, startable, ui" />
```

Assigning a work queue to specific InsuranceSuite application cluster servers

Suppose that you want to assign the management of the activity work queues in the InsuranceSuite application cluster to a subset of the cluster servers with the activityworkqueue role. By default, InsuranceSuite application distributes all other work queues to those servers with the default workqueue role.

For example, in file `work-queue.xml`, you define the following:

```
<?xml version="1.0"?>
<work-queues xmlns="http://guidewire.com/work-queue" defaultServer="#workqueue">
  <work-queue workQueueClass="com.guidewire.pl.domain.escalation.ActivityEscalationWorkQueue" ... >
    <worker server="#activityworkqueue"/>
  </work-queue>
  <work-queue workQueueClass="com.guidewire.pl.domain.geodata.geocode.GeocodeWorkQueue" ... >
    <worker/>
  </work-queue>
</work-queues>
```

In this example:

1. If a server has the `workqueue` role only, then that server:
 - a. Starts an executor for the `GeocodeWorkQueue` work queue.
 - b. Does not start an executor for the `ActivityEscalationWorkQueue` work queue.
2. If a server has the `activityworkqueue` role only, then that server:
 - a. Starts an executor for `ActivityEscalationWorkQueue` work queue.
 - b. Does not start an executor for the `GeocodeWorkQueue` work queue.
3. If a server has both the `activityworkqueue` and `workqueue` roles, then that server starts executors for both work queues.
4. If a server has neither the `activityworkqueue` nor the `workqueue` role, then the server does not start an executor for either of these work queues.

Performing custom actions after a process completes

You can use Process Completion Monitor processing to launch custom actions after a work queue or batch process completes a batch of items. For example, you might want to start the writer of a follow-on work queue during nightly batch processing.

Process Completion Monitor processing runs at schedulable intervals and examines the `ProcessHistory` table for all completed work queues and batch processes.

For each completed work queue that it finds, Process Completion Monitor:

- Determines if all the work items in that work queue have either completed or failed.
- Calls the `IBatchCompletedNotification` plugin implementation on a process if the process is complete and has no remaining available or checked-out work items.
- Sets `ProcessHistory.NOTIFICATIONSENT` to `true` to invoke the `IBatchCompletedNotification` plugin implementation a single time only for any given process.

The `IBatchCompletedNotification` interface has a `completed` method that you can override to perform specific actions if a work queue or batch process finishes a batch of work. The parameters of the `completed` method are the `ProcessHistory` entity and the number of failed items. The InsuranceSuite application considers work queue processing as complete if no work items remain on the queue, other than work items that failed. InsuranceSuite application considers a batch process as complete if the process stopped and its process history is available.

Schedule the Process Completion Monitor batch process

The Process Completion Monitor process runs at schedulable intervals and examines the `ProcessHistory` table for all completed work queues and batch processes.

About this task

In the ClaimCenter and PolicyCenter base configuration, the Process Completion Monitor process is not scheduled. To enable this process, you need to add the process to file `scheduler-config.xml`.

In the BillingCenter base configuration, the Process Completion Monitor process is scheduled to run every five minutes. You can alter the schedule by modifying the `<CronSchedule>` element for the `ProcessCompletionMonitor` process.

Procedure

1. In the Guidewire Studio **Project** window, expand **configuration > config > scheduler**.

- a) Open `scheduler-config.xml`.

- b) In ClaimCenter and PolicyCenter, add the following `<ProcessSchedule>` element:

```
<ProcessSchedule process="ProcessCompletionMonitor">
  <CronSchedule minutes="*/5"/>
</ProcessSchedule>
```

- c) In BillingCenter, modify the following `<ProcessSchedule>` element and set the `minutes` attribute to a value that meets your business needs:

```
<ProcessSchedule process="ProcessCompletionMonitor">
  <CronSchedule minutes="*/5"/>
</ProcessSchedule>
```

2. Save your changes.

Implement the `IBatchCompletedNotification` interface

The Process Completion Monitor calls the `IBatchCompletedNotification` plugin implementation on a process if the process is complete and has no remaining available or checked-out work items.

Procedure

1. In the Guidewire Studio **Project** window, expand **configuration > gsrc**.
2. Do one of the following:
 - If a package for your plugin implementation classes already exists within **gsrc**, navigate to that package, then skip to “step 6”.
 - If a package for your plugin implementation classes does not exist, continue to “step 3”.
3. Right-click **gsrc**, then click **New > Package**.
4. Enter a package name, such as `workqueue`.
5. Right-click your implementation class package and click **New > Gosu Class**.
6. Enter the name `IBatchCompletedNotification` for the gosu class.
7. Click **OK**.
8. Define your Gosu class, using the following framework:

```
package myCompany.plugin.workqueue
uses gw.plugin.workqueue.IBatchCompletedNotification

class IBatchCompletedNotification implements IBatchCompletedNotification {
  construct() { }
  override function completed(batch : ProcessHistory, numFailed : int) {
    //do something
    return
  }
}
```

9. Save your work.

Register a custom batch notification plugin

After you create your `IBatchCompletedNotification` plugin implementation, you need to register the plugin with InsuranceSuite application.

Procedure

1. In the Guidewire Studio **Project** window, expand **configuration > config > Plugins**.
2. Right-click **registry** and click **New > Plugin**.
3. In the **Plugin** dialog, enter `IBatchCompletedNotification` for the name of your plugin.
4. In the **Plugin** dialog, click ...
5. In the **Select Plugin Class** dialog, type `IBatchCompletedNotification` and select the **IBatchCompletedNotification** interface.
6. In the **Plugin** dialog, click **OK**.
Studio creates a GWP file under **Plugins > registry** with the name that you entered.
7. Click the **Add Plugin** icon and select **Add Gosu Plugin**.
8. For **Gosu Class**, enter your class, including the package.
9. Save your changes.

Troubleshooting work queues

It is possible for a work queue worker to encounter a problem that causes it to fail, before the worker completes all the items it checked out from the queue. For example, it is possible for a server to die, killing its workers in the middle of processing. This action can result in orphan work items. A work item becomes an orphan if a worker has an item checked out but does not complete processing the item within an allotted amount of time. The `progressinterval` attribute on the `<worker>` element in `work-queue.xml` defines this time span.

Workers treat orphans just as they do available items. The next worker that encounters the orphan item in the table adopts it for processing and resets the `LastUpdateTime`, `CheckedOutBy`, and `Status` fields on the orphan work item.

If a work queue is experiencing a large number of orphans, review log files to locate timeouts during processing. For example, a timeout can occur while a worker waits for an external server to return a value. If the log contains these type of timeouts, increase the `progressinterval` value for the work queue in `work-queue.xml` to give workers more processing time.

Sometimes, a problem inherent in the item itself causes the processing of the item to fail. For example, a batch process throws an exception. In such cases, the worker stops processing the item and goes on to the next. The item becomes an orphan and the next worker attempts to process it. In this way, a work queue attempts to process each item multiple times up to a limit configured for the work queue. If a work item exceeds the limit of processing attempts, the InsuranceSuite application changes the status of the work item to `failed`. Workers ignore items with a status of `failed` and no longer attempt to process them.

InsuranceSuite application base processes

In the base configuration, InsuranceSuite application provides a number of work queues and batch processes. Guidewire categorizes these work queues and batch processes into the following categories.

MaintenanceOnly

It is only possible to run the process if the application server on which it runs is at the maintenance (NODAEMONS) run level. For all other processes, the server run level must be at the DAEMONS run level or higher for a process to execute.

APIRunnable

It is possible to run the process from the command line using a `system_tools` command option, which calls the web services API to execute the process.

Schedulable

It is possible to schedule this process to run at regular intervals.

UIRunnable

It is possible to start or stop the process from the InsuranceSuite application user interface.

Common base configuration processes

Activity Escalation work queue

Code	ActivityEsc
Categories	UIRunnable, Schedulable
Implementation	Work queue
Workers	0 (ClaimCenter only)
Schedule	Every 30 minutes
Class	ActivityEscalationWorkQueue.java

The Activity Escalation work queue finds activities that meet certain escalation criteria and marks the activity for escalation. The work queue logic looks for activities that meet each of the following criteria:

- The activity has an escalation date.
- The escalation date is prior to today's date.
- The application has not previously escalated the activity.

If the Activity Escalation work queue finds an activity that meets all the criteria, it marks the activity as escalated and calls the activity escalation rules to determine any actions.

If you set your escalation deadline in days, then there is no reason to run activity escalation more than daily. However, if your escalation deadline is shorter, then run this process more frequently to take action on overdue activities in a timely manner. By default, the application runs Activity Escalation work queue every 30 minutes. As indicated, you can change this schedule as needed.

By default, the ClaimCenter base configuration disables the Activity Escalation work queue by setting the number of workers in `work-queue.xml` to 0. You must set the number of workers in this file to 1 or more to be able to run this work queue.

Data Distribution batch process

Code	DataDistribution
Categories	APIRunnable
Implementation	Batch process
Stoppable	No (single phase process)

Schedule	Not schedulable
Class	BCDataDistributionBatchProcess.java (In BillingCenter) CCDataDistributionBatchProcess.java (In ClaimCenter) PCDataDistributionBatchProcess.java (In PolicyCenter)

The Data Distribution batch process generates data on the distribution of various items in the InsuranceSuite application database. It is not possible to schedule this process. You must run this process from the **Data Distribution** screen of the Server Tools **Info Pages** or by using the maintenance_tools command line utility.

As this type of batch process can be very resource intensive, it has the possibility of adversely affecting the performance of the application. Before you run this process in a production environment, Guidewire recommends that you run the process first against a test environment that contains a full copy of production data.

Database Consistency Check work queue

Code	DBConsistencyCheck
Categories	Schedulable
Implementation	Work queue
Non-exclusive	Yes
Schedule	• First Sunday of every sixth month, at 4:00 p.m. (In ClaimCenter) • Not scheduled (In BillingCenter and PolicyCenter)
Schedule	
Class	DBConsistencyCheckWorkQueue.java

The Database Consistency Check work queue runs consistency checks on the InsuranceSuite application database.

Scheduling consistency checks

In the ClaimCenter base configuration, Guidewire provides a default schedule for execution of the the DBConsistencyCheck work queue in file scheduler-config.xml. To modify this schedule, modify the entry for this work queue in file scheduler-config.xml.

In the BillingCenter and PolicyCenter base configuration, Guidewire does not provide a default entry for the DBConsistencyCheck work queue in file scheduler-config.xml. You must add your own entry for this work queue if you want to automatically execute the work queue on a regular schedule. For example, you can add something similar to the following entry to file scheduler-config.xml, adjusting the schedule to meet your business needs.

```
<!-- Run DBConsistencyCheck on the first Sunday of every 6th month at 4PM -->
< ProcessSchedule process="DBConsistencyCheck">
  <CronSchedule month="/6" hours="16" dayofmonth="?" dayofweek="1#1"/>
</ProcessSchedule>
```

Running consistency checks

You use the Server Tools **Info Pages** > **Consistency Checks** screen to launch the database checks from the InsuranceSuite application. From this screen, you can set a number of check parameters, including the following:

- The tables or table groups to check, or, all tables
- The specific types of database checks to run, or, all database check types
- The number of work queue workers to use in running the database checks

You can also view past history reports of database checks from the **Consistency Checks** screen, as long as the historical report still exists in the database.

Running consistency checks manually

If you have the requisite access and permission, you can execute the DBConsistencyCheck work queue manually, using the following `system_tools` command:

```
system_tools -user user -password password -checkdbconsistency ...
```

You can also add optional information on which checks to run against which database tables.

Database Statistics work queue

Code	DBStats
Categories	Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	DBStatisticsWorkItemWorkQueue.java

The Database Statistics work queue generates database statistics about how the InsuranceSuite application and data model interact with the physical database. For example, database statistics store row counts in a table, how a table distributes the data, and much more. A database management system uses statistics to determine query plans to optimize performance.

Development mode

In development mode, it is possible to run the Database Statistics process in any of the following ways:

- From a command prompt, using the `-updatestatistics` option of the `system_tools` command
- From the **Execution History** tab of the Server Tools **Database Statistics** screen
- As a scheduled batch process

Production mode

In production mode, it is possible to run Database Statistics process in the following ways only:

- From a command prompt, using the `-updatestatistics` option of the `system_tools` command.
- As a scheduled batch process

Guidewire recommendation

Guidewire specifically recommends that you collect full statistics in the following circumstances:

- If there are significant changes to data such as after a major upgrade.
- If using the `zone_import` command.
- If using the `table_import -integritycheckandload` command in ClaimCenter or BillingCenter.
- If you are trying to troubleshoot performance problems.

In all other cases, Guidewire recommends that you collect INCREMENTAL database table statistics only.

Deferred Upgrade Tasks batch process

Code	DeferredUpgradeTasks
Categories	APIRunnable
Implementation	Batch process

Stoppable	No (single phase process)
Class	DeferredUpgradeBatchProcess.java

The Deferred Upgrade Tasks batch process creates the nonessential indexes that the InsuranceSuite application uses to improve performance. In addition, the batch process creates indexes on archived entities, if these types of entities exist.

The InsuranceSuite application runs the Deferred Upgrade Tasks process automatically after an upgrade if you set the following attribute on `<upgrade>` in `database-config.xml` to `true`:

```
<upgrade defer-create-nonessential-indexes="true" ...
```

If the DeferredUpgradeTasks process fails, manually run the process again during non-peak hours. To manually run the Deferred Upgrade Tasks process, use the following command:

```
maintenance_tools -server url -password password -startprocess DeferredUpgradeTasks
```

Note: To run this command, you must have permission to create indexes until after the DeferredUpgradeTasks batch process completes.

To check the status of the DeferredUpgradeTasks batch process, review the upgrade logs and the InsuranceSuite application Server Tools **Upgrade and Versions** screen.

[See also](#)

- *Deferring creation of nonessential indexes* in the *Upgrade Guide*

Production mode

Do not go into full production while the Deferred Upgrade Tasks process is still running. The lack of so many performance-related indexes can likely make the InsuranceSuite application unusable.

Until the Deferred Upgrade Tasks process has run to completion, the InsuranceSuite application reports errors during schema validation while starting. These include errors for column-based indexes existing in the data model but not in the physical database and mismatches between the data model and system tables.

Archive-enabled installations in ClaimCenter

If Guidewire enables data archiving, do not use the ClaimCenter archiving functionality until the Deferred Upgrade Tasks batch process completes successfully.

Destroy Contact for Personal Data work queue

Code	DestroyContactForPersonalData
Categories	Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	PersonalDataContactDestructionWorkQueue.gs

This work queue finds all `PersonalDataContactDestructionRequest` objects that have a status set to `New` or `ReRun` (category `ReadyToAttemptDestruction`). How far the destruction process went for the found contacts is determined by the `ContactDestructionStatus` returned from the `Destroyer`, the class that implements the `PersonalDataDestroyer` interface.

The contact destruction status is set to the returned status. If the status is `Completed`, `Partial`, or `NotDestroyed` (category `DestructionStatusFinished`), the date of completion is also populated.

An exception is thrown if return status is `New` or if you try to change the status from a typecode in the `DestructionStatusFinished` category.

By default, Guidewire does not provide an entry for this work queue in file `scheduler-config.xml` in the base configuration. To schedule this work queue, add an entry for it to `scheduler-config.xml`.

Geocode Writer work queue

Code	Geocode
Categories	• <code>UIRunnable</code> • <code>Schedulable</code> • <code>APIRunnable</code> (ClaimCenter only)
Implementation	Work queue
Schedule	Not scheduled
Class	<code>GeocodeWorkQueue.java</code>

The Geocode Writer work queue is the writer for the Geocode process in ClaimCenter and PolicyCenter. This work queue runs periodically to update geocoding information on user contact (`UserContact`) primary addresses and account locations. The `UserContact` entity represents a InsuranceSuite application user.

By default, in PolicyCenter, Guidewire comments out the entry for this work queue in file `scheduler-config.xml`. To schedule the Geocode Writer work queue, remove the comment marks around the entry for it in `scheduler-config.xml`.

Group Exception work queue

Code	GroupException
Categories	<code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Workers	0 (ClaimCenter only)
Schedule	• Not scheduled (In BillingCenter) • Once a day, at 4:00 a.m. (In ClaimCenter and PolicyCenter)
Class	<code>GroupExceptionWorkQueue</code>

The Group Exception work queue runs any defined group exception business rules on all groups in the system.

By default, the ClaimCenter base configuration disables the Group Exception work queue by setting the number of workers in `work-queue.xml` to 0. You must set the number of workers in this file to 1 or more to be able to run this process.

By default, the BillingCenter base configuration does not add an entry for this work queue in file `scheduler-config.xml`. To schedule this work queue, add an entry for it to `scheduler-config.xml`.

Inbound Chunk work queue

Code	InboundChunkWQ
Categories	<code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Not scheduled
Class	<code>InboundChunkWorkQueue.java</code>

InboundChunkWorkQueue is a work queue that processes records within one chunk. A chunk is a part of a record group in a database.

Inbound File batch process

Code	InboundFileBatchProcess
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Not scheduled
Class	InboundFileBatchProcess.java

The Inbound File batch process loads inbound files from the specified directory and creates records in the database. The process splits large groups of records into smaller chunks that it then passes to the InboundChunkWorkQueue work queue for processing.

Inbound File Purge work queue

Code	InboundFilePurgeWQ
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	InboundFilePurgeWorkQueue.java

The Inbound File Purge work queue checks to see if there are any inbound files for which the purge date has elapsed. If there are such files, the work queue removes the files, and their associated records, from the database.

Notify External System for Personal Data work queue

Code	NotifyExternalSystemForPersonalData
Categories	Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	PersonalDataDestructionNotifyExternalSystemsWorkQueue.gs

The Notify External System for Personal Data work queue finds all `PersonalDataDestructionRequest` objects for which both of the following are true:

- The object has a status that matches a typecode in the `DestructionStatusFinished` category.
- The object `RequestersNotified` field is set to false.

The work queue processes any found requests by sending a notification to all associated requesters. It then marks the `RequestersNotified` field as true. If the notification fails, the work queue throws an exception and the `RequestersNotified` field remains false.

A method on the `PersonalDataDestruction` plugin, `notifyExternalSystemsRequestProcessed`, is called by `PersonalDataDestructionNotifyExternalSystemsWorkQueue` to notify external systems when a personal data destruction request is completed. The original `RequestID` is passed to the method, which does nothing by default. You

are expected to implement this method to notify systems of interest. The RequestID is received when the destruction request is originally created through the PersonalDataDestructionAPI web service.

By default, Guidewire does not add an entry for this work queue to file `scheduler-config.xml` in the base configuration. To schedule this work queue, add an entry for it to `scheduler-config.xml`.

Notes

1. The implementation class for Notify External System for Personal Data work queue is `PersonalDataDestructionNotifyExternalSystemsWorkQueue`. In your implementation, you must verify that the notification was successful before marking `RequestersNotified` as `true`.
2. The base configuration includes one of the following implementation classes for the `PersonalDataDestruction` plugin:
 - `PCPersonalDataDestructionPlugin` in `PolicyCenter`
 - `CCPersonalDataDestructionSafePlugin` in `ClaimCenter`
 - `BCPersonalDataDestructionPlugin` in `BillingCenter`.

Outbound File batch process

Code	<code>OutboundFileBatchProcess</code>
Categories	<code>APIRunnable</code> , <code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Not scheduled
Class	<code>OutboundFileBatchProcess.java</code>

The Outbound File batch process loads outbound records from the database. Next, it creates outbound files and saves them to the specified location.

Outbound File Purge work queue

Code	<code>OutboundFilePurgeWQ</code>
Categories	<code>APIRunnable</code> , <code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Not scheduled
Class	<code>OutboundFilePurgeWorkQueue.java</code>

`OutboundFilePurgeWorkQueue` is a work queue that checks to see if there are any outbound files for which the purge date has elapsed. If there are such files, the work queue removes those files, and their associated records, from the database.

Outbound Record Purge work queue

Code	<code>OutboundRecordPurgeWQ</code>
Categories	<code>APIRunnable</code> , <code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Not scheduled

Class	OutboundRecordPurgeWorkQueue.java
--------------	-----------------------------------

OutboundRecordPurgeWorkQueue is a work queue that checks if there are any outbound records for which the purge date has elapsed and that have the SKIPPED status. If there are such records, the work queue removes them from the database.

Phone Number Normalizer work queue

Code	PhoneNumberNormalizer
Categories	UIRunnable
Implementation	Work queue
Workers	0 (BillingCenter only)
Class	CompactPhoneNormalizerWorkQueue.java

IMPORTANT: Run the Phone Number Normalizer work queue once only, after upgrading from earlier application versions to 8.0.0+. Disable the Phone Number Normalizer work queue in a production environment.

The Phone Number Normalizer work queue calls the registered plugin that implements the `IPhoneNumberNormalizer` interface. Use the Phone Number work queue to normalize phone numbers after upgrading from earlier versions of InsuranceSuite application to 8.0.0+.

By default, the BillingCenter base configuration disables the Phone Number Normalizer work queue by setting the number of workers in `work-queue.xml` to 0. You must set the number of workers in this file to 1 or more to be able to run this process.

Guidewire recommends that you use a substantial number of workers with the Phone Number Normalizer work queue. Using a small number of workers to normalize the phone numbers in a large database can take a very long time. The optimal number of workers to use varies according to the available hardware and the volume of the data involved. It is also possible to allocate workers to several different InsuranceSuite application servers rather than simply increasing the number of workers on a single server.

In BillingCenter, disable this work queue after the process completes normalizing all old phone numbers by resetting the number of workers in `work-queue.xml` back to 0. You never need run Phone Number Normalizer work queue more than once, after an upgrade to 8.0.0+.

In ClaimCenter and PolicyCenter, disable this work queue after the process completes normalizing all old phone numbers by setting the number of workers in `work-queue.xml` to 0. You never need run Phone Number Normalizer work queue more than once, after an upgrade to 8.0.0+.

Populate Search Columns batch process

Code	PopulateSearchColumns
Categories	APIRunnable
Implementation	Batch process
Stoppable	Yes (multi-phase process)
Schedule	Not schedulable
Class	PopulateSearchColumnBatchProcess.java

In ClaimCenter and PolicyCenter, the Populate Search Columns batch process populates denormalized searchColumn columns from their designated sourceColumn columns.

This process is only available from the `maintenance_tools` command or from a web service.

Process Completion Monitor batch process

Code	ProcessCompletionMonitor
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	• Every 5 minutes (InBillingCenter) • Not scheduled (In ClaimCenter and PolicyCenter)
Class	ProcessCompletionMonitor.java

The Process Completion Monitor batch process runs at schedulable intervals and examines the ProcessHistory table for all completed work queues and batch processes.

For each completed work queue that it finds, Process Completion Monitor:

- Determines if all the work items in that work queue have either completed or failed.
- Calls the IBatchCompletedNotification plugin implementation on a process if the process is complete and has no remaining available or checked-out work items.
- Sets ProcessHistory.NOTIFICATIONSENT to true, to prevent the process from invoking the IBatchCompletedNotification plugin implementation more than a single time for any given process.

The IBatchCompletedNotification interface has a completed method that you can override to perform specific actions after a work queue or batch process completes a batch of work.

By default, the ClaimCenter and PolicyCenter base configuration does not add an entry for this process to scheduler-config.xml. To schedule this process, add an entry for it to scheduler-config.xml.

Process History Purge batch process

Code	ProcessHistoryPurge
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	On the third day of the month, at 1:30 a.m.
Class	ProcessHistoryPurge.gs

The Process History Purge process purges batch process history data from the ProcessHistory table. It is important to periodically delete process history data. A large number of history records in the database can slow performance during use of the Server Tools **Batch Process Info** or **Work Queue Info** screens.

This process uses Gosu class ProcessHistoryPurge to read the value of the BatchProcessHistoryPurgeDaysOld parameter in config.xml. The process then uses this value to determine the history data to purge.

Note: The InsuranceSuite application also uses configuration parameter BatchProcessHistoryPurgeDaysOld to determine how many days to retain process history records, which the separate “Work Item Set Purge batch process” removes.

Purge Cluster Members batch process

Code	PurgeClusterMembers
-------------	---------------------

Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	On the first day of each month, at 2:00 a.m.
Class	PurgeClusterMembers.gs

The Purge Cluster Members batch process purges `ClusterMemberData` entities. This process uses Gosu class `PurgeClusterMembers` to read the value of the `ClusterMemberPurgeDaysOld` parameter in `config.xml`. The process then uses this value to determine to purge the `ClusterMemberData` entities that have a `LastUpdate` value prior to the current date minus the value of the `ClusterMemberPurgeDaysOld`.

Purge Failed Work Items batch process

Code	PurgeFailedWorkItems
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	• Every Saturday, at 1:00 a.m. (InBillingCenter) • On the first day of the month, at 1:00 a.m. (In ClaimCenter and PolicyCenter)
Class	PurgeFailedWorkItems.gs

The Purge Failed Work Items batch process purges failed work items from all existing work queues. This process uses Gosu class `PurgeFailedWorkItems` to determine which work items to delete. The `doWork` method of the `PurgeFailedWorkItems` class iterates across all work queues and deletes failed work items as it finds them.

During a scheduled execution of the batch process, the process deletes failed work items that are older than the last run date of the batch process. It then sets the last run date to the current date. Thus, if the scheduled execution of this batch process is monthly, the process deletes work items that are older than a month only. If the scheduled execution of this batch process is weekly, the process deletes work items that are older than a week only.

If you run this process manually and there are work items that are newer than the last run date, the batch process does not delete them. If you then run the batch process a second time on the same day, the process deletes work items that are older than the current date. This is the expected behavior.

Purge Message History batch process

Code	PurgeMessageHistory
Categories	• UIRunnable • Schedulable • APIRunnable (ClaimCenter only)
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	• Every night, at 4:20 a.m. (In BillingCenter) • Each Sunday, at 4:30 a.m. (In ClaimCenter) • On the 20th of the month, at 1:00 a.m. (In PolicyCenter)

Class	PurgeMessageHistory.gs
--------------	------------------------

The Purge Message History batch process purges old messages from the message history table. The `KeepCompletedMessagesForDays` parameter in `config.xml` specifies how many days a message can remain in the message history table before the Purge Message History process removes the message. The default value of `KeepCompletedMessagesForDays` is 90 days.

The `doWork` method on the `PurgeMessageHistory` class calls the `MessagingUtil.removeMessageHistory` method to delete message history that is older than the current date minus the value of `KeepCompletedMessagesForDays`.

Purge Old Contact Destruction Request work queue

Code	RemoveOldContactDestructionRequest
Categories	Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	RemoveOldContactDestructionRequestWorkQueue.gs

This work queue finds all `PersonalDataDestructionRequest`, `PersonalDataContactDestructionRequest`, and `PersonalDataDestructionRequester` objects that have the following values:

- The value of `RequestersNotified` is true.
- The value of the `PersonalDataContactDestructionRequest.DestructionDate` property plus the value of the `ContactDestructionRequestAgeForPurgingResults` configuration parameter is less than or equal to today's date.

The InsuranceSuite application removes each request that it finds for which the value of `AllRequestsFulfilled` is true.

By default, the `BillingCenter` and `ClaimCenter` base configuration does not provide an entry for this work queue in `scheduler-config.xml`. By default, the `PolicyCenter` base configuration comments out the entry for this work queue in `scheduler-config.xml`. To schedule this work queue, remove the comment marks from its entry in `scheduler-config.xml`.

Purge Old Transaction IDs batch process

Code	PurgeTransactionIDs
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Not scheduled
Class	PurgeTransactionIds.gs

The Purge Old Transaction IDs batch process deletes SOAP header transaction IDs generated by systems external to InsuranceSuite application. This process uses Gosu class `PurgeTransactionIds` to read the value of the `TransactionIdPurgeDaysOld` parameter in `config.xml`. The process then purges transaction IDs that have a creation date prior to the current date minus the value of the `TransactionIdPurgeDaysOld` parameter.

Guidewire does not schedule this batch process in the base configuration as the table that stores the transaction IDs takes very little space in the database. Unless there is a constant buildup of these transaction IDs, there is no real need to continually purge this data. In fact, if you do purge this data, it is then not possible to determine if a new transaction

is a duplicate of a transaction sent by the external system at an earlier date. There are other alternatives to purging this data. For example, you can partition the table by date.

By default, Guidewire does not provide an entry for this process in file `scheduler-config.xml` in the base configuration. To schedule this process, add an entry for it to `scheduler-config.xml`.

Purge Profiler Data batch process

Code	PurgeProfilerData
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	Yes (multi-phase process)
Schedule	• Not scheduled (In BillingCenter and PolicyCenter • Once a day, at 2:00 a.m. (In ClaimCenter)
Class	ProfilerDataPurgeBatchProcess.java

The Purge Profiler Data batch process purges profiler data at regularly specified intervals. This process uses the read-only `ProfilerDataPurgeBatchProcess` class to read the value of the `ProfilerDataPurgeDaysOld` parameter in `config.xml`. The process then uses the value of this parameter to determine how many days to retain profiler data before Purge Profiler Data batch process removes it.

Purge Workflows batch process

Code	PurgeWorkflows
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	On the first day of the month, at 1:30 a.m. And, in PolicyCenter, on the 10th of the month, at 1:00 a.m.
Class	PurgeWorkflows.gs

The Purge Workflows batch process deletes any completed workflow older than the specified number of days. The purge process deletes the following items:

- All completed workflows older than the specified date
- Any log messages associated with the deleted workflows
- The workflow that performs the purging itself

Note: Deletion of the workflow is permanent.

This process uses Gosu class `PurgeWorkflow` to read the value of configuration parameter `WorkflowPurgeDaysOld` in `config.xml`. The process then uses this value to determine the number of days to retain workflow data before purging it.

Purging unnecessary activity data

Each time the InsuranceSuite application creates an activity, it adds the activity to the following database tables:

- `xx_Workflow`
- `xx_WorkflowLog`
- `xx_WorkflowWorkItem`

In the database table name, `xx` is the application product code, such as `pc`, `bc`, or `cc`.

After a user completes the activity, the InsuranceSuite application sets the workflow status to completed. Thereafter, the application never uses the table entries for the activity ever again. These tables grow in size over time and can adversely affect performance as well as waste disk space. Excessive records in these tables also negatively impact the performance of the database upgrade.

Use the Purge Workflows batch process to purge older, completed, workflows and their logs. Purging workflows, workflow log entries, and workflow items for completed activities does the following:

- It recovers disk space.
- It improves database upgrade.
- It improves operational performance.
- It reduces the number of unused workflow log records.

Guidewire recommends that you purge the completed workflows and their logs periodically.

Alternatively, you can purge only the logs associated with completed workflows by running the Purge Workflow Logs batch process. The Purge Workflow Logs batch process ignores the workflow records and removes only the workflow log records.

Purge Workflow Logs batch process

Code	PurgeWorkflowLogs
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	On the first day of the month, at 2:30 a.m.
Class	PurgeWorkflowLogs.gs

The Purge Workflow Logs batch process purges completed workflows logs. This process uses Gosu class `PurgeWorkflowLogs` to read the value of `WorkflowLogPurgeDaysOld` parameter in `config.xml`. The process then uses this value to determine the number of days to retain workflow logs before purging them.

Solr Data Import batch process

Code	SolrDataImport
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Not scheduled
Class	SolrDataImportBatchProcess.gs

In ClaimCenter and PolicyCenter, the Solr Data Import batch process tests the operation of the free-text batch load command, especially its embedded SQL query. A batch process is limited to using memory available to the application server, which can deprive other processes of that memory on a production system. This batch process also requires write access to the local file system, which is typically not available on a production system. Only run the Solr Data Import process on development-mode servers.

IMPORTANT: Do not run this process in production to load and re-index the Guidewire Solr Extension. Instead, run the free-text batch load command (`batchload`) on the host on which the Guidewire Solr Extension resides.

IMPORTANT: In a development or production environment that use ClaimCenter or PolicyCenter on Guidewire Cloud, you run the free-text batch load command in Guidewire Home.

User Exception work queue

Code	UserException
Categories	• APIRunnable (ClaimCenter only) • UIRunnable • Schedulable
Implementation	Work queue
Workers	0 (ClaimCenter only)
Schedule	• Not scheduled (In BillingCenter) • Once a day, at 3:00 a.m. (In ClaimCenter and PolicyCenter)
Class	UserExceptionWorkQueue.java

The User Exception work queue runs the user exception rule set on all users in the system.

By default, ClaimCenter disables the User Exception work queue in base configuration by setting the number of workers in `work-queue.xml` to 0. You must set the number of workers in this file to 1 or more to be able to run this process.

Note: BillingCenter does not provide User Exception sample rules in the base configuration.

Work Item Set Purge batch process

Code	WorkItemSetPurge
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	On the second day of the month, at 1:30 a.m.
Class	WorkItemSetPurge.gs

The Work Item Set Purge batch process purges work item sets from the database. This process uses Gosu class `WorkItemSetPurge` to read the value of the `BatchProcessHistoryPurgeDaysOld` parameter in `config.xml`. The process then uses this value to determine the number of days to retain work item sets before purging them.

Note: The `BatchProcessHistoryPurgeDaysOld` parameter also configures how many days to retain process history records, which the separate Process History Purge batch process removes.

Work Queue Instrumentation Purge batch process

Code	WorkQueueInstrumentationPurge
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)

Schedule	On the second day of the month, at 2:30 a.m.
Class	WorkQueueInstrumentationPurge.gs

The Work Queue Instrumentation Purge batch process purges instrumentation data for work queues. This process uses Gosu class `WorkQueueInstrumentationPurge` to read the value of the `InstrumentedWorkerInfoPurgeDaysOld` parameter in `config.xml`. The process then uses this value to determine how long to retain work queue instrumentation data. The default value of `InstrumentedWorkerInfoPurgeDaysOld` is 45 (business) days.

Workflow work queue

Code	Workflow
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	• Every 10 minutes, starting at 5 minutes after the hour (5, 15, 25, 35, 45, 55). (In ClaimCenter) • Every 10 minutes, starting on the hour (0, 10, 20, 30, 40, 50). (In PolicyCenter)
Class	WorkflowDistributedWorkQueue.java

The Workflow work queue runs ClaimCenter and PolicyCenter workflow tasks. Workflow cannot advance any faster in the background than the work queue schedule. To increase the speed of the workflow, change the schedule of this work queue.

PolicyCenter base configuration processes

Account Holder Count work queue

Code	AccountHolderCount
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	AccountHolderCountWorkQueue.java

On Contact objects, the `AccountHolderCount` values sometimes are incorrect. In PolicyCenter, the Account Holder Count work queue finds Contact objects with incorrect `AccountHolderCount` values, and updates the value. If the value of `AccountHolderCount` is set incorrectly, it can affect search performance.

Based on the number of contacts that need updating, decide whether to run Account Holder Count manually or automatically as a scheduled batch process.

- If the number of contacts to update is relatively small and does not change often, consider running it manually. Periodically check the data distribution in case something later causes `Contact.AccountHolderCount` fields to be incorrect.
- If the number of contacts to update is large, or if contacts regularly have incorrect `AccountHolderCount` values, consider scheduling Account Holder Contact to run on a regular basis.

By default, PolicyCenter does not provide an entry for this work queue in file `scheduler-config.xml` in the base configuration. To schedule this work queue, add an entry for it to `scheduler-config.xml`.

Determine the number of incorrect AccountHolderCount contacts

About this task

In the base configuration, PolicyCenter does not schedule the Account Holder Count process. To decide whether to schedule the process or run it manually, do the following:

Procedure

1. Log into PolicyCenter using an administrative account.
2. Press ALT+SHIFT+T to open the Server Tools screen.
3. Navigate to **Info Screens > Data Distribution**.
4. In the **Collect distributions for all tables** field, select **Specify tables**.
5. Add the `pc_contact` table.
6. Click **Submit Data Distribution Batch Job**.
The `pc_contact` data distribution table reports the *Number of Contacts with incorrect AccountHolderCount field*.

Account Withdraw Evaluation work queue

Code	AccountWithdraw
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 2 a.m.
Class	AccountWithdrawWorkQueue.gs

In PolicyCenter, the Account Withdraw Evaluation work queue marks the account status as withdrawn (`Withdrawn`) if:

- There are no policies associated with the account.
- The `Account.CreateTime` or `Account.OriginationDate` is older than a configurable number of months in the past. The `AccountsWithdrawnAfterMonths` parameter in `config.xml` specifies the number of months. In the base configuration, this parameter is set to 37 months.
- There are no open activities associated with the account.

Apply Pending Account Data Updates work queue

Code	ApplyPendingAccountDataUpdates
Categories	UIRunnable, Schedulable
Implementation	Work queue
Stoppable	No (single phase process)
Schedule	Once a day, at 12:10 a.m.
Class	ApplyPendingAccountDataUpdatesWorkQueue.gs

In PolicyCenter, the Apply Pending Account Data batch process synchronizes changes made during future-dated jobs with the backing account information. For examples, suppose that a customer updates their personal auto policy to show that their marital status changes on some future date. This process applies that change to the corresponding account on the edit-effective date of the policy change job.

Asynchronous Quoting work queue

Code	AsyncQuoting
Categories	None
Implementation	Work queue
Class	AsyncQuotingWorkQueue.gs

The Asynchronous Quoting work queue rates and quotes the current policy period asynchronously. If two-step quoting is also enabled for a line of business, then this work queue quotes only.

PolicyCenter creates an asynchronous quoting work item (AsyncQuotingWorkItem) when a user starts an asynchronous quote. This work queue processes these work items. AsyncQuotingWorkItem is an entity that delegates to WorkItem. This work item has properties for the following:

RequestingUser

The user that requested the quote

PolicyPeriod

The policy period to quote

In work-queue.xml, the default progress interval for asynchronous quoting work queue is 600,000 milliseconds (10 minutes). Set this value to an interval longer than the expected time for an asynchronous quote to complete.

Handling quoting issues

If the asynchronous quote generates issues, most of these issues appear in the **Validation Results** tab next to the **Activity** tab. This tab displays both product model synchronization issues (ProductModelSyncIssue) and validation issues (ValidationIssue). Other more serious issues, such as rating or unexpected errors, appear on the policy transaction screen.

The AsyncQuotingWorkQueue.processWorkItem method processes asynchronous quoting work items. If exceptions occur while processing work items, this method creates one or more issues, AsyncQuoteIssue entity instances, for each exception. The MessageType property contains the issue type, defined in the AsyncQuoteIssueType typelist.

The processWorkItem method catches all exceptions thrown by the call to JobProcess.requestQuote. For each exception, PolicyCenter displays messages:

Exception	Message appears on...	Description
QuoteHaltedException	Validation Results tab	
EntityValidationException	Validation Results tab	This exception can contain multiple messages. The method creates one AsyncQuoteIssue entity instance for each message.
DisplayableException	Policy transaction screen	Displayable exceptions report failure such as out-of-sequence conflicts or rating failure. When you click to display the asynchronous quoting activity, PolicyCenter displays this exception on the policy transaction screen.
Unexpected throwables	Policy transaction screen	Are transferred to DisplayableException and appear on the policy transaction screen. Unlike the expected exceptions, after catching the throwable and creating the corresponding AsyncQuoteIssue entity, the work queue retries the work item. If the

Exception	Message appears on...	Description
		throwable still occurs during the last retry, the AsyncQuotingWorkQueue#workItemFailed method creates a failure activity.
UWAuthorityBlocksProgressException	Policy transaction screen with a link to the Risk Analysis screen	Normally, no message is created because the exception creates an underwriting issue (UWIssue) for the exception. In the unlikely event that an underwriting issue was not created, processWorkItem generates an AsyncQuoteIssue instance.

AsyncQuotingWorkItem entity

The asynchronous quoting work queue processes AsyncQuotingWorkItem work items. AsyncQuotingWorkItem is an entity that delegates to WorkItem. This work item has properties for the following:

RequestingUser

The user that requested the quote

PolicyPeriod

The policy period to quote

AsyncQuoteIssue entity

The asynchronous quoting work queue generates messages when something notable occurs while processing the quote. Each AsyncQuoteIssue entity instance corresponds to a message. Messages can be informational, warnings, errors, or exceptions. Some of the properties of this entity are:

PolicyPeriod

Foreign key to the policy period being quoted.

IssueType

Issue type for messages generated by asynchronous quoting. Values are defined in the AsyncQuoteIssueType typelist.

Description

Description of the issue.

IssueSequence

Indicates the order in which the issue was created.

Asynchronous Rating work queue

Code	AsyncRating
Categories	None
Implementation	Work queue
Class	AsyncRatingWorkQueue.gs

In PolicyCenter, the Asynchronous Rating work queue rates the current policy period asynchronously. If both asynchronous quoting and two-step quoting are enabled for a line of business, PolicyCenter creates an asynchronous rating work item (AsyncRatingWorkItem) when a user requests an asynchronous rate from PolicyCenter. This work queue processes AsyncRatingWorkItem work items. AsyncRatingWorkItem is an entity that delegates to WorkItem. This work item has properties for the following:

RequestingUser

The user that requested rating

PolicyPeriod

The policy period to rate

Audit Task work queue

Code	AuditTask
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 5:00 a.m.
Class	AuditTaskMonitorWorkQueue.java

In PolicyCenter, the Audit Task work queue starts scheduled audits. See the *Application Guide* for information on audits.

Bound Policy Exception work queue

Code	BoundPolicyException
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	BoundPolicyExceptionWorkQueue.java

In PolicyCenter, the Bound Policy Exception work queue runs policy exception rules on every bound PolicyPeriod that has not had exception rules run on it for more than a defined number of days.

Parameter BoundPolicyThresholdDays in config.xml defines the minimum number of days that must pass before PolicyCenter runs the Exception rules again on a bound PolicyPeriod. To be eligible for processing, the PolicyPeriod must be either in force or expired within the last year.

By default, PolicyCenter comments out the entry for this work queue in file scheduler-config.xml in the base configuration. To schedule this work queue, remove the comments marks from around its entry in scheduler-config.xml.

Bulk Submission Job batch process

Code	BulkSubmission
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single-phase process)
Schedule	Not scheduled
Class	BulkSubmission.gs

Bulk Submission Job batch processing is part of Smart Communications for PolicyCenter. Bulk Submission Job batch processing enables creation of documents asynchronously using SmartCOMM templates using the SmartCOMM Bulk API. In the base configuration, Bulk Submission Job passes the location of the files to SmartCOMM. SmartCOMM processes the files and creates the documents.

By default, PolicyCenter does not schedule this process in file scheduler-config.xml in the base configuration. To schedule this process, add an entry for it to scheduler-config.xml.

Clean Up Account Contact Role Table work queue

Code	CleanupAccountContactRole
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	CleanupAccountContactRoleWorkQueue.gs

In PolicyCenter, the Clean Up Account Contact Role Table work queue deletes retired AccountContactRole entity instances and any associated AccountContactRoleReplacement entity instances. The work queue marks AccountContactRole entity instances that are no longer in use as Retired.

By default, PolicyCenter comments out the entry for this work queue in file scheduler-config.xml in the base configuration. To schedule this work queue, remove the comment marks from around its entry in scheduler-config.xml.

Clean Up PurgedRootInfo work queue

Code	CleanupPurgedRootInfo
Categories	Schedulable, UIRunnable
Implementation	Work queue
Schedule	Not scheduled
Class	PurgeOldPurgedRootInfoWorkQueue.gs

When purging a Contact, Account, Policy, PolicyTerm, or PolicyPeriod, PolicyCenter adds a PersonalDataPurge event and adds a corresponding PurgedRootInfo entity. Guidewire DataHub can then extract the data from PurgedRootInfo.

The CleanupPurgedRootInfo work queue removes PurgedRootInfo entities whose PurgeType is PersonalData. For the work queue to consider an entity for clean up, the entity must have a purge date older than the number of days specified in the configuration parameter KeepPurgedRootForDays. The default value of this parameter is 180 days.

By default, PolicyCenter comments out the entry for this work queue in file scheduler-config.xml in the base configuration. To schedule this work queue, remove the comment marks from around its entry in scheduler-config.xml.

Clear Policy Renewal Check Dates batch process

Code	PolicyRenewalClearCheckDate
Categories	APIRunnable, MaintenanceOnly
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Not schedulable
Class	PolicyRenewalClearCheckDate.gs

In PolicyCenter, the Clear Policy Renewal Check Dates batch process clears (null out) the existing check date for all policies through a single direct database update statement. Because this is a direct update statement, this batch process is only available in maintenance mode. In other words, the server must be a run level of NO_DAEMONS or lower. Contact Guidewire Support to enable maintenance mode.

Only run this process if a configuration change to automated renewal contains a possible risk that the automated renewal process picks up some policies unacceptably late for renewal. Ultimately, the implementation of the `PolicyRenewalPlugin` plugin controls the lead time of any individual policy needs for renewal. By default, this process depends on the code of that plugin and the `NotificationConfig` system table accessed through the `NotificationConfigPlugin` plugin implementation. As you can overwrite the functionality of either plugin, which types of changes might significantly change the renewal start date can vary.

This process is not available from the PolicyCenter interface. It is also not possible to schedule this batch process. Instead, you must start this process using the `maintenance_tools -startprocess` command option with `PolicyRenewalClearCheckDate` or its equivalent web service command.

Run the Clear Policy Renewal Check Dates process

Run the Clear Policy Renewal Check Dates process to clear (null out) the existing dates for all policies through a single, direct, database update.

Before you begin

Only run this process if one or more of the following is true:

- If a configuration change to automated renewal contains a possible risk that the automated renewal process picks up some policies unacceptably late for renewal.
- After you bring the server back up after applying a configuration update.

Note: Contact Guidewire Support to enable maintenance mode.

Procedure

1. Place the PolicyCenter server in maintenance mode (`NO_DAEMONS` or lower).
2. Run the following `maintenance_tools` command a single time only.

```
maintenance_tools -password password -startprocess PolicyRenewalClearCheckDate
```

3. After the process completes, finish bringing the server all the way up to production mode.

Closed Policy Exception work queue

Code	<code>ClosedPolicyException</code>
Categories	<code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Not scheduled
Class	<code>ClosedPolicyExceptionWorkQueue.java</code>

In PolicyCenter, the Closed Policy Exception work queue runs policy exception rules on every closed `PolicyPeriod` that has not had exception rules run on it for a defined number of days.

Parameter `ClosedPolicyThresholdDays` in `config.xml` defines the minimum number of days that must pass before PolicyCenter runs the Exception rules again on a closed `PolicyPeriod`.

Create UW Rules for UW Issue Types work queue

Code	<code>CreateUnmappedUWRules</code>
Categories	<code>UIRunnable</code>
Implementation	Work queue

Class	CreateUnmappedUWRulesWorkQueue
--------------	--------------------------------

After a PolicyCenter upgrade, the Create UW Rules for the UW IssueTypes work queue runs at server start. For each UWIssueType object does not have a corresponding UWRule object, this process does the following:

- It creates a new UWRule object.
- It links the new UWRule object with its corresponding UWIssueType object.
- It marks the associated business rule as being externally managed.

Extract Rating Worksheets work queue

Code	ExtractWorkSheets
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	WorksheetExtractWorkQueue.java

In PolicyCenter, the Extract Rating Worksheets work queue extracts rating worksheet data from worksheet container (WorksheetContainer) objects to files in a specified directory on the server with the batch role. The process also marks WorksheetContainer objects for purging.

This process calls the implementation of the WorksheetExtractPlugin plugin, which sets the target directory.

By default, PolicyCenter does not provide an entry for this work queue in the scheduler-config.xml file in the base configuration. To schedule this work queue, add an entry for it to scheduler-config.xml.

Fix Job Group On Moved Policies work queue

Code	FixJobGroupOnMovedPolicies
Categories	UIRunnable
Implementation	Work queue
Schedule	Not schedulable
Class	FixJobGroupOnMovedPoliciesWorkQueue.gs

The Fix Job Group on Moved Policies work queue assigns the correct JobGroup category to Job objects associated with policies that have been moved between accounts. The correct JobGroup is also necessary during data destruction, purging accounts, purging policies, and other actions.

Form Text Data Delete batch process

Code	FormTextDataDelete
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Once a day, at 6:00 a.m.
Class	FormTextDataDeleteBatchProcess.java

In PolicyCenter, the Form Text Data Delete batch process deletes orphaned, purged, or archived FormTextData entities. A FormTextData object stores the text data that makes up the XML data for a Form.

Handle Unresolved Contingency work queue

Code	HandleUnresolvedContingency
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Every day at 2:00 a.m.
Class	HandleUnresolvedContingencyWorkQueue.gs

In PolicyCenter, the Handle Unresolved Contingency work queue initiates action on pending contingencies with an action start date of today or a date in the past for contingencies on which action has not yet started. The contingency Status is changed to Action_Initiated.

Adding contingency actions

You can also add a new contingency action by doing the following in PolicyCenter Studio:

- Modify Gosu class HandleUnresolvedContingencyWorkQueue to support the new contingency action.
- Add a typecode to the ContingencyAction type list for the new action.
- Add a start date for the new action to the updateActionStartDate method in Gosu enhancement ContingencyEnhancement.gsx.
- Add a property getter for the new job using isPolicyChangeAction as a model.

Impact Testing Export work queue

Code	ImpactTestingExport
Categories	UIRunnable
Implementation	Work queue
Schedule	Not schedulable
Class	ImpactTestingExportWorkQueue.java

In PolicyCenter, the Impact Testing Export work queue exports test periods to an Excel file if you click **Create Excel Export File** on the **Impact Results** screen.

It is only possible to run the Impact Testing Export work queue from the Server Tools **Batch Process Info** screen.

Impact Testing Test Case Preparation work queue

Code	ImpactTestingTestPrep
Categories	UIRunnable
Implementation	Work queue
Schedule	Not schedulable
Class	ImpactTestingTestPrepWorkQueue.java

In PolicyCenter, the Impact Testing Test Case Preparation work queue generates baseline policy periods on the selected policies if you click **Create Baselines** from the **Create Baseline** screen.

It is only possible to run the Impact Testing Test Case Preparation work queue from the Server Tools **Batch Process Info** screen.

Impact Testing Test Case Run work queue

Code	ImpactTestingTestRun
Categories	UIRunnable
Implementation	Work queue
Schedule	Not schedulable
Class	ImpactTestingTestRunWorkQueue.java

The Impact Testing Test Case Run work queue generates test policy periods rated using the selected rate books if you click **Quote Test Periods** from the **Testing Periods** screen.

It is only possible to run this process from the Server Tools **Batch Process Info** screen.

Job Expire work queue

Code	JobExpire
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 6:00 a.m.
Class	JobExpirationWorkQueue.java

In PolicyCenter, the Job Expire work queue expires a job if the job has had no action taken on it for a configurable period of time. This process changes the job status from New, Draft, or Quote status to Expired. In the default configuration, the process expires submissions in these statuses that are at least seven days past the effective date of the policy.

To configure the expiration threshold as the number of days past the effective date or the creation date, modify the corresponding configuration parameter in `config.xml`:

- `JobExpirationEffDateThreshold`
- `JobExpirationCreateDateThreshold`

Enable expiration for a specific job type

About this task

The Job Expire process examines all jobs meeting the date criteria. It then expires those jobs for which `job.canExpireJob` returns true.

Procedure

1. In the PolicyCenter, Studio **Project** window, expand **configuration > config**:
 - a) Open `config.xml`.
 - b) Search for `JobExpireCheck<JobType>`, for example, `JobExpireCheckAudit`.
 - c) Change the value from false to true.
2. (Audit only) In the **Project** window, expand **configuration > gwrc > gw > job**:
 - a) Open `AuditProcess.gs`.

- b) Modify the `canExpireJob` method to return true instead of false.

Often, business requirements do not permit the expiration of Audit jobs. To expire the audit job type, you need override the `AuditProcess.canExpireJob` method.

Results

Forcing the expiration of all jobs for which expiration is possible improves the performance of queries related to jobs.

Make util uppercase work queue

Code	<code>MakeUpperCaseUtilInCodeExpressionFragments</code>
Categories	<code>UIRunnable</code>
Implementation	Work queue
Schedule	Not scheduled
Class	<code>MakeUpperCaseUtilInExpressionFragmentsWorkQueue.java</code>

Run this work queue after upgrading from PolicyCenter 9 to PolicyCenter 10. In PolicyCenter 10.0.0, Guidewire removed the following line that specified that util was lower case in business rule definitions from file `gsrc/gw/bizrules/provisioning/contexts/AbstractUWRuleContextDefinition.gs`:

```
public final static var PARAM_LIB: String = "util"
```

As a result of this change, after upgrading from PolicyCenter 9 to PolicyCenter 10, PolicyCenter marks business rules using the older lower case util as invalid. This work queue prevents that problem by converting util to Util in `CodeExpressionFragment` and `GosuTemplateExpressionFragment` instances associated with business rules stored in the database.

See also

- *Upgrade Guide*

Open Policy Exception work queue

Code	<code>OpenPolicyException</code>
Categories	<code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Not scheduled
Class	<code>OpenPolicyExceptionWorkQueue.java</code>

in PolicyCenter, the Open Policy Exception work queue runs policy exception rules on every open, unlocked `PolicyPeriod` that has not had exception rules run on it for a definable number of days.

Parameter `OpenPolicyThresholdDays` in `config.xml` defines the minimum number of days that must pass before PolicyCenter runs the Exception rules again on an open and unlocked `PolicyPeriod`.

In the base configuration, PolicyCenter comments out the entry for the Open Policy Exception work queue in file `scheduler-config.xml`. To schedule this work queue for execution, remove the comment marks from around the entry for it in `scheduler-config.xml` and set the schedule appropriately for your business needs.

Overdue Premium Report work queue

Code	<code>OverduePremiumReport</code>
-------------	-----------------------------------

Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 4:30 a.m.
Class	OverduePremiumReportWorkQueue.java

In PolicyCenter, the Overdue Premium Report work queue runs premium report escalation rules on overdue premium reports.

Policy Hold Job Evaluation work queue

Code	PolicyHoldJobEval
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 2:00 a.m.
Class	PolicyHoldJobEvalWorkQueue.java

In PolicyCenter, the Policy Hold Job Evaluation work queue evaluates each job against the policy holds blocking it. The process finds policy hold jobs that have the following characteristics:

- Jobs that are open
- Jobs that have policy periods with an active blocking hold
- Jobs that have not been evaluated since the time the policy hold changed

The PolicyHoldJobEvalPlugin plugin implementation determines the actions to take on each job found.

Policy Locations Risk Assessment Temporary Store work queue

Code	PolicyLocationsRiskAssessment
Categories	APIRunnable
Implementation	Work queue
Schedule	Not scheduled
Class	PolicyLocationsRiskAssessmentWorkQueue.gs

In PolicyCenter, the Policy Locations Risk Assessment Temporary Store work queue starts risk assessments for all policy locations associated with a policy period. This work queue sends to the user requesting risk assessment an activity indicating whether the risk assessment succeeded or failed.

In the base configuration, PolicyCenter calls this process if you select **Update Risk Evaluations** on the **Risk Analysis** screen in a commercial property policy transaction.

This process retrieves risk assessment results from the Guidewire Spotlight risk assessment service for each location on a given PolicyPeriod and creates LocationRiskAssessment objects. Because PolicyCenter sets an effective date on each LocationRiskAssessment entity that it creates, it is only possible to run this process on an unlocked PolicyPeriod object.

If processing fails, the process does the following:

- It rolls back the entire transaction and does not persist any of the risk assessment results.
- It creates an activity and sends the activity to the list of configured user roles on the current job. You can configure the list of roles in the SpotlightNotificationActivityCreator class.

- It sends a notification activity to the requesting user, if the work item includes a reference to that user.

If processing succeeds, it does the following:

- It commits the transaction and persists the risk assessment results on the `PolicyPeriod`.
- It creates an activity and sends the activity to the list of configured user roles on the current job.
- It sends a notification activity to the requesting user, if the work item includes a reference to that user.

Policy Renewal Start work queue

Code	<code>PolicyRenewalStart</code>
Categories	<code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Once a day, at 1:00 a.m.
Class	<code>PolicyRenewalStartWorkQueue.java</code>

In PolicyCenter, the Policy Renewal Start work queue starts renewal processing for policies set to expire. The process determines the date on which to start the renewal process by subtracting the value of configuration parameter `RenewalProcessLeadTime` in `config.xml` from the policy expiration date.

Premium Ceding work queue

Code	<code>PremiumCeding</code>
Categories	<code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Once a day, at 3:30 a.m.
Class	<code>RICedingWorkQueue</code>

In PolicyCenter, the Premium Ceding work queue performs calculations related to premium ceding.

Purge work queue

Code	<code>Purge</code>
Categories	<code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Every day at 4:30 a.m.
Class	<code>PurgeWorkQueue.java</code>

In PolicyCenter, the Purge work queue purges jobs and prunes policy periods that meet the purge and prune criteria. This process deletes jobs and other entities from the database.

Note: PolicyCenter disables the Purge work queue in the base configuration.

Purge Orphaned Policy Periods work queue

Code	<code>PurgeOrphanedPolicyPeriods</code>
Categories	<code>UIRunnable</code> , <code>Schedulable</code>

Implementation Work queue

Schedule	Not scheduled
Class	PurgeOrphanedPolicyPeriodWorkQueue.java

In PolicyCenter, the Purge Orphaned Policy Periods work queue finds orphaned policy periods (policy periods not associated with a specific job) and deletes them. The process deletes policy periods and other entities from the PolicyCenter database.

Note: Guidewire disables the Purge work queue in the base configuration.

Purge Quote Clones work queue

Code	PurgeQuoteClones
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	PurgeQuoteClonesWorkQueue.java

Quote cloning creates copies of policy period quotes, known as quote clones. In PolicyCenter, the Purge Quote Clones work queue deletes quote clones that are marked as processed from the PolicyCenter database.

By default, PolicyCenter comments out the entry for this work queue in file `scheduler-config.xml` in the base configuration. To schedule this work queue, remove the comment marks from around its entry in `scheduler-config.xml`.

Purge Rate Book Export Result work queue

Code	PurgeRateBookExportResult
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Every day at 3 a.m.
Class	RateBookExportResultPurgeWorkQueue.gs

In PolicyCenter, the Purge Rate Book Export Result work queue removes spreadsheets and XML files resulting from exporting rate books to spreadsheet or to XML. This process removes `RateBookExportResult` Excel and XML files that are more than 60 days old. Specify the number of days in the `RateBookExportResultAgeForPurging` parameter in `config.xml`.

This process removes the files from the PolicyCenter database only. It does not remove files downloaded to the user's computer.

To disable this process, set the `PurgeRateBookExportResultEnabled` parameter in `config.xml` to `false`.

Purge Rating Worksheets work queue

Code	PurgeWorksheets
Categories	UIRunnable, Schedulable
Implementation	Work queue

Schedule	Not scheduled
Class	WorksheetPurgeWorkQueue

In PolicyCenter, the Purge Rating Worksheets work queue removes worksheet container (WorksheetContainer) objects that meet certain criteria. The RatingWorksheetContainerAgeForPurging parameter in config.xml specifies the minimum number days after the closure of a job before the WorksheetContainer associated with its policy is eligible for purging.

By default, PolicyCenter disables the schedule for this work queue by commenting out the entry for it in file scheduler-config.xml. To schedule this work queue, remove the comment marks from around its entry in scheduler-config.xml.

Purge Risk Assessment Temporary Store work queue

Code	PurgeRiskAssessmentTempStore
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Every Monday at 1:00 a.m.
Class	PurgeRiskAssessmentTempStoreWorkQueue.gs

In PolicyCenter, the Purge Risk Assessment Temporary Store work queue deletes temporary objects created for risk assessment. This process deletes objects if they are older than the number of days specified in the PurgeRiskAssessmentTempStoreDays parameter. In the base configuration, Guidewire sets the value of this parameter to 30 days.

The Purge Risk Assessment Temporary Store process purges temporary objects if the course of standard PolicyCenter operations does not remove them.

Note: If you enable and use Spotlight, Spotlight creates a temporary entity that stores personal data named OutboundLocationRiskAssessmentTempStore. The PurgeRiskAssessmentTempStore work queue deletes these entities as well.

Purge Temporary Policy Periods work queue

Code	PurgeTemporaryPolicyPeriods
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	PurgeTemporaryPolicyPeriodsWorkQueue.java

Sometimes policy period (PolicyPeriod) objects in a temporary (Temporary) status exist in the database. In PolicyCenter, the Purge Temporary Policy Periods work queue removes these temporary policy periods from the database.

Configuration parameter PurgeTemporaryPolicyPeriodsAfterDays sets the minimum number of days that must pass before it is possible to purge a temporary policy period. In the base configuration, Guidewire set the value of this parameter to 14 day.

To enable this work queue, set configuration parameter PurgeTemporaryPolicyPeriodsEnabled to true. If set to false, the work queue does not remove any temporary policy periods.

Rate Book Export work queue

Code	RateBookExport
Categories	APIRunnable
Implementation	Work queue
Class	RateBookExportWorkQueue.java

In PolicyCenter, the Rate Book Export work queue exports a rate book and its included rate table and rate routines to a file in either in spreadsheet format or in XML format. PolicyCenter initiates this process if a user selects one of the following in the Detail screen for a Rate Book:

Export > Export to Spreadsheet

Export > Export to XML

Recalculate Contingency Action Start Date work queue

Code	RecalculateContingencyActionStartDate
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	RecalculateContingencyActionStartDateWorkQueue.gs

In PolicyCenter, the Recalculate Contingency Action Start Date work queue recalculates the action start date on all unresolved contingencies. It is only necessary to run this process if you modify the code for Gosu enhancement `ContingencyEnhancement.gsx` or want to update the action start date of existing contingencies.

Reset Purge Status and Check Dates work queue

Code	ResetPurgeStatusAndCheckDates
Categories	UIRunnable
Implementation	Work queue
Schedule	Not schedulable
Class	ResetPurgeStatusAndCheckDatesWorkQueue.java

In PolicyCenter, the Reset Purge Status and Check Dates work queue resets the purge status and purge or prune dates on the jobs. Run this process if you have a need to reset the purge status and purge date.

For each job, this process:

- Sets the `Job.PurgeStatus` property to Unknown.
- Sets the `Job.NextPurgeCheckDate` to null.

For example, if a job has a `PurgeStatus` of `NoActionRequired` or `Pruned`, the process resets the `PurgeStatus` to Unknown.

Retire Activities work queue

Code	ActivityRetire
-------------	----------------

Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 30 minutes past midnight
Class	RetireActivityWorkQueue.java

In PolicyCenter, the Retire Activities work queue retires Activity entity records that are one of the following:

- Canceled
- Dismissed

Team Screens batch process

Code	TeamScreens
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Every hour from 6:00 a.m. until 9:00 p.m., at three minutes past the hour
Class	TeamScreenProcess

In PolicyCenter, the Team Screens batch process collects statistics for the PolicyCenter **Team** tab screens.

While configuring the Team Screens batch process, consider the following:

- The time window used to calculate user statistics.
- The logic for calculating the summary statistics.

See the *PolicyCenter Application Guide* for information on these topics.

Scheduling

The following lines in the `scheduler-config.xml` file set the frequency of the Team Screens batch process:

```
<!-- Hourly user statistics generation at three minutes past the hour -->
      <ProcessSchedule process="TeamScreens">
        <CronSchedule minutes="3"/>
      </ProcessSchedule>
```

The time and database workload required for running the Team Screens batch process may be significant and may adversely impact system performance if run too frequently.

ClaimCenter base configuration processes

Address Delete work queue

Code	AddressDelete
Implementation	Work queue
Schedule	Not schedulable
Class	AddressDeleteWorkQueue.java

AddressDeleteWorkQueue is an internal work queue that ClaimCenter uses to delete orphaned addresses. During every bundle commit, ClaimCenter identifies addresses that are potentially orphaned as a result of that bundle commit. Potentially orphaned addresses fall into the following categories:

- It is a newly inserted address that does not have any entity pointing to it.
- It is the original address on an entity, which now contains an updated address.

For each identified potential orphan, the work queue creates an AddressDeleteWorkItem work item and associates it with public ID of the address. As the work queue processes a work item, it attempts to delete the corresponding address row from the Address table. If there are any remaining foreign key references to the address from any other table, the delete operation fails silently. The work queue then continues to process any remaining work items.

During a bundle commit, ClaimCenter calls a writer that creates the actual work items. The writer creates the work items such that they only become available for processing one minute after creation, by default. This delay gives a user time to associate a newly orphaned address with a different entity, if desired. It is possible to configure this time delay through configuration parameter AddressDeletionDelay in file config.xml.

Aggregate Limit Calculations work queue

Code	AggLimitCalc
Categories	APIRunnable, UIRunnable
Implementation	Work queue
Schedule	Not schedulable
Class	RebuildAggLimitsWorkQueue.java

In ClaimCenter, the Aggregate Limit Calculations work queue forces a recalculation of the aggregate limits stored in the ClaimCenter database. (An aggregate limit is the maximum financial amount that an insurer must pay on a policy or coverage during a given policy period.) The work queue then repopulates the database tables with this updated data. Run Aggregate Limit Calculations work queue only if you encounter consistency check failures and cannot identify the reason for the inconsistency.

By default, ClaimCenter sets the number of workers in work-queue.xml for this work queue to 2 in the base configuration. However, Guidewire recommends that you use substantially more workers with the Aggregate Limit Calculations work queue. Using just a few workers in a large database can take a very long time.

The optimal number of workers to use varies according to the available hardware and the volume of the data involved. It is also possible to allocate work queue workers to several different ClaimCenter servers rather than simply increasing the number of workers on a single server.

You can run the Aggregate Limit Calculations work queue in the following ways:

- From the ClaimCenter Server Tools **Work Queue Info** screen available to administrators.
- From a command prompt using the `maintenance_tools -rebuildagglimits` command option. If using the `maintenance_tools` command, the following command options are also useful:

-forceall	Use to mark all aggregate limits as invalid to force a rebuild.
Note: The system must be in maintenance mode before you use the <code>-forceall</code> parameter. Contact Guidewire Support to enable maintenance mode.	
-startprocess	Use to monitor and review the progress of this process.
-processstatus	use to review the process execution after completion.

Bulk Invoice Escalation batch process

Code	BulkInvoiceEsc
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Twice a day, at 6:35 a.m. and 6:35 p.m.
Dependency	Run "T-accounts Escalation Batch Processing" batch process <i>before</i> running Bulk Invoice Escalation.
Class	BulkInvoiceEscalationMonitor.java

In ClaimCenter, the Bulk Invoice Escalation batch process initiates the escalation process for bulk invoices that meet the following criteria:

- A **status** of **Awaiting submission**
- A **send date** of the current date or earlier

This process changes the status of a bulk invoice to **Initiating escalation**, which invokes the Invoice Processing work queue.

Bulk Invoice Workflow Monitor batch process

Code	BulkInvoiceWF
Categories	UIRunnable, Schedulable
Implementation	Batch process
Schedule	Every 30 minutes
Class	BulkInvoiceWorkflowMonitor.java

In ClaimCenter, the Invoice Item Processing work queue processes the items in a Bulk Invoice. After the work queue completes its work on the last item to process, it updates the status of the Bulk Invoice.

In the rare occasion that the status update was not successful, the Bulk Invoice Workflow Monitor batch process runs as a safeguard every thirty minutes to update the bulk invoice's status if all items have finished processing. For example, if a bulk invoice unexpectedly remains in **Creating checks** status, this batch process transitions the bulk invoice to one of the following statuses:

- **Awaiting submission** – The bulk invoice contains only approved checks.
- **Invalid bulk invoice items** – The bulk invoice contains rejected checks.

Catastrophe Claim Finder batch process

Code	CatastropheClaimFinder
Categories	UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Once a day, at 4:30 a.m.
Class	CatastropheClaimFinderBatch.java

In ClaimCenter, the Catastrophe Claim Finder batch process finds possible claims related to a catastrophe. For each claim that it finds, Catastrophe Claim Finder creates an activity to review the claim. The purpose of the activity is to determine if there is an association between the claim and the catastrophe.

By default, this batch process operates on claims that meet the following criteria:

- The claim must not be already associated with a catastrophe.
- The claim loss date must be within the effective date range of the catastrophe.
- The claim loss type and loss cause must match the coverage perils for the catastrophe.
- The claim must not have skipped or completed the `catastrophe_review` activity.
- The claim must be active (not retired).

You can modify which claims the process operates on by changing the `findClaims()` method in `GW_CatastropheEnhancement.gsx`. Access this enhancement in Studio at the following location:

configuration > Classes > gw > entity

Catastrophe Policy Location Download batch process

Code	CatastrophePolicyLocationDownload
Categories	Not enabled
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Not scheduled
Dependencies	Requires integration with a policy administration system (PAS)
Class	CatastrophePolicyLocationDownload.gs

Example

IMPORTANT: The Catastrophe Policy Location Download batch process requires integration with a policy administration system. If that policy administration system is not Guidewire PolicyCenter, you must modify the batch process code to work with that system. Guidewire strongly recommends that you modify Gosu class `CatastrophePolicyLocationDownload` rather than base class `CatastrophePolicyLocationDownloadBase`.

In ClaimCenter, the Catastrophe Policy Location Download batch process looks for catastrophes for which the last modified date is later than the date of the last download of policy locations. For each catastrophe that qualifies, the process updates the policy locations in the modified area in the following phases:

Phase	Description
Location Query Phase	The process queries the policy administration system for locations within the area of interest through the <code>PolicyLocationsSearchAPI</code> web service that PolicyCenter publishes.
Claim Matching Phase	The process matches policy locations downloaded from the policy administration system with policies on claims that already are in ClaimCenter.

Note: Some, possibly many, policy locations can fail to match existing claims. The usual case is that there are many more policy locations than claims in an area of interest.

Scheduling

By default, ClaimCenter disables the Catastrophe Policy Location Download batch process in the base configuration. Generally, you want to schedule it to run regularly on a daily cycle. The process can take a long time to complete, depending on any of the following factors:

- The number of areas of interest

- The size of the areas
- The number of locations within these areas

For performance reasons, schedule the process to run during a time of day in which the use of ClaimCenter is low, such as late at night.

To schedule this batch process, you need to perform both of the following actions:

- Add a Schedulable category to this batch process in the BatchProcessType typelist.
- Add an entry for this batch process to file `scheduler-config.xml`.

Claim Contacts Calculations batch process

Code	ClaimContactsCalc
Categories	APIRunnable, MaintenanceOnly
Implementation	Batch process
Stoppable	Yes (multi-phase process)
Schedule	Not schedulable
Class	ClaimContactsCalculationBatchProcess.java

Example

IMPORTANT: You can run the Claim Contacts Calculations batch process only if the InsuranceSuite application server is in maintenance mode. Contact Guidewire Support to enable maintenance mode.

In the base configuration, ClaimCenter denormalizes several contact fields into different objects for performance reasons. The Claim Contacts Calculations batch process recalculates the denormalization fields in case the fields ever fall out of synchronization.

This process updates the following fields:

- Exposure.ClaimantDenorm
- Claim.ClaimantDenorm
- Claim.InsuredDenorm
- CheckPayee.PayeeDenorm
- Recovery.PayerDenorm

Only run this process if you encounter consistency check failures. The server must be at the maintenance run level to run Claim Contacts Calculations. You cannot run this process from the ClaimCenter user interface or schedule the process. You can only run Claim Contacts Calculations by calling MaintenanceToolsAPI web service.

Claim Exception work queue

Code	ClaimException
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 2:00 a.m.
Class	ClaimExceptionMonitorWorkQueue.java

In ClaimCenter, the Claim Exception work queue runs claim exception rules on the following types of claims:

- The claim contains updates since the process last ran.
- The exposures, transactions, claim contacts, claim contact roles, or matters associated with the claim contain updates since the process last ran.

The Claim Exception work queue also runs claim exception rules on claims created since Claim Exception processing last run.

IMPORTANT: The Claim Exception work queue runs on a claim each time that it finds a triggering reason. This means that if the work queue finds multiple triggering reasons on a claim, it runs once for each triggering reason.

Configuration parameter `SeparateIdleClaimExceptionMonitor` controls whether the Idle Claim Exception and Claim Exception work queues run independently or together. If `SeparateIdleClaimExceptionMonitor` is `false`, then Idle Claim Exception and Claim Exception run claims concurrently through claim exception rules for recently modified and idle claims.

ClaimCenter sets configuration parameter `SeparateIdleClaimExceptionMonitor` to `true` in the default configuration. If you set the parameter to `false`, also disable the schedule for running Idle Claim Exception work queue.

Because running claim exceptions is processing intensive, Guidewire recommends that you schedule Claim Exception work queue during periods of low activity.

Claim Health Calculations batch process

Code	ClaimHealthCalc
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Not scheduled
Class	ClaimHealthCalculatorBatch.gs

In ClaimCenter, the Claim Health Calculations batch process calculates health indicators and metrics for all claims for which no calculated metrics exist. The value of `MaxClaimResultsPerClaimHealthCalcBatch` configuration parameter in `config.xml` determines the maximum number of claims that Claim Health Calculations processes in a single run.

Guidewire recommends that you run the Claim Health Calculations batch process in the following circumstances:

- After an upgrade from a version of ClaimCenter that did not have Claim Metrics, meaning ClaimCenter 6.0 or earlier.

The Bulk Claim Validation batch process also forces the creation of Claim Health Metrics on the claims that it processes if metrics do not exist.

Claim Health Calculations does not update metrics on claims that already have metrics. Guidewire provides a sample claim exception rule that you can use to force creation of new claim metrics, exposure metrics, and claim indicators on every claim. The name of the sample rule is **CER04000 - Recalculate claim metrics**.

ClaimCenter (SPM) Completed Review Synchronization work queue

Code	ReviewSync
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Work queue

Schedule	Not scheduled
Class	ReviewSyncWorkQueue.java

The ClaimCenter (SPM) Completed Review Sync work queue transmits completed reviews of service providers to Guidewire ContactManager. ContactManager uses this information to construct a ReviewSummary object. After the process submits a review to ContactManager, the process sets the AddressBookUID field on the ClaimCenter Review object, indicating the transmission of the review.

By default, Guidewire disables the schedule for this work queue by commenting it out in file `scheduler-config.xml`. To schedule this work queue, remove the comment marks from around its entry in `scheduler-config.xml`.

Contact Auto-synchronization work queue

Code	ContactAutoSync
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Work queue
Schedule	Twice a day, at 5:00 a.m. and 5:00 p.m.
Dependencies	Run the Contact Auto-synchronization work queue <i>before</i> Financials Escalation batch process to prevent that process from encountering validation errors due to out-of-synchronization contacts.
Class	CCContactAutoSyncWorkQueue.java

The Contact Auto-synchronization work queue synchronizes contact information between Guidewire ContactManager and Guidewire ClaimCenter. You can configure ClaimCenter to synchronize contact information whenever a user updates a contact, or you can schedule the Contact Auto-synchronization process to synchronize contact information at a set time.

To modify or update the schedule for the ContactAutoSync work queue, update file `scheduler-config.xml`. To manually execute the work queue, navigate to the (Server Tools) **Work Queue Info** screen and click **Run Writer** for the ContactAutoSync work queue.

Configuration parameters

The following configuration parameters in file `config.xml` affect the behavior of the ContactAutoSync work queue:

- ContactAutoSyncBundleCommitSize
- ContactAutoSyncWorkItemChunkSize
- InstantaneousContactAutoSync

Contact Retire work queue

Code	ContactRetire
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 1:00 a.m.
Class	ContactRetireWorkQueue.java

In ClaimCenter, the Contact Retire work queue checks for contacts that are potentially eligible for retirement and removes them. On bundle commit, ClaimCenter checks for any entities that have a foreign key link to the contact and creates ContactRetireWorkItem objects for any contacts being affected by the following:

- The owning entity selected for retirement.
- The property being changed, and a new value being set

As the Contact Retire work queue runs, it calls the `ContactRetireHelper.retireContact()` method to attempt to retire the contact referenced by each `ContactRetireWorkItem`.

Dashboard Statistics batch process

Code	DashboardStatistics
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Once a day, at 1:00 a.m.
Class	DashboardStatsBatchProcess.java

In ClaimCenter, the Dashboard Statistics batch process recalculates the statistics for data shown in the ClaimCenter **Dashboard** tab that is available to supervisors only.

Encryption Upgrade work queue

Code	EncryptionUpgrade
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 1:00 a.m.
Class	SnapshotUpgradeWorkQueue.java

The Encryption Upgrade work queue upgrades encryption fields for previous entity versions. ClaimCenter supports the encryption of sensitive data, such as tax IDs, on claim snapshots.

The `ClaimSnapshot` entity includes an `EncryptionVersion` integer column. This column stores a number representing the version of the `IEncryption` plugin implementation used to encrypt the snapshot fields. ClaimCenter also stores the current encryption version. If the encryption metadata or plugin algorithm change, ClaimCenter increments this version number.

The Encryption Upgrade work queue recalculates encrypted field values for any `ClaimSnapshot` that has a value for `EncryptionVersion` less than the current encryption version. ClaimCenter decrypts the encrypted value by using the original plugin implementation. Then, ClaimCenter encrypts the value with the new plugin implementation. Finally, ClaimCenter updates the `EncryptionVersion` of the snapshot to mark it current.

You can adjust the number of snapshots that this process upgrades at one time by modifying the `SnapshotEncryptionUpgradeChunkSize` parameter in `config.xml`. A value of 0 for `SnapshotEncryptionUpgradeChunkSize` sets no limit to the number of snapshots upgraded at once.

Exchange Rate batch process

Code	ExchangeRate
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)

Schedule	Not scheduled
-----------------	---------------

Class	ExchangeRateBatchProcess.java
--------------	-------------------------------

The Exchange Rate batch process uses a class that implements the `IExchangeRateSetPlugin` interface to populate exchange rate data in ClaimCenter.

By default, ClaimCenter comments out the entry for this work queue in file `scheduler-config.xml`. To schedule the Exchange Rate batch process, remove the comment marks from around its entry in `scheduler-config.xml`.

Financials Calculations batch process

Code	FinancialsCalculations
-------------	------------------------

Categories	MaintenanceOnly
-------------------	-----------------

Implementation	Batch process
-----------------------	---------------

Stoppable	No (single phase process)
------------------	---------------------------

Schedule	Not schedulable
-----------------	-----------------

Class	FinancialsCalculationBatchProcess.java
--------------	--

IMPORTANT: Before you run this batch process, first run the “Database Consistency Check” work queue and verify that there are no consistent children failures.

Financials Calculations batch process recalculates denormalized financial values. ClaimCenter denormalizes several financial fields into different objects for performance reasons. In ClaimCenter, the Financials Calculations process recalculates denormalization fields in case they ever become unsynchronized. This process updates the following entities:

- CheckRpt
- ClaimRpt
- ExposureRpt

Each of these entities stores denormalized financials totals to improve performance if ClaimCenter displays the entity. These entities are kept current by the normal operation of ClaimCenter. However, issues in a custom configuration can cause the values to become unsynchronized and cause database consistency checks to fail. To help address such issues, it is possible for Guidewire to ask you to run this batch process.

Never add new claims to ClaimCenter with the web service APIs while the Financials Calculations batch process is running. This applies to both `addFNOL()` and `migrateClaim()` methods.

This batch process can only be run from the command prompt while the server is in maintenance mode.

Note: Contact Guidewire Support to enable maintenance mode.

Warning

The Financials Calculations process recalculates denormalized financial values in two distinct steps:

1. It first zeros out all denormalization fields in all the RPT tables.
2. It then recalculates the denormalization fields in the RPT tables.

If the Financials Calculations process does not complete successfully, there is a risk that ClaimCenter can leave all denormalization (transaction summary) fields in the RPT tables zeroed out. If this is the case, ClaimCenter users can see this result in the financial summary screens.

There are multiple reasons that the Financials Calculations process can fail:

- Restarting the database server while the Financials Calculations process is executing.
- Manually stopping the Financials Calculations process before the process completes, perhaps because the process is taking longer than the scheduled maintenance time window.
- Encountering a triangle inconsistency DBCC error for a claim, in which the child objects on the Claim object do not consistently point to the same claim.

If the Financials Calculations process fails for any reason, do the following:

1. Determine the root cause of the process failure and fix it.
2. Rerun the Financials Calculations process and ensure that it completes successfully.

Financials Escalation work queue

Code	FinancialsEsc
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Work queue
Stoppable	Yes
Schedule	Twice a day, at 6:00 a.m. and 6:00 p.m.
Dependencies	• Run T-accounts Escalation batch process <i>before</i> running Financials Escalations. • Run Contact Auto-Synchronization work queue <i>before</i> running Financials Escalations to prevent validation errors due to out-of-synchronization contacts.
Class	FinancialsEscalationWorkQueue.gs

ClaimCenter periodically runs a Financials Escalation work queue that processes checks that are ready for submission. A check is automatically eligible for submission:

- If it has a TransactionStatus of awaitingsubmission
- If it has a scheduled send date that is either *today* or earlier
- If it is not part of a bulk invoice

Note: If a check is marked for PendEscalationForBulk, then it will *not* be submitted.

The Financials Escalation process changes the status of unissued payments that have passed their date requirements from Awaiting Submission to Submitting. ClaimCenter can then send the unissued payments to the financial system.

Although the intention for many payments is to produce a check as soon as possible, it is possible to schedule the issuance of a check for a future date. This is very common if creating a schedule of recurring payments, for example.

Idle Claim Exception work queue

Code	IdleClaim
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a week, at Sunday noon (12:00 p.m.)
Class	IdleClaimExceptionMonitorWorkQueue.java

In ClaimCenter, the Idle Claim Exception work queue runs the claim exception rules on open claims that are idle. ClaimCenter considers a claim to be idle if the exception rules have not been run on it in the number of days specified by the IdleClaimThresholdDays configuration parameter.

The `SeparateIdleClaimExceptionMonitor` configuration parameter controls whether the Idle Claim Exception and Claim Exception processes run independently or together. This parameter can take the following values:

<code>false</code>	The Idle Claim Exception and Claim Exception work queues run claims concurrently through the claim exception rules for recently modified and idle open claims.
<code>true</code>	The Idle Claim Exception and Claim Exception work queues do not run claims concurrently through the claim exception rules.

ClaimCenter sets the value of `SeparateIdleClaimExceptionMonitor` to `true` in the default configuration. If you set the value of this parameter to `false`, also disable the schedule for running the Idle Claim Exception work queue.

Idle Closed Claim Exception work queue

Code	<code>IdleClosedClaim</code>
Categories	<code>APIRunnable</code> , <code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Unscheduled
Class	<code>IdleClosedClaimExceptionMonitorWorkQueue.java</code>

In ClaimCenter, the Idle Closed Claim Exception work queue runs the claim exception rules on closed claims that are idle. ClaimCenter considers a claim to be idle if the exception rules have not been run on it in the number of days specified by the `IdleClosedClaimThresholdDays` configuration parameter. By default, Guidewire disables the Idle Closed Claim Exception work queue in the scheduler.

Invoice Item Processing work queue

Code	<code>InvoiceItemProcessing</code>
Categories	<code>UIRunnable</code>
Implementation	Work queue
Workers	2
Schedule	Not schedulable
Class	<code>InvoiceItemWorkQueue.java</code>

In ClaimCenter, the Invoice Item Processing work queue processes bulk invoice items in parallel. The work queue's workers pick up work items (`InvoiceItemWorkItem` entities) and process bulk invoice items when the invoice item's `AwaitingProcessing` flag is set to `true`.

Processing work items

After the Invoice Processing work queue completes, the Invoice Item Processing work queue handles the processing of bulk invoice items.

The `BulkInvoiceStatus.tti` typelist contains a `ParallelProcessing` typefilter to identify this set of bulk invoice statuses. Parallel Processing begins after all work items are created for that `BulkInvoice`. This work queue processes the work items and performs the particular operation on each `BulkInvoiceItem`. For example, set an invoice item's status to Pending void and void the placeholder check associated with the invoice item.

When processing is complete, the invoice item's `AwaitingProcessing` flag is set to `false` and the `BulkInvoice` status changes to the next status as shown in the following table:

Parallel processing status	Resulting status after processing
Creating checks	Awaiting Submission

Parallel processing status	Resulting status after processing
Processing escalation	Requesting
Processing void	Pending void
Processing stop	Pending stop
Processing delete	Retired

If a bulk invoice remains in one of the parallel processing statuses listed in this table and all workers have finished, you must notify the Invoice Item Processing work queue's workers by selecting **Notify** in the **Work Queue Info** screen in Server Tools. If that is not successful, run the Bulk Invoice Workflow Monitor batch process, and, then run the Invoice Item Processing work queue again.

Configuring the Messaging plugin

To communicate with downstream operations, you can use any of the following statuses to configure the Messaging plugin:

- Requesting
- Pending void
- Pending stop

Configure the Messaging plugin to call the appropriate acknowledge method to update to the appropriate status:

- Requested
- Voided
- Stopped

This is similar to the same statuses on Check.

Note: You need to configure both Message creation in the Event Fired rules and the Messaging plugin. ClaimCenter does not implement this functionality in the base configuration.

Archiving and work queue processing

It is possible for the work queue to fail if there is an attempt to archive a claim while work queue is processing a bulk invoice associated with that claim. ClaimCenter logs the failure as an error. The error also appears in the **Work Queue Info** screen in Server Tools.

To resolve this work queue issue, restore the claim from the archive, and retry the failed work item in the **Work Queue Info** screen. After the Invoice Item processing work queue completes successfully, it is possible to archive the claim.

Invoice Processing work queue

Code	InvoiceProcessing
Categories	UIRunnable
Implementation	Work queue
Workers	2
Schedule	Not schedulable
Class	InvoiceWorkQueue.java

Creation of work items

In ClaimCenter, the Invoice Processing work queue processes each BulkInvoice when a bulk invoice has one of the following statuses:

- Initiating check creation
- Initiating escalation
- Initiating void
- Initiating stop
- Initiating delete

This work queue is usually invoked when a user performs an action on a bulk invoice such as submitting or voiding the bulk invoice. Automated processes such as invoice escalation also invoke this work queue. The Invoice Processing work queue runs against the bulk invoice and creates a work item (an `InvoiceItemWorkItem` entity) for each `BulkInvoiceItem` on the `BulkInvoice`. The invoice item's `AwaitingProcessing` flag is then set to `true`.

After creating all work items, the work queue sets the bulk invoice to one of the following statuses:

- Creating checks
- Processing escalation
- Processing void
- Processing stop
- Processing delete

The `BulkInvoiceStatus.tti` typelist contains a typefilter (`AsyncProcessing`) to identify this set of bulk invoice statuses. After the work queue creates work items, processing of each bulk invoice item is handled by the Invoice Item Processing work queue.

Recalculate Claim Metrics work queue

Code	<code>RecalculateMetrics</code>
Categories	<code>APIRunnable</code> , <code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Once a day, at 3:30 a.m.
Class	<code>RecalculateClaimMetricsWorkQueue.java</code>

In ClaimCenter, the Recalculate Claim Metrics work queue recalculates claim metrics on claims for which the metric update time has already passed.

Service Request Metric Escalation work queue

Code	<code>ServiceRequestMetricEscalation</code>
Categories	<code>APIRunnable</code> , <code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	At 10 minutes and 40 minutes past the hour
Class	<code>ServiceRequestMetricEscalationWorkQueue</code>

In ClaimCenter, the Service Request Metric Escalation work queue finds time metrics for service requests that meet the following conditions:

- The time metric is open and has not yet escalated.
- The time metric does not have a `ReachRedTime` status before the current time.
- The time metric has not already been selected for processing.

For each metric the work queue finds, the process creates an activity from a metric-specific activity pattern assigned to the service request's assigned user. It then marks the metric as escalated.

Statistics batch process

Code	Statistics
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Every hour, at three minutes past the hour
Class	StatisticsBatchProcess.java

The Statistics batch process calculates the work activity statistics that ClaimCenter shows in the Dashboard **Statistics** screen. The process also calculates aging values on claims and exposures.

Only users with the Supervisor role in ClaimCenter are able to view work activity statistics for the team in the **Statistics** screen. This screen shows counts of open claims, activities, exposures, and matters for each user.

By default, this process runs every hour at three minutes past the hour. You can schedule this process to run more frequently. However, monitor system performance for possible negative impact if you change the schedule.

T-account Escalation batch process

Code	TaccountEsc
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Batch process
Stoppable	No (single phase process)
Schedule	Every night, at one minute past midnight
Dependencies	Run this batch process <i>before</i> running the Financials Escalation and Bulk Invoice Escalation batch processes.
Class	TAccountEscalationMonitor.java

In ClaimCenter, the T-account Escalation batch process updates the T-accounts of the payments on a future-dated check that has now reached its scheduled send date. After this update, the payment amount no longer contributes to future payments. Instead, the payment contributes to calculations that include payments scheduled for today, such as total payments, open reserves, remaining reserves, and available reserves.

The T-account Escalation process updates financials calculations in the event that the schedule for the Financials Escalation or Bulk Invoice Escalation processes runs after the end of the business day. That way, the financial calculations are current for the day scheduled to send the check, but, the check is still editable as it has not yet been escalated. T-account Escalation ensures that T-account balances and dependent calculated financials values are correct between midnight and the first scheduled run of Financials Escalation or Bulk Invoice Escalation for that day.

Schedule the T-account Escalation process to run as close to just past midnight as possible and before the Financials Escalation and Bulk Invoice Escalation batch processes run.

Scheduling the Financials Escalation work queue

In the default ClaimCenter application, the Financials Escalation work queue runs twice a day, at 6:05 a.m. and 6:05 p.m. If you do not run the T-account Escalation batch process earlier in the day, then the ClaimCenter **Financials Summary** screen can potentially display out-of-date values. The values are out-of-date from midnight until the time that the Financials Escalation work queue does run.

Depending on your implementation, you can schedule these two escalation processes differently, for example:

- Schedule one of these two processes to run before the calculated values need to be up-to-date. This could be before people arrive for work, for example.
- Run the T-account Escalation process as soon after midnight as possible and schedule the Financials Escalation process at your midday time to keep checks editable until sometime during the last available working day.
- Schedule the Financials Escalation process just after midnight if you do not care about not being able to edit future-dated checks that have reached their send date before the Financials Escalation process runs. In this case, you need not run the T-account Escalation process.

User Workload Update work queue

Code	UserWorkloadUpdate
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	UserWorkloadUpdateWorkQueue.java

In ClaimCenter, the User Workload Update work queue updates workload data for ClaimCenter users. Run this process any time a change is made that affects existing weighted workload values. In particular, run this process to recalculate workload classifications and assignments to groups after any change to classifications. Otherwise, it is likely that the stored computed workload weights are out of date.

By default, ClaimCenter does not add an entry for this work queue to file `scheuduler-config.xml` in the base configuration. To schedule this work queue for execution, you must add an entry for it to `scheuduler-config.xml`.

BillingCenter base configuration processes

Account Inactivity work queue

Code	AccountInactivity
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 2:35 a.m.
Class	AccountInactivityWorkQueue.java

In BillingCenter, the Account Inactivity work queue creates notification activities for accounts that are inactive for longer than a defined number of days and do not already have an open `AcctInactiveActivity` activity.

The Account Inactivity work queue operates on accounts that:

- Do not have an open Activity of type `AcctInactiveActivity`.
- Do not have any `AccountHistory` records created within the account activity period with the `CountsAsActivity` flag set to `True`.

The Account Inactivity work queue creates a notification activity that is a subtype of `Activity` called `AcctInactiveActivity` with a status of open. If the process detects an open `AcctInactiveActivity`, it does not create additional `AcctInactiveActivity` activities for that account. If a user changes the status to something other than open, then `AccountInactivity` creates a new `AcctInactiveActivity` for that account.

BillingCenter constructs a time window in which it considers an account active through the use of the following:

- Configuration parameter `DaysBeforeAccountIsConsideredInactive`
- The creation time of the history records

Guidewire defines the time window as the following:

```
today >= history event >= (today - DaysBeforeAccountIsConsideredInactive)
```

For each account that the Account Inactivity work queue finds, it calls the following static Gosu method:

```
gw.activity.AccountInactivityUtil.maybeCreateAcctInactiveActivity
```

Advance Expiration work queue

Code	AdvanceExpiration
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 2:15 a.m.
Class	AdvanceExpirationWorkQueue.java

In BillingCenter, the Advance Expiration work queue checks for advance commission payments that are no longer maintained and expires them. The producer then owes any remaining advance balance.

The Advance Expiration work queue query returns `AdvanceCmsnPayment` entities that meet the following criteria:

- Have not yet expired
- Have a `MaintainUntil` date that is before or equal to the current date and time

For each `AdvanceCmsnPayment` found, the Advance Expiration work queue calls the `Producer.expireAdvances()` method. This method expires the advance commission payment. This is done by removing any amount in excess of the total of all maintained advances from the producer's `CommissionsAdvance` T-account through the `ProducerAdvanceExpired` transaction.

For each `AdvanceCmsnPayment` that `AdvanceExpiration` finds, the work queue sets `AdvanceCmsnPayment.Expired` to `True`.

Agency Suspense Payment work queue

Code	AgencySuspensePayment
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 3:30 a.m.
Class	AgencySuspensePaymentWorkQueue.java

In BillingCenter, the Agency Suspense Payment work queue finds all active agency bill suspense payment items. A suspense item is a place to temporarily put funds that are meant for a policy period that has not yet been created. This is necessary as agency cycle payments and promises can target policy periods that do not yet exist in BillingCenter but might already exist in the policy administration system.

For each item that the work queue finds, the process calls the `matchAgencyBillSuspensePolicyPeriod` method in the `EventHandler` plugin implementation to match the item with a policy period. If the batch process does not find a match with a policy period, it leaves the item unchanged.

If the method finds a matching `PolicyPeriod`, it sets the `MatchingPolicy` field of the suspense item to that policy period. The work queue then calls the `processAgencyBillSuspenseItemAfterPolicyMatch()` method of the `EventHandler` plugin implementation. The default implementation of this method does nothing at all. However, you

can provide custom logic for this method if you want to perform any special processing after the batch process matches a suspense item with a policy period. For example, you might want to apply the funds associated with the suspense item to the policy period.

Allot Funds for Funds Tracking work queue

Code	FundsAllotment
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 4:10 a.m.
Class	FundsAllotmentWorkQueue.java

In BillingCenter, the Allot Funds for Funds Tracking work queue operates on Account entity instances that have funds to allot. The work queue then calls the `Account.allotAllFunds` method for each Account instance in the result set. The `allotAllFunds` method finds source trackers and use trackers on the account that are not fully allotted. It packages the source and use trackers into two sets, which it passes as parameters to the `allotFunds` method on the IFundsTracking plugin implementation. The plugin implementation performs an allotment process against each Account instance.

It is possible to specify the number of Account entities to process in each work item in configuration parameter `FundsAllotmentAccountsPerWorkItem`.

Automatic Disbursement work queue

Code	AutomaticDisbursement
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 4:00 a.m.
Class	AutomaticDisbursementWorkQueue.java

In BillingCenter, the Automatic Disbursement batch process operates on accounts on all accounts that have the following characteristics:

- The account has an unapplied balance.
- The account has a positive balance in its Unapplied T-account.

For each account, the process performs the following operations:

- Distributes any existing payments and credits.
- Applies funds stored in default and designated Unapplied T-accounts to pay charges.

If any Unapplied funds remain after paying all charges, the funds are returned to the customer by either creating a new disbursement or modifying an existing one.

Charge ProRata Transaction work queue

Code	ChargeProRataTx
Categories	UIRunnable, Schedulable
Implementation	Work queue

Schedule	Once a day, at 2:20 a.m.
Class	ChargeProRataTxnWorkQueue.java

WARNING: Guidewire recommends tracking earned premium in the policy administration system and not in BillingCenter. Consequently, Guidewire recommends against enabling this work queue.

In BillingCenter, the Charge ProRata Transaction work queue operates on ChargeProRataTx entities that match the following criteria:

- The ChargeProRataTx entity has a status of Planned.
- The date specified by EarnDate has already passed.

The work queue creates a ProRataChargeEarned transaction for each ChargeProRataTx entity that it finds, using ChargeProRataTx.Amount. The transaction moves funds from the Unearned T-account for the charge to its Earned T-account.

BillingCenter creates ChargeProRataTx entities as it creates a Charge with a ChargePattern subtype of ProRataCharge.

Enable the ChargeProRataTx Work queue

By default, BillingCenter disables the ChargeProRataTx work queue in the base configuration. You must enable this work queue to be able to use it.

About this task

Perform the following steps in Guidewire Studio.

Procedure

1. Open file work-queue.xml and set the number of workers for the ChargeProRataTx work queue to one (1) or more.
2. Open typelist BatchProcessType and add the categories of Schedulable and UIRunnable to the entry for ChargeProRataTx.
3. Open file config.xml and set the value of configuration parameter EnableChargeProRataTxCreation to true.

IMPORTANT: If you set the value of configuration parameter EnableChargeProRataTxCreation to true and start the application server, you cannot change the value of this parameter thereafter.

4. Open file scheduler-config.xml and uncomment the entry for ChargeProRataTx.

If there is no entry for ChargeProRataTx in file scheduler-config.xml, you need to create the entry.

Collateral Requirement Effective work queue

Code	CollEffective
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 2:45 a.m.
Class	CollEffectiveWorkQueue.java

In BillingCenter, the Collateral Requirement Effective work queue operates on pending CollateralRequirement entities with an EffectiveDate value that is on or before the current date. It does the following:

- The process changes the Compliance status from Pending to Not Compliant.
- The process flags the collateral for processing by the collateral allocation algorithm.
- The collateral allocation algorithm tries to allocate existing funds so as to satisfy all active requirements associated with a collateral requirement.

The Collateral Requirement Effective work queue operates on CollateralRequirement entities that meet the following criteria:

- Have a Compliance of Pending
- Have an EffectiveDate that is before or on the current date

For each qualifying CollateralRequirement entity, the process updates CollateralRequirement.Compliance to Not Compliant, and sets a flag on its related Collateral entity. Upon commit, BillingCenter runs the collateral allocation algorithm, which uses available collateral, such as cash or letters of credit, to fulfill the various collateral requirements.

Collateral Requirement Expiration work queue

Code	CollExpiration
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 2:50 a.m.
Class	CollExpirationWorkQueue.java

In BillingCenter, the Collateral Requirement Expiration work queue operates on CollateralRequirement entities with an ExpirationDate that is on or before the current date. It does the following:

- The process changes the Compliance status to Closed.
- The process flags the collateral for processing by the collateral allocation algorithm.
- The collateral allocation algorithm then tries to allocate existing funds.
- The collateral allocation algorithm tries to allocate existing funds so as to satisfy all active requirements associated with a collateral requirement.

The Collateral Requirement Expiration work queue operates on CollateralRequirement entities that meet the following criteria:

- Have a Compliance value of Compliant or Not Compliant
- Have an ExpirationDate that is before or on the current date

For each qualifying CollateralRequirement entity, the process updates CollateralRequirement.Compliance to Closed and sets a flag on its related Collateral entity. Upon commit, BillingCenter runs the collateral allocate algorithm, which uses available collateral, such as cash or letters of credit, to fulfill the various collateral requirements.

Commission Payable Calculations work queue

Code	CmsnPayable
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 3:50 a.m.
Class	CommissionPayableWorkQueue.java

In BillingCenter, the Commission Payable Calculations work queue makes commission payable on direct bill policies. The process looks for commissions that a producer is eligible to earn. It does the following:

1. For each eligible commission found, the process creates a reserve commission earned transaction.
2. The transaction debits the policy's CommissionReserved T-account and credits the policy's CommissionPayable T-account.

In most cases, a commission becomes payable as soon as the PayableCriteria on the associated CommissionSubPlan occurs. A criterion, such as receiving a payment or billing an invoice, serves as the triggering event to make commission immediately payable. However, the following criteria do not automatically trigger commission earnings:

- On Effective Date
- On Expiration Date
- Custom

Commission Payment work queue

Code	CommissionPmt
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 2:30 a.m.
Class	CommissionPaymentWorkQueue.java

In BillingCenter, the Commission Payment work queue executes manual commission payments that have a schedule for execution on a specific date.

Commission Payment technical details

Commission Payment processing operates on DelayedCmsnPayment entities that:

- Have a PayOn date that is before or on the current date.
- Have a PaymentTime setting of payondate.
- Have a Paid status of False.
- Have an ApprovalStatus of Approved.

For each qualifying DelayedCmsnPayment entity, the Commission Payment process creates a ManualPayment entity and sets ManualPayment.Producer to the Producer associated with the DelayedCmsnPayment.

Each DelayedCmsnPayment entity then calls execute(ManualPayment), which creates commission payment transactions that differ for each subtype of DelayedCmsnPayment.

The CommissionPayment supertype execute method is called, which sets the DelayedCmsnPayment's Paid status to True and sets the ProducerPayment field to the newly created ManualPayment entity.

The AdvanceCmsnPayment subtype creates a ProducerAdvancePayable transaction, which debits the Producer's CommissionAdvance T-account, and credits the Producer's CommissionsPayable T-account.

The BonusCmsnPayment subtype creates a ProdCodeBonusEarned transaction, which debits the BonusCmsnPayment.ProducerCode CommissionExpense T-account and credits the BonusCmsnPayment.ProducerCode CommissionsPayable T-account. It then creates a ProdCodeCmsnPayable transaction, which moves the funds just added to the ProducerCode's CommissionsPayable T-account from that account to the Producer's CommissionsPayable T-account.

The ManualCmsnPayment subtype creates a ManualCmsnEarned transaction, which debits the ManualCmsnPayment.ProducerCode's CommissionExpense T-account and credits the ManualCmsnPayment.ProducerCode's CommissionsPayable T-account. It then creates a ProdCodeCmsnPayable

transaction, which moves the funds just added to the ProducerCode's CommissionsPayable T-account from that account to the Producer's CommissionsPayable T-account.

The StandardCmsnPayment subtype just moves the balance of the ProducerCode's CommissionsPayable T-account to the Producer's CommissionsPayable T-account using the ProdCodeCmsnPayable transaction, when the PaymentType of the StandardCmsnPayment is All Current. If the PaymentType is Other Amount, then BillingCenter attempts to move the amount specified on the StandardCmsnPayment to the Producer's CommissionsPayable T-account. If the amount requested is greater than the balance of the ProducerCode's CommissionsPayable T-account, then the balance is obtained from the ProducerCode's PolicyProducerCodes and PolicyProducerCodeCharges CommissionsPayable balances. If there are still insufficient funds, then whatever amount was found is transferred to the Producer's CommissionsPayable T-account.

Delete Account Commissions Expense TAccount work queue

Code	DeleteAccountCommissionsExpense
Categories	APIRunnable, Schedulable
Implementation	Work queue
Schedule	Once only, if required
Class	DeleteAccountCommissionsExpenseWorkQueue

IMPORTANT: Run this work queue once only, after upgrading to BillingCenter 10.0.1 or later. Run this process with exactly one worker as it is not safe to run with multiple workers. Disable this work queue in a production environment.

The Delete Account Commissions Expense TAccount work queue deletes Account Commissions Expense T-accounts as BillingCenter no longer uses these T-accounts. Run the Delete Account Commissions Expense TAccount work queue once only, after upgrading from any BillingCenter version previous to 10.0.1.

In the base configuration, BillingCenter disables this work queue by setting the number work queue workers to 0 in file work-queue.xml. To enable this work queue, set the number of workers for this work queue to one (1). Use only a single worker for this work queue. After you run this work queue a single time, disable the work queue by setting the number of workers back to zero (0).

Disbursement work queue

Code	Disbursement
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 4:05 a.m.
Class	DisbursementWorkQueue.java

The Disbursement work queue issues disbursement payments, during which BillingCenter creates and executes the appropriate transactions. The work queue selects a disbursement for processing if the disbursement meets either of the following criteria:

- The disbursement has been approved
- The disbursement is an automatic disbursement that is awaiting approval.

The process also filters out all account disbursements that have a disbursement hold, meaning that the account has a trouble ticket with a disbursement hold.

Disbursement technical details

For each disbursement selected for payment, BillingCenter performs the following operations in the order described.

- The disbursement is passed to the Event Handler plugin method `updateDisbursementStatusAndAmount`. The method's `isAboutToExecuteDisbursement` argument is passed a value of `true` if the current date is on or after the disbursement's due date; otherwise, the argument is `false`. The plugin method can modify the disbursement's monetary amount or status, including canceling the disbursement.
- The disbursement is executed if the current date is on or after the disbursement's due date and either of the following criteria is met. (The criteria are identical to that used to originally select the disbursement, but the plugin may have changed the disbursement's status.)
 - The disbursement has been approved (`Disbursement.Status` field is `TC_APPROVED`).
 - The disbursement is awaiting approval and is an automatic disbursement (`Disbursement.Status` field is `TC_AWAITINGAPPROVAL` and `Disbursement.Reason` field is `TC_AUTOMATIC`).

When a disbursement is executed, the following operations occur:

- If the disbursement's monetary amount is not a positive, non-zero value, an `IllegalStateException` is thrown.
- An appropriate BillingCenter transaction is created, added to the account's bundle, and executed.

BillingCenter supports several types of disbursements and each type creates its own transaction type. Each disbursement type extends the `Disbursement` class.

Disbursement type	Transaction type
AccountDisbursement	DisbursementPaid If funds tracking is enabled, the transaction is recorded in the funds tracking system.
AgencyDisbursement	AgencyDisbursementPaid
CollateralDisbursement	CollateralDisbPaid
SuspenseDisbursement	SuspPymtDisbPaid If the monetary amount of the disbursement is not equal to the suspense payment amount, an <code>IllegalArgumentExpection</code> is thrown.

- An `OutgoingDisbPmnt` object is created and added to the account's bundle. The following object fields are set.
 - The `Currency` field is set to the account's currency.
 - The `MailTo`, `MailToAddress`, and `PayTo` fields are set to the current values in the `Disbursement` object's `MailTo`, `Address`, and `PayTo` fields, respectively.
 - The `IssueDate` field is set to the current date.

In addition, the `OutgoingDisbPmnt` object is stored in the `Disbursement.OutgoingPayment` field.

- Finally, BillingCenter updates the following `Disbursement` object fields:
 - The `Status` field is set to `TC_SENT`.
 - The `CloseDate` field is set to the current date.

Full Pay Discount work queue

Code	FullPayDiscount
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 3:00 a.m.

Class	FullPayDiscountWorkQueue.java
--------------	-------------------------------

In BillingCenter, the Full Pay Discount work queue operates on PolicyPeriod entities that meet the following criteria:

- EligibleForFullPayDiscount set to true
- FullPayDiscountEvaluated set to false
- FullPayDiscountUntil date before the current date

For each qualifying PolicyPeriod, the process calls the Policy Period plugin's applyFullPayDiscount method. The default implementation of the method uses the PolicyPeriod.DiscountedPaymentThreshold property to determine whether to apply a discount and the discount amount. Modify this plugin method to fit your requirements for full-pay discounts.

Use configuration parameter FullPayDiscountPolicyPeriodsPerWorkItem to specify the number of PolicyPeriod entities to process in each work item.

Invoice work queue

Code	Invoice
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 3:20 a.m.
Class	InvoiceBilledWorkQueue.java

In BillingCenter, the Invoice work queue operates on AccountInvoice entities that have met the following criteria:

- Status field equal to Planned
- EventDate plus the cutoff time is less than the current date

The process sets the Status of matching account invoice entities to Billed and executes the necessary transactions on the invoice items.

BillingCenter uses time value of configuration parameter InvoicePeriodCutoff to calculate the time of day that an invoice period ends. The process subtracts the InvoicePeriodCutoff time from the current date and time. BillingCenter considers for billing any unbilled invoice with a bill date prior to the resulting date and time.

Invoice technical details

In BillingCenter, invoice batch processing operates on AccountInvoice entities that have met the following criteria:

- Status field equal to Planned
- EventDate plus the cutoff time is less than the current date

The process sets the Status of matching account invoice entities to Billed and executes the necessary transactions on the invoice items.

InvoicePeriodCutoff is a configuration parameter that takes a time value and is used to calculate the time of day that an invoice period ends. The process subtracts the InvoicePeriodCutoff time from the current date and time. Any unbilled invoice with a bill date prior to the resulting date and time is considered for billing.

The first step in billing an AccountInvoice is to push any held InvoiceItems forward to a future AccountInvoice. The set of held InvoiceItems includes any non-separate InvoiceItems that the IInvoice.shouldHoldDirectBillInvoiceItem() plugin method determines must be held.

The next step is to determine if the entire AccountInvoice is carried forward. This is determined by calling IInvoice.shouldCarryForwardInvoice(AccountInvoice). If this plugin method returns true, then all

InvoiceItems on the current AccountInvoice are added to the next planned AccountInvoice, and the current AccountInvoice.Status is marked as Carried Forward.

If the AccountInvoice meets the following conditions, then a writeoff is created for the AccountInvoice and AccountInvoice.Status is marked as Written Off. The conditions are:

- AccountInvoice.Account.BillingPlan.SuppressLowBalInvoices is set to true.
- AccountInvoice.Amount is less than AccountInvoice.Account.BillingPlan.LowBalanceThreshold.
- AccountInvoice.Account.BillingPlan.LowBalanceMethod equals Write Off.

If the AccountInvoice is neither written off nor carried forward, then invoice fees are added as necessary. Invoice fees are not added to account invoices that have been pre-paid. The amount of the invoice fee is determined by calling the IInvoice.getInvoiceFeeAmount() plugin implementation method. If the invoice fee amount is non-zero, then a Charge with the Invoice Fee ChargePattern is added to the AccountInvoice.Account.

If AccountInvoice.Account.BillingPlan.SkipInstallmentFees is false, then installment fees are added as necessary. Installment fees are only added for InvoiceItems that are linked to a PolicyPeriod. Installment fees are added for each PolicyPeriod on a given AccountInvoice. An installment fee is also added for the deposit if the PolicyPeriod payment plan's SkipFeeForDownPayment flag is false. The amount of the installment fee for a given PolicyPeriod is determined by calling the IInvoice.getInstallmentFeeAmount() plugin method.

If the AccountInvoice.Account.PaymentMethod is **CreditCard**, **ACH**, or **Wire**, then a PaymentRequest entity is created. A PaymentRequest schedules a future payment that can be used to delay an imminent delinquency. A PaymentRequest does not affect the financial totals because it has not been received yet. When the actual payment is received, a Payment entity is created to process it. The StatusDate, RequestDate, and DraftDate for the PaymentRequest are initialized to be the AccountInvoice.EventDate plus the AccountInvoice.Account.BillingPlan.RequestInterval. Initially, the PaymentRequest.Status is Created. Payment Request batch processing later updates PaymentRequest.Status.

The AccountInvoice.Status is updated from Planned to Billed.

Each InvoiceItem on the AccountInvoice executes a billed transaction. If InvoiceItem.Amount is positive, then the ChargeBilled transaction is used. This transaction moves funds from the Unbilled T-account to the Billed T-account of the InvoiceItem.Charge. If InvoiceItem.Amount is negative, then the NegativeChargeBilled transaction is used. This transaction moves funds from the Unbilled T-account of the InvoiceItem.Charge, through its Billed and Due T-accounts to the T-account owner's Unapplied T-account. In other words, the NegativeChargeBilled transaction pushes the credit represented by the negative charge into the Unapplied T-account, where that credit can then be used to pay positive charges.

After all invoicing transactions have been executed, the charge payment algorithm is called on all T-account owners related to the AccountInvoice. This call allows positive invoice items to be paid by the credit from negative invoice items.

After all invoice items have been billed and credits applied, snapshot data is generated for the AccountInvoice. This snapshot data captures the balances of the AccountInvoice at the time it was billed. The snapshot data is stored on the AccountInvoice itself, and is populated by calls to the following plugin methods on IInvoice:

- getOutstandingAmount()
- getRemainingBalance()
- getUnappliedAmount()
- getCollateralOutstandingAmount()
- getCollateralRemainingBalance()
- getCollateralUnappliedAmount()
- setPolicyPeriodInvoiceSnapshot() for each PolicyPeriod related to the AccountInvoice.

Invoice Due work queue

Code	InvoiceDue
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 3:15 a.m.
Class	InvoiceDueWorkQueue.java

In BillingCenter, Invoice Due batch processing marks appropriate invoices as past due. BillingCenter selects an invoice for processing based on its due date and an adjusted current date and time. The current date and time is adjusted by subtracting the time specified in the InvoicePeriodDueCutoff configuration parameter. The default time of InvoicePeriodDueCutoff is 12:00 a.m. All invoices with a due date prior to the adjusted time are selected for additional processing.

If a selected invoice has a Status of Billed and its due date has passed, Invoice Due batch processing changes its Status to Due. The batch processing also starts a delinquency process for any past due invoice amounts.

If a selected invoice has a Status of Carried Forward or Written Off and meets the following criteria, Invoice Due batch processing creates the transactions:

- The invoice due date has passed.
- The invoice contains items without ChargeDue transactions.
- Invoice Due batch processing leaves the invoice Status unchanged.

Legacy Agency Bill work queue

Code	LegacyAgencyBill
Categories	UIRunnable, Schedulable
Implementation	Work queue
Workers	0
Schedule	Once only, if needed
Class	LegacyAGBLWorkQueue.java

IMPORTANT: Run the Legacy Agency Bill work queue once only, after loading staging tables during a conversion project. Run this process with exactly one worker as it is not safe to run with multiple workers. Disable the Legacy Agency Bill work queue in a production environment.

In BillingCenter, the Legacy Agency Bill work queue starts workflows after successful completion of a staging table load. The process determines those agency cycle processes for which the agency bill plan is Legacy Agency Bill Plan. For those agency cycle process, the process sets the AgencyBillWorkflow attribute of the agency cycle process.

By default, BillingCenter disables the Legacy Agency Bill work queue in the base configuration by setting the number of workers in work-queue.xml to 0. You must set the number of workers in this file to 1 to be able to run this process.

Disable this process after work queue completes its work by resetting the number of workers back to 0.

Legacy Collateral work queue

Code	LegacyCollateral
-------------	------------------

Categories	UIRunnable, Schedulable
Implementation	Work queue
Workers	0
Schedule	Once only, if needed
Class	LegacyCollateralWorkQueue.java

IMPORTANT: Run the Legacy Collateral work queue once only, after loading data from staging tables during a conversion process. Run this work queue with exactly one worker as it is not safe to run with multiple workers. Disable the Legacy Collateral work queue in a production environment.

Run the Legacy Collateral work queue after importing `Collateral` or `LetterOfCredit` entities into `BillingCenter` through staging tables. The process runs the collateral allocation method `CollateralImpl.touchAndMarkComputedAmountsDirty` on all `Collateral` entities in the system. This action updates the denormalization fields on the `Collateral` entities, which are otherwise too complicated to update in a loader callback.

By default, `BillingCenter` disables the Legacy Collateral work queue in the base configuration by setting the number of workers in `work-queue.xml` to 0. You must set the number of workers in this file to 1 to be able to run this process.

Disable this process after work queue completes its work by resetting the number of workers back to 0.

Legacy Delinquency work queue

Code	LegacyDelinquency
Categories	UIRunnable, Schedulable
Implementation	Work queue
Workers	0
Schedule	Once only, if needed
Class	LegacyDelinquencyWorkQueue.java

IMPORTANT: Run the Legacy Delinquency work queue once only, after loading data from staging tables during a conversion process. Run this batch process with exactly one worker as it is not safe to run with multiple workers. Disable the Legacy Delinquency work queue in a production environment.

Before you run this process, create a delinquency plan in `BillingCenter` based closely on the logic used in the legacy system. This logic must correspond to the events in the workflow section of the delinquency plan.

In `BillingCenter`, the Legacy Delinquency work queue operates on delinquency processes that meet the following criteria:

- Has an associated delinquency plan with the name `Legacy Delinquency Plan`
- Has a status of `Open`
- Has no associated workflow

After the load, this process starts a workflow for each active delinquency process. Delinquency processes with an `Open` status are active. Delinquency processes with a `Closed` status are past. They are for historical purposes, so their events have happened already. You can see these past events from the **Delinquency** screen for the account.

By default, Guidewire disables the Legacy Delinquency work queue in the base configuration by setting the number of workers in `work-queue.xml` to 0. You must set the number of workers in this file to 1 to be able to run this process.

Disable this process after work queue completes its work by resetting the number of workers back to 0.

Letter of Credit work queue

Code	LetterOfCredit
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 2:40 a.m.
Class	LetterOfCreditWorkQueue.java

In BillingCenter, the Letter Of Credit work queue operates on LettersOfCredit entities with a status of Current using the LetterOfCreditStatus.TC_CURRENT.get() method and an ExpirationDate that is before the current date. For qualifying LettersOfCredit entities, the process changes their status to Expired. This causes the collateral allocation algorithm to run.

The collateral allocation algorithm then uses available collateral, such as cash or letters of credit, to fulfill the various collateral requirements.

New Payment work queue

Code	NewPayment
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 3:10 a.m.
Class	NewPaymentWorkQueue.java

In BillingCenter, the New Payment work queue allocates automatic payments on an account to the corresponding account or policy charges. The process also allocates payments entered manually into the BillingCenter **Enter Multiple Payments** screen. This process distributes available money according to the account distribution settings.

The New Payment process distributes the money as follows:

- Searches for direct bill money received by BillingCenter (DirectBillMoneyRcvd), but which BillingCenter has not yet distributed, and for which there is no reversal.
- Calls the distribute method on each DirectBillMoneyRcvd entity found

Payment Request work queue

Code	PaymentRequest
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 3:25 a.m.
Class	PaymentRequestWorkQueue.java

In BillingCenter, the Payment Request work queue selects PaymentRequest objects that match the following criteria:

- The value of the payment request StatusDate must fall on or before the current date.
- The value of the payment request Status is not Closed or Canceled.

For those requests the process selects, it advances the values of their StatusDate and Status fields.

The RequestDate field is shown in some parts of the user interface. However, the batch process uses the value in the StatusDate field to select payment requests. Best practice recommends that the RequestDate and StatusDate fields

reference the same date. Whenever you make a change to one field, Guidewire recommends that you assign the other field the equivalent setting.

The number of `PaymentRequest` objects processed by each work item can be specified by the configuration parameter `PaymentRequestsPerWorkItem`.

Policy Closure work queue

Code	<code>PolicyClosure</code>
Categories	<code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Once a day, at 4:15 a.m.
Class	<code>PolicyClosureWorkQueue.java</code>

In `BillingCenter`, the Policy Closure work queue checks each policy period to see if it meets the criteria necessary to be closed. It is not possible to bypass any of the Policy Closure criteria. However, it is possible to test for additional criteria by configuring the Event Handler plugin method `canClosePolicyPeriod`.

`BillingCenter` closes policy periods that meet all the following criteria (plus any additional criteria implemented in the `canClosePolicyPeriod` method):

- The policy period is currently open.
- The policy period has no remaining billed, unbilled, or due balances.
- If the policy period has not been canceled then:
 - The current date must be equal to or later than the policy period's expiration date plus the number of days specified in the configuration parameter `PolicyClosureBufferDays`.
- If the policy period has been canceled then:
 - The cancellation date equals the policy period's effective date. In other words, the policy period was flat-canceled.
 - All invoices for the policy period are due.
 - The current date must be equal to or later than the policy period's effective date plus the number of days specified in the configuration parameter `PolicyClosureBufferDays`.
- If the configuration parameter `EnableChargeProRataTxCreation` is `true`, then all premium has been earned.

Policy Period Closure criteria technical details

Checking closure criteria

In `BillingCenter`, the Policy Closure batch process checks each policy period for the following criteria.

- `PolicyPeriod.ClosureStatus` is `OPEN`.
- `PolicyPeriod` fields `BilledAmount`, `UnbilledAmount`, and `DueAmount` are zero.
- If `PolicyPeriod.CancelStatus` is not `CANCELED` then:
 - The current date must be equal to or later than the `PolicyPeriod.PolicyPerExpirDate` plus the number of days specified in the configuration parameter `PolicyClosureBufferDays`.
- If `PolicyPeriod.CancelStatus` is `CANCELED` then:
 - The cancellation date equals the `PolicyPeriod.PolicyPerEffDate`. In other words, the policy period was flat-canceled.
 - The `Invoice.Status` field of all invoices for the policy period is `DUE`.

- The current date must be equal to or later than the `PolicyPeriod.PolicyPerEffDate` plus the number of days specified in the configuration parameter `PolicyClosureBufferDays`.
- If the configuration parameter `EnableChargeProRataTxCreation` is `true` then each of the policy period's `ChargePattern` objects of Subtype `PRORATACHARGE` (for example, a premium charge) must have no remaining unearned amount.

Some of the criteria are checked by the batch process's writer, other criteria are checked by the worker, and some criteria are checked by both the writer and the worker.

If the criteria checked by the writer are met, the process adds a work item to the `bc_policyclosureworkitem` table.

If the criteria checked by the worker are not met, a `CannotCloseException` is logged and the related policy period is not closed.

If all the criteria are met, the worker calls the Event Handler plugin method `canClosePolicyPeriod`. If the method returns `false` or `null` then the policy period is not closed. The base configuration method always returns `true`.

Closing the policy period

While closing a policy period that has met the required criteria, the following operations are performed by the worker in the order listed.

- Set each `CollateralRequirements.Compliance` to `CLOSED`.
- For each `PolicyCommission`:
 - Calculate and distribute any earned incentive bonus. Calls the Incentive Calculation plugin method `calculatePolicyBasedIncentiveBonus`.
 - Refresh the commission earnings for commissions that are payable on `EffectiveDate` or `ExpirationDate` or are based on a Custom payment criteria. The intention of this operation is to handle any funds remaining in Commission Reserve.
- Set `PolicyPeriod.CloseDate` to the current date.
- Set `PolicyPeriod.ClosureStatus` to `CLOSED`.
- Calculate and set `PolicyPeriod.NextArchiveCheckDate` which is the date when the policy period becomes eligible to be archived. The check date is the current date plus the number of days specified in the configuration parameter `ArchivePolicyPeriodDays`.

Post Batch Payments work queue

Code	<code>PostBatchPayments</code>
Categories	<code>APIRunnable</code> , <code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Not scheduled
Class	<code>PostBatchPaymentsWorkQueue.java</code>

In `BillingCenter`, the Post Batch Payments work queue identifies all batch payments that meet the following criteria:

- The batch payment status is *Posting*.
- The batch payment is not currently being processed by any other instance of a process.

For each batch payment that the work queue identifies:

- The work queue creates a payment or suspense payment as needed.
- The work queue change the status of the batch payment to *Posted* after the work is complete.

Note: Note that the payments are not allocated. The money remains in the appropriate unapplied fund.

You can configure the frequency and exact time that the Post Batch Payment work queue runs. In the base configuration, BillingCenter does not schedule or enable this work queue.

Premium Reporting Report Due work queue

Code	PremiumReportReportDue
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 2:55 a.m.
Class	PremiumReportDDWorkQueue.java

In BillingCenter, the Premium Reporting Report Due work queue checks to see whether a PremiumReportDueDate entity became past due without receipt of a matching PremiumReportBI for that policy. If the process finds an entity for which this is true, the process calls the `shouldStartFailureToReportDelinquency` method in the PremiumReport plugin implementation to determine whether to start a delinquency.

In the base configuration, method `shouldStartFailureToReportDelinquency` returns true if all of the following are true:

- The PremiumReportDueDate entity due date has past.
- There is no matching PremiumReportBI.

If the method does return true, InsuranceSuite application starts a FailureToReport delinquency on the PremiumReportDueDate.PolicyPeriod.

Producer Payment work queue

Code	ProducerPayment
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 3:55 a.m.
Class	ProducerPaymentWorkQueue.java

In BillingCenter, the Producer Payment work queue manages payments to producers. Commissions are the most common type of producer payment. The Producer Payment process generates a producer statement that accumulates the payments for a specific time period. An external system integrated with BillingCenter then creates the physical payment in the form of a check, ACH transfer, or other monetary mechanism.

Processing earned commission

Commission payments are typically paid automatically according to an automatic payment schedule. A producer's automatic payment schedule is defined by two fields on the producer **Summary** screen: **Payment Frequency** and **Commission Day of Month**. The **Payment Frequency** specifies the payment interval at which to generate producer statements, such as monthly or quarterly. The **Commission Day of Month** field specifies the day of the month to generate the producer statement.

Earned commissions are calculated and processed whenever a transaction that generates an earned commission occurs. In addition, all earned commissions are immediately transferred to and accumulated in the Commissions Payable T-account at the Producer level of the commission hierarchy. Commissions earned at the lower levels do not remain at that level. Instead, they are transferred immediately to the Producer level. As a result, the Commissions Payable T-accounts at the ProducerCode, PolicyCommission, and ChargeCommission levels always maintain a balance of zero. Because of this behavior, earned commissions are maintained only at the Producer level in the Commissions Payable

T-account. Finer granularities of earned commissions, such as at the policy period level or charge level are not available.

The Commissions Payable T-account of the Producer accumulates earned commissions until the producer's statement date. At that time, a producer statement is generated and the accumulated earned commission is actually paid. Producer statements are managed and generated by the Producer Payment batch process.

A scheduled producer statement can be skipped and never generated if the statement's payable amount does not meet a minimum threshold. However, exceptions exist that will force a producer statement to be created even if the minimum threshold is not met. One such exception is the existence of a bonus commission or advance commission payment that has been scheduled with the **Pay With Next Payment** setting. All entries on a skipped statement are carried over to the next scheduled producer statement.

Producer Payment technical details

In BillingCenter, the Producer Payment batch process performs two core operations.

- It moves transactions from the collecting statement to the appropriate pending statement. Pending statements act as in-progress producer statements.
- On the appropriate date, it processes an in-progress producer statement to generate a final producer statement.

The following sections describe the operations performed by the batch process. The operations are described in the order in which they occur.

Selecting producers to process

The Producer Payment writer retrieves each producer that has a transaction on its collecting statement.

A special case exists with the `ProducerTransaction` subtype of `ProducerPayableTransferTxn`. Such transactions affect two producers: a source producer and a destination producer. The transaction must be included on the producer statement of each producer. The source producer will already have the transaction on its collecting statement. The destination producer is retrieved separately and then added to the group of producers to process.

Finally, each producer is retrieved who must have their producer statement generated on the current day. The generation of the statement is determined if a `ProducerStatement` owned by a `Producer` meets the following criteria.

- The producer statement has not already been processed (`ProducerStatement.Processed` field is `false`).
- The current date is either on or after the day on which to generate the statement (`ProducerStatement.StatementDate` $\leq$ current date).
- The producer statement date is either on or before the last date for which producer statements are generated (`ProducerStatement.StatementDate` $\leq$ `Producer.FinalRecurDate`).

All subsequent operations on the retrieved producers are performed by batch process workers.

Moving transactions from the collecting statement to the pending statement

Pending statements are in-progress producer statements. They accumulate producer payment transactions incrementally as each transaction occurs. Pending statements are updated whenever the Producer Payment batch process is run.

Pending statements are stored in the `Producer.Statements` array. The same array also stores final processed producer statements. Pending statements are differentiated from final processed producer statements by the `ProducerStatement.Processed` field. Pending statements have a field value of `false`, while processed producer statements have a value of `true`. Processed producer statements cannot be modified.

A batch process worker moves each transaction on the producer's collecting statement to the appropriate pending statement. In terms of class fields, the `Producer.CollectingStatement` transactions are moved to the appropriate `Producer.Statements` array element. A transaction is placed on the pending statement that covers the time period in which the transaction occurred. In most cases, the transaction is placed on the current in-progress pending statement which will be generated at the next statement date.

Transactions moved to the pending statement no longer exist on the collecting statement. As a result, at the completion of performing the operation, the collecting statement is empty.

After moving all transactions from the collecting to the pending statement, the modified Producer is committed to the database.

Generating the producer statement

Each producer is examined to determine whether to generate a producer statement. The process of generating a producer statement is initiated if the following criteria are met.

- The current date is on or after the `ProducerStatement.StatementDate`.
- Either the `Producer.FinalRecurDate` is null or the `ProducerStatement.StatementDate` is on or before the `Producer.FinalRecurDate`. The `FinalRecurDate` is the last date for which producer statements are generated. For example, assume statement dates are generated on the first day of every month and the `FinalRecurDate` is February 28. If the batch process is run on March 15, it can generate statements for any day prior to February 28, such as February 1. Also, notice that even though the batch process was run after the `FinalRecurDate`, statements dated up to the final date are still generated.

During statement generation, the batch process can still determine to skip the statement. If the statement is skipped, all transactions on the statement are moved to the next pending statement. The skipped statement is then removed from the `Producer.Statements` array and also from the database.

When a producer statement is generated, the following operations occur in the order in which they are described.

1. Execute any delayed commission payments scheduled to occur with the next payment.

Delayed commission payments must meet the following criteria:

- The scheduled time of the payment is “pay with next payment” (`DelayedCmsnPayment.PaymentTime = TC_NEXTPAYMENT`).
- The commission payment has not already been paid (`DelayedCmsnPayment.Paid = false`).
- The payment has been approved (`DelayedCmsnPayment.ApprovalStatus = TC_APPROVED`).
- The time of the payment (`DelayedCmsnPayment.CreateTime`) is within the time frame covered by the generated statement.

After the delayed commission payment is executed, the following fields are updated and the amount of the payment is added to the statement’s payment balance.

- The `DelayedCmsnPayment.ProducerPayment` field is assigned the processed `ProducerPayment` object.
- `DelayedCmsnPayment.Paid` is set to true.

2. Determine whether to generate or skip the statement.

A statement is generated, regardless of other criteria, if it contains a delayed commission payment.

A statement is skipped if it meets the following criteria. Otherwise, the statement is generated.

- The option is enabled to hold a statement if its payment balance is not within a specified threshold range (`Producer.HoldStatement` is true).
- The statement payment balance $\geq$ `Producer.StatementHoldNegativeLimit`.
- The statement payment balance $\leq$ `Producer.StatementHoldPositiveLimit`.

3. Determine whether to make a payment.

The payment is skipped if any of the following criteria are met, otherwise, the payment is made.

- The statement balance is not positive.
- There is no delayed commission payment on the statement *and* the statement balance does not meet the minimum threshold returned by the System Parameters plugin method `getProducerAutoPaymentThreshold`. In the base configuration, the method returns null, which specifies that no threshold exists. For additional details, see the *Plugins, Prebuilt Integrations, and SOAP APIs*.
- The balance of the producer’s Commissions Payable T-account is not positive.

4. Make the payment

The payment is made using the following operations:

- Any advance amount that was repaid as of the statement date is processed by creating a `ProducerAdvanceRepaid` transaction. The balance in the Commissions Payable T-account is also applied to repay any advance. The transaction is added to the producer statement.
- A `ProducerPaymentSent` transaction is created.
- The `ProducerStatement.StatementNumber` field is assigned a generated statement number. The number is generated by calling the Number Generator plugin method `generateStatementNumber`. For details, see the *Plugins, Prebuilt Integrations, and SOAP APIs*.
- The `ProducerStatement.Processed` field is set to `true`.
- The `ProducerStatement.NetAmountSentAndReceived` field is set to the total net payments sent and received within the time period covered by the statement. Payments include bonuses, advances, standard commission payments, and automatic payments. Subtracted from these outgoing payments are any incoming producer payments. Reversals of any of these items are also included.
- The `ProducerStatement.Balance` field is set to any remaining amount that was not paid. The field can be non-zero if the net payment is negative or does not meet the minimum threshold for sending the payment.
- The modified Producer is committed to the database.

Collecting producer payments

As described in the previous section, `BillingCenter` manages and accumulates producer payments incrementally during the period covered by a producer statement. The incremental accumulation is made possible by making commissions and other producer payments payable as soon as they are earned. The payments are then moved immediately to the producer level. `BillingCenter` takes this approach to maximize performance.

When the commission is made payable, the relevant transaction is also stored on the producer's "collecting statement." The collecting statement acts as a temporary storage location. The transactions on the collecting statement are subsequently moved by the Producer Payment batch process to an in-progress producer statement. The collecting statement can be accessed from the `Producer.CollectingStatement` field. It is persisted in the `bc_prodcolctngstmt` table.

The collecting statement can contain only transactions extended from the abstract classes listed below. Each of the abstract classes extends the abstract `ProducerTransaction` class. Also listed are the instantiable transaction types extended from the abstract classes.

Base Abstract Class	Instantiable Transaction Classes
CommissionEarned	PolicyIncentivesEarned
	ProdCodeBonusEarned
	ProducerAdvanceExpired
	ProducerAdvancePayable
	ProducerAdvanceRepaid
	ProducerPayableTransferTxn
	ReserveCmsnEarned
CommissionPayable	PolicyCmsnPayable
	ProdCodeCmsnPayable
ProducerPaymentTxn	ProducerPaymentRecd
	ProducerPaymentSent

Release Charge Holds work queue

Code	ReleaseChargeHolds
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 2:10 a.m.
Class	ReleaseChargeHoldWorkQueue.java

In BillingCenter, the Release Charge Holds work queue operates on charge holds with release dates that have passed and releases them. Each charge has a date, defined by `HoldReleaseDate`, that marks the charge release date. After release, the charge is available for invoicing.

For charge holds that have expired, the process sets the `Hold` field to `ChargeHoldStatus.TC_None`.

Release Trouble Ticket Hold Types work queue

Code	ReleaseTtktholdTypes
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 2:05 a.m.
Class	ReleaseTtktholdTypesWorkQueue.java

In BillingCenter, the Release Trouble Ticket Hold Types work queue operates on `HoldTypeEntry` entities on trouble tickets with release dates that have passed and releases those holds.

Reverse Batch Payments work queue

Code	ReverseBatchPayments
Categories	APIRunnable, UIRunnable, Schedulable
Implementation	Work queue
Schedule	Not scheduled
Class	ReverseBatchPaymentsWorkQueue.java

In BillingCenter, the Reverse Batch Payments work queue identifies all batch payments that meet the following criteria:

- The batch payment status is *Reversing*.
- The batch payment is not currently being processed by any other process.

For each batch payment that the work queue identifies:

- The work queue attempts to reverse the payment.
- If there is a distribution, the work queue attempts to reverse the distribution.
- The work queue change the status of the batch payment to *Reversed* after the work is complete.

You can configure the frequency and exact time that the Post Batch Payment work queue runs. In the base configuration, BillingCenter does not schedule or enable this work queue.

Statement Billed work queue

Code	StatementBilled
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 3:35 a.m.
Class	StatementBilledWorkQueue.java

In BillingCenter, the Statement Billed work queue changes the agency bill `StatementInvoice.Status` to `Billed` for all `StatementInvoices` the meet the following criteria:

- `StatementInvoice.Planned` equals `true`
- `StatementInvoice.EventDate` is equal to or before the date that the batch process runs

For each qualifying `StatementInvoice`, the process calls the `makeBilled` method to:

- Push any invoice items that are the target of a hold to a future invoice.
- Add any installment fees for the `StatementInvoice`.
- Set a line number for each invoice item on the statement: starting at 1, in increments of 1, for each invoice item.
- Create charge invoicing transaction entities, such as `ChargeBilled` or `NegativeChargeBilled`, for each invoice item.
- Recalculate the following denormalized fields for the `StatementInvoice`: `Amount`, `AmountDue`, `NetAmount`, and `NetAmountPaid`.
- Start the agency bill workflow for the `StatementInvoice`.

Statement Billed technical details

In BillingCenter, Statement Billed batch processing changes the agency bill `StatementInvoice.Status` to `Billed` for all `StatementInvoices` the meet the following criteria:

- `StatementInvoice.Planned` equals `true`
- `StatementInvoice.EventDate` is equal to or before the date that the batch process runs

For each qualifying `StatementInvoice`, the process calls the `makeBilled()` method to:

- Push any invoice items that are the target of a hold to a future invoice.
- Add any installment fees for the `StatementInvoice`.
- Set a line number for each invoice item on the statement: starting at 1, in increments of 1, for each invoice item.
- Create charge invoicing transaction entities, such as `ChargeBilled` or `NegativeChargeBilled`, for each invoice item.
- Recalculate the following denormalized fields for the `StatementInvoice`: `Amount`, `AmountDue`, `NetAmount`, and `NetAmountPaid`.
- Start the agency bill workflow for the `StatementInvoice`.

Statement Due work queue

Code	StatementDue
Categories	UIRunnable, Schedulable
Implementation	Work queue

Schedule	Once a day, at 3:40 a.m.
Class	StatementBilledWorkQueue.java

In BillingCenter, the Statement Due work queue changes the status of agency bill StatementInvoices to Due for all StatementInvoice entities in which:

- StatementInvoice.PaymentDueDate is before or on the date that the Statement Due batch process runs.
- StatementInvoice.Billed is true.

For qualifying StatementInvoice entities, the Statement Due process calls the makeDue method to do the following:

- It pushes any invoice items that are the target of a hold to a future invoice.
- It create ChargeDue transactions for each invoice item on the StatementInvoice.
- It recalculate the following denormalized the fields for the StatementInvoice: , , , and .
 - Amount
 - AmountDue
 - NetAmount
 - NetAmountPaid

Suspense Payment work queue

Code	SuspensePayment
Categories	UIRunnable, Schedulable
Implementation	Work queue
Schedule	Once a day, at 3:05 a.m.
Class	SuspensePaymentWorkQueue.java

In BillingCenter, the Suspense Payment work queue searches for the payer of specific payments stored in Unapplied T-accounts. Specifically, payments received by BillingCenter always specify the payer. The payer can be either an account, a producer, or a policy period associated with an account or producer.

It is possible for BillingCenter to receive payments before it receives information about the payer. In such cases, the account, producer, or policy period does not yet exist in BillingCenter. But, after the receipt of the required information, BillingCenter creates the account, producer, or policy period.

If BillingCenter receives a payment from an unknown payer, it places the payment in an Unapplied T-account of type SuspensePayment. BillingCenter places each payment with an unknown payer in its own, separate Unapplied T-account. For example, if ten payments with unknown payers arrive, BillingCenter creates ten Unapplied SuspensePayment T-accounts.

The Suspense Payment process searches for the payer of each payment stored in these Unapplied T-accounts, checking to see if the account or producer exists. If the payer exists in BillingCenter, the process move the payment from the Unapplied SuspensePayment T-account to the appropriate Unapplied T-account of the payer.

The Suspense Payment plugin implementation manages all operations performed by the Suspense Payment work queue. The work queue passes the suspense payments that it identifies to the plugin implementation for processing. It is possible to modify the plugin implementation to configure the functionality of the Suspense Payment work queue to meet your business needs.

Suspense Payment technical details

In BillingCenter, Suspense Payment batch processing searches for the payer of specific payments stored in Unapplied T-accounts. Specifically, payments received by BillingCenter always specify the payer. The payer can be either an account, a producer, or a policy period that is associated with an account or producer.

It is possible for BillingCenter to receive payments before it has been informed about the payer. In such cases, the account, producer, or policy period does not yet exist in BillingCenter, but will presumably be created in the near future.

When a payment is received and the payer is unknown to BillingCenter, the payment is placed in an Unapplied T-account of type `SuspensePayment`. Each payment with an unknown payer is placed in its own, separate Unapplied T-account. For example, if ten payments with unknown payers arrive then BillingCenter creates ten Unapplied `SuspensePayment` T-accounts.

Thus, `Suspense Payment` batch processing searches for the payer of each payment stored in these Unapplied T-accounts, checking to see if the account or producer has been created yet. If the payer is found to exist in BillingCenter, the payment is moved from the Unapplied `SuspensePayment` T-account to the appropriate Unapplied T-account of the payer.

All operations performed by the `Suspense Payment` process are implemented in the implementation of the configurable `Suspense Payment` plugin. Each `suspense payment` handled by the process is passed to the plugin implementation for processing. By modifying the implementation of the `Suspense Payment` plugin, you can configure the functionality of `Suspense Payment` batch processing to perform the desired operations.

Trouble Ticket Escalation work queue

Code	<code>TroubleTicketEsc</code>
Categories	<code>UIRunnable</code> , <code>Schedulable</code>
Implementation	Work queue
Schedule	Once a day, at 2:25 a.m.
Class	<code>TroubleTicketEscalationWorkQueue.java</code>

In BillingCenter, the `Trouble Ticket Escalation` work queue escalates trouble tickets that have a ticket status (`TroubleTicket.TicketStatus`) of `Open`. This process operates on unescalated trouble tickets with escalation dates that are equal to the current date or earlier and flags them as escalated.

Upgrade Producer Statement Hold Limits batch process

Code	<code>UpgradeProducerStatementHoldLimits</code>
Categories	<code>APIRunnable</code> , <code>UIRunnable</code>
Implementation	Batch process
Stoppable	No (single phase process)
Class	<code>UpgradeProducerStatementHoldLimits.gs</code>

In BillingCenter, run the `Upgrade Producer Statement Hold Limits` batch process once only after you upgrade to BillingCenter 9.0.0 to provide default values for the following fields on `Producer`:

- `HoldStatement`
- `StatementHoldNegativeLimit`
- `StatementHoldPositiveLimit`

The default logic for the `Upgrade Producer Statement Hold Limits` batch process populates these values using the `getProducerAutoPaymentThreshold` method on the `ISystemParameters` plugin. However, it is also possible to override the batch process implementation class to arrive at the default values for each producer through custom logic.

Workflow Writer work queue

Code	Workflow
Categories	UIRunnable, Schedulable
Implementation	Work queue
Workers	0
Schedule	Once a day, at 3:45 a.m.
Class	WorkflowDistributedWorkQueue.java

In BillingCenter, the Workflow Writer work queue runs BillingCenter workflow worker tasks. BillingCenter disables this work queue in the base configuration by setting the number of workers for the work queue to 0 in file `work-queue.xml`. To execute this work queue, you must set the number of workers for the work queue to 1 or more.

Write-off Staging work queue

Code	WriteoffStaging
Categories	UIRunnable, Schedulable
Implementation	Work queue
Workers	0
Schedule	Not scheduled
Class	WriteoffStagingWorkQueue.java

IMPORTANT: Run the Write-off Staging work queue once only, after loading staging tables during a conversion project. Run this process with exactly one worker as it is not safe to run with multiple workers. Disable the Write-off Staging work queue in a production environment.

In BillingCenter, run the Write-off Staging work queue only after completing the process of staging table loading. The process distributes any undistributed write-offs and reversals that result from the staging table load process. BillingCenter generates the appropriate T-accounts and transactions automatically during the loading process.

By default, BillingCenter disables Write-off Staging work queue in the base configuration by setting the number of workers in `work-queue.xml` to 0. You must set the number of workers for this work queue to 1 to be able to run this process.

Disable this process after work queue completes its work by resetting the number of workers back to 0.

Other processes

Unused and internal batch processes

InsuranceSuite applications use some batch process types either internally, or, in a few cases, not at all. You cannot run these processes from an InsuranceSuite application, `maintenance_tools`, or a web services API.

Unused processes

The Batch Process Type typelist (`BatchProcessType.tti`) includes a few Guidewire platform processes that PolicyCenter has officially retired. Guidewire indicates this status by setting the `retired` flag on the process typecode to `true` and placing a line through the typecode the typelist in Studio. You can ignore these processes.

In addition, there are other processes listed in the Batch Process Type typelist that Guidewire no longer uses but which you can still see in the typelist, depending on the application version. Examples of these types of processes include the following:

- Aggregate Limit Loader Calculations (Retired and in ClaimCenter only)
- Bulk Purge (In PolicyCenter and retired in BillingCenter)
- Contact Auto Synch (In PolicyCenter and BillingCenter)
- Geocode Writer (BillingCenter only)
- Post Batch Payments (BillingCenter only, do not use without consultation with Guidewire)
- Stat Report (Retired and in BillingCenter only)

Internal processes

InsuranceSuite applications use some Guidewire batch processes internally. For example, an InsuranceSuite application runs some processes to generate database performance reports only. You cannot run internal processes separately. Do not attempt to use the following internal processes manually:

- Bulk Claim Validation Work Queue (In ClaimCenter)
- Business Rule Export/Import
- Business Rule Validator
- Create Perf Only Indexes
- Find Usages Of Upgraded Typecodes
- Microsoft DMV Report
- Staging Table Import

Archive-related processes

Guidewire must specifically enable data archiving in your InsuranceSuite cloud application before you are able to use the archive functionality.

The following batch processes affect data archiving:

- Deferred Upgrade Tasks
- Form Text Data Delete (PolicyCenter only)
- Retired Policy Graph Disconnecter (ClaimCenter only)

The following work queues affect data archiving:

- Archive Item Writer (ClaimCenter only)
- Archive Policy Periods (BillingCenter only)
- Archive Policy Terms (PolicyCenter only)
- Archive Reference Tracking Synchronization (ClaimCenter only)
- Bulk Purge (ClaimCenter only)
- Invoice Item Processing (ClaimCenter only)
- Mark Archived Entity Purge Ready (ClaimCenter only)
- Restore Archived Claims
- Retrieve Policy Period (BillingCenter only)
- Retrieve Policy Term (PolicyCenter only)

For a discussion of how data archiving uses these processes, see *Data Archiving*.

Custom processing

It is possible to implement a custom process to supplement the standard processes that Guidewire provides in the base configuration. In creating a custom process, consider the following factors in deciding between developing a custom work queue and developing a custom batch process:

- The volume of items in a typical batch
- The duration of the batch processing window

For business oriented batch processing, such as invoicing or aging, Guidewire recommends that you develop a custom work queue. Work queues process items in parallel tasks distributed across all servers in a InsuranceSuite application cluster that have the workqueue server role.

IMPORTANT: Regardless of the volume of items or the duration of the processing window, Guidewire recommends that you implement all custom batch processing as a custom work queue.

Manually executing InsuranceSuite application processes

InsuranceSuite applications provide the following Server Tools screens for use by administrators in manually executing the different application processes:

- **Batch Process Info**
- **Work Queue Info**

Batch processing typecodes

InsuranceSuite applications identify and manage application batch processing by typecodes in the `BatchProcessType` typelist. Each type of batch processing, whether implemented as a batch process or a work queue, has a unique typecode in the typelist. Whenever you develop a type of custom batch processing, begin by defining a typecode for it the `BatchProcessType` typelist.

You use the typecode for your type of custom batch processing in the following ways:

- Arguments to some of the methods in your custom work queue or batch process class
- An argument to the maintenance tools command and web service that enable you to start a batch processing run
- Categorizing how your custom batch process can be run, from the administrative user interface, on a schedule, or from the maintenance tools command prompt or web service
- A case clause in the Batch Processing Completion plugin for your type of batch processing

Regardless the mode of batch processing you implement, first define a new typecode in the `BatchProcessType` typelist for your type of batch processing..

About scheduling InsuranceSuite application processes

It is possible to schedule many, if not most, of the InsuranceSuite application processes for execution at periodic intervals. Guidewire recommends that you take the following information into account in setting up a processing schedule.

Night-time batch processing

Processes business transactions accumulated at the end of specific business periods, such as business days, months, quarters, or years.

Day-time batch processing

Defers to periodic asynchronous background processing complex transactional processing triggered by user actions.

Depending on your custom process, certain implementation details can vary.

The scheduler configuration file

File `scheduler-config.xml` defines the default schedule that InsuranceSuite applications use to launch many of the processes, including writer processes for work queues. You can find this file in the following location in the Studio **Project** window:

configuration > config > scheduler

It is possible to define your own schedule for a process.

Night-time processing

Night-time processing typically processes relatively large batches of work, such as processing premium payments that accumulate during the business day. Each type of night-time processing runs once during the night, when users are not active in the application.

Because most kinds of night-time processing typically operate on large batches and have rigid processing windows, develop your night-time process as a custom work queue. Night-time processing often requires processing items in parallel to achieve sufficient throughput. Large workloads of nightly processing are typically too severe for the servers in a InsuranceSuite application cluster that have the batch server role.

For night-time batch processing, you typically configure a custom work queue table to segregate the work items of different work queues from each other. Each type of night-time processing typically has many worker tasks simultaneously accessing the queue for available work items. Using the same work queue table for all types of nightly work queues can degrade processing throughput due to table contention. In addition, segregating work items from different work queues into separate tables can ease recovery of failed work items before the nightly processing window closes.

Night-time processing frequently requires chaining so that completion of one type of batch processing starts another type of follow-on processing. Regardless of implementation mode – work queue or batch process – you most likely must develop custom process completion logic for your type of night-time processing.

Daytime processing

Typically, daytime processes are relatively small batches of work, such as reassigning activities escalated by users. Daytime processes run frequently during the day, while users are active in the application. You might schedule different types of daytime processes to run every hour or even every few minutes.

Even though daytime processing typically operates on small batches and lacks rigid processing windows, develop your type of daytime batch processing as a custom work queue. Although daytime processing often achieves sufficient throughput by processing items sequentially, its workload on the servers with the batch server role can slow overall performance of the application. As a work queue, worker tasks on multiple servers (with the workqueue server role), can perform the work in parallel.

If you develop your daytime processing as a custom work queue, you typically can use the standard work queue table for your work items. Different types of daytime processing run intermittently and in different bursts of relatively short work. So, table contention between various types of daytime batch processing often is minimal.

Daytime processes seldom requires chaining. Completion of one type of daytime process seldom starts another type of follow-on processing. Regardless of implementation mode, you most likely do not need to develop custom process completion logic for your type of daytime processing.

Custom work queues

A work queue is code that runs without human intervention as a background process on multiple servers to process the units of work for a batch in parallel. Develop a custom work queue if you require the processing of units of work in parallel to achieve an acceptable throughput rate.

About custom work queues

A custom work queue comprises the following components.

Writer

The `findTargets` method on a Gosu class that extends the `WorkQueueBase` class that selects the units of work for a batch and writes work items for them in the work queue table.

Work queue

An entity type that implements the `WorkItem` delegate, such as the `StandardWorkItem` entity, to establish the database table for the work queue and its work items.

Worker

The `processWorkItem` method on the same Gosu class that extends the `WorkQueueBase` base class that processes the units of work identified on the work queue by work items

Starting the writer initiates a run of the type of batch processing that a work queue performs. The batch is complete when the workers exhaust the queue of all work items in the batch, except those they fail to process successfully.

About custom work queue classes

You implement the code for your writer and your workers as methods on a single Gosu class. You must derive your custom work queue class from the `WorkQueueBase` class. This base class and the work queue framework provide most of the logic for managing the work items in your work queue. You typically need override only few methods in the base class to implement the code for your writer and your workers.

Work queue writer

You implement the writer for your work queue by overriding the `findTargets` method inherited from the base class. Your code selects the units of work for a batch and returns an iterator for the result set. The work queue framework then uses the iterator to create and insert work items into your work queue. Each work item holds the instance ID of a unit of work in your result set.

For example, your custom work queue sends email about overdue activities to their assignees. In this example, instances of the `Activity` entity type are the units of work for the work queue. In your `findTargets` methods, you query the database for activity instances in which the last viewed date of an open activities exceeds 30 days. You return an iterator to the result set, and then the framework creates a work item for each element in the result.

Work queue workers

You implement the workers of your work queue by overriding the `processWorkItem` method inherited from the base class. The work queue framework calls the method with a work item as the sole parameter. Your code accesses the unit of work identified in the work item and processes it to completion. Upon completion, your code returns from the method, and the work queue framework deletes the work item from the work queue.

For example, the units of work for your work queue are open activities that have not been viewed in the past 30 days. In your `processWorkItem` method, you access the activity instance identified by the work item. Then, you generate and send an email message to the assignee of that activity. After your code sends the email, it returns from the method. The framework then deletes the work item and updates the count of successfully processed work items in the process history for the batch.

Work queues and work item entity types

A work item is an instance of entity type that implements the `WorkItem` delegate, such as `StandardWorkItem`. The entity type provides the database table in which the work items for your custom work queue persist. Work items in the

work queue table are accessible from all servers in an InsuranceSuite application cluster. Thus, workers can access the work items asynchronously, in parallel, and distribute the work items to servers with the workqueue server role.

Work items have many properties that the work queue framework uses to manage work items and the process histories of work queue batches. If you use `StandardWorkItem` for your work queue, the only field on a work item that your custom code uses is the `Target` field. The `Target` field holds the ID of a unit of work that your writer selected. The code for your writer and your workers can safely ignore the other properties on work item instances. However, you can define a custom work item type with additional fields for your custom code to use.

Custom work item types

Guidewire recommends that you define your custom work item entities as `keyable`, rather than `retireable`. If you define a custom work item as `retireable`, then you also need to create a custom batch process to purge the work items after the InsuranceSuite application processes them and they become old and stale. This batch process also needs to clean up failed work items as well. Any delay in purging failed or stale work items can potentially prevent further operations.

Work queues that use `StandardWorkItem`

Guidewire supports creating multiple work queues that use the same work item type only if that work item is the `StandardWorkItem` type.

In the data model, the `StandardWorkItem` entity type implements the `WorkItem` delegate. The `StandardWorkItem` entity type thus inherits the following from the `WorkItem` delegate:

- Methods to manipulate the work item
- Entity fields that store such items as `Status`, `Priority`, `Attempts`, and other similar types of information

Guidewire marks the `StandardWorkItem` entity type as `final`. Thus, it is not possible to subtype this entity type.

The `StandardWorkItem` entity type contains a `QueueType` typekey, which obtains its value from the `BatchProcessType` typelist. It is the use of this typekey that makes it possible for multiple work queues to use the `StandardWorkItem` type simultaneously.

For example, to query for all `StandardWorkItem` work items of a certain type, use a query that is similar to the following:

```
var target = Query.make(StandardWorkItem).compare(StandardWorkItem#QueueType, Equals, BatchProcessType.TC_CUSTOMWORKITEM)
```

This query returns all work items for the process that match the filter criteria. The query includes failed work items and work items in progress, not just the work items available for selection.

Error: Two work queues cannot use the same work item type

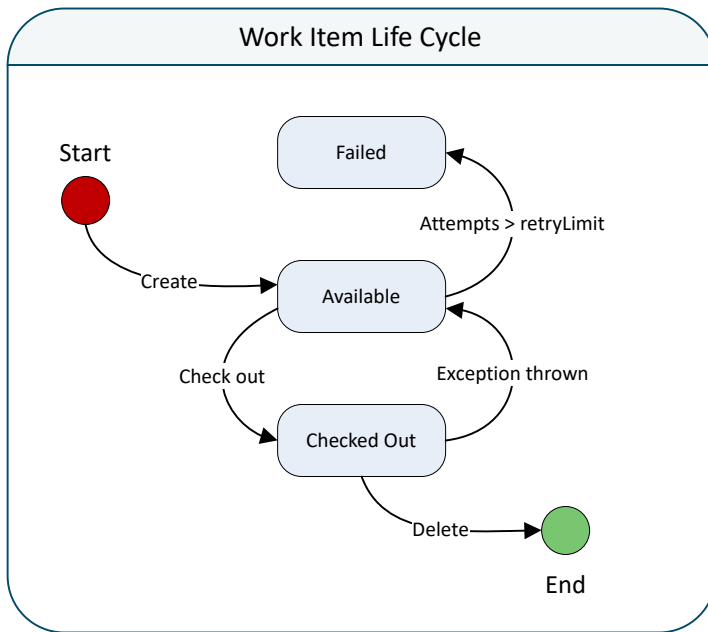
Guidewire does not support creating multiple work queues that use the same work item type, except for the `StandardWorkItem` type. If you attempt to do so using a work item type other than `StandardWorkItem`, the InsuranceSuite application throws the following error:

```
Two work queues cannot use the same work item type at runtime: workItemType.Name
```

It is not possible for workers from different work queues to process work items from the same work item database table. In this circumstance, it is not possible for the different sets of workers to determine which work items to process. In any case, it is very likely that there can be database contention issues along with race conditions between the workers in picking up the work items from the queue.

Lifecycle of a work item

The `Status` field of a work item records the current state of its lifecycle. The work queue framework manages this field and the lifecycle of work items for your writer and workers. The following diagram illustrates the lifecycle.



A work item begins life after the writer selects the units of work for a batch and returns an iterator for the collection. The framework then creates work items that reference the units of work for a batch. The initial state of a work item is available. At the time the framework checks out a quota of work items for a worker, the state of the work items becomes checkedout. The framework then hands the quota of work items to the worker one at a time. After a worker finishes processing a work item successfully, the framework deletes it from the work queue and updates the statistics in the process history for the batch.

Returning work items to the work queue

Sometimes a worker cannot finish processing a work item successfully. For example, a network resource may be temporarily unavailable. Or, a concurrent data change exception (CDCE) can occur if multiple workers simultaneously update common entity data in the domain graphs of two separate units of work. Whenever errors occur, the worker throws an exception to return the work item to the work queue as available again.

Whenever a worker returns a work item to the work queue, the InsuranceSuite application increments the `Attempts` property on the work item. If the value of `Attempts` remains below the retry limit for the worker, the state of the work item remains available. The work item is subsequently checked out in a quota for the same or another worker. If the temporary error condition clears, the next worker completes it successfully and the InsuranceSuite application removes the work item from the work queue.

Work items that fail

Whenever a worker returns a checked out work item and the `Attempts` property exceeds the work item retry limit, the state of the work item becomes failed. Failed work items end their lifecycle in this state. Workers ignore items with a status of failed and no longer attempt to process them. Someone must determine the reason for the failure of a work item and take corrective action. They can then remove the failed work item from the work queue.

Considerations for developing a custom work queue

Before you develop the Gosu code for your custom work queue, you need to decide which of the following you want to use for your work queue table:

- A `StandardWorkItem` entity type
- A custom work item type

The following reasons outline why you would want to define a custom work item type:

- Include custom fields on work items not available on the `StandardWorkItem` entity type
- Avoid table contention with other work queues that typically process large batches at the same time

Whenever you define a custom work item type, you must implement the `createWorkItem` method that your custom work queue inherits from `WorkQueueBase`.

If you create a custom work queue, your writer can pass more fields to the workers than a target field. On `StandardWorkItem`, the `Target` field is an object reference to an entity instance, which represents the unit of work for the workers. In a custom work item, you can define as many fields as you want to pass to the workers. Your custom work item itself can be the unit of work.

Guidewire recommends that you use custom work queue types for work queues with typically large batches that potentially run at the same time as other work queues with large batches. Especially if developing a custom work queue for night-time processing, consider using a custom work queue subtype instead of using `StandardWorkItem`. If you create a custom work item subtype to avoid table contention, you often define a single field for the writer to set, much like the `Target` field on `StandardWorkItem`. By convention, you name the single field with the same name as the entity type, not the generic name `Target`.

IMPORTANT: Do not create multiple work queues that use the same work item type other than the `StandardWorkItem` type. If you attempt to do so, InsuranceSuite application throws an error.

Linking custom work items to target entities

In creating a custom work item, you need to create a link from the custom work item table to the entity table that is the target of the work item. To create this link, add a column element to your custom work item and use the following parameters.

Name	Value
name	Target
type	softentityreference
nullok	false

For `Target`, enter the name of the entity on which the work items acts. Use `softentityreference` (a soft link) rather than `foreignkey` (a hard link). A `softentityreference` is a foreign key for which the InsuranceSuite application does not enforce integrity constraints in the database. The use of a hard foreign key in this context would do the following:

- Require that you delete the work item row from the database before you delete the linked entity row .
- Can force InsuranceSuite application to include the custom work item table in the main domain graph.

Keyable work item entities

Guidewire recommends that you define your custom work item entities as `keyable`, rather than `retireable`. If you define a custom work item as `retireable`, then you also need to create a custom batch process to purge the work items after the InsuranceSuite application processes them and they become old and stale. This batch process also needs to clean up failed work items as well. Any delay in purging failed or stale work items can potentially prevent further operations.

Define a typecode for a custom work queue

About this task

Before you begin developing the Gosu code for your custom work queue, you need to define the custom work queue type. For your custom type, you need to add a typecode for it in the `BatchProcessType` typelist. The constructor of your custom work queue class requires the typecode as an argument.

Procedure

1. Open Guidewire Studio

2. In the Studio **Project** window, expand **configuration** > **config** > **Extensions** > **Typelist**.
3. Open `BatchProcessType.ttx`.
4. Click the green plus button, , to add a typecode.
5. In the panel of fields on the right, specify appropriate values for the following fields:

code	Specify a code value that uniquely identifies your work queue. This code value identifies the work queue in configuration files.
name	Specify a human readable identifier for your work queue. This name appears for the work queue writer on the Server Tools Batch Process Info screen.
desc	Provide a short description of what your custom process accomplishes. This description appears for your work queue on the Server Tools Batch Process Info screen.

Define a custom work item type

Procedure

1. Open Guidewire Studio.
2. Add a new entity for your custom work item type:
 - a) In the Studio **Project** window, expand **configuration** > **config** > **Extensions** > **Entity**.
 - b) Right-click **Entity**, and then select **New** > **Entity**.
 - c) In the **Entity** dialog box, enter the appropriate values.
For example, if creating a new work item entity called `MyWorkItem`, enter the following values:

Field	Example value
Entity	<code>MyWorkItem</code>
Entity Type entity	
Desc	Custom work item
Table	<code>myworkitem</code>
Type	<code>keyable</code>

Accept all the other default values in the dialog box.

- d) Click **OK**.
3. Set the entity to implement the correct work item delegate:
 - a) In the toolbar, in the drop-down list next to the green plus button, , select **implementsEntity**.
 - b) In the panel of fields on the right, select `WorkItem` from the **name** drop-down list.
4. Add a soft reference to the unit of work for your custom work queue:
 - a) Select the **column** field type in the drop-down list next to the green plus button, . Studio inserts a new **column** element in the element table.
 - b) In the panel of fields on the right, enter the entity name for the unit of work for the **name** value.
For example, if the unit of work is an instance of an `Activity` object, add the following values at the panel at the right:

Field	Example value
name	<code>Activity</code>
type	<code>Activity</code>
nullok	<code>false</code>

5. (Optional) Add other fields to your custom entity as meets your business needs.

What to do next

Whenever you define a custom work item for a work queue, your custom work queue class must implement the `createWorkItem` method to write the work item to the queue.

Creating a custom work queue class

After you define a typecode for your custom work queue and possibly create a custom work item type for it, you are ready to create your custom work queue class. This class contains the programming logic for the writer and the workers of your work queue. You must derive your class from `WorkQueueBase`, and you generally must override the following two methods.

<code>findTargets</code>	Logic for the writer, which selects units of work for a batch and returns an iterator for the result set
<code>processWorkItem</code>	Logic for the workers, which operates on a single unit of work selected by the writer

The `WorkQueueBase` provides other methods that you can override such as `shouldProcessItem`. However, you can generally develop a successful custom work queue by overriding the two required methods `findTargets` and `processWorkItem`.

WARNING: Do not implement multi-threaded programming in custom work queues derived from `WorkQueueBase`.

Custom work queue class declaration

In the declaration of your custom work queue class, you must include the `extends` clause to derive your work queue class from `WorkQueueBase`. Because the base class is a template class, your class declaration must specify the entity types for the unit of works and for the work items in your work queue.

The following example code declares a custom work queue type that has `Activity` as the target, or unit of work, type. It also declares that `StandardWorkItem` as the work queue type.

```
class MyWorkQueue extends WorkQueueBase <Activity, StandardWorkItem> { ... }
```

Custom work queue class constructor

You must implement a constructor in your custom work queue class. The constructor that you implement calls the constructor in the `WorkQueueBase` class. Your constructor associates the custom work queue class at runtime with its typecode in the `BatchProcessType` typelist, its work queue item type, and its target type.

The following example code is the constructor for a custom work queue. It associates the class at runtime with its batch process typecode `MyWorkQueue`. The code associates the work queue with the `StandardWorkItem` entity type. So, the work queue shares its work queue table with many other work queues. The code associates the work queue with the `Activity` entity type as its target unit of work type. So, the writer and the workers operate on `Activity` instances.

```
construct () {
    super (typekey.BatchProcessType.TC_MYWORKQUEUE, StandardWorkItem, Activity)
}
```

Do not include any code in the constructor of your custom work queue other than calling the super class constructor.

Developing the writer for your custom work queue

In your custom work queue class, override the `findTargets` method that your custom work queue class inherits from `WorkQueueBase` to provide your specific logic for a writer task.

IMPORTANT: Do not operate on any of the units of work in a batch within your writer logic. Otherwise, process history statistics for the batch will be incorrect.

In the `findTargets` method logic, return a query builder iterator with the results you want as targets for the work items in a batch. InsuranceSuite application uses the iterator to write the work items to the work queue. The `findTargets` method is a template method for which you specify the target entity type, the same type for which you make a query builder object.

The following example code is a writer for a work queue that operates on `Activity` instances. The query selects activities that have not been viewed for five days or more and returns the iterator.

```
override function findTargets (): Iterator <Activity> {
    // Query the target type and apply conditions: activities not viewed for 5 days or more
    var targetsQuery = Query.make(Activity)
    targetsQuery.compare(Activity#LastViewedDate, LessThanOrEquals, java.util.Date.Today.addBusinessDays(-5))
    return targetsQuery.select().iterator()
}
```

Returning an empty iterator

Generally, if your writer returns an iterator with items found by the query, the InsuranceSuite application creates a `ProcessHistory` instance for the batch run. Sometimes the query in your writer finds no qualifying items. If your writer returns an empty iterator, the InsuranceSuite application does not create a process history for that batch processing run.

Writing custom work item types

Suppose that you are creating a custom work queue class and decide not to use the `StandardWorkItem` work item and instead define a custom work item type. If you create your own class, you must override the `createWorkItem` method inherited from `WorkQueueBase` to write new work items to your custom work queue table. The `createWorkItem` method has two parameters.

- **target** – An object reference to an entity instance in the iterator returned from the `findTargets` method.
- **safeBundle** – A transaction bundle provided by InsuranceSuite application to manage updates to the work queue table.

Your method implementation must create a new work item of the custom work item type that you defined. Pass the `safeBundle` parameter in the new work item statement. Then, assign the `target` parameter to the target unit of work field that you defined for your custom work item type. If you defined additional fields, assign values to them, as well.

The return type for the `createWorkItem` method is any entity type that implements the `WorkItem` delegate. Return your new custom work item type.

The following Gosu example code creates a new custom work item that has a single custom field, an `Activity` instance. The implementation gives the target field in the parameter list the name `activity` to clarify the code. The custom work item type is `MyWorkItem`.

```
override function createWorkItem (activity : Activity, safeBundle : Bundle) : MyWorkItem {
    var customWorkItem = new MyWorkItem(safeBundle)
    customWorkItem.Activity = activity
    return customWorkItem
}
```

Developing workers for your custom work queue

In your custom work queue class, override the `processWorkItem` method that your custom work queue class inherits from `WorkQueueBase` to provide the programming logic for a worker task. The workers of a work queue are inherently single threaded. Do not attempt to improve the processing rate of individual workers by spawning threads from within your `processWorkItem` method.

The `processWorkItem` has a work item as its single parameter. You access the target, or unit of work, for the worker through the `WorkItem.Target` property. The type and target type of the work item are the types that you specified in the constructor of your custom work queue class derived from `WorkQueueBase`. At the same time that the InsuranceSuite application calls the `processWorkItem` method, it sets the `Status` field of the work item to `checkedout`.

Successful work items

In your custom work queue class, return from the `processWorkItem` method after a worker finishes operating on a target instance. The InsuranceSuite application then deletes the work item from the work queue.

The following example code extracts the targeted unit of work, an `Activity` instance, from the work item parameter. Then, the code sends the assigned user an email message.

```
override function processWorkItem (WorkItem : StandardWorkItem): void {
    // Extract the unit of work: an Activity instance
    var activity = extractTarget(WorkItem)

    if (activity.AssignedUser.Contact.EmailAddress1 != null) {
        // Send an email to the user assigned to the Activity.
        mailUtil.sendEmailWithBody(null,
            activity.AssignedUser.Contact,
            null,
            "Activity not viewed for five days",
            "Take a look at activity " + target.Subject + ", due on " + target.TargetDate + ".")
    }
    return
}
```

In your override of the base `processWorkItem` method, specify the same work item entity type that you specified in the constructor of your custom work queue class.

Updates to entity data

The abstract `WorkQueueBase.processWorkItem` method does not have a bundle to manage database transactions related to targeted units of work. A bundle does exist at the time the InsuranceSuite application calls your custom `processWorkItem` method, but the InsuranceSuite application uses that bundle for updates to the work item. To modify entity data, you must create a bundle using the `Transaction.runWithNewBundle` API in your custom work queue class.

The following example code uses the `runWithNewBundle` transaction method to update the `EscalationDate` on the `Activity` instance from the work items that is processes.

```
uses gw.transaction.Transaction
...
override function processWorkItem (WorkItem : StandardWorkItem): void {
    // Extract the unit of work: an Activity instance
    var activity = extractTarget(WorkItem)

    // Update the activity escalation date
    Transaction.runWithNewBundle( \ bundle -> {
        activity = bundle.add(activity)
        activity.EscalationDate = java.util.Date.Today } ) // add the activity to the new bundle // update the escalation date

    return
}
```

Managing failed work items

If your custom worker code encounters an error while operating on a target instance, throw an exception. The InsuranceSuite application detects and throws some types of exceptions automatically, such as a concurrent data change exception (CDCE). These types of exceptions generally resolve themselves quickly and automatically. However, it is also possible that your code detects other logic errors that require human intervention to resolve. If so, implement a custom exception and throw it whenever your worker code detects the exceptional situation.

The InsuranceSuite application catches exceptions thrown by code within the scope of your custom `processWorkItem` method. Whenever the InsuranceSuite application catches an exception, it increments the `Attempts` property on the

work item. If the value of the `Attempts` property does not exceed the value of the `WorkItemRetryLimit` parameter set in `config.xml`, it sets the `Status` property of the work item to `available`. Otherwise, it sets the `Status` property of the work item to `failed`.

Retrying work items that cause exceptions

InsuranceSuite application makes work items that trigger exceptions available again on the work queue, because many exceptions resolve themselves quickly. For example, a CDCE exception often occurs whenever two worker tasks attempt to update common data related to their two separate units of work. By the time the InsuranceSuite application gives a work item that encountered an exception to another worker task, the exceptional condition often resolves itself. The second worker then process the work item to completion successfully.

Handling exceptions

In your custom work queue class, provide an implementation of the `handleException` method inherited from `WorkQueueBase` to augment the actions taken whenever code in the `processWorkItem` method throws an exception. For example, it is possible for your worker code to throw an exception whenever it encounters a logic error that cannot resolve itself without human intervention.

Bulk insert work queues

Guidewire recommends that you implement a custom work queue class that extends `BulkInsertWorkQueueBase`, rather than `WorkQueueBase` if it is possible for a query to determine the work items to create.

The use of the `BulkInsertWorkQueueBase` class provides the following performance optimizations:

- The work queue writer does not add duplicate entries to the work queue.
- The use of the bulk insert method eliminates the need to fetch and commit each work item individually.

As a consequence, a work queue that subclasses `BulkInsertWorkQueueBase` performs its work much more quickly than a work queues that subclasses `WorkQueueBase`.

Overview of bulk insert work queues

If your type of process works extremely large work batches, it is possible for the `WorkQueueBase.findTargets` method to return an iterator that exceeds the memory capacity of the work queue server. In some cases, the method returns an iterator for batches that typically number hundreds of thousands of units of work.

Thus, Guidewire recommends that you implement your custom work queue as a class that extends `BulkInsertWorkQueueBase` instead of `WorkQueueBase`. In your class, implement the `BulkInsertWorkQueueBase.buildBulkInsertSelect` method for your query logic, instead of the `WorkQueueBase.findTargets` method.

The InsuranceSuite application calls the `buildBulkInsertSelect` method with a query object that has no restrictions on the target entity type that you specify in the class constructor. However, bulk insert work queues support standard work items only. Thus, do not attempt to use custom fields on work items to pass any data other than target entity instances to the workers of your bulk insert work queue.

Use the query builder APIs in your code to add restrictions, including restrictions based on joins, that select the targets for a batch. After your code returns from `buildBulkInsertSelect`, the InsuranceSuite application submits a `SELECT` statement based on the query object directly to the database. The database then uses its native bulk insert capabilities to insert standard work items for the batch directly into the standard work queue table.

Bulk insert work queues are suitable if you can reduce the selection criteria for the targeted units of work to a single query builder query object. Because the InsuranceSuite application creates and inserts work items at the database level, it is not necessary to implement the following methods:

- `createWorkItem`
- `shouldProcessItem`

The `BulkInsertWorkQueueBase` base class is a subclass of `WorkQueueBase`, so you must also implement method `processWorkItem` for the worker logic of a bulk insert work queue.

Bulk insert work queue declaration and constructor

In the declaration of your bulk insert work queue class, you must include the `extends` clause to derive your work queue class from abstract class `BulkInsertWorkQueueBase`. Your custom class must include a constructor that calls the constructor in the base class using the `super` syntax. Your constructor associates your bulk insert work queue class at runtime with the following items:

- The work queue typecode in the `BatchProcessType` typelist
- The work queue item type
- The user under which the database transaction runs

The following example code declares a bulk insert work queue type and implements a constructor. Unlike a work queue class that you derive from `WorkQueueBase`, the constructor does not specify the entity type of the `Target` fields on the work items for the work queue.

```
class MyBulkInsertWorkQueue extends BulkInsertWorkQueueBase <User, StandardWorkItem> {
    construct () {
        super(BatchProcessType.TC_MYBULKINSERTWORKQUEUE, StandardWorkItem, User)
    }
    ...
}
```

Querying for targets of a bulk insert work queue

You provide the query logic for the writer thread of a bulk insert work queue by overriding the abstract `buildBulkInsertSelect` method that your custom work queue class inherits from `BulkInsertWorkQueueBase`.

The following Gosu example code selects the units of work for a batch run, which are users with a status of `On Vacation`.

```
override function buildBulkInsertSelect(query : InsertSelectBuilder, args: List<Object>) {
    query.SourceQuery.compare( User#VacationStatus.PropertyInfo.Name, Relop.Equals,
        VacationStatusType.TC_ONVACATION )
}
```

The following Gosu example code selects the units of work for a batch run, which are activities that no one has viewed in five days or more. Notice that this implementation of the method uses the `extractSourceQuery` method to generate the query result.

```
override function buildBulkInsertSelect(builder : Object, args: List<Object>) {
    extractSourceQuery(builder).compare( Activity#LastViewedDate.PropertyInfo.Name, Relop.LessThanOrEquals,
        java.util.Date.Today.addBusinessDays(-5) )
}
```

Eliminating duplicate items in bulk insert work queue queries

In constructing the query for your custom work queue class, Guidewire recommends that you include the following method, which excludes duplicate `StandardWorkItem` work items from the query:

```
excludeDuplicatesOnStandardWorkItem(java.lang.Object opaqueBuilder, boolean allowFailedDuplicates)
```

The `excludeDuplicatesOnStandardWorkItem` method takes the following parameters:

<code>opaqueBuilder</code>	The query builder to use.
<code>allowFailedDuplicates</code> (Boolean)	Whether to recreate a failed work item.

The `excludeDuplicatesOnStandardWorkItem` helper method excludes `StandardWorkItem` work items only. If you query on another work item type, use custom code similar to the following to exclude the duplicate work items:

```
builder.mapColumn(XXWorkItem#Target.PropertyInfo as IEntityPropertyInfo,
  XX#Id.PropertyInfo as IEntityPropertyInfo)

var subQuery = Queries.createQuery(XXWorkItem)
if allowFailedDuplicates subQuery.compare(XXWorkItem#Status.PropertyInfo.Name, Relop.NotEquals,
  WorkItemStatusType.TC_FAILED)

builder.SourceQuery.subselect(XX#Id.PropertyInfo.Name, CompareNotIn, subQuery,
  XXWorkItem#Target.PropertyInfo.Name)
```

Processing work items in a custom bulk insert work queue

You provide the programming logic for the writer thread of a bulk insert work queue by implementing the abstract `processWorkItem` method that your custom work queue class inherits from `BulkInsertWorkQueueBase`. The `processWorkItem` method takes a single argument, which is the work item type to use as the unit of work.

The following Gosu example code processes the units of work for a batch run, which are activities that no one has viewed in five days or more.

```
override function processWorkItem(workItem: StandardWorkItem) {
  // Extract an object reference to the unit of work: an Activity instance
  var activity = extractTarget(workItem) // Convert the ID of the target to an object reference

  if (activity.AssignedByUser.Contact.EmailAddress1 != null) {
    // Send an email to the user assigned to the activity
    EmailUtil.sendEmailWithBody(null, activity.AssignedUser.Contact,
      null,
      "Activity not viewed for five days",
      "See activity" + activity.Subject + ", due on " + activity.TargetDate + ".")
    // To:
    // From:
    // Subject:
    // Body:
  }
}
```

Example work queue that bulk inserts its work items

The following Gosu code is an example of a custom work queue that uses bulk insert to write work items for a batch to its work queue. This custom work queue class derives from the base class `BulkInsertWorkQueueBase` instead of from `WorkQueueBase`. The code provides an implementation of the `buildBulkInsertSelect` method instead of `findTargets` for its writer logic. The code provides an implementation of the `processWorkItemMethod` for its worker logic.

Bulk insert work queues like this one use the `StandardWorkItem` entity type for its work queue table. To use a custom work queue, you must implement `createBatchProcess` method to tell the InsuranceSuite application in which table to insert the work items created from the SELECT statement.

The unit of work for this work queue is an activity that has not been viewed for five days or more. The process sends an email to the user assigned to the activity, and then it sets the escalation date on the activity to the current date. The process updates entity data, so the `processWorkItemMethod` uses the `runWithNewBundle` API.

```
uses gw.api.database.Query
uses gw.api.database.Relop
uses gw.api.email.EmailUtil
uses gw.processes.BulkInsertWorkQueueBase

/**
 * An example of a process implemented as a work queue that sends email
 * to assignees of activities that have not been viewed for five days or more.
 */
class MyOptimizedActivityEmailWorkQueue extends BulkInsertWorkQueueBase <Activity, StandardWorkItem> {

  /**
   * Let the base class register this type of custom batch processing by
   * passing the batch processing type, the entity type the work queue item, and
   * the target type for the units of work.
   */

  construct() {
    super ( BatchProcessType.TC_NOTIFYUNVIEWEDACTIVITIES, StandardWorkItem, Activity )
  }
}
```



```

/**
 * Select the units of work for a batch run: activities that have not been
 * viewed for five days or more.
 */
override function buildBulkInsertSelect(builder : Object, args: List<Object>) {
    excludeDuplicatesOnStandardWorkItem(builder, true); // if using StandardWorkItem
    extractSourceQuery(builder).compare(
        Activity#LastViewedDate.PropertyInfo.Name, Relop.LessThanOrEquals,
        java.util.Date.Today.addBusinessDays(-5) )
}

/**
 * Process a unit of work: an activity not viewed for five days or more
 */
override function processWorkItem(workItem: StandardWorkItem) {
    // Extract an object referent to the unit of work: an Activity instance
    var activity = extractTarget(workItem) // Convert the ID of the target to an object reference

    if (activity.AssignedByUser.Contact.EmailAddress1 != null) {
        // Send an email to the user assigned to the activity
        EmailUtil.sendEmailWithBody(null, activity.AssignedUser.Contact, // To:
            null, // From:
            "Activity not viewed for five days", // Subject:
            "See activity" + activity.Subject + ", due on " + activity.TargetDate + ".") // Body:
    }
}
}

```

Examples of custom work queues

Example work queue that extends WorkQueueBase

The following Gosu code is an example of a simple custom work queue. It uses the `StandardWorkItem` entity type for its work queue table. Its unit of work is an activity that no one has viewed for five days or more. The process simply sends an email to the user assigned to the activity. The process does not update entity data, so the `processWorkItemMethod` does not use the `runWithNewBundle` API.

Note: In general, Guidewire recommends that your custom work queue class extend `BulkInserWorkQueueBase` rather than `WorkQueueBase`. The following example is for illustration only.

```

uses gw.api.database.Query
uses gw.api.database.Relop
uses gw.api.email.EmailUtil
uses gw.processes.WorkQueueBase
uses java.util.Iterator

/**
 * An example of batch processing implemented as a work queue that sends email
 * to assignees of activities that have not been viewed for five days or more.
 */
class MyActivityEmailWorkQueue extends WorkQueueBase <Activity, StandardWorkItem> {

    /**
     * Let the base class register this type of custom process by
     * passing the process type, the entity type the work queue item, and
     * the target type for the units of work.
     */

    construct() {
        super ( BatchProcessType.TC_NOTIFYUNVIEWEDACTIVITIES, StandardWorkItem, Activity )
    }

    /**
     * Select the units of work for a batch run: activities that have not been
     * viewed for five days or more.
     */

    override function findTargets() :Iterator <Activity> {
        // Query the target type and apply conditions: activities not viewed for 5 days or more
        var targetsQuery = Query.make(Activity)
        targetsQuery.compare( Activity#LastViewedDate.PropertyInfo.Name, Relop.LessThanOrEquals,
            java.util.Date.Today.addBusinessDays(-5) )
        return targetsQuery.select().iterator()
    }

    /**
     * Process a unit of work: an activity not viewed for five days or more
     */
    override function processWorkItem(WorkItem: StandardWorkItem) {
        // Extract an object referent to the unit of work: an Activity instance
        var activity = extractTarget(WorkItem) // Convert the ID of the target to an object reference
    }
}

```

```

    if (activity.AssignedByUser.Contact.EmailAddress1 != null) {
        // Send an email to the user assigned to the activity
        EmailUtil.sendEmailWithBody(null,
            activity.AssignedUser.Contact,
            null,
            "Activity not viewed for five days",
            "See activity" + activity.Subject + ", due on " + activity.TargetDate + ".")
        //To:
        //From:
        //Subject:
        //Body:
    }
}
}

```

Notice that this work queue implementation uses a `findTargets` method to return an iterator of target objects on which to base the work items.

Example work queue for updating entities

The following Gosu code is an example of a custom work queue that updates entity data as part of processing a unit of work. It uses the `StandardWorkItem` entity type for its work queue table. Its unit of work is an activity that has not been viewed for five days or more. The process sends an email to the user assigned to the activity, and then it sets the escalation date on the activity to the current date. The process updates entity data, so the `processWorkItemMethod` uses the `runWithNewBundle` API.

```

uses gw.api.database.Query
uses gw.api.database.Relop
uses gw.api.email.EmailUtil
uses gw.processes.WorkQueueBase
uses gw.transaction.Transaction
uses java.util.Iterator

/**
 * An example of batch processing implemented as a work queue that sends email
 * to assignees of activities that have not been viewed for five days or more.
 */
class MyActivityEscalationWorkQueue extends WorkQueueBase <Activity, StandardWorkItem> {

    /**
     * Let the base class register this type of custom process by
     * passing the process type, the entity type the work queue item, and
     * the target type for the units of work.
     */
    construct () {
        super (typekey.BatchProcessType.TC_NOTIFYUNVIEWEDACTIVITIES, StandardWorkItem, Activity)
    }

    /**
     * Select the units of work for a batch run: activities that have not been
     * viewed for five days or more.
     */
    override function findTargets (): Iterator <Activity> {
        // Query the target type and apply conditions: activities not viewed for 5 days or more
        var targetsQuery = Query.make(Activity)
        targetsQuery.compare(Activity#LastViewedDate, Relop.LessThanOrEquals,
            java.util.Date.Today.addBusinessDays(-5))
        return targetsQuery.select().iterator()
    }

    /**
     * Process a unit of work: an activity not viewed for five days or more
     */
    override function processWorkItem (WorkItem : StandardWorkItem) {
        // Extract an object referent to the unit of work: an Activity instance
        var activity = extractTarget(WorkItem)

        // Send an email to the user assigned to the activity.
        if (activity.AssignedUser.Contact.EmailAddress1 != null) {
            EmailUtil.sendEmailWithBody(null,
                activity.AssignedUser.Contact,
                null,
                "Activity not viewed for five days",
                "See activity " + activity.Subject + ", due on " + activity.TargetDate + ".")
            // To:
            // From:
            // Subject:
            // Body:
        }

        // Update the escalation date on the assigned activity
        Transaction.runWithNewBundle( \ bundle -> {
            activity = bundle.add(activity) // Add the activity to the new bundle
            activity.EscalationDate = java.util.Date.Today } ) // Update the escalation date
    }
}

```

Example work queue with a custom work item type

The following example creates a custom work queue that uses a custom work item type for its work queue table. The name of the custom work item type is `MyWorkItem`. The code provides an implementation of the `createWorkItem` method in addition to the `findTargets` method to add work items for a batch to the work queue.

The unit of work for the work queue is an activity that has not been viewed for five days or more. The process sends an email to the user assigned to the activity, and then it sets the escalation date on the activity to the current date.

Define custom work item `MyWorkItem`

It is necessary to create a custom work item if using a custom work queue.

Procedure

1. Open Guidewire Studio.
2. In the Studio **Project** window, expand **configuration** > **config** > **Extensions** > **Entity**.
3. Right-click **Entity** and select **New** > **Entity**.
4. Enter the following information in the **Entity** dialog and click **OK**.

Entity	MyWorkItem
Entity Type	Entity
Description	Custom work item
Type	keyable

Note: See “Considerations for developing a custom work queue” on page 264 for a discussion as to why Guidewire recommends the use of `softentityreference` instead of `foreignkey` here.

For all other fields, accept that dialog defaults.

5. Using the add functionality at the top of the left-hand column (the plus sign), add the following subelements to the `MyWorkItem` entity.

Element	Element attribute
column	• <code>name</code> – Activity • <code>type</code> – <code>softentityreference</code> • <code>nullok</code> – false
implementsEntity	• <code>name</code> – <code>Workitem</code>

What to do next

To use this custom work item, you need to implement custom work queue `MyActivitySetEscalationWorkQueue`.

Implement `MyActivitySetEscalationWorkQueue`

To create a custom work queue, implement a class that extends class `WorkQueueBase`.

Before you begin

This example requires that you first create custom work item `MyWorkItem`.

Procedure

1. Open Guidewire Studio

2. In the Studio **Project** window, expand **configuration** > **gsrsrc**.
3. Create a new package directory named **workflow**.
4. Within the **workflow** folder, create a new Gosu class named **MyActivitySetEscalationWorkQueue**.
5. Populate this class with the following code.

```
package workflow

uses gw.processes.WorkQueueBase
uses gw.api.database.Query
uses gw.api.database.Relop
uses java.util.Iterator
uses gw.transaction.Transaction
uses gw.api.email.EmailUtil
uses gw.pl.persistence.core.Bundle

/**
 * An example of batch processing implemented as a work queue that sends email
 * to assignees of activities that have not been viewed for five days or more.
 */
class MyActivitySetEscalationWorkQueue extends WorkQueueBase <Activity, MyWorkItem> {

    /**
     * Let the base class register this type of custom process by
     * passing the process type, the entity type the work queue item, and
     * the target type for the units of work.
     */
    construct () {
        super (typekey.BatchProcessType.TC_NOTIFYUNVIEWEDACTIVITIES, MyWorkItem, Activity)
    }

    /**
     * Select the units of work for a batch run: activities that have not been
     * viewed for five days or more.
     */
    override function findTargets (): Iterator <Activity> {
        // Query the target type and apply conditions: activities not viewed for 5 days or more
        var targetsQuery = Query.make(Activity)
        targetsQuery.compare(Activity#LastViewedDate.PropertyInfo.Name, Relop.LessThanOrEquals,
            java.util.Date.Today.addBusinessDays(-5))
        return targetsQuery.select().iterator()
    }

    /**
     * Write a custom work item
     */
    override function createWorkItem (activity : Activity, safeBundle : Bundle) : MyWorkItem {
        var customWorkItem = new MyWorkItem(safeBundle)
        customWorkItem.Activity = activity
        return customWorkItem
    }

    /**
     * Process a unit of work: an activity not viewed for five days or more
     */
    override function processWorkItem (workItem : MyWorkItem): void {
        // Get an object reference to the activity
        var activity = workItem.Activity

        // Send an email to the user assigned to the activity
        if (activity.AssignedUser.Contact.EmailAddress1 != null) {
            EmailUtil.sendEmailWithBody(null,
                activity.AssignedUser.Contact, // To:
                null, // From:
                "Activity not viewed for five days", // Subject:
                "See activity " + activity.Subject + ", due on " + // Body:
                    activity.TargetDate + ".")

        }

        // Update the escalation date on the assigned activity
        Transaction.runWithNewBundle( \ bundle -> {
            activity = bundle.add(activity) // add the activity to the new bundle
            activity.EscalationDate = java.util.Date.Today } ) // update the escalation date

        return
    }
}
```

Example of the real-time creation of work items

Generally, BillingCenter and ClaimCenter generate work items through the use of a writer in an implementation of the createWorkItem method on class WorkQueueBase. A batch job typically triggers the action of the writer.

However, there are situations in which it is better to create work items in real time, such as when executing code in a Gosu rule. In any case, if the event point is already known, it does not make sense to perform additional queries about the event in a batch job.

BillingCenter example

You can configure BillingCenter to create work items in real time that update the account service tier level in the BillingCenter database. For example, whenever a user updates the service tier level:

- BillingCenter calls the Account Preupdate rule set.
- Preupdate rule logic creates a work item for the update action.
- Periodically, at a time interval frequency defined by its `maxpolinterval` value, a worker scans the queue for new work items.
- The worker retrieves the new work item.
- The worker performs the work.

ClaimsCenter example

You can configure ClaimCenter to create work items in real time that update the claim description in the ClaimCenter database. For example, whenever a user updates the description of a claim within ClaimCenter:

- ClaimCenter calls the Claim Preupdate rule set.
- Preupdate rule logic creates a work item for the update action.
- Periodically, at a time interval frequency defined by its `maxpolinterval` value, a worker scans the queue for new work items.
- The worker retrieves the new work item.
- The worker performs the work.

Note: It is important to understand that the bundle used during work item creation is part of the current transaction. Thus, if the code throws an exception in the Preupdate rule as part of the creation of the work item, ClaimCenter do not persist the work item to the database.

Complete the following steps to implement these examples:

1. “Create a ProcessWorkItem entity” on page 277
2. “Add custom typecodes to the BatchProcessType typelist” on page 278
3. “Create a preupdate rule for processing work items” on page 279
4. “Create custom ProcessWQ class” on page 280
5. “Update the work queue configuration file” on page 281
6. “Test your custom work queue” on page 282

Create a ProcessWorkItem entity

This procedure creates a custom work item extension entity.

Before you begin

Before starting this procedure, read and understand the concepts outlined in “Example of the real-time creation of work items” on page 276.

Procedure

1. Open Guidewire Studio.
2. In the Studio **Project** window, expand **configuration > config > Extensions**.

3. Select **Entity** and right-click to open the context menu.
4. Select **New > Entity**.
5. In the **Entity** dialog, enter the following attribute information and press **OK**.

Entity	ProcessWorkItem
Entity Type	entity
Desc	• Work item to use for processing account service tier updates (For BillingCenter example) • Work item to use for processing Claim updates (For ClaimCenter example)
Table	processworkitem
Type	keyable

6. Select **column** from the drop-down list near the top of the screen and click +.
7. Enter the following attribute information for the new column.

name	• Account (For BillingCenter) • Claim (For ClaimCenter)
type	softentityreference
nullok	false

Note: See “Considerations for developing a custom work queue” on page 264 for a discussion as to why Guidewire recommends the use of `softentityreference` instead of `foreignkey` here.

8. Select the new entity row.
9. Using the right-click context menu, select the following:
Add new... > implementsEntity
10. Select the new **implementsEntity** row and then select **WorkItem** from the **name** drop-down list.
11. Save your work.

What to do next

Continue to “Add custom typecodes to the BatchProcessType typelist” on page 278.

Add custom typecodes to the BatchProcessType typelist

Before you begin

Before starting this procedure, perform the steps listed in “Create a ProcessWorkItem entity” on page 277.

About this task

This step adds a `ProcessWQLoggers` typecode to the `BatchProcessType` typelist.

Procedure

1. Open Guidewire Studio.
2. In the Studio **Project** window, expand **configuration > config > Extensions > Typelist**.
3. Open the **BatchProcesstype** typelist for editing.
4. Select **typecode** from the drop-down list near the top of the screen and click +.

5. Enter the following attribute information for the new column.

code	ProcessWQLoggers
name	Process Work Item Loggers
desc	Batch process type for processing work items

6. Select the newly created **typecode** row.
7. From the right-click context menu, select the following:
Add new... > typecode > category
8. Enter the following attribute information for the new category.

code	UIRunnable
typelist	BatchProcessTypeUsage

9. Select the typecode again and create a new category with the following attribute information.

code	Schedulable
typelist	BatchProcessTypeUsage

10. Save your work.

What to do next

Continue to “Create a preupdate rule for processing work items” on page 279.

Create a preupdate rule for processing work items

Before you begin

Before starting this procedure, perform the steps listed in “Add custom typecodes to the BatchProcessType typelist” on page 278.

About this task

Based on which InsuranceSuite application you are configuring, this step completes the following task:

- In BillingCenter, this step creates a new Account Preupdate rule that processes any work items associated with an update to an account service tier level.
- In ClaimCenter, this step creates a new Claim Preupdate rule that processes any work items associated with an update to a claim.

Procedure

1. Open Guidewire Studio.
2. In the Studio **Project** window, expand the following .
 - For BillingCenter: **configuration > config > Rule Sets > Preupdate > AccountPreupdate**
 - For ClaimCenter: **configuration > config > Rule Sets > Preupdate > ClaimPreupdate**
3. In the center pane, select one of the following:
 - For BillingCenter, select **AccountPreupdate**
 - For ClaimCenter, select **ClaimPreupdate**
4. From the right-click context menu, select **New Rule** and enter the following information.

-
- Rule name**
- Work Item Rule (In BillingCenter)
 - CPU90000 - Work Item Rules (In ClaimCenter)
-

5. Under **USES**, the following class name:

- For BillingCenter:

```
gw.api.system.BCLoggerCategory
```

- For ClaimCenter:

```
gw.api.system.CCLoggerCategory
```

6. Under **ACTION**, enter the following Gosu code.

- For BillingCenter:

```
BCLoggerCategory.RULE_HIERARCHY_ROOT.info("*** ENTERING " + actions.Rule.DisplayName + " ***")
var workItem = new ProcessWorkItem(account.getBundle())
workItem.Account = account.ID.Value
workItem.initialize()
```

- For ClaimCenter:

```
CCLoggerCategory.RULE_HIERARCHY_ROOT.info("*** ENTERING " + actions.Rule.DisplayName + " Rule")
var workItem = new ProcessWorkItem(claim.getBundle())
workItem.Claim = claim.ID.Value
workItem.initialize()
```

7. Under **CONDITION**, enter the following return value.

- For BillingCenter:

```
return account.isFieldChanged(Account#ServiceTier)
```

- For ClaimCenter:

```
return claim.isFieldChanged(Claim#Description)
```

8. Save your work.

What to do next

Continue to “Create custom ProcessWQ class” on page 280.

Create custom ProcessWQ class

Before you begin

Before starting this procedure, perform the steps listed in “Create a preupdate rule for processing work items” on page 279.

About this task

Based on which InsuranceSuite application you are configuring, this step completes the following task:

- In BillingCenter, this step creates a Gosu class to process the account update work item.
- In ClaimCenter, this step creates a Gosu class to process the claim update work item.

Procedure

1. Open Guidewire Studio.
2. In the Studio **Project** window, expand **configuration** > **gsrsrc**.
3. Select **gsrsrc** and select **New Package** from the right-click context menu.

4. For the new package name, enter `myWorkQueuePackage`, and click **OK**.
5. Select the new package, right-click and select **New > Gosu Class**.
6. Enter the following information.

Name `myProcessWQClass`

Kind `Class`

7. For class `myProcessWQClass`, enter the following code.

To process the account update work item in `BillingCenter`:

```
package myWorkQueuePackage

uses gw.processes.WorkQueueBase
uses gw.pl.persistence.core.Bundle
uses gw.api.system.BCLoggerCategory

class myProcessWQClass extends WorkQueueBase <Account, ProcessWorkItem> {

    construct() {
        super(BatchProcessType.TC_PROCESSWQLOGGERS, ProcessWorkItem, Account)
    }

    override function processWorkItem(workItem: ProcessWorkItem) {
        BCLoggerCategory.RULE_HIERARCHY_ROOT.info("*** PROCESSING workItem ${workItem.Account} ***")
    }

    override function createWorkItem(account: Account, safeBundle: Bundle) : ProcessWorkItem {
        var workItem = new ProcessWorkItem(safeBundle)
        workItem.Account = account.ID.Value
        return workItem
    }
}
```

To process the claim update work item in `ClaimCenter`:

```
package myWorkQueuePackage

uses gw.processes.WorkQueueBase
uses gw.pl.persistence.core.Bundle
uses gw.api.system.CCLoggerCategory

class myProcessWQClass extends WorkQueueBase <Claim, ProcessWorkItem> {

    construct() {
        super(BatchProcessType.TC_PROCESSWQLOGGERS, ProcessWorkItem, Claim)
    }

    override function processWorkItem(workItem: ProcessWorkItem) {
        CCLoggerCategory.RULE_HIERARCHY_ROOT.info("*** PROCESSING workItem ${workItem.Claim} ***")
    }

    override function createWorkItem(claim: Claim, safeBundle: Bundle) : ProcessWorkItem {
        var workItem = new ProcessWorkItem(safeBundle)
        workItem.Claim = claim.ID.Value
        return workItem
    }
}
```

8. Save your work.

What to do next

Continue to “Update the work queue configuration file” on page 281.

Update the work queue configuration file

Before you begin

Before starting this procedure, perform the steps listed in “Create custom ProcessWQ class” on page 280.

About this task

This step adds a `ProcessWQLoggers` work queue to the work queue configuration file.

Procedure

1. Open Guidewire Studio.
2. In the Studio **Project** window, expand **configuration** > **config** > **workqueue**.
3. Open **work-queue.xml** for editing.
4. Add the following new work queue to the work queue configuration file.

```
<!-- My new work queue-->
<work-queue workQueueClass="myWorkQueuePackage.myProcessWQClass" progressinterval="60000">
  <worker instances="1" batchsize="250"/>
</work-queue>
```

5. Save your work.

What to do next

Continue to “Test your custom work queue” on page 282.

Test your custom work queue

Before you begin

Before starting this procedure, perform the steps listed in “Update the work queue configuration file” on page 281.

Procedure

1. Log into the Guidewire InsuranceSuite application where you want to test the custom work queue.
2. To test the BillingCenter example, update the Service Tier field in BillingCenter:
 - a) Navigate to the **Account Details** screen.
 - b) Click **Edit**.
 - c) Set the **Service Tier** field to a different level.
3. To test the ClaimCenter example, edit the loss details Description field in ClaimCenter:
 - a) In the **Desktop**, select a claim.
 - b) Navigate to the claim **Loss Details** screen.
 - c) Click **Edit**.
 - d) Update the text in the **Description** field.
4. Click **Update** to save your work.
5. Verify that the following records exist in the rule execution log.

- In BillingCenter:

```
*** ENTERING Work Item Rule ***
```

And, a few minutes later...

```
*** PROCESSING workItem 1002 ***
```

- In ClaimCenter:

```
*** ENTERING CPU90010 - Process Work Item Rule
```

And, a few minutes later...

```
*** PROCESSING workItem 1 ***
```

Delayed processing of real-time work items

In BillingCenter and ClaimCenter, you can create work items in real time and also delay the processing of those work items until a later time.

For example, suppose that you write a Gosu rule to create work items in response to a user-initiated event in BillingCenter or ClaimCenter. However, you want to defer the processing of these work items until the middle of the night. At this point, the load on the processing server is lighter and you can use multiple worker threads to process the work items.

There are multiple technical reason for why you would want to do this, for example:

- The processWorkItem method that runs for each work item can require large amounts of processing power.
- The executor of each work queue performs a polling of the work queue multiple times during the day to determine if there are new work items. This work can place a heavy load on the work queue server.

Use scheduled batch process to execute work items

In BillingCenter and ClaimCenter, you can delay processing a certain work item type by disabling the polling of the queue for those work items. A batch process, instead, notifies the queue workers that there is work available for processing at a later time.

Before you begin

Review the steps involved in creating work items in real time through Gosu rules before undertaking this example.

Procedure

1. Open Guidewire Studio.
2. Create a Gosu rule that creates work items in real time.
3. Create a class that extends WorkQueueBase to manage the work queue that processes these work items.
4. In this class, add a createBatchProcess method that creates a batch process to execute the work items created by the Gosu rule.

For example, you can do something similar to the following code in your work queue class.

```
package gwservices.myWorkQueuePackage
uses gw.api.webservice.cc.maintenanceTools.CCMaintenanceToolsImpl
uses gw.processes.BatchProcess
uses gw.processes.BatchProcessBase
uses gw.processes.WorkQueueBase
uses gw.pl.persistence.core.Bundle
uses gw.api.system.CCLoggerCategory

class myProcessWQClass extends WorkQueueBase <Claim, ProcessWorkItem> {

    construct() {
        super(BatchProcessType.TC_PROCESSWQLOGGERS, ProcessWorkItem, Claim)
    }

    override function processWorkItem(workItem: ProcessWorkItem) {
        CCLoggerCategory.RULE_HIERARCHY_ROOT.info("*** PROCESSING workItem ${workItem.Claim} ***")
    }

    override function createWorkItem(claim: Claim, safeBundle: Bundle) : ProcessWorkItem {
        var workItem = new ProcessWorkItem(safeBundle)
        workItem.Claim = claim.ID.Value
        return workItem
    }

    /**
     * Associated batch process for this Workqueue.
     * No need to create WorkItems in the batch process as workitems are already created at run time in the Gosu rule
     * This batch file simply notifies the queue workers to process available workitems
     */

    function createBatchProcess(args : Object[]): BatchProcess {
        return new BatchProcessBase(getQueueType()) {
            function doWork() {
                new CCMaintenanceToolsImpl().notifyQueueWorkers(getQueueType().getCode())
            }
        }
    }
}
```

```

    };
  }
}

```

5. Set the maximum polling interval for the work queue to 0 in file `work-queue.xml`.

The following example code sets the maximum polling interval to 0, which effectively disables work queue polling, and thus disables work item processing.

```

<!-- My new work queue -->
<work-queue workQueueClass="gw.myWorkQueuePackage.myProcessWQClass" progressinterval="60000">
  <worker instance="10" maxpollinterval="0" batchsize="1" />
</work-queue>

```

6. Schedule the batch process created by your work queue class to run at a time of your choosing.

Set when work queue server is available for processing

In `BillingCenter` and `ClaimCenter`, you can delay processing a certain work item type by setting when the work queue server that processes the work item is available. Do this by assigning the work queue to a specific server with a required work queue role, then bringing up that server only at specific times. In other words, you limit the work queue to certain servers that are active at specific times, such as after the business day ends.

Before you begin

Review the steps involved in creating work items in real time through Gosu rules before undertaking this example.

Procedure

1. Open Guidewire Studio.
2. Open file `config.xml` for editing.
3. Add a specific role for the work queue to the `<registry>` element and assign it to a specific server or servers. The following example code adds the `customRole` role to the `<registry>` element and assigns to server `prod2`.

```

<registry roles="...", customRole">
  <server serverid="prod2" roles="customRole"/>
</registry>

```

You can also assign the required `customRole` role to a server as you start the server from a command prompt.

4. Define work queue parameters for the `customRole` server in file `work-queue.xml`.

```

<work-queue workQueueClass="gwservices.myWorkQueuePackage.myProcessWQClass" progressinterval="30000">
  <worker instances="3" server="#customRole"/>
</work-queue>

```

The hash mark in front of `customRole` indicates that the value that follows the hash mark is a server role and not a server ID.

5. Start the server with the specified work queue role at a specific time, either manually or through some automated process.

Set when work items are available for processing

In `BillingCenter` and `ClaimCenter`, you can delay processing a certain work item type by setting when that work item is available to perform work.

Before you begin

Review the steps involved in creating work items in real time through Gosu rules before undertaking this example.

Procedure

1. Open Guidewire Studio.

2. Create a Gosu rule that creates work items in real time.
For example, create a Gosu rule that creates work items to update activity events in real time.
3. Create a class that extends `WorkQueueBase` to process the work items created by the Gosu rule.
4. In this class, override the `createWorkItem` method and set the `availableSince` value to a future time.
For example, do something similar to the following code in your class.

```
override function createWorkItem(activity: Activity, safeBundle: Bundle) : ProcessWorkItem {  
    var workItem = new ProcessWorkItem(activity.getBundle())  
    workItem.Activity = activity.ID.Value  
    workItem.initialize()  
  
    //Set the 'available since' value to a future time  
    var lng = DateUtil.currentDate().getToday().addHours(19).getTime()  
    workItem.setAvailableSinceAsLong(lng)  
  
    return workItem  
}
```

This code delays processing of the work item until 1900 hours in a 24-hour clock.

Even though the affected work queue server does not process the work items until the specified time, the executor of the work queue continues to poll the queue throughout the day, even though no work is done.

Developing custom batch process

A batch process is code that runs without human intervention as background process on a single server instance to process the units of work in a batch sequentially.

IMPORTANT: Regardless of the volume of items or the duration of the processing window, Guidewire strongly recommends implementing any type of custom batch processing as a custom work queue.

WARNING: Do not implement multi-threaded programming in custom batch processes derived from `BatchProcessBase`.

Custom batch processes are Gosu classes that extend the `BatchProcessBase` base class. The `BatchProcessBase` base class includes code that the InsuranceSuite application uses to manage the processing of your custom batch process class.

Note: Custom batch processes are not substitutes for shell scripts or batch files. Do not attempt to automate a batch of system commands by developing custom batch processes.

You extend the `BatchProcessBase` base class by providing your own implementation of the `doWork` abstract method that `BatchProcessBase` inherits from its super class, `SinglePhaseBatchProcess`.

Custom batch processes are inherently single threaded. Do not attempt to improve the throughput of units of work by spawning threads from within your `doWork` method. In particular, the inherited methods `incrementOperationsCompleted` and `incrementOperationsFailed` are not thread-safe. Entity instances in transactional bundles on separate threads can cause problems.

IMPORTANT: If your type of batch process requires processing units of work in parallel to achieve sufficient throughput, develop a custom work queue instead of a custom batch process.

Custom batch processes, bundles, and users

Most likely, any custom batch process that you create needs to create, delete, or modify one or more business objects. Your batch process class must create a new bundle to perform this work. If you do not explicitly associate a user with the bundle, the InsuranceSuite application uses a default user for bundle creation and commit.

Do not use the Guidewire sys user (System User), or any other default user, for this purpose. Instead, create your own application system user and limit the permissions assigned to that user to what is appropriate to successfully perform the necessary operation. You then use the following syntax to create a bundle with a specific user.

```
gw.transaction.Transaction.runWithNewBundle(\ bundle -> BLOCK_BODY, user)
```

Create a custom batch process

About this task

Creating a custom batch process is a multistep process:

Procedure

1. In Studio, edit the `BatchProcessType` typelist and add a typecode to represent your custom batch process.
Note: Creating this new typecode adds support for the new batch process within the `BatchProcessInfo` PCF and the `IMaintenanceToolAPI` web service.
2. Create a class that extends the `BatchProcessBase` class (`gw.processes.BatchProcessBase`). The only method you must override is the `doWork` method, which takes no arguments. Perform your batch process work in this method.
3. In Studio, in the Plugins editor, register your new plugin for the `IProcessesPlugin` plugin interface.
4. To schedule your custom batch process to run regularly on a schedule instead of on-demand, add an entry for the batch process to the XML file `scheduler-config.xml`.

Define a typecode for a custom batch process

About this task

Before you begin developing the Gosu code for your custom batch process, you need to define the batch process type. For your custom type, you need to add a typecode for it in the `BatchProcessType` typelist. The constructor of your batch process implementation class requires the typecode as an argument.

Procedure

1. Open Guidewire Studio.
2. In the Studio **Project** window, expand **configuration** > **config** > **Extensions** > **Typelist**.
3. Open `BatchProcessType.ttx`.
4. Click the green plus button, , to add a typecode.
5. In the panel of fields on the right, specify appropriate values for the following fields:

code	Specify a code value that uniquely identifies your custom batch process. This code value identifies the custom batch process in configuration files.
name	Specify a human readable identifier for your custom batch process. This name appears for batch processing type on the Server Tools Batch Process Info screen.
desc	Provide a short description of what your custom batch process accomplishes. This description appears for your batch process on the Server Tools Batch Process Info screen.

6. Select the newly added typecode.
7. Select **Add new... > typecode > category** from the right-click context menu. This action adds a new **category** element under the selected typecode.
8. For this category, define the following fields:

typelist	Select BatchProcessTypeUsage from the drop-down list.
code	Select one of the available choices from the drop-down list. You must add at least one category or your batch process cannot run.

Results

Creating this new typecode adds support for the new batch process within the BatchProcessInfo PCF and the IMaintenanceToolAPI web service.

Batch process base class

Your custom batch process must extend the BatchProcessBase Gosu class. You must override the abstract doWork method, which takes no arguments. You can override other methods, but it is not strictly necessary because the base class defines meaningful defaults.

The doWork method does not have a bundle to track the database transaction. To modify data, you must use the Transaction.runWithNewBundle method to create a bundle.

Ensure that your main doWork method frequently checks the TerminateRequested flag. If it is true, exit from your code. For example, if you are looping across a database query, exit from the loop.

IMPORTANT: The InsuranceSuite application calls the requestTermination method in a different thread from the thread that runs the doWork method of your batch process.

Useful properties on class BatchProcessBase

The following list describes some of the useful properties on class BatchProcessBase.

Property	Description
DetailStatus	Returns the detailed status for the batch type. Class BatchProcessBase defines a simple default implementation. Override the default property getter to provide more useful detail information about the status of your batch process for the InsuranceSuite application Administration screens. The details might be important if your class experiences any error conditions. For Java implementations, you must implement this property getter as the method getDetailStatus, which takes no arguments.
Exclusive	Sets whether another instance of this batch process can start while the current process is still running. The base class implementation of the isExclusive method always returns true. Override the isExclusive method if you need to customize this behavior. This value does not affect whether other batch process classes can run. It only affects the current batch process class. For maximum performance, be sure to set the property value to false, if possible. For example, if your batch process takes arguments in its constructor, it might be specific to one entity such as a single Account entity in BillingCenter. If you want to permit multiple instances of your batch process to run in parallel, you must ensure your batch process class implementation returns false. For example, <pre> override property get Exclusive() : boolean { return false } </pre> For Java implementations, implement this property getter as the method isExclusive, which takes no arguments.
Finished	Returns a Boolean value to indicate whether the process completed. Completion says nothing about the errors, if any. For Java implementations, implement this property getter as the isFinished method.
OperationsCompleted	Returns a count of how many operations are complete, as an integer value. For Java implementations, implement this property getter as the getOperationsCompleted method.

Property	Description
OperationsExpected	Returns a count of how many operations InsuranceSuite application expects the batch process to perform, as an integer value. For Java implementations, implement this property as the <code>getOperationsExpected</code> method.
OperationsFailed	Returns an internal count for the number of operations that failed. For Java implementations, implement property getter as the <code>getOperationsFailed</code> method.
Progress	Dynamically returns the progress of the batch process. The base class returns text in the form <code>x of y</code> , with <code>x</code> being the amount of work completed and <code>y</code> being the total amount of work. If the value of <code>y</code> is unknown, the property merely returns <code>x</code> . The InsuranceSuite application determines the values of <code>x</code> and <code>y</code> from the <code>OperationsExpected</code> and <code>OperationsCompleted</code> properties. For Java implementations, implement the property getter as the <code>getProgress</code> method.
TerminateRequested	Returns a Boolean value that is the return value from the <code>requestTermination</code> method.
Type	Returns the type of the current batch process. There is no need to override the default <code>BatchProcessBase</code> implementation of this property. For Java implementations, implement the property getter as the <code>getType</code> method, which takes no arguments.

Useful methods on class BatchProcessBase

Class `BatchProcessBase` contains a number of useful methods:

- `checkInitialConditions`
- `getDescription`
- `incrementOperationsCompleted`
- `incrementOperationsFailed`
- `incrementOperationsFailedReasons`
- `requestTermination`
- `setChunkingById`

checkInitialConditions

The `BatchProcessBase.checkInitialConditions` method has the following signature.

```
checkInitialConditions()
```

The `checkInitialConditions` method is a no-argument class method. It is possible for the batch process manager to call this method multiple times for the same batch job as it processes that job.

The InsuranceSuite application instantiates batch process classes at startup on servers with the batch server role. Later, the InsuranceSuite application calls the `checkInitialConditions` method to determine whether to start a batch process:

- If the method returns `true`, InsuranceSuite application starts the batch process by calling its `doWork` method.
- If the method returns `false`, the initial conditions are not met and InsuranceSuite application does not call the `doWork` method, skipping the batch process for the time being.

The base class implementation of the `checkInitialConditions` method always returns `true`. You must override this method if you want to provide a conditional response. If you override the `checkInitialConditions` method, be certain your code completes and returns quickly. Do not include long running code, such as queries of the database. The intent of the method is to determine environmental conditions, such as server run level. If you want to check initial conditions of data in the database, perform the query in the `doWork` method.

The batch process base class automatically ensures the server is at the maintenance run level if you configure your batch process typecode with the `MaintenanceOnly` category. You can perform additional checks of the current server run level by overriding the `checkInitialConditions` method.

If you override the `checkInitialConditions` method, forward the call to the superclass before returning. If the superclass returns `false`, you must return `false`. If the superclass returns `true`, then perform your additional checks of environmental conditions and return `true` or `false` appropriately.

Returning false from the `checkInitialConditions` method

Guidewire recommends that you do not return `false` from the `checkInitialConditions` method unless the condition that triggered the `false` return value is likely to be temporary. If your overridden `checkInitialConditions` method returns `false`, the InsuranceSuite application:

- Freezes any on-going work in the batch process.
- Sets the **Last Run Status** value in the Server Tools **Batch Process Info** screen to **Running**.
- Executes the `checkInitialConditions` method once a minute unless the returned value becomes `true`, or you click **Stop** in the **Batch Process Info** screen for this batch process, or the repeat attempts time out.
- Stops executing the `checkInitialConditions` method after 10 minutes and generates the following error message in the InsuranceSuite application log.

No one started batch process after 600 seconds.

For example, suppose that your implementation of the `checkInitialConditions` method checks whether the necessary files are available for processing. In some cases, it can be necessary to schedule a batch process to run more often than the actual schedule of input files arriving for processing. In such a case, it is better for the `checkInitialConditions` method to return `true` to indicate that the batch job is ready to run. You then log an INFO or WARN message to the effect that the necessary files are missing.

getDescription

The `BatchProcessBase.getDescription` method has the following signature.

```
getDescription()
```

The `getDescription` method is a no-argument method that retrieves the description of the current batch type. The base class retrieves this string from the batch type typecode description. Override this method if you need to customize this behavior.

incrementOperationsCompleted

The `BatchProcessBase.incrementOperationsCompleted` method has the following signature.

```
incrementOperationsComplete()
```

The `incrementOperationsCompleted` method is a no-argument method that increments an internal counter for how many operations are complete. For example, suppose your batch process iterates across entities of some type with special conditions and specially handle each found item. For each entity you modify, call this method once to track the progress of this batch process.

This method returns the current number of operations that are complete. It is possible to use the returned value in the following ways:

- In the InsuranceSuite application user interface to show the progress of the batch process
- In debugging code to track the progress of the batch process

Note: For each entity for which you have an error condition, call this method once to track the progress of this batch process.

incrementOperationsFailed

The `BatchProcessBase.incrementOperationsFailed` method has the following signature.

```
incrementOperationsFailed()
```

The `incrementOperationsFailed` method is a no-argument method that increments an internal counter for how many operations failed. For example, suppose your batch process iterates across entities of some type with special conditions and specially handle each found item. For each entity for which you have an error condition, call this method once to track the progress of this batch process.

This method returns the current number of operations failed. It is possible to use the returned value in the following ways:

- In the InsuranceSuite application to show the progress of the batch process
- In debugging code to track the progress of the batch process

IMPORTANT: For any operations that fail, call both the `incrementOperationsFailed` method and the `incrementOperationsCompleted` method. Also, call the `incrementOperationsFailedReasons` method at the same time as well.

incrementOperationsFailedReasons

The `BatchProcessBase.incrementOperationsFailedReasons` method has the following signature.

```
incrementOperationsFailedReasons(String msg)
```

The `incrementOperationsFailedReasons` method takes a single `String` argument that states the reason that a particular operation failed. Calling this method adds that reason to the `operationsFailedReasons` array property. Use public method `getOperationsFailedReasons` to retrieve the `operationsFailedReasons` array.

If you call the `incrementOperationsFailed` method, also call the `incrementOperationsFailedReasons` at the same time so that the operation of these two methods are in synchronization.

requestTermination

The `BatchProcessBase.requestTermination` method has the following signature.

```
requestTermination()
```

The `requestTermination` method is a no-argument class method.

Whenever you click the **Stop** button on the Server Tools **Batch Process Info** screen, the InsuranceSuite application calls the `requestTermination` method on that batch process to terminate the batch process, if possible.

Your custom batch process must shut down any necessary systems and stop your batch process if you receive this message. If you cannot terminate your batch process, return `false` from this method.

The `BatchProcessBase` class implementation of the `requestTermination` method always returns `false`, which means that the request did not succeed. The base class also sets an internal `TerminateRequested` flag that you can check to see if a terminate request was received.

IMPORTANT: The InsuranceSuite application calls the `requestTermination` method in a different thread from the thread that runs your batch process `doWork` method.

For typical implementations, use the following pattern:

- Override the `requestTermination` method and have it return `true`. Whenever a user requests termination of the batch process, the InsuranceSuite application calls your overridden version of the method.
- Ensure that the `doWork` method in your custom class frequently checks the value of the `TerminateRequested` flag. In your `doWork` code, exit from your code if that flag is set. For example, if you are looping across a database query, exit from the loop and return.

Override the `requestTermination` method and return `false` if you genuinely cannot terminate the process soon. Be warned that if you do this, you risk the server shutting down before your process completes.

WARNING: Although you can return `false` from `requestTermination`, Guidewire strongly recommends you design your batch process so that you actually can terminate the action. It is critical to understand that returning either value does not prevent the application from shutting down or reducing run level. The application delays shutdown or change in run level for a period of time. However, eventually the application shuts down or reduces run level independent of this batch process setting. For maximum reliability and data integrity, design your code to frequently check and respect the `TerminateRequested` property.

The InsuranceSuite application writes a line to the application log to indicate whether it is possible to terminate the batch process. In other words, the log line includes the result of your `requestTermination` method.

If your batch process can only run one instance at a time, returning `true` does not remove the batch process from the internal table of running batch processes. This means that another instance cannot run until the previous one completes.

setChunkingById

The `BatchProcessBase.setChunkingById` method has the following signature.

```
setChunkingById(IQueryResult queryResult, int chunkingSize)
```

The `setChunkingById` method returns nothing.

The default the InsuranceSuite application behavior for query builder result sets is to retrieve all entries from the database into the application server. However, retrieving a large dataset as a single block can cause problems. To mitigate this issue in working with batch processes, use the `setChunkingById` method to set the query retrieve chunk size.

After setting the chunking factor, the batch process iterates the Gosu query as usual. Beneath the surface, the query retrieves the data in chunks, using a series of separate SQL queries, rather than retrieving the data all at once in one massive block. The effect is very similar to the existing `setPageSize` method on the query API. However, the differences between the two methods include the following:

- Method `Query.select().setPageSize` uses `ROWNUM` (or its equivalent) to do the chunking.
- Method `BatchProcessBase.setChunkingById` orders the results by ID and uses `WHERE ID > lastChunkMaxId` to do the chunking.

Using ID chunking is much more robust than using the `setPageSize` method if your process alters the input to the original query, as batch processes often do. For example, suppose that your batch process looks for all unprocessed items of type X and then processes them. If you use `ROWNUM` chunking then the query for the first chunk functions correctly:

```
WHERE ROWNUM < chunksize
```

After processing the returned items, the Gosu query will issue another SQL query:

```
WHERE ROWNUM >= chunksize AND ROWNUM < chunksize * 2
```

Unfortunately, this query skips `chunksize` unprocessed items because processing the first chunk of data alters the input to the subsequent query.

In general, the use of ID chunking is not useful if the query `SELECT` statement selects for certain specific columns only. Row-based chunking is frequently more useful in querying entities, if there is no change to the entities that affects subsequent queries.

The following code sample illustrates the use of the `setChunkingById` method.

```
uses gw.processes.BatchProcessBase
uses gw.processes.ProcessHistoryPurge
uses gw.transaction.Transaction

class ChunkingTestBatch extends BatchProcessBase {
    construct() {
        super(BatchProcessType.TC_TESTCHUNKING)
    }

    private var _daysOld = 5
    private var _batchSize = 1024
```

```

override final function doWork(): void {
    var query = new ProcessHistoryPurge().getQueryToRetrieveOldEntries(_daysOld)

    setChunkingById(query, _batchSize)
    OperationsExpected = query.getCount()

    var itr = query.iterator()

    while (itr.hasNext() and not TerminateRequested) {
        Transaction.runWithNewBundle(\b -> {
            var cnt = 0
            while (itr.hasNext() and cnt < _batchSize and not TerminateRequested) {
                cnt = cnt + 1
                incrementOperationsCompleted()
                b.delete(itr.next())
            }
        }, "su")
    }
}

```

In this code, notice the use of the following BatchProcessBase properties and methods:

- Property OperationsExpected
- Property TerminateRequested
- Method setChunkingById
- Method incrementOperationsCompleted

Notice also that you must create a TestChunking typecode on BatchProcessType.

Examples of custom batch processes

Example batch process for a background task

The following Gosu code is an example of a custom batch process that operates as a background task instead of one that operates on a batch of units of work. Its process history does not track the number of items that processed successfully or that failed, because it has no formal units of work.

The following Gosu batch process purges old workflows. It takes one parameter that indicates the number of days for successful processes. Pass this parameter in the constructor to this batch process class. In other words, your implementation of IProcessesPlugin must pass this parameter such as the new MyClass(myParameter) when it instantiates the batch process. If the parameter is missing or null, it uses a default system setting.

Notice also that the second constructor uses the TC_PURGEWORKFLOWS typecode on the BatchProcessType typelist. If this typecode does not exist, you must create it.

```

package gw.processes

uses gw.processes.BatchProcessBase
uses java.lang.Integer
uses gw.api.system.PLConfigParameters
uses gw.api.admin.WorkflowUtil

class PurgeWorkflows extends BatchProcessBase {
    var _succDays = PLConfigParameters.WorkflowPurgeDaysOld.Value

    construct() {
        this(null)
    }

    construct(arguments : Object[]) {
        super(typekey.BatchProcessType.TC_PURGEWORKFLOWS)
        if (arguments != null) {
            _succDays = arguments[0] != null ? (arguments[0] as Integer) : _succDays
        }
    }

    override function doWork() : void {
        WorkflowUtil.deleteOldWorkflowsFromDatabase( _succDays )
    }
}

```

IMPORTANT: You must use the `runWithNewBundle` API if you want to modify entity data in your custom batch process.

Example batch process for unit of work processing

The following Gosu code is an example of a type of a batch process that processes units of work rather than operating as a background task. Its process history tracks the number of items that were processed successfully or that failed. Its units of work are urgent activities.

The batch process implements a notification scheme for urgent activities such that:

- The activity owner must respond to an urgent activity within 30 minutes.
- If the activity owner does not handle the issue within that time limit, the batch process notifies a supervisor.
- If the supervisor fails to resolve the issue in 60 minutes, the batch process sends a message further up the supervisor chain.

If there are no qualified activities, the batch process returns `false` so that it does not create a process history. If there are items to handle, the batch process increments the count. The application uses this count to display batch process status as "n of t" or a progress bar. If there are no contact email addresses, the task fails and the application flags it as a failure.

This example checks the value of the `TerminateRequested` flag to terminate the loop if the user or the application requested to terminate the process.

Notice also, that if the `TestBatch` typecode on the `BatchProcessType` typelist does not exist, you must create it.

The code in this Gosu example for `BillingCenter` does not actually send the email. Instead, the code prints the email information to the console. You can change the code to use real email APIs.

```
package sample.processes

uses gw.api.database.Query
uses gw.api.email.Email
uses gw.api.email.EmailContact
uses gw.api.email.EmailUtil
uses gw.api.profiler.Profiler
uses gw.api.profiler.ProfilerTag
uses gw.api.util.DateUtil
uses gw.processes.BatchProcessBase
uses java.lang.StringBuilder
uses java.util.HashMap
uses java.util.Map

class TestBatch extends BatchProcessBase {
    static var tag = new gw.api.profiler.ProfilerTag("TestBatchTag1")

    //If BatchProcessType typecode TestBatch does not exist, you must create it.
    construct() {
        super( BatchProcessType.TC_TESTBATCH )
    }

    override function requestTermination() : boolean {
        super.requestTermination() // set the TerminationRequested flag
        return true // return true to signal that we will attempt to terminate in our doWork method
    }

    override function doWork() : void { // no bundle
        var frame = Profiler.push(tag)

        try {
            var activityQuery = Query.make(Activity)
            activityQuery.compare(Activity#Priority, Equals, Priority.TC_URGENT)
            activityQuery.compare(Activity#Status, Equals, ActivityStatus.TC_OPEN)
            activityQuery.compare(Activity#CreateTime, LessThan, DateUtil.currentDate().addMinutes(-30))
            OperationsExpected = activityQuery.select().getCount()
            var map = new HashMap<Contact, String>()

            for (activity in activityQuery.select()) {
                if (TerminateRequested) {
                    return
                }
                incrementOperationsCompleted()
                var haveContact = false
                var msgFragment = constructFragment(activity)
                haveContact = addFragmentToUser(map, activity.AssignedUser, msgFragment) or haveContact
            }
        }
    }
}
```

```

    var group = activity.AssignedGroup

    if (activity.CreateTime < DateUtil.currentDate().addMinutes( -60 )) {
        while (group != null) {
            group = group.Parent
            haveContact = addFragmentToUser(map, group.Supervisor, msgFragment) or haveContact
        }
    }

    if (!haveContact) {
        incrementOperationsFailed()
        addFragmentToUser(map, User.util.UnrestrictedUser, msgFragment)
    }
}

if (not TerminateRequested) {
    for (addressee in map.Keys) {
        sendMail(addressee, "Urgent activities still open", map.get(addressee).toString())
    }
} finally {
    Profiler.pop(frame)
}
}

private function constructFragment(activity : Activity) : String {
    return formatAsURL(activity) + "\n\t" + " Subject: " + activity.Subject + " AssignedTo: "
        + activity.AssignedUser + " Group: " + activity.AssignedGroup + " Supervisor: "
        + activity.AssignedGroup.Supervisor + "\n\t" + activity.Description
}

private function formatAsURL(activity : Activity) : String {
    // TODO: YOU MUST ADD A PCF ENTRYPOINT THAT CORRESPONDS TO THIS URL TO DISPLAY THE ACTIVITY.
    return "http://BillingCenter:8580/bc/Activity.go(${activity.ID})"
}

private function addFragmentToUser(map : HashMap<Contact, String>, user : User,
    msgFragment : String) : boolean {
    if (user != null) {
        var email = user.Contact.EmailAddress1

        if (email != null and email.trim().length > 0) {
            var sb = map.get(email)

            if (sb == null) {
                sb = new StringBuilder()
                map.put(user.Contact, sb)
            }

            sb.append(msgFragment)
            return true
        }
    }
    return false
}

private function sendMail(contact : Contact, subject : String, body : String) {
    var email = new Email()
    email.Subject = subject
    email.Body = "<!DOCTYPE html PUBLIC \"-//W3C//DTD HTML 4.01 Transitional//EN\">\n" +
        "<html>\n" +
        "  <head>\n" +
        "    <meta http-equiv=\"content-type\">\n" + "content=\"text/html; charset=UTF-8\">\n" +
        "    <title>${subject}</title>\n" +
        "  </head>\n" +
        "  <body>\n" +
        "    <table>\n" +
        "      <tr><th>Subject</th><th>User</th><th>Group</th><th>Supervisor</th></tr>\n" +
        "    </table>\n" +
        "  </body>\n" +
        "</html>"
    email.addToRecipient(new EmailContact(contact))
    EmailUtil.sendEmailWithBody(null, email)
}
}

```

To use this entry point, create the following PCF entry point file with name Activity.

```

<PCF>
  <EntryPoint authenticationRequired="true" id="Activity" location="ActivityForward(actvtIdNum)">
    <EntryPointParameter locationParam="actvtIdNum" type="int"/>
  </EntryPoint>
</PCF>

```

In Studio, create this Activity.pcf file in the following location: **configuration > config > Page Configuration > pcf > entrypoints**

Also, include the following ActivityForward PCF file as the ActivityForward.pcf in each place in the PCF hierarchy that you need it:

```
<PCF>
  <Forward id="ActivityForward">
    <LocationEntryPoint signature="ActivityForward(actvtIdNum : int)"/>
    <Variable name="actvtIdNum" type="int"/>
    <Variable name="actvt" type="Activity">initialValue="find(a in Activity
      where a.ID == new Key(Activity, actvtIdNum))"/>
    <ForwardCondition action="AccountForward.go(actvt.Accounts);
      ActivityDetailWorksheet.goInWorkspace(actvt)"/>
    </Forward>
  </PCF>
```

IMPORTANT: You must use the `runWithNewBundle` transaction method to modify entity data in your custom batch process. For more information, see the *Gosu Reference Guide*.

Working with custom processing

Before your custom processing can run, you must perform some, or all, of the following steps.

Categorize the process typecode

You must associate the process typecode with one or more appropriate categories in the `BatchProcessType` typelist.

Update the work queue configuration

You must add a work-queue element for your work queue in the `workqueue-config.xml` file.

Provide an implementation for the `IProcessesPlugin` plugin

You must provide an implementation of the `IProcessesPlugin` plugin that references the typecode from the `BatchProcessType` typelist for your custom process.

The steps to perform depend on whether you implemented a custom work queue or a custom batch process.

Categorizing a process typecode

For your type of custom process to run, you need to associate its typecode with appropriate categories in the `BatchProcessType` typelist. You must categorize the typecode whether you implemented a custom work queue or a custom batch process. You must add at least one runnable category or your type of custom batch processing cannot run.

The following list describes the valid runnable categories.

UIRunnable	The process is runnable both from the user interface and from the web service APIs.
APIRunnable	The process is runnable only from web service APIs.
Schedulable	It is possible to schedule the process to execute at specific dates and times.
MaintenanceOnly	It is only possible to run the process if the system is at maintenance run level.

The **Work Queue Info** screen shows your custom work queue, regardless how you categorize your process type code. Categorizing a process type code affects only whether the **Batch Process Info** screen displays custom batch processes and writers for work queues and their runnable characteristics.

Updating the work queue configuration

InsuranceSuite applications instantiate custom work queues that derive from `WorkQueueBase` or `BulkInsertWorkQueueBase` at start up, for servers in a InsuranceSuite application cluster with the `workqueue` server role. For the InsuranceSuite application to instantiate your custom work queue, you must add a work-queue element for it in the `workqueue-config.xml` file.

The work-queue element in `workqueue-config.xml` has the following syntax.

```
<work-queue workQueueClass="string" progressinterval="decimal">
  <worker instances="1" />
</work-queue>
```

The work-queue element

The work-queue element identifies your custom work queue to the InsuranceSuite application. For the `workQueueClass` attribute, specify the fully qualified class name of your custom work queue Gosu class.

The value of the `progressinterval` attribute is measured in milliseconds, and there is no default. It specifies the interval of time that the InsuranceSuite application waits for a worker task to complete an allotment, or batch, of work items checked out to it from the work queue. If a worker does not complete an allotment within the time specified, the remaining work items become orphaned, and the InsuranceSuite application assigns them to another worker task to complete.

An interval that is too short can result in many orphaned work items, which can create processing overhead by frequently reassigning work items. The default allotment, or batch size, is 10 work items. Start with a value of 600000 for `progressinterval`. If you see many orphaned work items, increase the value. A value that exceeds the actual interval required causes no processing delays.

The worker element

The XML schema requires a worker element for your custom work queue to operate. The `instances` attribute sets the number of worker tasks to create initially for the work queue at server startup. Generally a value of 1 is sufficient.

Implementing IProcessesPlugin

InsuranceSuite applications instantiate custom batch processes that derive from `BatchProcessBase` at startup, for servers in a InsuranceSuite application cluster with the batch server role. Although the InsuranceSuite application instantiates your class on all servers that have the batch server role, a single run of your custom batch process executes on only one the servers.

For an InsuranceSuite application to instantiate your custom batch process, your implementation of the `IProcessesPlugin` plugin must reference the typecode from the `BatchProcessType` typelist for your custom process. In the base configuration, the InsuranceSuite application provides a `Gosu ProcessesPlugin` class that implements the `IProcessesPlugin` interface in the following location in Guidewire Studio™:

configuration > config > gsrc > gw > plugin > processes

For each typecode in the `BatchProcessType` that represents a custom batch process, the InsuranceSuite application calls the `createBatchProcess` method of the `Processes` plugin. The `createBatchProcess` method takes a typecode from the `BatchProcessType` as a parameter. The method uses a `switch` statement and a `case` clause to determine which process type to instantiate.

The following example code demonstrates how to implement the `IProcessesPlugin` plugin. The code instantiates the individual custom batch process in the `switch` case statements.

```
uses gw.plugin.processing.IProcessesPlugin
uses gw.processes.BatchProcess

@Export
class ProcessesPlugin implements IProcessesPlugin {

  construct() { }

  override function createBatchProcess(type : BatchProcessType, arguments : Object[]) : BatchProcess {
    switch(type) {
      case BatchProcessType.TC_PURGEMESSAGEHISTORY:
        return new PurgeMessageHistory(arguments)
      case BatchProcessType.TC_CUSTOMBATCHPROCESS:
        return new CustomBatchProcess()
      default:
        return null
    }
  }
}
```


Passing initialization parameters to batch process constructors

Every time that an InsuranceSuite application executes a batch process, whether it is a scheduled process, or requested manually or through web services, the InsuranceSuite application creates a new instance of the batch process class. Thus, it is important the class constructor not perform programming logic.

However, it is possible to pass one or more initialization parameters to a constructor that creates the batch process. For example, in BillingCenter, the implementation class for the PurgeMessageHistory batch process type contains two constructors, one that takes no arguments and one that does take arguments.

```
class PurgeMessageHistory extends BatchProcessBase {
    var _ageInDays : int

    construct() {
        this({BCConfigParameters.KeepCompletedMessagesForDays.Value})
    }

    construct(arguments : Object[]) {
        super(BatchProcessType.TC_PURGEMESSAGEHISTORY)
        if (arguments.length == 1 and arguments[0] typeis Integer) {
            _ageInDays = Coercions.makeIntFrom(arguments[0])
        } else {
            _ageInDays = BCConfigParameters.KeepCompletedMessagesForDays.Value
        }
    }
    ...
}
```

Notice that the constructor with arguments performs no actual programming logic. It simply determines the value to use in initializing the batch process.

Starting processes with initialization parameters from command prompt

Use the `-startprocess` command, available with the `maintenance_tools` command prompt utility, to start a batch process. Using this command, it is possible to start the batch process with, or without, initialization parameters. Use the following syntax.

```
maintenance_tools -password password -startprocess process [-args arg1 arg2 ...]
```

To use arguments in instantiating custom batch processing, use the `createBatchProcess(type, args)` method on the `ProcessesPlugin` class. In this way, you can pass initialization parameters to the batch process constructor called by the new operation.

Starting processes with initialization parameters from web services

Class `MaintenanceToolsAPI` defines a `startBatchProcess` method that accepts initialization parameter and that returns a `ProcessID` value:

```
function startBatchProcessWithArguments(processName : String, arguments : String[]) : ProcessID {
    return getDelegate().startBatchProcess( processName, arguments )
}
```

Use this method to start a batch process with initialization parameters, for example:

```
processID = maintenanceToolsAPI.startBatchProcessWithArguments("myCustomBatchProcess", args)
```

The typecode for the batch processing type must include the `APIRunnable` category in order to start your custom batch process with the `MaintenanceToolsAPI` web service.

Retrying failed batch process items

In the base configuration, ClaimCenter provides abstract class `RetryingBatchProcess` that you can extend to create a custom batch process that retries items that failed during execution of the batch process.

Abstract class `RetryingBatchProcess`

Class `RetryingBatchProcess` provides an abstract extension class for `BatchProcessBase`, with the added ability that the batch process attempts to retry items that failed during the initial execution of the batch process. After the batch process completes its initial execution, the process retries all items that failed during the execution of the process. If any of the retried items still fail, the process does not retry those items again. Instead, the process logs the retry failure in the application log.

The default `ClaimCenter` implementation of the `RetryingBatchProcess` class retries a failed item only if the failure was due to a concurrent data change exception. Class logic considers all other exceptions as unrecoverable failures.

Public methods on class `RetryingBatchProcess`

Class `RetryingBatchProcess` contains the following public methods. Some of these methods are themselves abstract.

`getChunkSize()`

The `getChunkSize` method returns the value set for chunking the results of the query used for fetching items for processing. The default value is 100. Guidewire recommends that you do not change this value without thought. Especially, do not change this value to either 0 or an extremely high number.

`fetchItemsForProcessing()`

The `fetchItemsForProcessing` method returns the objects for processing as an `IQueryBeanResult<T>` object. You must provide an overridden method body for this abstract class method in your custom batch process.

`processItem(item)`

The `processItem` method defines the actual work to perform on each individual batch item. The item for processing is the passed-in `item` value. You must provide an overridden method body for this abstract class method in your custom batch process.

Guidewire guarantees that class logic wraps each item for processing in a writable bundle and active transaction. Therefore, any implementation of this method need not try to create its own transaction or move the item to any other bundle.

`doWork()`

`ClaimCenter` marks the `doWork` class method as `protected` and `final`. It is not possible to override the `ClaimCenter` implementation of this class method.

The `doWork` method does the following:

1. It calls class method `fetchItemsForProcessing`, which returns a set of `IQueryBeanResult` objects.
2. It processes each returned item in its own bundle by calling private class method `processItemAndCommit` (which calls class method `processItem` on each item).
3. If the `processItem` method throws an exception, the `doWork` method calls class method `shouldRetryItem` on that item.
4. If the `shouldRetryItem` returns `true` for an item (which means the exception was a concurrent data change exception), the `doWork` method adds the item to the internal list of items for retry.
5. If the `shouldRetryItem` returns `false` (for all other exception types), the `doWork` method calls private method `handleUnrecoverableException` on the exception.
6. After the `doWork` method processes all fetched items, it calls class method `retryProcessing` on each failed item (which method calls the `processItem` method on each item again, by default).

`shouldRetryItem(processingEx, item)`

The `shouldRetryItem` method determines if it is possible to recover from an exception that happened during the processing of a specific item. The passed-in method arguments have the following meaning:

- `item` - The item that failed its initial processing.
- `processingEx` - The exception thrown during the initial processing of the item.

The method returns `true` if it is possible to recover from the exception, and thus, it is possible to retry processing the item. The method returns `false` otherwise.

In the default implementation, the method returns `true` only if the processing failure throws a concurrent data change exception. The method considers all other types of exceptions to be unrecoverable failures. It is possible for your custom implementation of this method to override this basic implementation logic.

prepareFailedItemForRetry(item)

The `prepareFailedItemForRetry` method determine if an item queued for retry is still eligible for processing. The `doWork` method calls the `prepareFailedItemForRetry` method during execution of that method.

It is necessary to call this method as it is possible for the item to have been modified externally, with those modifications rendering the item ineligible for reprocessing. The method returns one of the following:

- The object on which to retry processing
- `null`, if the method determination that it is not possible to retry processing on the object

In the default implementation, the method simply refetches the passed-in `item` from the application cache or database. Guidewire recommends that your custom implementation of this method override this basic implementation logic to impose specific eligibility requirements on the item. In your implementation of this class, call `super.prepareFailedItemForRetry`.

retryProcessing(item)

The `retryProcessing` method retries processing a specific item that had originally failed processing due to a recoverable exception. In the default implementation, the method retries processing the passed-in `item` only once. If the item processing fails again during the retry, the method considers the failure to be a permanent failure and does not attempt any further processing of the item. The method logs the retry failure to the application log.

It is possible for your custom implementation of this method to override this basic implementation logic. Be aware that the default implementation of this method calls an internal method to wrap the action in a bundle. The method then increments the number of completed operations. In your implementation of this class, call `super.retryProcessing`, if you can. Otherwise, you need to create your own bundle and track the number of completed operations in your code.

getItemDescription(item)

The `getItemDescription` method returns a human-friendly description (`String`) of the passed-in `item` for use in log statements.

Creating a custom retry batch process class

Your custom class must do the following:

- Extend abstract class `RetryingBatchProcess`.
- Provide overridden method bodies for all `RetryingBatchProcess` class methods declared abstract.
- Provide overridden method bodies for those default method implementations that do not meet your business needs.

Your custom Gosu class must include a class constructor of the following form:

```
construct () {
    //If BatchProcessType typcode CustomWorkQueue does not exist, you must create it.
    super (typekey.BatchProcesstype.TC_CUSTOMWORKQUEUE)
}
```

You must create a typecode in typelist `BatchProcessType` for your custom batch process.

Monitoring InsuranceSuite application processes

Although InsuranceSuite application processes run without human intervention as background tasks, it is important to manage the processes and monitor their progress. InsuranceSuite provides a number of features for monitoring default and custom processes.

Monitoring InsuranceSuite application processing using administrative screens

InsuranceSuite provides two Server Tools administration screens that you can use to manage and monitor batch processes and work queues. They are:

- **Batch Process Info** screen
- **Work Queue Info** screen

The Batch Process Info screen

System administrators can use the (Server Tools) **Batch Process Info** screen to start and view information about InsuranceSuite application processes, including writer processes for work queues.

The Work Queue Info screen

System administrators can use the (Server Tools) **Work Queue Info** screen to control and view information about InsuranceSuite application work queues. From this **Work Queue Info** screen, you can also download process history information on individual work queues.

Monitoring InsuranceSuite application processes for completion

Night-time processing frequently requires chaining, meaning that the completion of one type of process starts another type of follow-on process. Regardless of implementation mode – work queue or batch process – you most likely must develop custom process completion logic for your type of nightly batch processing.

Characteristics of a completed process run vary depending on the implementation mode:

Work queues

Complete if no work items for a batch remain in the work queue, other than work items that failed

Batch processes

Complete if the batch process stopped and its process history is available

In the base InsuranceSuite application configuration, Guidewire provides an `IBatchCompletedNotification` interface that you can implement and call from the Process Completion Monitor process.

```
public interface IBatchCompletedNotification extends InternalPlugin {
    void completed(ProcessHistory var1, int var2);
}
```

The completed method takes the following parameters.

var1 Process history

var2 Number of failed work items

In your implementation of this interface, override the `completed` method to perform specific actions after a work queue or batch process completes a batch of work. Add a case clause to the method for your type of custom batch processing by specifying its typecode from the `BatchProcessType` typelist, for example:

```
switch(type) {
    case BatchProcessType.TC_ACTIVITYESC:
        return new ActivityEscalationWorkQueue()
    case BatchProcessType.TC_MYCUSTOMBATCHPROCESS:
        return new MyCustomBatchProcess()
    ...
    default:
        return null
}
```

Monitoring batch processes using maintenance tools

You can start, terminate, or retrieve the status of batch processes, including writers for work queues, by using the `maintenance_tools` command or the `MaintenanceToolsAPI` web service. The web service provides additional functions

not available with the command, such as accessing and modifying configuration properties for work queues and notifying workers of work on the queue.

For information on web services, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Managing user entity updates in batch processing

It is possible for an update to the User entity to null out additional fields on the entity if the executing user is a InsuranceSuite application System user. This situation can arise during batch processing or during the execution of code in the Studio Gosu tester. For example, if the code sets the value of `User.ExternalUser` to `true`, The InsuranceSuite application also changes a number of other fields within the entity automatically. These additional changes include:

- Setting the value of `User.UserType` to `Other`
- Setting the value of `User.Organization` to null
- Setting the Address fields on the associated Contact entity to null

Specify a Current User that belongs to the same organization as the User executing the code so that the User inherits the organization information of the Current User. The Current User must be an external user as well.

Set the external user field in a batch process

About this task

If you need to set the `User.ExternalUser` field in a batch process, Guidewire recommends that you take the following steps to manage this process.

Procedure

1. Set the `User.ExternalUser` property to the specified value.
2. Retrieve the value of Current User that the bundle specifies.
3. If the User is not an external user, use the existing organization to which the user belongs. In other words, do not clear this field.
4. If the User is an external user:
 - Remove all the insurance company's internal roles for the User, meaning all roles for which `Role.isCarrierInternalRole` is true.
 - Set the User's organization to the Current User's organization.
5. If the User's organization is not the same as the Current User's organization, remove all the groups and producer codes associated with the User.
6. If the User's organization is not an insurance company, set the `User.UseProducerCodeSecurity` property to true.
7. If the value of `User.UseOrgAddress` is true, set the user's contact address information to the organization address.
8. Set the `User.UserType` property to `Other`.

